



SPRING

COURSE MATERIAL

Volume 2



NARESH



TM

Opp. Satyam Theatre, Ameerpet, Hyderabad - 16
Ph: 040-23746666, 23734842 Cell: 9000994007 / 08

technologies

B.O. Kaba Plaza, 3rd Floor, LB Road, Adyar, Chennai - 20. Phone : 044-6555 5557, 6555 5558

info@nareshit.com www.nareshit.com nareshit nareshitech



SPRING

COURSE MATERIAL

Volume 2



NARESH



Opp. Satyam Theatre, Ameerpet, Hyderabad - 16
Ph: 040-23746666, 23734842 Cell: 9000994007 / 08

technologies

B.O. Kaba Plaza, 3rd Floor, LB Road, Adyar, Chennai - 20. Phone : 044-6555 5557, 6555 5558

info@nareshit.com www.nareshit.com [nareshit](https://www.facebook.com/nareshit) [nareshitech](https://www.twitter.com/nareshitech)

26. 100% java code based spring MVC

176 - 191

Abstract Controller

192 - 220

27. Dynamic Registration of Servlet

221 - 224

28. Spring Boot MVC

224 - 264

Spring Security:

1. Introduction

265 - 284

2. 100% Code Based Spring Security

285 - 291

3. LDAP Configuration

292 - 317

4. Spring Security Boot

318 - 324

Spring Social:

1. Introduction

325 - 340

2. Accessing face book data

341 - 351

3. Accessing twitter data

352 - 361

4. Spring social Example

362 - 405

SPRING ORM:

1. Introduction

406 - 454

Spring Batch:

1. Introduction

455 - 495

SPRING FAQs

496-524

Spring Index

Spring MVC Module:

1. Introduction	1 - 7
2. Controller Classes	7 - 9
3. View Resolver	10 - 10
4. Controller	11-11
5. ParameterizableViewController	12 - 16
6. UriFileNameViewController	17 - 23
7. Abstract Controller	24 - 35
8. Command Class, AbstractCommandController	36 - 50
9. Internal flow of Command Controller	51 - 51
10. SimpleFormController	52 - 60
11. Form Validation using SimpleFormController	62 - 71
12. Internal flow of form Validation	72 - 86
13. ReferenceData	87 - 96
14. MultiActionController	97 - 102
15. AbstractWizardFormController	103 - 110
16. HandlerMapping	111 - 118
17. HandlerInterceptor	119 - 122
18. ViewResolver	123 - 134
19. Internationalization (i18n)	135 - 138
20. Annotation driven MVC	139-145
21. Annotation base AbstractCommandController	146 - 150
22. Annotation base Simple Form Controller	151 - 160
23. File Uploading and Downloading	161 - 167
24. MVC with Tiles	168 - 173
25. Exception Handling	174 - 175

Spring MVC

Different web app developments

<u>Model1</u>	<u>Model2</u>
Here either jsp or Servlets will be taken as main web resource programs having multiple logics.	In model 2 we take multiple technologies in multiple layers to develop various logic. This gives clean separation between logics.
Since every servlet/jsp contains multiple logics, there is no separation between logics. So this architecture is not suitable for large scale web application.	In MVC2 architecture we take separate resources for view, separate resources for model and separate resource for controller
MVC1 architecture we take single resource (server/jsp) having view, controller logics and we take separate resources for model layer logics.	

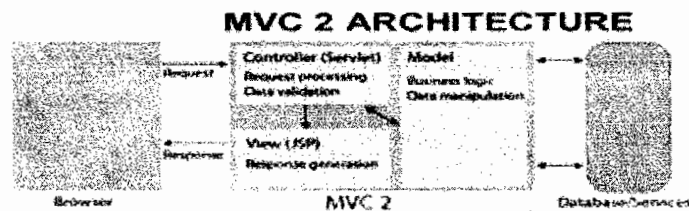
M---->Model---->Data+B.logic+persistence logic

V---->View----->persistence logic

C----->controller--->integration logic/controller logic.

Controller maintains logic all the activities of web application like takes the request, passes the request to appropriate model component, gathers requests from model and passes the results to view.

MVC2 Architecture:



- * MVC 2 isolates the presentation logic from business logic.
- * In MVC 2 controller is servlet and model is java class.
- * Supports re usability of application components.

Diagram:

Multiple backend software together is called EIS.

The project that is developed by using old technology or old versions of existing technologies is called Legacy project.

Eg: COBOL project, Java 1.0 project

Advantages of MVC2:

- >Because of multiple layer we can get clean separations between logics.
- >The modification done in layer logics does not affect other layer logics.
- >Maintainance and enhancement, so gives good productivity.
- >While using EJB, spring in the model layer we can take advantage of built-in middleware services.

Disadvantages:

- >Knowledge on more technologies are required.
- >For parallel development more programmers are required.
- >Every web application contains some common logic to handle along with application logics.

Importance Terminologies**⇒ Request graphing/wrapping:**

Reading data from the form comps of form pages and placing that data in a java class object.

⇒ Request Submission/form submission:

We get request from different places of web pages like through hyperlinks, through submit buttons, through other components. These request may come with or without data, so we should handle all the request properly.

⇒ Form Validation/Form handling:

Verifying the pattern, format of form data and displaying same form page with errors having old values in form component.

If we develop web application by using JEE technologies directly we need to write the above common logics in multiple servlet programs, which leads boiler plate code problem.

These are like system services (security, logging and etc...).

If we develop MVC2 architecture web applications by using JEE technologies directly (Servlet, JSP) we need to develop all the logics of all the layers manually moreover we need to write both common and application specific logics.

MVC2 Design pattern/Architecture rules:

- a) Every layer is designed to have specific logics, so place only those logics don't place any additional logics.
- b) All the operations of application must be take place under the control or monitoring of

controller servlet.

- c) There can be multiple resources in view layer, multiple resources in model layer but there must be only 1 servlet acting as controller. This must be taken as front controller.
- d) The view and model layer comps must not interact with each other directly. They must interact through controller.

If we develop mvc2 architecture based web application manually. All the logics of all the layers (including common, application specific logics) must be taken care by programmer.

If we develop mvc2 architecture based web application by using web framework then the common logics will be taken care by the framework by giving one built-in servlet as frontController so the programmer just need to develop application specific logics.

The common logics are like request wrapping, form/request submission, form handling services.

The web frameworks are:-

- ⇒ Struts -----> from apache
- ⇒ JSF----->from SunMS (Oracle) (2)
- ⇒ Webwork-----> from opson symphony
- ⇒ ADF----->from Oracle corp (3)
- ⇒ SpringMVC-----> from inteface21 (1)

Q) While developing a MVC2 architecture based web application can we change the roles of various technologies that are used?

Ans) Yes, we can change but it is not recommended.

Servlet as View

Presentation logic changes at regular intervals during modification in JSP is quite easy compare to doing modification in servlet components. So it is recommended to use jsp as view...

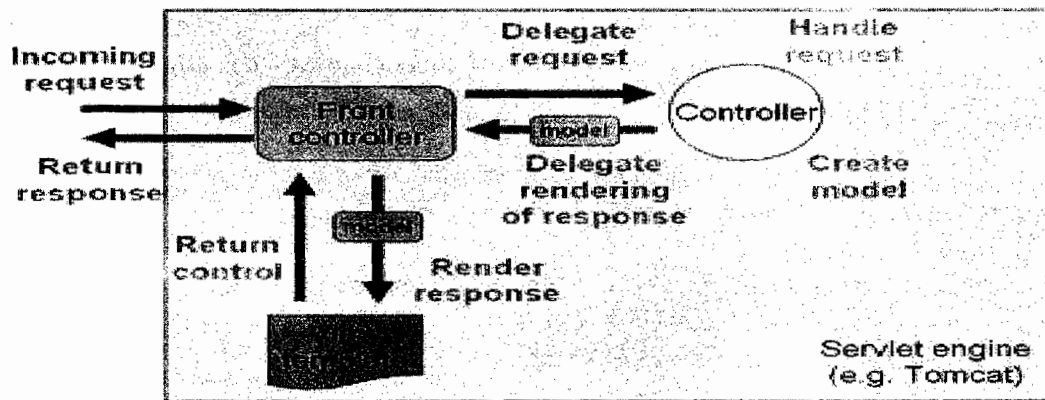
JSP as Controller

Writing Java code in JSP is a bad practice when we use JSP as a controller we should place JAVA code in JSP to interact with model layer business component.

So it is recommended to use servlet /Servlet filter as controller where placing java code is not a problem.

Java class /EJB component/Spring bean as View and controller components

This thought is wrong because they cannot take requests from clients and they cannot deliver response to browser.

Front ControllerDiagram:

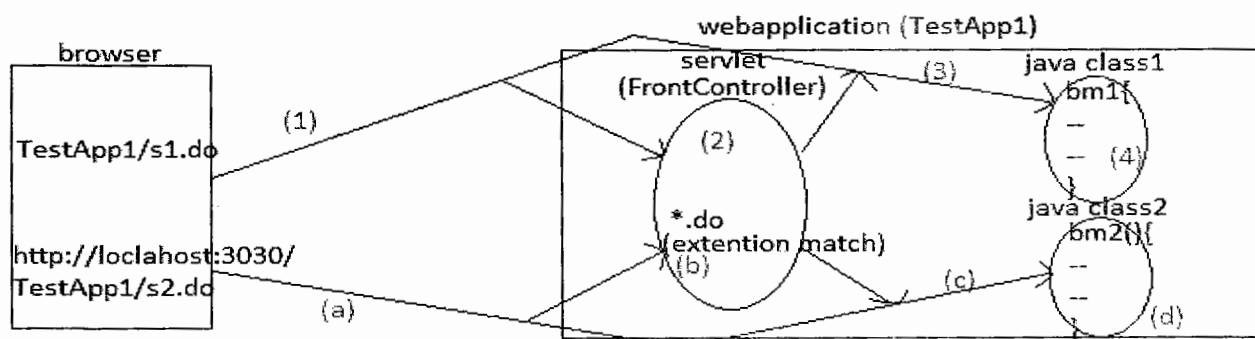
- ⇒ The special web component of web application that acts as entry point for all the request and applies all common system services for all the request like security, logging and etc is called front controller.
- ⇒ We can take either servlet/JSP for front controller. To make front controller trapping and taking multiple request coming from clients we must configure it either with directory match or extension match url pattern.

=> *.do, *.htm, *.http and etc are extension match url patterns.
 => /x/*, /x/y/* and etc are directory match url pattern.

In struts ActionServlet is built-in front controller servlet.

In spring MVC DispatcherServlet is built-in front controller servlet.

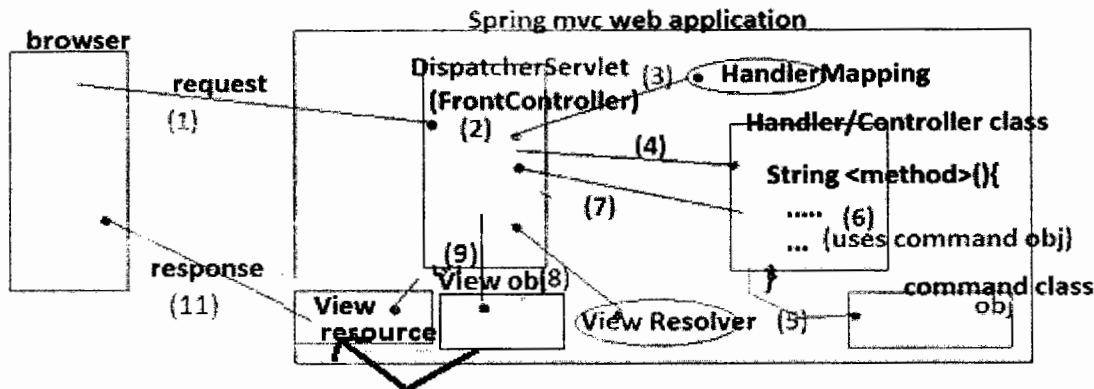
- ⇒ Java classes of web application cannot take request directly from browser. So we make those request getting trapped by front controller to delegates the request to appropriate Java class...



- We take ServletFilter as InterceptingFilter having optional servicers to apply on web app having the ability to enable or disable (like allowing only certain browser request and etc...).
- Use FrontController to apply common system services on all the request (like security, logging and etc...).

- Use *InterceptingFilter* to apply optional services having the ability to enable or disable.
- Don't configure *FrontController* servlet with */** url pattern to allow certain request directly going to web comps without applying system services.
- If needed we can place both *FrontControllers*, *InterceptingFilters* in one web application development.

Spring MVC web application Flow



(1) Browser gives request to web application

(2) As frontcontroller DispatcherServlet traps and takes the request and applies the common system services like security and etc....

(3) DispatcherServlet uses HandlerMapping component to decide the Handler class to utilize based on the incoming request uri.

(4) DispatcherServlet passes the control to Handler class by calling method.

(5) Handler class internally writes the received form data to command class object.

(6) Handler class process the request and generates the output, if needed it also uses command object work with form data. Sometimes handler class delegates the request to other external components like *ejb*, *webservices* and etc...

(7) Handler class returns logical view name back to DispatcherServlet.

(8) DispatcherServlet uses view Resolver to get view object having view layer technology and resource name.

(9) DispatcherServlet uses the view object to pass the control to view resource.

(10 & 11) The view resource format the results and sends the output to browser (response to browser).

Spring MVC features:

- ⇒ Allows to use various technologies in the view layer like html, jsp, velocity, freemarker, xslt, etc...
- ⇒ Allows to design command class (whose object holds form data) as pojo class (Need not to extend or implement spring api class/interfaces)
- ⇒ Allow to take any type properties in command class to hold the form data.
- ⇒ Supports internationalization (i18n)
(making our app working for different Locales).
- ⇒ Allows to configure themes for rendering better presentation.
- ⇒ Allows to develop b.logics, form validation logics and other logics as reusable logics.
- ⇒ Provides clean separation between logics/comps by involving different handler mapping, views, viewresolver, command classes, handler/controller classes and etc...
- ⇒ Allows to integrate with other modules of spring framework and etc..

Spring MVC is designed around **DispatcherServlet** because various operations on the request will be done by DispatcherServlet by using different components to process the Request and to deliver the response.

Even though it is predefined servlet its configuration in web.xml is mandatory. We configure this servlet with either extension match or directory match url pattern to make that servlet as frontcontroller to take multiple requests.

This servlet internally activates IOC container of type `WebApplicationContext` by taking **<DispatcherServlet logical name given in web.xml>-servlet.xml** or specified filename (as namespace init param value) in spring bean configuration file.

Spring bean configure file contains Handler mapping, Controller class, viewresource and etc...

web.xml

```
<web-app>
<servlet>
  <servlet-name>dispatcher </servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <int-param>
    <param-name>namespace </param-name>
```



```
<param-value>myConfig</param-value>
</init-param>
<load-on-startup>2 </load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher </servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>
```

If this namespace init parameter is not configured the IOC container created by DispatchServlet (ds) takes **dispatcher-servlet.xml** as springbean configure file. If it is configured the IOC container takes **myConfig.xml** as spring bean configuration file.

It is recommended to place jsp files inside WEB-INF folder or in its sub folders. The reasons are:-

- 1) We can hide the jsp technology that is used in the webapplication development from enduser.(protection from hackers)
- 2) If jsp page is displaying request attributes given by servlet the direct request to jsp will display ugly values. We can avoid this ugly messages by not allowing direct request to jsp for that place jsp inside the WEB-INF folder.
- 3) If session is not logged out properly hackers can access previous session by appending sessionid to the request url of jsp collected from browser history. This can be avoided by keeping jsp's inside the WEB-INF folder and also tacking the support of filter.
- 4) When jsp's are place inside WEB-INF folder the underlying server/container and other web components of web application access it directly even though it is not configured in web.xml having URL pattern. Spring MVC provides to locate jsp components of WEB-INF folder for other components of application.

Controller classes:-

In order to develop our controller/handler class we must take a class implementing `org.springframework.web.servlet.mvc.ControllerInterface` and should provide implementation for `handleRequest (-,-)` method having logic to process the request and to return ModelAndView object having logical view name.

Ex:

```
public class MyController implements Controller{
    public ModelAndView handleRequest (HttpServletRequest req, HttpServletResponse res)
    {
        .....
        .....
        return ModelAndView ("home");
    }
}
```

Every controller/handler class must be configured in spring configuration file having bean id and that bean id must be mapped with incoming request url by using one or other `HandlerMapping`. `HandlerMapping` is responsible to map incoming request urls to handler classes (controller classes).

Every `HandlerMapping` is a class that implements `org.springframework.web.servlet.HandlerMapping` interface. The most regularly used handler mapping class is `"SimpleUrlHandlerMapping"`.

Sample Code:

```
<!--controller class-->
<bean id="wc" class="com.nt.controller.WelcomeController"/>
<!--HandleMapping-->
<bean id="surl" class="org.springframework.web.servlet.SimpleUrlHandlerMapping">
    <property name="mappings">
        <props>
            <prop Key="home.htm">wc</prop>
        </props>
    </property>
</bean>
```

1st spring mvc web application that displays "welcome.jsp" of WEB-INF/pages folder upon receiving the request

- MVCFirstWeb
 - ▶ JAX-WS Web Services
 - Java Resources
 - src
 - com.nt.controller
 - ▶ WelcomeController.java
 - Libraries
 - ▶ commons-logging.jar - C:\Users\dell\Des
 - EAR Libraries
 - ▶ JRE System Library [JavaSE-1.7]
 - MVCLib
 - ▶ spring-aop-4.1.6.RELEASE.jar - C:\User
 - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Us
 - ▶ spring-context-4.1.6.RELEASE.jar - C:\
 - ▶ spring-context-support-4.1.6.RELEASE
 - ▶ spring-core-4.1.6.RELEASE.jar - C:\Use
 - ▶ spring-expression-4.1.6.RELEASE.jar - C
 - ▶ spring-web-4.1.6.RELEASE.jar - C:\User
 - ▶ spring-webmvc-4.1.6.RELEASE.jar - C\
 - Web App Libraries
 - ▶ JavaScript Resources
 - ▶ build
 - WebContent
 - ▶ META-INF
 - WEB-INF
 - lib
 - pages
 - welcome.html
 - dispatcher-servlet.xml
 - web.xml

Request URL:

Request URL : <http://localhost:3030/MVCFirstWeb/home.htm>

web.xml:

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="wc" class="com.nt.controller.WelcomeController"/>
  <bean id="surl"
class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
```

```
<property name="mappings">
  <props>
    <prop key="home.htm">wc</prop>
  </props>
</property>
</bean>
<bean id="vr"
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>
```

welcome.jsp:

```
<h1><center> HOME PAGE</center></h1>
  ${requestScope.msg}
```

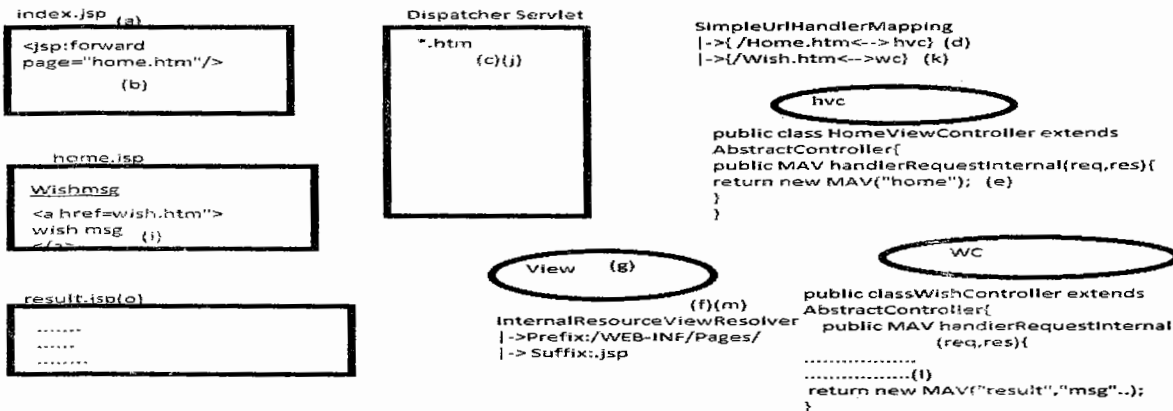
WelcomeController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;
public class WelcomeController implements Controller {
    @Override
    public ModelAndView handleRequest(HttpServletRequest req,
    HttpServletResponse res) throws Exception {
        return new ModelAndView("welcome", "msg", "Welcome To Spring MVC");
    }
}
```

ViewResolver:

- ⇒ This view Resolver takes the logical view name given by controller class and resolves physical view name by adding prefixes, suffixes and returns View interface implementation class object back to DispatcherServlet having physical view name.
- ⇒ DispatcherServlet calls render() method on the given View implementation class object and this render() method passes the control to physical view resource.
- ⇒ Every ViewResolver is a class that implements org.springframework.web.servlet.ViewResolver interface class. Spring has given multiple built-in ViewResolver classes in that InternalResourceViewResolver is useful to return view implementation class object having servlet, jsp programs placed in the WEB-INF folder of the web application (internal resource).

Spring MVC web app2 that launches homepage and process the request based on the hyperlink clicked in homepage.

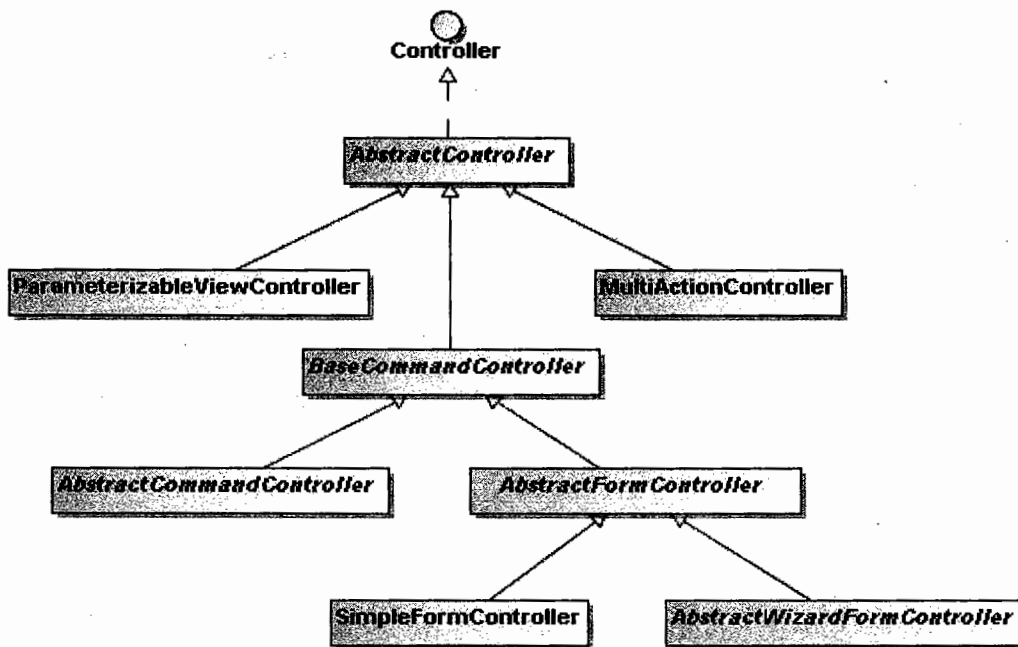


Note:

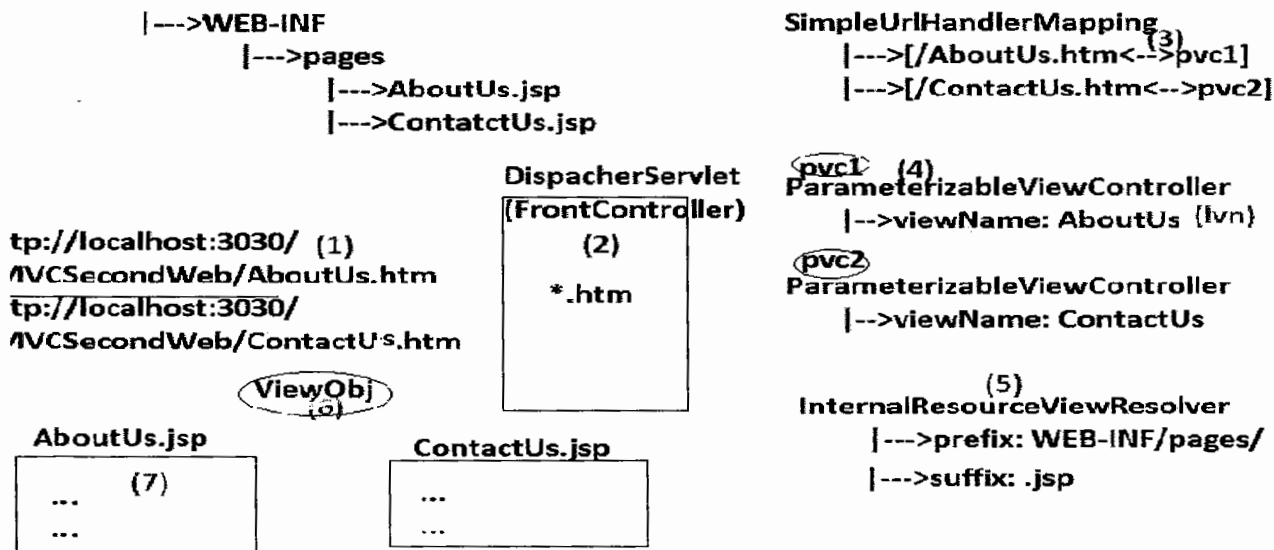
- ⇒ In spring mvc application only welcome pages will be placed outside the WEB-INF Folder and remaining jsps are recommended to place inside WEB-INF folder for security reason. Welcome page executes automatically for every request given to web application and gives implicit request by using `<jsp:forward>` tag or `response.sendRedirect(-)` method to launch other pages that are there inside WEB-INF folder through controller/handler classes.
- ⇒ It is always recommended to develop our controller/handler class by extending from predefined `XxxController` classes. That means it is not recommended to developer same class by implementing Controller interface directly.

Controller/Handler:

- ⇒ The java class that implements `org.springframework.servlet.mvc.Controller()` is called controller/Handler. It is mainly useful to render logical viewname and data that is required to display in view layer resource.
- ⇒ Placing b.logic/request processing logic in controller class is not recommended process. It is recommended to place such logics in service classes and communicate with them from controller class.
- ⇒ Spring API has supplied lots of built-in controller classes implementing "Controller" interface. These classes are given to handle the request coming from form pages, hyperlinks and from other form components...

The Controller Classes are:**ParameterizableViewController:**

- ⇒ In order to display webpage without processing request, then instead of taking separate user defined controller class we can configure this class by specifying the destination view name using "viewName" property value. This is useful to display webpage for hyperlink generated request without having any request processing logics
- ⇒ Eg: clicking "AboutUs", "ContactUs" hyperlinks. For multiple request of this kind we need to configure this class for multiple times.

Diagram:

SimpleUrlHandlerMapping

```
|--->[/Aboutus.htm<->pvc1]  
|--->[/contact.htm<->pvc2]
```

PVC1**ParameterizableViewController**

```
|----->ViewName: AboutUs (logical viewname)
```

PVC2**ParameterizableViewController**

```
|----->ViewName: ContactUs
```

InternalResourceViewResolver

```
|----->Prefix:/WEB-INF/Pages/  
|----->Softfix:.jsp
```

- ▲  MVCSecondWeb
 - ▷  JAX-WS Web Services
 - ▷ Deployment Descriptor: MVCSecondWeb
 - ▲  Java Resources
 - ▲  src
 - ▲  com.nt.controller
 - ▷  ViewHomeController.java
 - ▷  WishController.java
 - ▷  Libraries
 - ▷  JavaScript Resources
 - ▷  build
 - ▲  WebContent
 - ▷  META-INF
 - ▲  WEB-INF
 - ▷  lib
 - ▲  pages
 - ▷  AboutUs.jsp
 - ▷  ContactUs
 - ▷  home
 - ▷  result
 - ▷  welcome.jsp
 - ▷  dispatcher-servlet.xml
 - ▷  web.xml
 - ▷  index.isn

```
HttpServletResponse res) throws Exception {  
    // get System date and time  
    Calendar cl=Calendar.getInstance();  
    //get current hour of the day  
    int hour=cl.get(Calendar.HOUR_OF_DAY);  
    String msg=null;  
    if(hour<12)  
        msg="Good morning";  
    else if(hour<16)  
        msg="Good afternoon";  
    else if(hour<20)  
        msg="Good evening";  
    else  
        msg="Good Night";  
    return new ModelAndView("result","wmsg",msg);  
}  
}
```

ViewHomeController.java

```
package com.nt.controller;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractController;  
public class ViewHomeController extends AbstractController {  
    @Override  
    protected ModelAndView handleRequestInternal(HttpServletRequest req,  
        HttpServletResponse res) throws Exception {  
        return new ModelAndView("home");  
    }  
}
```

WishController.java

```
package com.nt.controller;  
import java.util.Calendar;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractController;  
public class WishController extends AbstractController {  
    @Override  
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
```


index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="surl" class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="home.htm">vhc</prop>
        <prop key="wish.htm">wc</prop>
        <prop key="AboutUs.htm">ufv</prop>
      </props>
    </property>
  </bean>
  <bean id="vhc" class="com.nt.controller.ViewHomeController"/>
  <bean id="wc" class="com.nt.controller.WishController"/>
  <bean id="ufv" class="org.springframework.web.servlet.mvc.UrlFilenameViewController"/>
  <!-- <bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="AboutUs"/>
  </bean>
  <bean id="pvc2" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="ContactUs"/>
  </bean> -->
  <bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
```


UrlFileNameViewController:

- ⇒ If you want to return the virtual path like (AboutUs) of incoming request url as logical view name to render view/webpages without having any request processing logic then we can use this controller.
- ⇒ This is alternate to using multiple ParameterizableViewController classes configuration to render multiple webpages/views without having any request processing for multiple hyperlink generated request like aboutus, contactus and etc....
- ⇒ While configuring this class no need of specifying any property because it takes the virtual path of request url (AboutUS.htm) and return it as logical viewname(AboutUs) automatically.

```
SimpleUrlHandlerMapping
```

```
|----->[/AboutUs.htm<--> UFNVC]
```

```
UFNVC
```

```
UrlFileNameViewController.
```

```
MVCSecondWeb
```

```
|----->webcontent
```

```
|--->WEB-INF
```

```
|--->pages
```

```
|--->AboutUs.jsp
```

```
|--->ContactUs.jsp
```

```
SimpleUrlHandlerMapping
```

```
|--->[/AboutUs.htm<-->vfc1 ]
```

```
|--->[/ContactUs.htm<-->vfc1 ]
```

```
vfc1 (4)
```

```
UrlFileNameViewController
```

```
|-->viewName: AboutUs (/vn)
```

```
(5)  
InternalResourceViewResolver
```

```
|--->prefix: /WEB-INF/pages/
```

```
|--->suffix: .jsp
```

```
http://localhost:3030/ (1)
```

```
MVCSecondWeb/AboutUs.htm
```

```
http://localhost:3030/
```

```
MVCSecondWeb/ContactUs.htm
```

```
DispatcherServlet
```

```
(FrontController)
```

```
(2)
```

```
*.htm
```

```
ViewObj  
(6)
```

```
AboutUs.jsp
```

```
(7)
```

```
...
```

```
...
```

```
ContactUs.jsp
```

```
...
```

```
...
```

-  MVCWishApp-UFVC4
 - >  Deployment Descriptor: <web app>
 - ✓  Java Resources
 - ✓  src/main/java
 - ✓  com.nt.controller
 - >  WishController.java
 -  src/main/resources
 - >  src/test/java
 - ✓  Libraries
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 - >  JavaScript Resources
 - >  Deployed Resources
 - ✓  src
 - ✓  main
 - >  java
 -  resources
 - ✓  webapp
 - ✓  WEB-INF
 - ✓  pages
 -  about_us.jsp
 -  contact_us.jsp
 -  result.jsp
 -  welcome.jsp
 -  dispatcher-servlet.xml
 -  web.xml
 -  index.jsp
 - >  test
 - >  target
 -  MVCUrlFileNameViewController.png
 -  pom.xml

WishController.java:

```
package com.nt.controller;

import java.util.Calendar;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;

public class WishController extends AbstractController {

    @Override
    public ModelAndView handleRequestInternal(HttpServletRequest req, HttpServletResponse
res) throws Exception {
        Calendar cal=null;
        int hour=0;
        String wMsg=null;
```

```
//get System Date
cal=Calendar.getInstance();
//get Current hour of the day
hour=cal.get(Calendar.HOUR_OF_DAY);
//generate Wish Message...
if(hour<=12)
    wMsg="Good Morning";
else if(hour<=16)
    wMsg="Good AfterNoon";
else if(hour<=20)
    wMsg="Good Evening";
else
    wMsg="Good Night";
//create and return MAV
return new ModelAndView("result","msg",wMsg);
} //handleRequestInternal(-,-)
} //class
```

about_us.jsp:

```
<h1 style="color:red;text-align:center">About Us Page</h1>
<i>Naresh i Technologies, a 15 year old IT training and placement
organization trained 10 lac students far with track recors of
scuccess(placements). This organization provides training for
All IT Freshers and for Experienced Professions(Showing gap as
experience)</i>
<br>
<a href='welcome.htm'>home</a>
```

result.jsp:

```
<h1 style="color:red;text-align:center">Result page</h1>
<h4> Wish Message ::: ${msg}</h4>
<br>
<a href="welcome.htm">home</a>
```

```
<h1 style="color:red;text-align:center">Contact Us Page</h1>
<h2>Naresh i Technologies,</h2>
<br>Ameerpet,
<br>Near satyam Theatre,
<br>ContactNo: 040-23746666.
<br>
<a href='welcome.htm'>home</a>
```

```
<h1 style="color:red;text-align:center">Home Page</h1>  
  
<a href="wish.htm">Get Wish Message</a>  
&nbsp;&nbsp;&nbsp;&nbsp;<a href='contact_us.htm'>contactUs</a>  
&nbsp;&nbsp;&nbsp;&nbsp;<a href='about_us.htm'>AboutUs</a>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  id="WebApp_ID" version="3.1">
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <!-- <init-param>
      <param-name>namespace</param-name>
      <param-value>myconfig</param-value>
    </init-param> -->
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
</web-app>
```

dispatcher-servlet.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<!-- Handler mapping -->
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
  <property name="mappings">
    <props>
      <prop key="welcome.htm">ufvc</prop>
      <prop key="wish.htm">wsc</prop>
      <prop key="contact_us.htm">ufvc</prop>
      <prop key="about_us.htm">ufvc</prop>
    </props>
  </property>
</bean>

<!-- Controller class -->
<bean id="ufvc" class="org.springframework.web.servlet.mvc.UrlFilenameViewController"/>

<bean id="wsc" class="com.nt.controller.WishController"/>

<!-- View Resolver -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>

</beans>
```

index.jsp:

```
<jsp:forward page="welcome.htm"/>
```

pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>nit</groupId>
  <artifactId>MVCWishApp-PVC3</artifactId>
```

```
<packaging>war</packaging>
<version>0.0.1-SNAPSHOT</version>
<name>MVCWishApp-PVC3 Maven Webapp</name>
<url>http://maven.apache.org</url>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.2.2.RELEASE</version>
</dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context-support -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context-support</artifactId>
  <version>4.2.2.RELEASE</version>
</dependency>

  <!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>4.2.2.RELEASE</version>
</dependency>

  <!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>

  <version>3.1.0</version>
</dependency>

</dependencies>
<build>
  <finalName>MVCWishApp-PVC3</finalName>
</build>
</project>
```


Note: If request is there without form data (from hyperlinks) and no request processing is required but displaying views is required, then go for ParameterizableViewController (or) UriFileNameViewController (like working with AboutUs, ContactUs hyperlink).

- ⇒ In real-time every spring application will be designed to have 2 IOC containers as parent, child containers because it is recommended to maintain presentation tier components like HandlerMapping, ViewResolver, Controller class in one container (WebApplicationContextContainer) given by DispatcherServlet and business tier component like (service class, DAO class, AOP, Aspect class) in another container (ApplicationContext) IOC container given by ContextLoaderListner.
- ⇒ ContextLoaderListner is a spring supplied predefined ServletContextListner which activates ApplicationContext container when servletContext object is created (either during server startup or during the deployment of web application) by taking contextConfigLocation contextParamValue as the springbean configuration file.

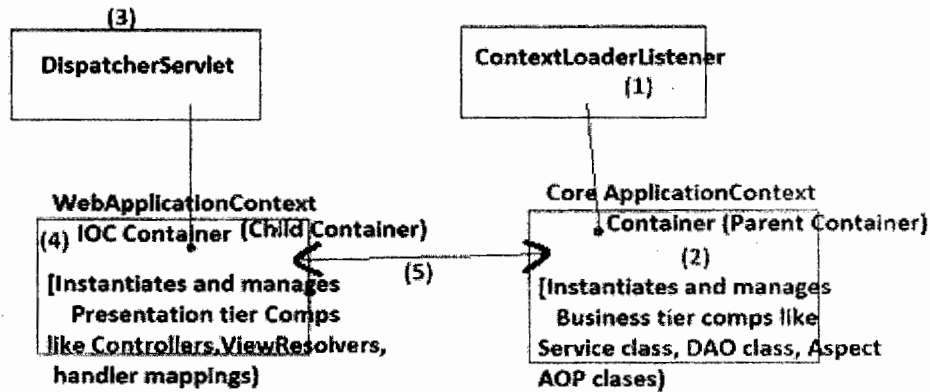
Code in web.xml:

```
<web-app>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet </servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <context-param>
    <param-name>contextConfigLocation</param-name >
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListner</listener-class>
  </listener>
</web-app>
```

- ⇒ If namespace InitParameter is not configure for DispatchServlet then the activated webapp context container takes <servlet-name>-servlet.xml as spring bean configure file. (dispatcher-servlet.xml).
- ⇒ Generally ServletListner object will be created first compare to the servlet object on which load-on-startup is enabled, but some servers creates serlvet object first before Listner object if <load-on-startup> priority value is 1(or)0.
- ⇒ We want to see Dispatcher servlets webApplicationContext container as child container to contextLoaderListner's ApplicationContext container in all situations and servers for this it is

recommended to enable load-on-startup on DispatcherServlet with priority value other than 1(or)0.

- ⇒ Since controller classes are managed as beans in child container so they can be injected with service class object/beans managed by parent container...



Since Controller classes are managed as beans in child container so they can be injected with Service class objs/beans managed by Parent Container.

AbstractController:

- ⇒ Take this class to develop controller class when there is no form submission in request but request processing required to display dynamic webpage as output.
- ⇒ Generally for hyperlink generated request to get dynamic content having request processing we should go for AbstractController class.

Usecases:

Getting current user profile in facebook when "viewprofile" link is clicked.
 Getting all the jobs that are posted when "current" trends link is clicked.
 Getting all employee details when "List of Employees" hyperlink is clicked.

Here no form submission, so there will not be any request wrapping form handling.

Since there is no form submission the `handleRequestInternal(-,-)` does not take any command object as the argument.

`handleRequestInternal(-,-)` returns `ModelAndView` object having logical view name and model Data that has to be displayed as dynamic content.

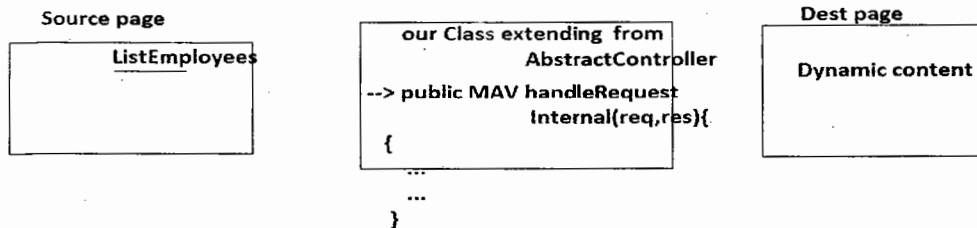
Note:

The process of storing form data into an object is called request wrapping.

Validating form data and displaying same form page with old values when form validation errors are there is called Form handling.

Note:

Writing b.logic in controller class is not a good practice, always place this logic in service class and communicate from controller class.

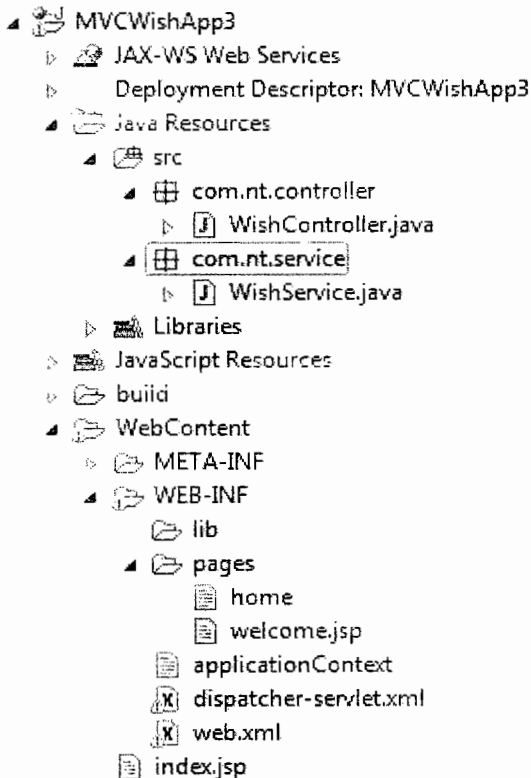
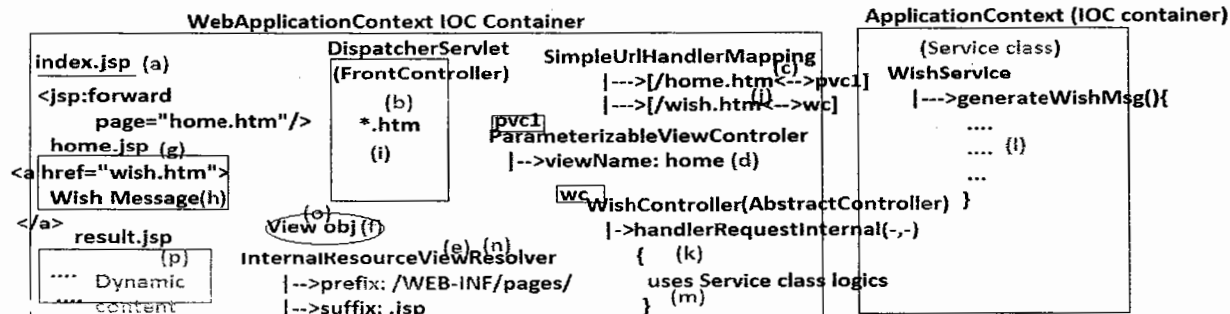
Diagram:

- Here no form submission, So there will not be any request grapping, form handling.
- Since there is no form submissin the `handleRequestInternal(-,-)` does not take any Command obj as argument.
- `handleRequestIntenal(-,-)` returns ModelAndView obj having logical view name and Model Data that has to be displayed as dynamic contennt.

note: ->The process of storing form data into an obj is called request grapping.

->Vaiidating form data and displaying same form page with old when form validation errors are there is called Form handling.

note: Writing b.logic in controller class is not a good pratice ,Always place this logic in Service class communicate from Controller class.



WishController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
import com.nt.service.WishService;
public class WishController extends AbstractController {
    private WishService service;
    public void setService(WishService service) {
        this.service = service;
    }
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        //Use Service class b.logic
        String wmsg=service.generateWishMsg();
        //create and return ModelAndView obj
        return new ModelAndView("result","wresult",wmsg);
    }
}
```

WishService.java:

```
package com.nt.service;
import java.util.Calendar;
public class WishService {
    public String generateWishMsg() {
        //get System date and time
        Calendar cl=Calendar.getInstance();
        //get current hour of the day
        int h=cl.get(Calendar.HOUR_OF_DAY);
        String msg=null;
        if(h<=12)
            msg="Good morning";
        else if(h<=16)
            msg="GoodAfter noon";
        else if(h<=20)
            msg="Good Evening";
        else
            msg="Good Night";
        return msg;
    }
}
```

index.jsp:

```
<jsp:forward page="home.htm"/>
```

web.xml:

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="surl"
  class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
  <props>
    <prop key="/home.htm">pvc1</prop>
    <prop key="/wish.htm">wc</prop>
  </props>
</property>
</bean>
<bean id="wc" class="com.nt.controller.WishController">
  <property name="service"><ref parent="wService"/></property>
</bean>
<bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="home"/>
</bean>
<bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="wService" class="com.nt.service.WishService"/>
</beans>
```

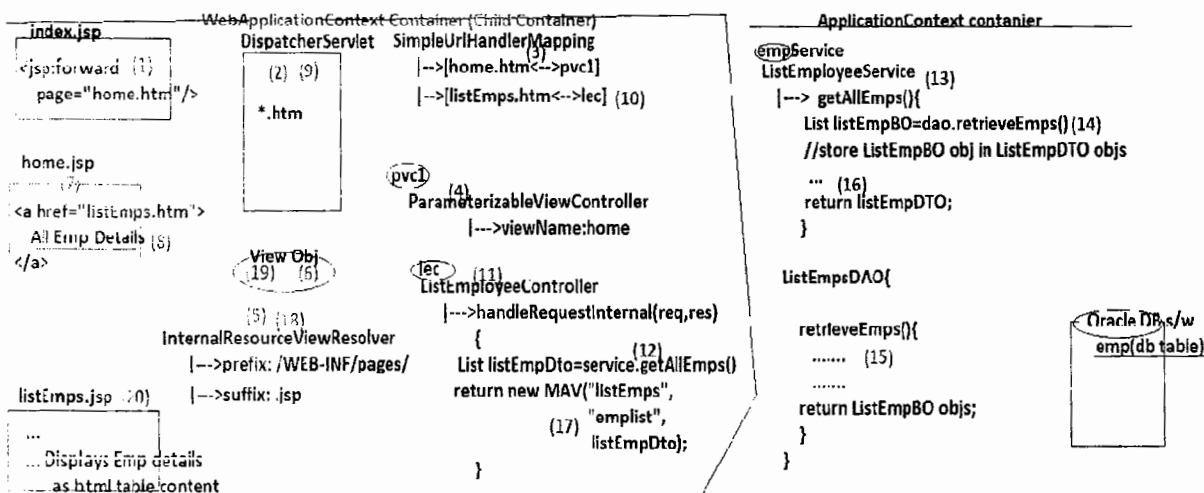
result.jsp:

```
<h1><center>Result Page</center></h1>
<br><br>
<font color="red"> ${wresult}</font>
<br>
<a href="home.htm">home</a>
```

home.jsp:

```
<H1><center> HOME PAGE</center></H1>
<br>
<br>
<a href="wish.htm">Wish Message</a>
```

⇒ In real time beans of service layer will be configured with separate xml(service-beans.xml) file, similarly the beans of persistence layer will be configured in separate xml file and all this xml will be imported in main applicationContext.xml.

Diagram:

- ▲  MVCApp4AC
 - ▷  JAX-WS Web Services
 - ▷ Deployment Descriptor: MVCApp4AC
 - ▲  Java Resources
 - ▲  src
 - ▲  com.nt.bo
 - ▷  EmpBO.java
 - ▲  com.nt.controller
 - ▷  ListEmpController.java
 - ▲  com.nt.dao
 - ▷  ListEmpDAO.java
 - ▲  com.nt.dto
 - ▷  EmpDTO.java
 - ▲  com.nt.service
 - ▷  ListEmpService.java
 - ▷  Libraries
 - ▷  JavaScript Resources
 - ▷  build
 - ▲  WebContent
 - ▷  META-INF
 - ▲  WEB-INF
 -  lib
 - ▲  pages
 -  home.jsp
 -  listEmps.jsp
 -  applicationContext.xml
 -  dispatcher-servlet.xml
 -  persistence-beans.xml
 -  service-beans.xml
 -  web.xml
 -  index.iso

Jars:- MVC library+DAO library+JSTL.jar+standard.jar

EmpBO.java:

```
package com.nt.bo;
public class EmpBO {
    private int no;
    private String name,job;
    private int salary;
    public EmpBO() {
        System.out.println("EmpBO:0-param constructor");
    }
    public EmpBO(int no, String name, String job, int salary) {
        System.out.println("EmpBO:4-param constructor");
        this.no = no;
        this.name = name;
        this.job = job;
        this.salary = salary;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```


ListEmpDAO.java:

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.EmpBO;
public class ListEmpDAO {
    private static final String GET_ALL_EMP_DETAILS="SELECT EMPNO,ENAME,JOB,SAL FROM
EMP";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public List<EmpBO> retrieveEmps(){
        List<EmpBO> listEmpBO=(List<EmpBO>)jt.query(GET_ALL_EMP_DETAILS,
                                                    new EmpRowMapper());
        return listEmpBO;
    }
    private class EmpRowMapper implements RowMapper<EmpBO>{

        @Override
        public EmpBO mapRow(ResultSet rs, int pos) throws SQLException {
            //Place ResultSet obj record into BO class obj
            EmpBO empbo=new EmpBO(rs.getInt(1),
                                   rs.getString(2),
                                   rs.getString(3),
                                   rs.getInt(4));

            return empbo;
        }
    }
}
//inner class//clas
```

EmpDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class EmpDTO implements Serializable {
    private int no;
    private String name,job;
    private int salary;

    public EmpDTO() {
        System.out.println("EmpDTO:0-param constructor");
    }
    public EmpDTO(int no, String name, String job, int salary) {
        System.out.println("EmpDTO:4-param constructor");
        this.no = no;
    }
}
```

```
        this.name = name;
    this.job = job;
        this.salary = salary;
    }

    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

ListEmpService.java:

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.EmpBO;
import com.nt.dao.ListEmpDAO;
import com.nt.dto.EmpDTO;
public class ListEmpService {
    private ListEmpDAO dao;
    public void setDao(ListEmpDAO dao) {
        this.dao = dao;
    }
    public List<EmpDTO> getAllEmps(){
        //use DAO class
        List<EmpBO> listEmpBO=dao.retrieveEmps();
        // Copy List of BO objs to List of DTO objs
    }
}
```

```
List <EmpDTO> listEmpDTO=new ArrayList<EmpDTO>();
for(EmpBO bo:listEmpBO){
    EmpDTO empdto=new EmpDTO(bo.getNo(),
    bo.getName(),bo.getJob(),bo.getSalary());
    listEmpDTO.add(empdto);
} //for
return listEmpDTO;
} //method
} //class
```

ListEmpController.java:

```
package com.nt.controller;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
import com.nt.dto.EmpDTO;
import com.nt.service.ListEmpService;
public class ListEmpController extends AbstractController {
    private ListEmpService service;

    public void setService(ListEmpService service) {
        this.service = service;
    }

    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        //use service class logics
        List <EmpDTO> listEmpDTO=service.getAllEmps();
        return new ModelAndView("list_emps","empList",listEmpDTO);
    }
}
```

index.jsp:

```
<jsp:forward page="home.htm"/>
```

web.xml:

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
```

```
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

service-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="empService" class="com.nt.service.ListEmpService">
    <property name="dao" ref="empDao"/>
  </bean>
</beans>
```

persistence-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
  <bean id="empDao" class="com.nt.dao.ListEmpDAO">
    <property name="jt" ref="template"/>
  </bean>
</beans>
```

dispatcher-servlet.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="/home.htm">pvc1</prop>
<prop key="/listEmps.htm">lec</prop>
</props>
</property>
</bean>
<bean id="lec" class="com.nt.controller.ListEmpController">
<property name="service" ref="empService"/>
</bean>
<bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
<property name="viewName" value="home"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/pages/">
<property name="suffix" value=".jsp"/>
</bean>
</beans>

```

applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<import resource="persistence-beans.xml"/>
<import resource="service-beans.xml"/>
</beans>

```

home.jsp:

```

<h1><Center>HOME PAGE</Center></h1>
<br>
<a href="listEmps.htm"> All Employees</a>

```

list_emps.jsp:

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<table border="1" bgcolor="red">
<tr bgcolor="blue"><th>eno</th><th>ename</th><th>JOB</th><th>Salary</th></tr>

```

```
<c:forEach var="emp" items="${empList}">
<tr bgcolor="cyan">
<td><c:out value="${emp.no}" /></td>
<td><c:out value="${emp.name}" /></td>
<td><c:out value="${emp.job}" /></td>
<td><c:out value="${emp.salary}" /></td>
</tr>
</c:forEach>
</table>
```

Command Class:

- ⇒ The java bean class that would be instantiated by spring mvc to store form data or form page data is called command class. The command class properties names and the form page component name must match. This avoids programmers burden of calling request.getParameter(-) and writing form data to java bean object.
- A command class properties can be of any type hold different types of values given by form page.

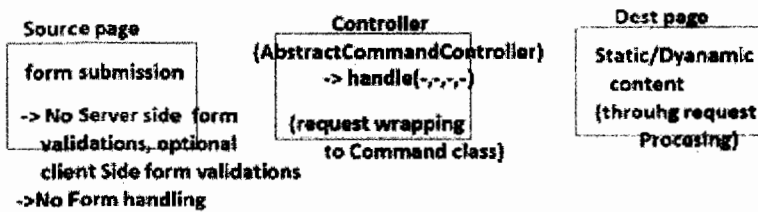
AbstractCommandController:

- ⇒ This controller is capable of reading form data of form page and writes that to command class, i.e it takes care of request graphing/wrapping.
- ⇒ Use this controller when source page having form submission with no serverside form validation, client side form validation optional and no form handling (no need of displaying same form page with old value if error occurs) and request wrapping is required, destination page should be static or dynamic page that should come after request processing. (Request processing is there) When we develop controller class extending from AbstractCommandController we must implement handle(-,-) method

```
public ModelAndView handle(HttpServletRequest req, HttpServletResponse res, Object cmd,
                          BindException be)
```

Note:

Since AbstractCommandController, SimpleFormController classes are removed from Spring 4.x In support of annotations, So use **Spring 3.2.x + Servlet 3.0.1** in the development of the Application

Diagram:**Source page:**

- > Form with submit button or java script based form submission having optional client side form validations
- > Since form is there request wrapping/graphing is required
- > No Form handling

Controller

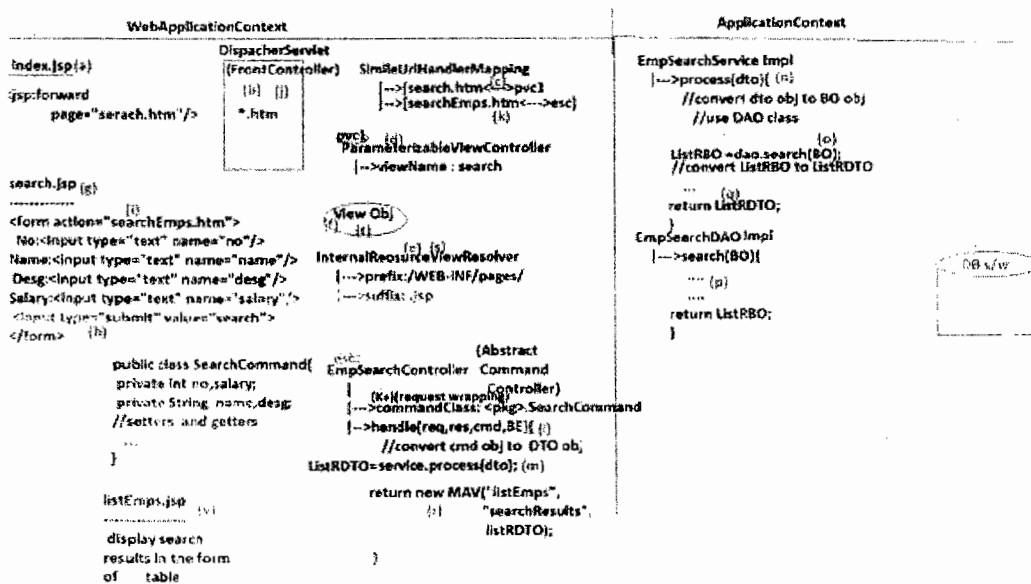
- => Recives form data and stores in command class obj (request wrapping/graphing)
- => Process the request by taking support of Service, DAO classes
- => Returns ModelAndView obj having logical view name and Model data (results)

Dest page

- => Static /dynamic content that comes after request processing.

Usease: Searching employee details based given no details or 1/2/3.. details in a form .

=> While c/f Controller class whose type is AbstractCommandController we must specify property "commandClass" having fully qualified Command class name as the value.



- ▶  MVCEmpSearchApp5ACC
 - ▶  JAX-WS Web Services
 - ▶ Deployment Descriptor: MVCEmpSearchApp.
 - ▶  Java Resources
 - ▶  src
 - ▶  com.nt.bo
 - ▶  SearchBO.java
 - ▶  SearchResultBO.java
 - ▶  com.nt.command
 - ▶  SearchCommand.java
 - ▶  com.nt.controller
 - ▶  EmpSearchController.java
 - ▶  com.nt.dao
 - ▶  EmpSearchDAO.java
 - ▶  EmpSearchDAOImpl.java
 - ▶  com.nt.dto
 - ▶  SearchDTO.java
 - ▶  SearchResultDTO.java
 - ▶  com.nt.service
 - ▶  EmpSearchService.java
 - ▶  EmpSearchServiceImpl.java
 - ▶  Libraries
 - ▶  JavaScript Resources
 - ▶  build
 - ▶  WebContent
 - ▶  META-INF
 - ▶  WEB-INF
 - ▶  lib
 - ▶  pages
 - ▶  aop-beans.xml

SearchBO.java:

```
package com.nt.bo;
public class SearchBO {
    private int no;
    private String name, desg;
    private int salary;
    public SearchBO() {
    }
    public SearchBO(int no, String name, String desg, int salary) {
        this.no = no;
        this.name = name;
        this.desg = desg;
        this.salary = salary;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDesg() {
        return desg;
    }
    public void setDesg(String desg) {
        this.desg = desg;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

SearchResultBO.java:

```
package com.nt.bo;
public class SearchResultBO {
    private int no;
    private String name,desg;
    private int salary,deptno,mgrno;
    public SearchResultBO() {
    }
    public SearchResultBO(int no, String name, String desg, int salary,
        int deptno, int mgrno) {
        this.no = no;
        this.name = name;
        this.desg = desg;
        this.salary = salary;
        this.deptno = deptno;
        this.mgrno = mgrno;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

```
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
public int getDeptno() {  
    return deptno;  
}  
public void setDeptno(int deptno) {  
    this.deptno = deptno;  
}  
  
public int getMgrno() {  
    return mgrno;  
}  
public void setMgrno(int mgrno) {  
    this.mgrno = mgrno;  
}  
}
```

EmpSearchDAO.java:

```
package com.nt.dao;  
import java.util.List;  
import com.nt.bo.SearchBO;  
import com.nt.bo.SearchResultBO;  
public interface EmpSearchDAO {  
    public List<SearchResultBO> search(SearchBO sbo);  
}
```

EmpSearchDAOImpl.java:

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;
public class EmpSearchDAOImpl implements EmpSearchDAO {

    private static final String EMP_SEARCH_QUERY="select empno,ename,job,sal,mgr,deptno
        from emp "+ " where (empno is not null and empno=? )"
        + "or (ename is not null and ename like ?)"
        + "or (job is not null and job like ?)"
        + "or (sal is not null and sal=?)";

    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }

    @Override
    public List<SearchResultBO> search(SearchBO sbo) {
        List <SearchResultBO> listRBO=jt.query(EMP_SEARCH_QUERY, new EmpRowMapper(),
        sbo.getNo(),
        sbo.getName(),
        sbo.getDesg(),
        sbo.getSalary());
        return listRBO;
    }

    private static final class EmpRowMapper implements RowMapper<SearchResultBO>{
        @Override
        public SearchResultBO mapRow(ResultSet rs, int pos) throws SQLException {
            //copy ResultSet record into SearchResultBO obj
            SearchResultBO rbo=new SearchResultBO();
            rbo.setNo(rs.getInt(1));
            rbo.setName(rs.getString(2));
            rbo.setDesg(rs.getString(3));
            rbo.setSalary(rs.getInt(4));
            rbo.setMgrno(rs.getInt(5));
            rbo.setDeptno(rs.getInt(6));
            return rbo;
        }
    }
}
```

SearchDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class SearchDTO implements Serializable {
    private int no;
    private String name,desg;
    private int salary;
    public SearchDTO() {
    }
    public SearchDTO(int no, String name, String desg, int salary) {
        this.no = no;
        this.name = name;
        this.desg = desg;
        this.salary = salary;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDesg() {
        return desg;
    }
    public void setDesg(String desg) {
        this.desg = desg;
    }
    public int getSalary() {
```

```
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

SearchResultDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class SearchResultDTO implements Serializable {
    private int no;
    private String name,desg;
    private int salary,mgrno,deptno;
```

```
public SearchResultDTO() {  
}  
public SearchResultDTO(int no, String name, String desg, int salary,  
    int mgrno, int deptno) {  
    this.no = no;  
    this.name = name;  
    this.desg = desg;  
    this.salary = salary;  
    this.mgrno = mgrno;  
    this.deptno = deptno;  
}  
public int getNo() {  
    return no;  
}  
public void setNo(int no) {  
    this.no = no;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
public int getMgrno() {  
    return mgrno;  
}  
public void setMgrno(int mgrno) {  
    this.mgrno = mgrno;  
}  
public int getDeptno() {  
    return deptno;  
}  
public void setDeptno(int deptno) {  
    this.deptno = deptno;  
}  
}
```

EmpSearchService.java:

```
package com.nt.service;
import java.util.List;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;
public interface EmpSearchService {
    public List<SearchResultDTO> process(SearchDTO sdto);
}
```

EmpSearchServiceImpl.java:

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;
import com.nt.dao.EmpSearchDAO;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;
public class EmpSearchServiceImpl implements EmpSearchService {
    private EmpSearchDAO dao;
    public void setDao(EmpSearchDAO dao) {
        this.dao = dao;
    }
    @Override
    public List<SearchResultDTO> process(SearchDTO sdto) {
        //Convert SearchDTO obj to SearchBO obj
        SearchBO sbo=new SearchBO();
        sbo.setNo(sdto.getNo());
        sbo.setName(sdto.getName());
        sbo.setDesg(sdto.getDesg());
        sbo.setSalary(sdto.getSalary());
        //Use DAO
        List<SearchResultBO> listRBO=dao.search(sbo);
        //Convert ListRBO to ListRDTO obj
        List<SearchResultDTO> listRDTO=new ArrayList<SearchResultDTO>();
        for(SearchResultBO rbo:listRBO){
            SearchResultDTO rdto=new SearchResultDTO();
            rdto.setNo(rbo.getNo());
            rdto.setName(rbo.getName());
            rdto.setDesg(rbo.getDesg());
            rdto.setSalary(rbo.getSalary());
            rdto.setMgrno(rbo.getMgrno());
            rdto.setDeptno(rbo.getDeptno());
            listRDTO.add(rdto);
        }
        return listRDTO;
    }
    //process
}
//class
```

SearchCommand.java:

```
package com.nt.command;
public class SearchCommand {
    private int no;
    private String name,desg;
    private int salary;
    public SearchCommand() {
        System.out.println("SearchCommand:0-param constructor");
    }

    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDesg() {
        return desg;
    }
    public void setDesg(String desg) {
        this.desg = desg;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
```

```
        this.salary = salary;
    }
}
```


EmpSearchController.java:

```
package com.nt.controller;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractCommandController;
import com.nt.command.SearchCommand;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;
import com.nt.service.EmpSearchService;
public class EmpSearchController extends AbstractCommandController {
    private EmpSearchService service;
    public void setService(EmpSearchService service) {
        this.service = service;
    }
    @Override
    protected ModelAndView handle(HttpServletRequest req,
        HttpServletResponse res, Object cmd, BindException be)
        throws Exception {
        //get Command obj
        SearchCommand command=(SearchCommand)cmd;
        //convert Command obj into DTO obj
        SearchDTO dto=new SearchDTO();
        dto.setNo(command.getNo());
        dto.setName(command.getName());
        dto.setDesg(command.getDesg());
        dto.setSalary(command.getSalary());
        //Use Service class
        List <SearchResultDTO> listRDTO=service.process(dto);
        return new ModelAndView("listEmps","searchResults",listRDTO);
    }
}
```

index.jsp:

```
<jsp:forward page="search.htm"/>
```

web.xml:

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

service-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="esService" class="com.nt.service.EmpSearchServiceImpl">
    <property name="dao" ref="esDao"/>
  </bean>
</beans>
```

persistence-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
  </bean>
  <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
  </bean>
```

```
<bean id="esDao" class="com.nt.dao.EmpSearchDAOImpl">
  <property name="jt" ref="template"/>
</bean>

</beans>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/search.htm">pvc1</prop>
        <prop key="/searchEmps.htm">esc</prop>
      </props>
    </property>
  </bean>
  <bean id="esc" class="com.nt.controller.EmpSearchController">
    <property name="commandClass" value="com.nt.command.SearchCommand"/>
    <property name="service" ref="esService"/>
  </bean>
  <bean id="pvc1" class="
    org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="search"/>
  </bean>
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <import resource="persistence-beans.xml"/>
  <import resource="service-beans.xml"/>
  <import resource="aop-beans.xml"/>
</beans>
```

aop-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="txMgr" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="dataSource"/>
</bean>

<tx:advice id="txAdvice" transaction-manager="txMgr">
<tx:attributes>
  <tx:method name="process" propagation="REQUIRED"
              read-only="true"/>
</tx:attributes>
</tx:advice>
<aop:config>
  <aop:pointcut id="ptc1"
                expression="execution(* com.nt.service.EmpSearchService.process(..))"/>
  <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
</aop:config>
</beans>
```

search.jsp

```
<center><H1>Search Employees</H1></center>
<form action="searchEmps.htm" method="POST">
  Emp no: <input type="text" name="no"><br>
  Emp Name: <input type="text" name="name"><br>
  Emp Desg: <input type="text" name="desg"><br>
  Emp Salary: <input type="text" name="salary"><br>
  <input type="submit" value="Search"></form>
```

listEmps.jsp:

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<center><h1>Search Results</h1></center>
<c:if test="${not empty searchResults}">
<table>
<tr>
<th>Emp No</th>
<th>Emp Name</th>
<th>Salary</th>
<th>Desg</th>
<th>Dept no</th>
<th>Manger No</th>
</tr>
<c:forEach items="${searchResults}" var="result">
<tr>
<td><c:out value="${result.no}"/></td>
<td><c:out value="${result.name}"/></td>
<td><c:out value="${result.salary}"/></td>
<td><c:out value="${result.desg}"/></td>
<td><c:out value="${result.deptno}"/></td>
<td><c:out value="${result.mgrno}"/></td>
</tr>
</c:forEach>
</table>
</c:if>

```

Internal flow of abstract command controller:-

Dispatcher Servlet

```

(h)
controller.handle (a)
Request(req,res);

```

```

public class AbstractCommandController implements Controller{
    (b)
    public MAV handlerRequest(req,res){
        ->create command class obj writes the received from data to the obj. (c)
        ->creates an Empty BinderException class ob.
        ->calls handler(-,-,-)with req,res,command obj,BindException obj. (d)
        -> return MAV obj given by handler(-,-,-)call (g)
    }
    ->public abstract MAV handle(req,res,Object cmd,BindException)
    }
    public class EmpSearchController extends
    AbstractCommandController {
        (e)
        public MAV handler(req,res,Object cmd,BindException be){
            .....
            .....
            .....
            return new MAV(-,-,-);(f)
        }
    }
}

```

- ⇒ *The moment we enable the Transaction management the underlying database software will be applied with read committed isolation level. i.e means it allows to read only committed data from database.*
- ⇒ *For every request given our AbstractCommandController class, one object of command class will be created for request wrapping.*
- ⇒ *DispatcherServlet always gets our controller class objects from webApplicationContext container by using ctx.getBean (-) and calls handleRequest(req, res) on that object. If not available then the super class handleRequest(-,-) executes.*
- ⇒ *DispatcherServlet doesn't create command class object. The handleRequest(-,-) of superclass to our controller class like AbstractCommandController/SimpleFormController Will create this command class object.*

SimpleFormController:

- ⇒ *It is a multipurpose controller which can be used for form submission, form validations, request wrapping, solving double posting and etc....*
- ⇒ *If our source page having form page with form validation, form handling and looking to get Dest page either as static/dynamic page having request processing then use SimpleFormController.*

Diagram:

SimpleFormController

Source page

Form page /form submission
request wrapping
Form validations
Form handling

-->commandClass
-->commandName
-->formView
-->succesView

MAV onSubmit()

Dest page

static /Dyanamic
web page

The SimpleFormController displays form page based on "formView" value if the request is initialphase request (request method:GET) i.e there is no need taking separte Parameterizable ViewController for launching form page.

The same controller becomes ready to process the request if the request is PostBack request (request method: POST)

- 1) DispatcherServlet calls `handleRequest(req,res)` on our SimpleFormController class obj
- 2) Since not available super class (SimpleFormController) `handleRequest(-,-)` method executes
- 3) The `handleRequest(-,-)` checks weather request is initialphase request or post back request

Initial Phase Request (if `req.getMethod.equals("GET")`)

- a) Creates an empty Command class obj
- b) Keeps Command class obj either in Session/request scope
- c) gather `formView` property value and returns it as logical View name to DispatcherServlet
- d) ViewResolver takes lvn (logical view name) and gives View obj to render form page to browser .This form page can display the initial values by collecting from Command obj

Post back Request (if `req.getMethod.equals("POST")`)

- a) creates /Locates Comand class obj and writes form data to it (request wrapping)
- b)creates an empty BindException obj
- c) calls `onSubmit()` having req,res,command,BinException objs
- d) `onSubmit()` our Controller class will process the request and returns MAV obj having logical view name
- e) ViewResolver takes lvn and gives view obj to render result page.

Source page

Form page /form submission
request wrapping
Form validations
Form handling

SimpleFormController

```
|-->commandClass
|-->commandName
|-->formView
|-->succesView

MAV onSubmit()
```

Dest page

static /Dyanamic
web page

The SimpleFormController displays form page based on "formView" value if the request is initialphase request (request method:GET) i.e there is no need taking seperate Parameterizable ViewController for launching form page.

The same controller becomes ready to process the request if the request is PostBack request (request method: POST)

- 1) DispatcherServlet calls `handleRequest(req,res)` on our SimpleFormController class obj
- 2) Since not available super class (SimpleFormController) `handleRequest(-,-)` method executes
- 3) The `handleRequest(-,-)` checks weather request is initialphase request or post back request

Initial Phase Request (if req.getMethod.equals("GET")){

- a) Creates an empty Command class obj
- b) Keeps Command class obj either in Session/request scope
- c) gather formView property value and returns it as logical View name to DispatcherServlet
- d) ViewResolver takes lvn (logical view name) and gives View obj to render form page to browser .This form page can display the initial values by collecting from Command obj

Post back Request (if req.getMethod.equals("POST")){

- a) creates /Locates Comand class obj and writes form data to it (request wrapping)
- b)creates an empty BindException obj
- c) calls `onSubmit()` having req,res,command,BinException objs
- d) `onSubmit()` our Controller class will process the request and returns MAV obj having logical view name
- e) ViewResolver takes lvn and gives view obj to render result page.

SimpleFormController

```
|----->CommandClass
|----->CommandName
|----->formView:form page name
|----->successView: Result page name
|----->sessionForm and etc.....
|----->true---->keep cmd obj in session obj.
|----->false---->keep cmd obj in requestScope.
```

Methods

```
|----->public MAV onSubmit (Object cmd)
|----->public MAV onSubmit (Object cmd, BindException be)
|----->public MAV onSubmit (req,res, Object cmd, BindException be)
```

4 param method internally call 2 param method and 2 param method internally calls 1 param method

Note:

- ⇒ working with SimpleFormController our formpage request method must be "POST".
- ⇒ In form handling we need to display the same page with old values when form validation error are there. In this process we need to write lots of additional code for this if we work with traditional HTML pages.
- ⇒ If we use spring supplies tag library tags then automatically it take care of this process which is nothing but reading old values from command object and displaying in the form page.

Spring gives 2 jsp tag libraries to develop that jsps of web application:**Generic jsp tag library**

```

|----->general tags
|----->tld File=spring.tld
|----->taglib uri:http://www.springframework.org/tags

```

Form jsp tag library

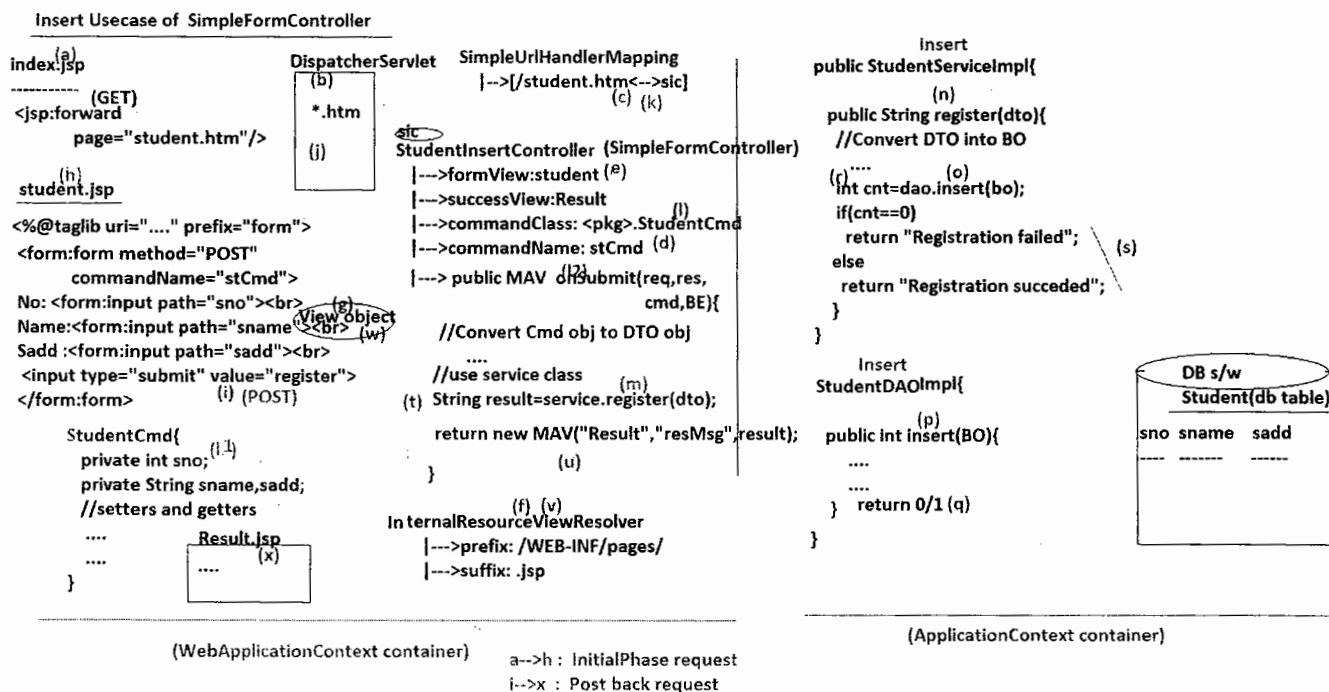
```

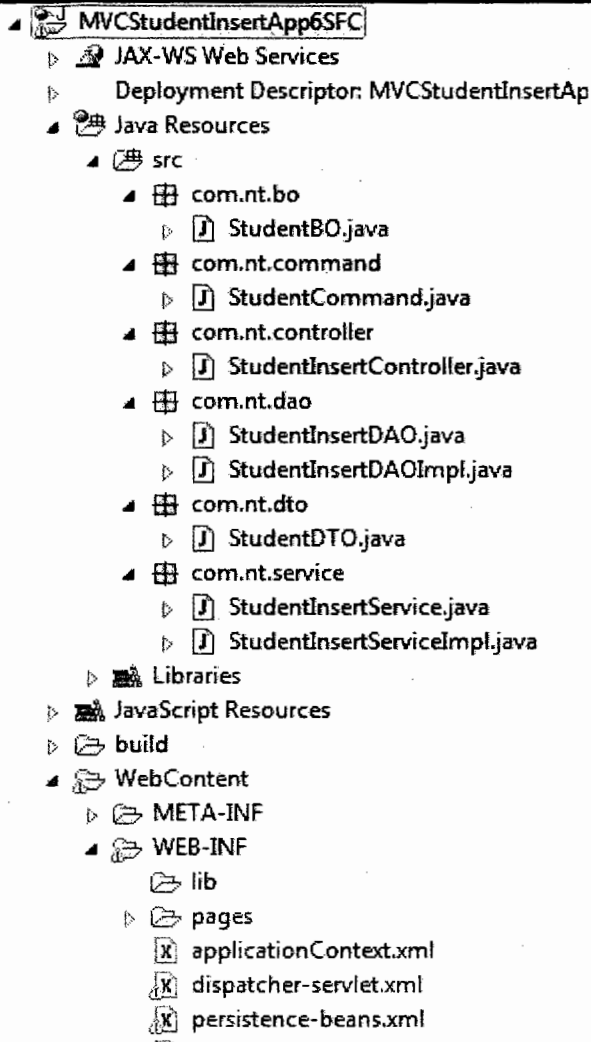
|-----> form tags
|----->tld File: Spring_form.tld
|----->taglib uri:http://www.springframework.org/tags/form

```

Note:

AbstractCommandController, SimpleFormController classes are deprecated in spring 3.x and removed from spring 4.x in support to annotation based controllers.

Diagram:

**StudentBO.java:**

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
```

```
return sadd;
}
public void setSadd(String sadd) {
    this.sadd = sadd;
}
}
```

StudentInsertDAO.java:

```
package com.nt.dao;
import com.nt.bo.StudentBO;
public interface StudentInsertDAO {
    public int insert(StudentBO sbo);
}
```

StudentInsertDAOImpl.java:

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.bo.StudentBO;
public class StudentInsertDAOImpl implements StudentInsertDAO {
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT values(?,?,?)";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int insert(StudentBO sbo) {
        int cnt=jt.update(INSERT_STUDENT_QRY,sbo.getSno(),
            sbo.getSname(),
            sbo.getSadd());
        return cnt;
    }
}
```

StudentDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
}
```

```
}  
    public void setName(String sname) {  
        this.sname = sname;  
    }  
    public String getSadd() {  
        return sadd;  
    }  
    public void setSadd(String sadd) {  
        this.sadd = sadd;  
    }  
}
```

StudentInsertService.java:

```
package com.nt.service;  
import com.nt.dto.StudentDTO;  
public interface StudentInsertService {  
    public String register(StudentDTO dto);  
}  
package com.nt.service;  
import com.nt.bo.StudentBO;  
import com.nt.dao.StudentInsertDAO;  
import com.nt.dto.StudentDTO;  
public class StudentInsertServiceImpl implements StudentInsertService{  
    private StudentInsertDAO dao;  
    public void setDao(StudentInsertDAO dao) {  
        this.dao = dao;  
    }  
    @Override  
    public String register(StudentDTO dto) {  
        // Convert DTO obj BO obj  
        StudentBO sbo=new StudentBO();  
        sbo.setSno(dto.getSno());  
        sbo.setName(dto.getName());  
        sbo.setSadd(dto.getSadd());  
        //Use DAO  
        int cnt=dao.insert(sbo);  
        if(cnt==0)  
            return " Student Registration Failed";  
        else  
            return " Student Registration Succeded with no"+sbo.getSno();  
    }  
}
```

StudentInsertController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentInsertService;
public class StudentInsertController extends SimpleFormController {
    private StudentInsertService service;
    public void setService(StudentInsertService service) {
        this.service = service;
    }
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        //type cast cmd obj
        StudentCommand cmd=(StudentCommand)command;
        //Convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //Use Service class
        String result=service.register(dto);
        //return ModelAndView obj
        return new ModelAndView(this.getSuccessView(),"resMsg",result);
    }
}
```

StudentCommand.java:

```
package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
}
```

```
public void setSname(String sname) {
    this.sname = sname;
}
public String getSadd() {
    return sadd;
}
public void setSadd(String sadd) {
    this.sadd = sadd;
}
}
```

index.jsp:

```
<jsp:forward page="student.htm"/>
```

web.xml:

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>

<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
                <prop key="/student.htm">sic</prop>
            </props>
        </property>
    </bean>
</beans>
```

```
</property>
</bean>
<bean id="sic" class="com.nt.controller.StudentInsertController">
  <property name="formView" value="student"/>
  <property name="successView" value="Result"/>
  <property name="commandClass" value="com.nt.command.StudentCommand"/>
  <property name="commandName" value="stCmd"/>
  <property name="sessionForm" value="true"/>
  <property name="service" ref="stService"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <import resource="persistence-beans.xml"/>
  <import resource="service-beans.xml"/>
</beans>
```

service-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="stService" class="com.nt.service.StudentInsertServiceImpl">
    <property name="dao" ref="stDao"/>
  </bean>
</beans>
```

persistence-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  </bean>
</beans>
```

```

<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>

<property name="username" value="scott"/>
<property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentInsertDAOImpl">
  <property name="jt" ref="template"/>
</bean>
</beans>

```

student.jsp:

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
<h1><center> Register Student Details</center></h1>
<form:form method="POST" commandName="stCmd">
  Number: <form:input path="sno"/><br>
  Name : <form:input path="sname"/><br>
  Address : <form:input path="sadd"/><br>
  <input type="submit" value="register"/>
</form:form>

```

result.jsp:

```

<h1><center> Result Page</center></h1>
${resMsg}

```

Form validationz in SimpleFormController:

- ⇒ Verifying the format and pattern of form data is called form validation. Spring MVC allows to write form validation logic in separate class so that it can be linked with multiple controller classes having reusability.

Sample Code:

- ⇒ Develop a validator class implementing validation(i) and implementsupports(-) and validate (-,-) methods.
- ⇒ Develop properties file having validation Error messages.
- ⇒ Configure the properties file in "dispatcher-servlet.xml"
- ⇒ Configure validator class in dispatcher-Servlet.xml and link that class with controller class using the property "validator"
- ⇒ Place <form:errors path="*" />tag in student.jsp to display the form validation error messages.

The SimpleFormController can handle 3 types of validation:

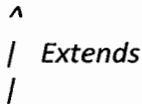
- a) Form Validations (like number, name can't be blank).
- b) typeMismatch errors (like storing string value to Sno properties).
- c) Application logic errors (like not allowing "delhi" as address for student) Instead of hardcoding error msgs directly in validator class and controller class it is recommended to

place in properties file having our choice keys and values, but typemismatch errors key should be "typeMismatch.<propertyname>"

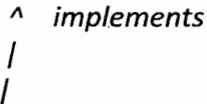
Q) What is the use of implementing supports (Class clazz) method in validator class of SimpleFormController?

=>This supports (-) method holds the configured "CommandClass" as "Class" parameter and we can check whether correct command class is configured or not by using Class. isAssignableFrom(-) method, if we configure correctly then it returns true, so validate (-, -) method executes, if not configured correctly then it returns false, so validate(-, -) will not execute. Due to this we can avoid mis configuration in spring bean configure file.

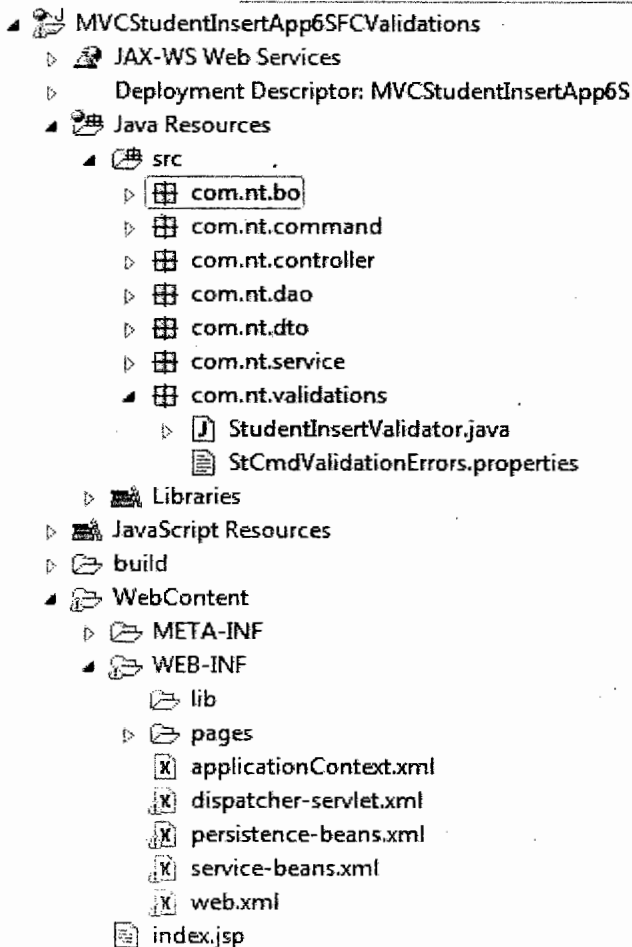
org.springframework.validation.Errors(i)



org.springframework.validation.BindingResults(i)



org.springframework.validation.BindException(c)



StudentBO.java:

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentInsertDAO.java:

```
package com.nt.dao;
import com.nt.bo.StudentBO;
public interface StudentInsertDAO {
    public int insert(StudentBO sbo);
}
```

StudentInsertDAOImpl.java:

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.bo.StudentBO;
public class StudentInsertDAOImpl implements StudentInsertDAO {
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT values(?,?,?)";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int insert(StudentBO sbo) {
        int cnt=jt.update(INSERT_STUDENT_QRY,sbo.getSno(),
            sbo.getSname(),
            sbo.getSadd());
    }
}
```

```
return cnt;
}
}
```

StudentInsertService.java:

```
package com.nt.service;
import com.nt.dto.StudentDTO;
public interface StudentInsertService {
    public String register(StudentDTO dto);
}
```

StudentDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname,sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentInsertServiceImpl.java:

```
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentInsertDAO;
import com.nt.dto.StudentDTO;
public class StudentInsertServiceImpl implements StudentInsertService{
    private StudentInsertDAO dao;
    public void setDao(StudentInsertDAO dao) {
        this.dao = dao;
    }
}
```

```
@Override
public String register(StudentDTO dto) {
    // Convert DTO obj BO obj
    StudentBO sbo=new StudentBO();
    sbo.setSno(dto.getSno());
    sbo.setSname(dto.getSname());
    sbo.setSadd(dto.getSadd());
    //Use DAO
    int cnt=dao.insert(sbo);
    if(cnt==0)
        return " Student Regisration Failed";
    else
        return " Student Regisration Succeded with no"+sbo.getSno();
}
```

StudentCommand.java:

```
package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StCmdValidationErrors.properties

typeMismatch.sno=student no can not empty or can not be String
sno.invalid=Student no must be positive value
sname.required=Student name is mandatory
sadd.required=Student address is mandatory
sadd.minlength= Addrds can not be less than 3 chars
sadd.city=Address can be not "delhi".

StudentInsertValidator.java:

```
package com.nt.validator;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.StudentCommand;
public class StudentInsertValidator implements Validator {

    @Override
    public boolean supports(Class<?> clazz) {

        return clazz.isAssignableFrom(StudentCommand.class);
    }

    @Override
    public void validate(Object command, Errors errors) {
        //type casting
        StudentCommand cmd=(StudentCommand)command;
        //form validations
        if(cmd.getSno()<=0)
            errors.rejectValue("sno", "sno.invalid");
        if(cmd.getSname().equals("") || cmd.getSname()==null)
            errors.rejectValue("sname", "sname.required");
        if(cmd.getSadd().equals("") || cmd.getSadd()==null)
            errors.rejectValue("sadd", "sadd.required");
        else{
            if(cmd.getSadd().length()<3){
                errors.rejectValue("sadd", "sadd.minlength");
            }
        }
    }
}
```

StudentInsertController.java:

```
package com.nt.controller;
import javax.servlet.RequestDispatcher;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentInsertService;
public class StudentInsertController extends SimpleFormController {
    private StudentInsertService service;
    public void setService(StudentInsertService service) {
        this.service = service;
    }
}
```

```
@Override
protected ModelAndView handleInvalidSubmit(HttpServletRequest req,
    HttpServletResponse res) throws Exception {
    return new ModelAndView("dblpst");
}

@Override
protected ModelAndView onSubmit(HttpServletRequest request,
    HttpServletResponse response, Object command, BindException errors)
    throws Exception {
    //type cast cmd obj
    StudentCommand cmd=(StudentCommand)command;
    // B.logic related validation
    if(cmd.getSadd().equals("delhi")){
        errors.rejectValue("sadd", "sadd.city");
        return showForm(request,response, errors);
    }
    //Convert Command obj to DTO obj
    StudentDTO dto=new StudentDTO();
    dto.setSno(cmd.getSno());
    dto.setSname(cmd.getSname());
    dto.setSadd(cmd.getSadd());
    //Use Service class
    String result=service.register(dto);
    //return ModelAndView obj
    return new ModelAndView(this.getSuccessView(),"resMsg",result);
}

@Override
protected Object formBackingObject(HttpServletRequest request)
    throws Exception {

    int no=Integer.parseInt(request.getParameter("id"));
    StudentCommand cmd=new StudentCommand();
    cmd.setSno(no);
    cmd.setSname("raja111");
    cmd.setSadd("hyd111");
    return cmd;
}
}
```

StudentCommand.java:

```
package com.nt.command;
public class StudentCommand {
    private int sno=101;
    private String sname="raja",sadd="hyd";
}
```

```
public int getSno() {  
    return sno;  
}  
public void setSno(int sno) {  
    this.sno = sno;  
}  
public String getSname() {  
    return sname;  
}  
public void setSname(String sname) {  
    this.sname = sname;  
}  
public String getSadd() {  
    return sadd;  
}  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}
```

index.jsp:

```
<jsp:forward page="student.htm"/>
```

web.xml:

```
<web-app>  
<servlet>  
    <servlet-name>dispatcher</servlet-name>  
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>  
    <load-on-startup>2</load-on-startup>  
</servlet>  
<servlet-mapping>  
    <servlet-name>dispatcher</servlet-name>  
    <url-pattern>*.htm</url-pattern>  
</servlet-mapping>  
<listener>  
    <listener-class>org.springframework.web.context.ContextLoaderListener  
</listener-class>  
</listener>  
<context-param>  
    <param-name>contextConfigLocation</param-name>  
    <param-value>/WEB-INF/applicationContext.xml</param-value>  
</context-param>  
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="/student.htm">sic</prop>
</props>
</property>
</bean>
<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
<property name="basename" value="com/nt/validator/StCmdValidationErrors"/>
</bean>
<bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>
<bean id="sic" class="com.nt.controller.StudentInsertController">
<property name="formView" value="student"/>
<property name="validator" ref="stValidator"/>
<property name="successView" value="Result"/>
<property name="commandClass" value="com.nt.command.StudentCommand"/>
<property name="commandName" value="stCmd"/>
<property name="sessionForm" value="true"/>
<property name="service" ref="stService"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/pages"/>
<property name="suffix" value=".jsp"/>
</bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<import resource="persistence-beans.xml"/>
<import resource="service-beans.xml"/>
</beans>
```

service-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
```



```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="stService" class="com.nt.service.StudentInsertServiceImpl">
<property name="dao" ref="stDao"/>
</bean>
</beans>
```

persistence-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
<property name="username" value="scott"/>
<property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentInsertDAOImpl">
<property name="jt" ref="template"/>
</bean>
</beans>
```

student.jsp:

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<h1><center> Register Student Details</center></h1>
<form:form method="POST" commandName="stCmd">
<!-- <font color='red'> -->
<%-- <form:errors path="*" /> --%>
<!-- </font> -->
<!-- <br> -->
Number: <form:input path="sno"/><form:errors path="sno"/><br>
Name : <form:input path="sname"/><form:errors path="sname"/><br>
Address : <form:input path="sadd"/><form:errors path="sadd"/><br>
<input type="submit" value="register"/>
</form:form>
```

result.jsp:

```
<h1><center> Result Page</center></h1>
${resMsg}
```

dbpost.jsp:

```
<h1> Warning!!!!</h1>
<p style="color:red"> double posting is not allowed </p>
```

Internal flow of form validation:

⇒ Form page Submits the form (post back request)---->Dispatcher servlet traps request and uses handler mapping to map the request to our SimpleFormController and calls handleRequest (-,-)-
----->this method checks wheather request is "initialphase" or "postback" request. Since the request is post back request.

a) Creates an empty BindException object

b) Creates or locates command class object and writes form data to it.

(request wrapping)

c) Checks "validation" is configured or not and it notices configured in controller class.

d) get validator object and calls supports(-) with Class object that points command Class if this method returns true then validate(-,-) will be called having Errors Object (BindException), command object otherwise validate(-,-) will not be called.

e) validate(-,-) perform form validations and stores validationErrors in Errors object.

f) handleRequest(-,-) checks the size of Error object to know wheather validation Errors are there or not there.

If Errors obj size=0 (Then no validation errors)

Then {

Calls onSubmit(-,-,-) having req,res,command and BindException objs.

onSubmit(-,-,-) will process the request

}

If (Error obj size>0) [form validation errors]

{

⇒ keep Error objs in request scope.

⇒ Returns MAV obj having formview property value as lvn, commandClassName, CommandClas obj as model attribute.

⇒ DispatcherServlet keeps the received command class obj in req scope and use ViewResolver to get view obj having form page as physical view Name.

⇒ Form page will be displayed. In that <form:input> tags collect command object data to show old values in comps and <form: errors> uses "Error obj" data to show validation Error messages.

}

Q)What is double posting problem and duplicate form submission problem and how can we solve that problem?

⇒ Pressing refresh button on the result page of form submission submits the form one more time, clicking on submit button for multiple times continuously also raises this problem. This leads duplicate form submission and gives problem like duplicate registrations, deducting amount from

credit card, debit card accounts for multiple times and etc..... This is also called as "doublePosting" problem.

- ⇒ We can overcome this problem with tokens support, when form page is submitted one token will be created and will be maintained as session attribute. Once onSubmit(-,-) method process the request it removes the token from session. If any request comes without token because of double posting problem error will be raised.
- ⇒ In spring MVC all these activities are handle internally if any request comes without token, the handleInvalidSubmit (-,-) method will be called. So we can place logic to render error page in that method.

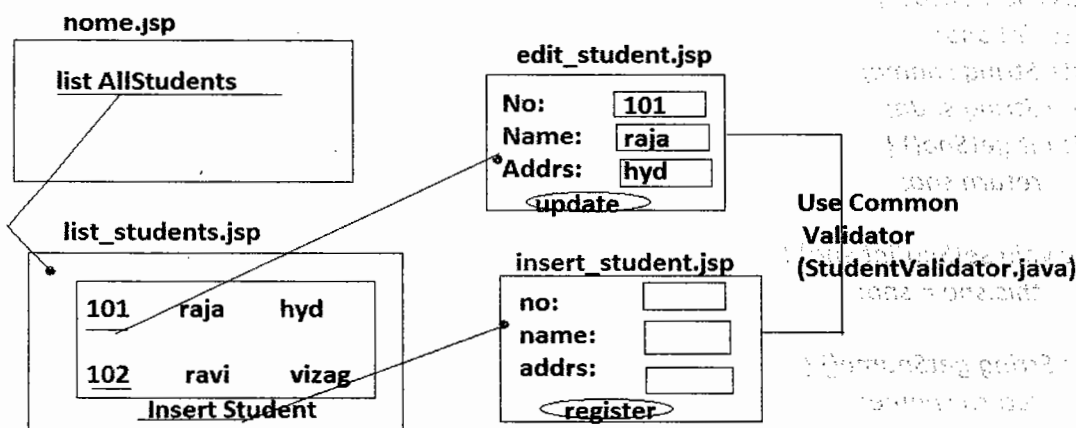
Importance of formBackingObject ()

- ⇒ Generally when handleRequest method is processing initial phase request it internally calls formBackingObject() to create command class object. The predefined method always creates empty command object. If you want to create command object with dynamic data according to application requirement then we need to override this method in our controller class that extends from SimpleformController.

Usecase:

- ⇒ When we clicked on student no hyperlink from list of student details, if we want to display edit.jsp having that student details initial as values.

Diagram:



MVCMinProjApp8

JAX-WS Web Services

Deployment Descriptor: MVCMinProjApp8

Java Resources

src

com.nt.bo

com.nt.command

com.nt.controller

com.nt.dao

com.nt.dto

com.nt.service

com.nt.validator

Libraries

JavaScript Resources

build

WebContent

META-INF

WEB-INF

lib

pages

aop-beans.xml

applicationContext.xml

dispatcher-servlet.xml

persistence-beans.xml

service-beans.xml

web.xml

index.jsp

StudentBO.java:

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd; } }
}
```

StudentDAO.java:

```
package com.nt.dao;
import java.util.List;
import com.nt.bo.StudentBO;
public interface StudentDAO {
    public List<StudentBO> getAllStudents();
    public int insert(StudentBO bo);
    public StudentBO getStudent(int no);
    public int update(StudentBO bo);
}
```

StudentDAOImpl.java:

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import com.nt.bo.StudentBO;
public class StudentDAOImpl implements StudentDAO {
    private static final String GET_ALL_STUDENTS_QRY="SELECT SNO,SNAME,SADD FROM STUDENT ";
    private static final String INSERT_STUDENT_QRY="INSERT INTO STUDENT VALUES(?,?,?)";
    private static final String GET_STUDENT_BY_SNO="SELECT SNO,SNAME,SADD FROM STUDENT WHERE SNO=?";
    private static final String UPDATE_STUDENT_BY_NO="UPDATE STUDENT SET SNAME=?, SADD=? WHERE SNO=?";

    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public List<StudentBO> getAllStudents() {
        List<StudentBO> listBO=jt.query(GET_ALL_STUDENTS_QRY, new StudentRowMapper());
        return listBO;
    }
    private class StudentRowMapper implements RowMapper<StudentBO>{
        @Override
        public StudentBO mapRow(ResultSet rs, int pos) throws SQLException {
            StudentBO bo=new StudentBO();
            bo.setSno(rs.getInt(1));
            bo.setSname(rs.getString(2));
            bo.setSadd(rs.getString(3));
            return bo;
        }
    }
}
```

```
    } //inner class
    @Override
    public int insert(StudentBO bo) {
        int cnt=jt.update(INSERT_STUDENT_QRY, bo.getSno(),
        bo.getSname(),
        bo.getSadd());
        return cnt;
    }
    @Override
    public StudentBO getStudent(int no) {
        StudentBO bo=jt.queryForObject(GET_STUDENT_BY_SNO, new
        StudentRowMapper(), no);
        return bo;
    }
    public int update(StudentBO bo){
        int cnt=jt.update(UPDATE_STUDENT_BY_NO,
        bo.getSname(),bo.getSadd(),bo.getSno());
        return cnt;
    }
}
```

StudentDTO.java:

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setName(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentService.java:

```
package com.nt.service;
import java.util.List;
import com.nt.dto.StudentDTO;
public interface StudentService {
    public List<StudentDTO> listStudents();
    public String registerStudent(StudentDTO dto);
    public StudentDTO searchStudentBySno(int no);
    public String updateStudentBySno(StudentDTO dto);
}
```

StudentServiceImpl.java:

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDAO;
import com.nt.dto.StudentDTO;
public class StudentServiceImpl implements StudentService{
    private StudentDAO dao;
    public void setDao(StudentDAO dao) {
        this.dao = dao;
    }
    @Override
    public List<StudentDTO> listStudents() {
        //use dao
        List<StudentBO> listBO=dao.getAllStudents();
        //Copy ListBO to List DTO
        List<StudentDTO> listDTO=new ArrayList<StudentDTO>();
        if(listBO!=null){
            for(StudentBO bo:listBO){
                StudentDTO dto=new StudentDTO();
                dto.setSno(bo.getSno());
                dto.setSname(bo.getSname());
                dto.setSadd(bo.getSadd());
                listDTO.add(dto);
            }
        }
        return listDTO;
    }
    @Override
    public String registerStudent(StudentDTO dto) {
        //Convert DTO obj BO obj
        StudentBO bo=new StudentBO();
        bo.setSno(dto.getSno());
        bo.setSname(dto.getSname());
        bo.setSadd(dto.getSadd());
    }
}
```

::78::

StudentInsertValidator.java:

```
package com.nt.validator;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.StudentCommand;
public class StudentInsertValidator implements Validator {
    @Override
    public boolean supports(Class<?> clazz) {
        return clazz.isAssignableFrom(StudentCommand.class);
    }
    @Override
    public void validate(Object command, Errors errors) {
        //type casting
        StudentCommand cmd=(StudentCommand)command;
        //form validations
        if(cmd.getSno()==null || cmd.getSno()<=0)
            errors.rejectValue("sno","sno.invalid");
        if(cmd.getSname().equals("") || cmd.getSname()==null)
            errors.rejectValue("sname", "sname.required");
        if(cmd.getSadd().equals("") || cmd.getSadd()==null)
            errors.rejectValue("sadd", "sadd.required");
        else{
            if(cmd.getSadd().length()<3){
                errors.rejectValue("sadd","sadd.minlength");
            }
        }
    }
}
```

ListStudentsController.java:

```
package com.nt.controller;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;
public class ListStudentsController extends AbstractController {
    private StudentService service;
    public void setService(StudentService service) {
        this.service = service;
    }
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        //use Service class
    }
}
```

```
List<StudentDTO> listDTO=service.listStudents();
    //create MAV obj
    ModelAndView mav=new ModelAndView();
    mav.setViewName("list_students");
    mav.addObject("stList",listDTO);
    return mav;
}
}
```

StudentInsertController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;

public class StudentInsertController extends SimpleFormController {
    private StudentService service;

    public void setService(StudentService service) {
        this.service = service;
    }

    @Override
    protected ModelAndView onSubmit(HttpServletRequest req,
        HttpServletResponse res, Object command, BindException errors)
        throws Exception {
        // type cast command
        StudentCommand cmd=(StudentCommand)command;
        //convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //use service
        String result=service.regisiterStudent(dto);
        return new ModelAndView("insert_result","resMsg",result);
    }

    @Override
    protected ModelAndView handleInvalidSubmit(HttpServletRequest request,
        HttpServletResponse response) throws Exception {
        return new ModelAndView("dblpst");
    }
}
```

StudentUpdateController.java

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;
public class StudentUpdateController extends SimpleFormController {
    private StudentService service;

    public void setService(StudentService service) {
        this.service = service;
    }
    @Override
    protected Object formBackingObject(HttpServletRequest request)
        throws Exception {
        //read sno
        int no=Integer.parseInt(request.getParameter("id"));
        //use Service class
        StudentDTO dto=service.searchStudentBySno(no);
        //create Command class obj and place DTO obj
        StudentCommand cmd=new StudentCommand();
        cmd.setSno(dto.getSno());
        cmd.setSname(dto.getSname());
        cmd.setSadd(dto.getSadd());
        return cmd;
    }
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        // typecast command
        StudentCommand cmd=(StudentCommand)command;
        //convert Command obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(cmd.getSno());
        dto.setSname(cmd.getSname());
        dto.setSadd(cmd.getSadd());
        //use Service class
        String updMsg=service.updateStudentBySno(dto);
        return new ModelAndView(getSuccessView(),"updResult",updMsg);
    }
}
```

index.jsp:

```
<jsp:forward page="home.htm"/>
```

web.xml:

```
<web-app>
<welcome-file-list>
<welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<servlet>
<servlet-name>dispatcher</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>dispatcher</servlet-name>
<url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="/home.htm">pvc1</prop>
<prop key="/listStudents.htm">stLc</prop>
<prop key="/insert.htm">sic</prop>
<prop key="/edit.htm">suc</prop>
</props>
</property>
</bean>
<bean id="pvc1"
class="org.springframework.web.servlet.mvc.ParameterizableViewController">
<property name="viewName" value="home"/>
</bean>
```

```
<bean id="stLc" class="com.nt.controller.ListStudentsController">
<property name="service" ref="stService"/>
</bean>
<bean id="messageSource"
    class="org.springframework.context.support.ResourceBundleMessageSource">
    <property name="basename" value="com/nt/validator/StCmdValidationErrors"/>
</bean>

<bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>
<bean id="sic" class="com.nt.controller.StudentInsertController">
<property name="formView" value="insert_student"/>
<property name="successView" value="insert_result"/>
<property name="commandClass" value="com.nt.command.StudentCommand"/>
<property name="commandName" value="stCmd"/>
<property name="validator" ref="stValidator"/>
<property name="sessionForm" value="true"/>
<property name="service" ref="stService"/>
</bean>
<bean id="suc" class="com.nt.controller.StudentUpdateController">
<property name="formView" value="edit_student"/>
<property name="successView" value="edit_result"/>
<property name="commandClass" value="com.nt.command.StudentCommand"/>
<property name="commandName" value="stCmd"/>
<property name="validator" ref="stValidator"/>
<property name="sessionForm" value="true"/>
<property name="service" ref="stService"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/pages/">
<property name="suffix" value=".jsp"/>
</bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <import resource="persistence-beans.xml"/>
    <import resource="service-beans.xml"/>
    <import resource="aop-beans.xml"/>
</beans>
```

persistence-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="jofb" class="org.springframework.jndi.JndiObjectFactoryBean">
  <property name="jndiName" value="DsJndi"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="jofb"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDAOImpl">
  <property name="jt" ref="template"/>
</bean>
</beans>
```

service-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="stService" class="com.nt.service.StudentServiceImpl">
  <property name="dao" ref="stDao"/>
</bean>
</beans>
```

aop-beans.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="txMgr"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="jofb"/>
</bean>
<tx:advice id="txAdvice" transaction-manager="txMgr">
<tx:attributes>
  <tx:method name="list*" propagation="REQUIRED">
```

```
        read-only="true"/>
<tx:method name="register*" propagation="REQUIRED"
        read-only="false"/>
<tx:method name="search*" propagation="REQUIRED"
        read-only="true"/>
<tx:method name="update*" propagation="REQUIRED"
        read-only="false"/>
</tx:attributes>
</tx:advice>
<aop:config>
<aop:pointcut id="ptc1"
        expression="execution(* com.nt.service.StudentService.*(..))"/>
<aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
</aop:config>
</beans>
```

dblpost.jsp:

```
<h1 style="color:red"><center> Double Posting is not Alloed </center></h1>
<a href="home.htm">home</a>
```

edit_result.jsp:

```
<h1><center> Result Page</center></h1>
${updResult}
<a href="home.htm">home</a>
```

edit_student.jsp:

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<h1><center> Update Student Details</center></h1>
<form:form method="post" commandName="stCmd">
    Sno : <form:input path="sno"/><span style="color:red"><form:errors path="sno"/></span>
    <br>
    Sname : <form:input path="sname"/><span style="color:red"><form:errors path="sname"/>
    </span><br>
    Sadd : <form:input path="sadd"/><span style="color:red"><form:errors path="sadd"/>
    </span><br>
    <input type="submit" value="update"/>
</form:form>
```

home.jsp:

```
<h1> Home Page</h1>
<a href="listStudents.htm">List All Students</a>
```

result.jsp:

```
<h1><center> Result page</center></h1>
${resMsg}
<br>
<a href="home.htm">home</a>
```

insert_student.jsp:

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<h1><center> Register Student Details</center></h1>
<form:form method="post" commandName="stCmd">
  Sno : <form:input path="sno"/><span style="color:red"><form:errors path="sno"/></span>
  <br>
  Sname : <form:input path="sname"/><span style="color:red"><form:errors path="sname"/>
  </span><br>
  Sadd : <form:input path="86ad"/><span style="color:red"><form:errors path="86ad"/>
  </span><br>
  <input type="submit" value="register"/>
</form:form>
```

list_students.jsp:

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<h1><center> Student Details</center></h1>
<table bgcolor=red border=1 width="100%" >
<tr bgcolor="green"><th>Sno </th><th>Sname</th><th>sadd</th></tr>
<c:choose>
<c:when test="{stList ne null}">
<c:forEach var="st" items="{stList}">
<tr>
<td><a href="edit.htm?id={st.sno}"><c:out value="{st.sno}" /></a></td>
<td><c:out value="{st.sname}" /></td>
<td><c:out value="{st.sadd}" /></td>
</tr>
</c:forEach>
</c:when>
<c:otherwise>
<span style="color:red"> No Records Found </span>
</c:otherwise>
</c:choose>
</table>
<br><br>
<center>
<a href="insert.htm"> Insert Student</a>
</center>
<br>
<br>
<a href="home.htm">home</a>
```

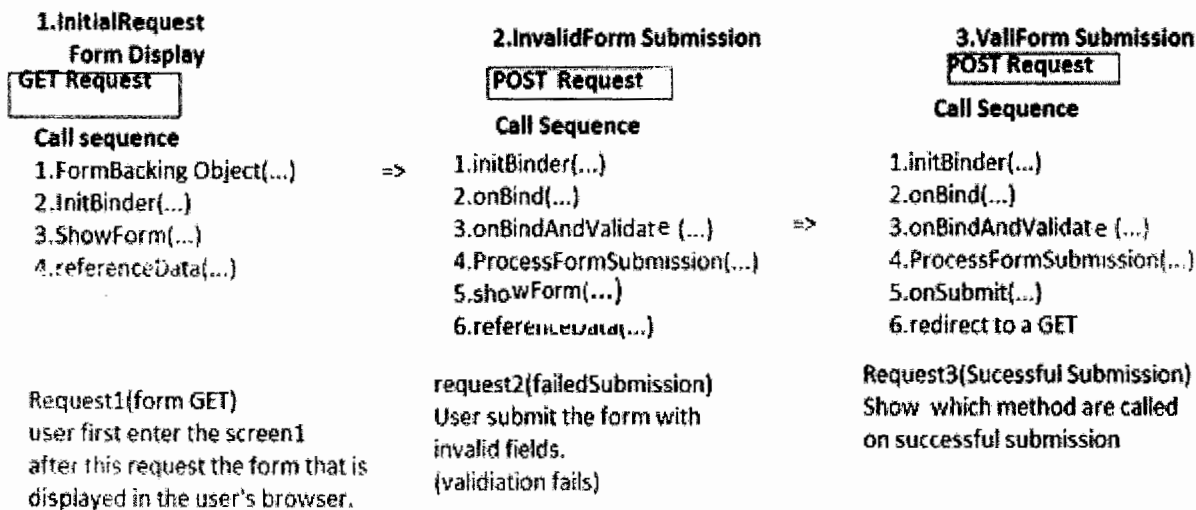

referenceData (HttpServletRequest)

- ⇒ While displaying form page as the response of initial phase request or as the response of post back request when form validation errors are there. if you want to display additional data apart from the data that is there in command object, we can override referenceData(-) in our SimpleFormController class to construct that data dynamically. This method returns Map object having data in the form of values List with key.
- ⇒ DispatcherServlet keeps this Map obj in request scope. In order to display this Map object data in certain component we need the key of Map obj in the "items" attribute of <form:options>, <form:checkboxes> and etc tags...

Method signature:

```
Public Map referenceData(HttpServletRequest request)
```

Understanding how various methods will be called internally from handleRequest() of SimpleFormController:



1. ShowForm(...) method collects and returns formview property value as logical viewname.
2. onBind(...) method reads the form data and stores into commandobject(RequestWrapping).
3. onBindAndValidate(...) method performs form validations and stores errors in Bindexception object.
4. ProcessFormSubmission(...) method takes Bindexception obj and check's weather from validation errors are there (or) not there .

If errors are there it calls ShowFrom(...) method otherwise it calls onSubmit(...) method.

Note:

All methods given in the above three boxes are actually called from handleRequest(-, -) method of SimpleFormController class.s

- ⇒ handleInvalid(-, -) method executes only double posting is occurred.

Note:

The spring tags generated radio buttons, List boxes, Select box and Checkboxes can't display initial/old values collected from command object for this we need to construct that additional/old values for these components by overriding `referenceData(-)`.

- ▲  MVCStudentApp7SPRINGFRAMEWORKC
 - ▶  JAX-WS Web Services
 - ▶ Deployment Descriptor: MVCStudentApp7SPRINGFRAMEWORKC.xml
 - ▲  Java Resources
 - ▲  src
 - ▲  com.nt.command
 - ▶  RegistrationCommand.java
 - ▲  com.nt.controller
 - ▶  StudentRegistrationController.java
 - ▲  com.nt.validations
 - ▶  StudentInsertValidator.java
 - ▶  StCmdValidationErrors.properties
 - ▶  Libraries
 - ▶  JavaScript Resources
 - ▶  build
 - ▲  WebContent
 - ▶  META-INF
 - ▲  WEB-INF
 - ▶  lib
 - ▶  pages
 - ▶  applicationContext.xml
 - ▶  dispatcher-servlet.xml
 - ▶  persistence-beans.xml
 - ▶  service-beans.xml
 - ▶  web.xml
 - ▶  index.jsp

Add Jars: Spring 3.2.6 jar files

StudentRegistrationController.java:

```
package com.nt.controller;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.validation.BindException;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
public class StudentRegistrationController extends SimpleFormController {
```

@Override

```
protected Map referenceData(HttpServletRequest request) throws Exception {
    System.out.println("SRC: referenceData(-)");
    Map<String, List> map=new HashMap<String, List>();
    List <String> courses=new ArrayList();
    courses.add("spring"); courses.add("hibernate");
    courses.add("adv.java"); courses.add("webservices");
    map.put("courseList",courses);
    return map;
}
```

@Override

```
protected ModelAndView onSubmit(HttpServletRequest request,
    HttpServletResponse response, Object command, BindException errors)
    throws Exception {
    return new ModelAndView("register_result");
}
}
```

RegistrationCommand.java:

```
package com.nt.command;  
public class RegistrationCommand {  
    private int sno;  
    private String sname;  
    private String course;  
    public int getSno() {  
        return sno;  
    }  
    public void setSno(int sno) {  
        this.sno = sno;  
    }  
    public String getSname() {  
        return sname;  
    }  
    public void setSname(String sname) {  
        this.sname = sname;  
    }  
    public String getCourse() {  
        return course;  
    }  
    public void setCourse(String course) {  
        this.course = course;  
    }  
}
```

StCmdValidationErrors.java:

```
ypeMismatch.sno=student no can not empty or can not be String  
sno.invalid=Student no must be positive value  
sname.required=Student name is mandatory
```

StudentInsertValidator.java

```
package com.nt.validator;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
import com.nt.command.RegistrationCommand;
public class StudentInsertValidator implements Validator {
    @Override
    public boolean supports(Class<?> clazz) {
        return clazz.isAssignableFrom(RegistrationCommand.class);
    }
    @Override
    public void validate(Object command, Errors errors) {
        //type casting
        RegistrationCommand cmd=(RegistrationCommand)command;
        //form validations
        if(cmd.getSno()<=0)
            errors.rejectValue("sno", "sno.invalid");
        if(cmd.getSname().equals("") || cmd.getSname()==null)
            errors.rejectValue("sname", "sname.required");
    }
}
```

index.jsp:

```
<jsp:forward page="register.htm"/>
```

web.xml:

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
    <listener-
class>org.springframework.web.context.ContextLoaderListener</listener-class>
    </listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean
class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="/register.htm">src</prop>
</props>
</property>
</bean>
<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
<property name="basename" value="com/nt/validator/StCmdValidationErrors"/>
</bean>
<bean id="stValidator" class="com.nt.validator.StudentInsertValidator"/>
<bean id="src" class="com.nt.controller.StudentRegistrationController">
<property name="formView" value="register"/>
<property name="validator" ref="stValidator"/>
<property name="successView" value="register_result"/>
<property name="commandClass"
value="com.nt.command.RegistrationCommand"/>
<property name="commandName" value="regCmd"/>
<property name="sessionForm" value="true"/>
</bean>
<bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/pages"/>
```

result.jsp:

```
<property name="suffix" value=".jsp"/>
</bean>
</beans>
```

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
<form:form method="POST" commandName="regCmd">
<form:input path="sno"/>
<br>
No : <form:input path="sno"/><br>
Name : <form:input path="sname"/><br>
Courses :
<form:select path="course">
<form:options items="{courseList}"/><br>
</form:select>
<input type="submit" value="register"/>
</form:form>

```

register.jsp

```

<b><center> Registration Successspringframeworkul</center></b>

```

Use case:

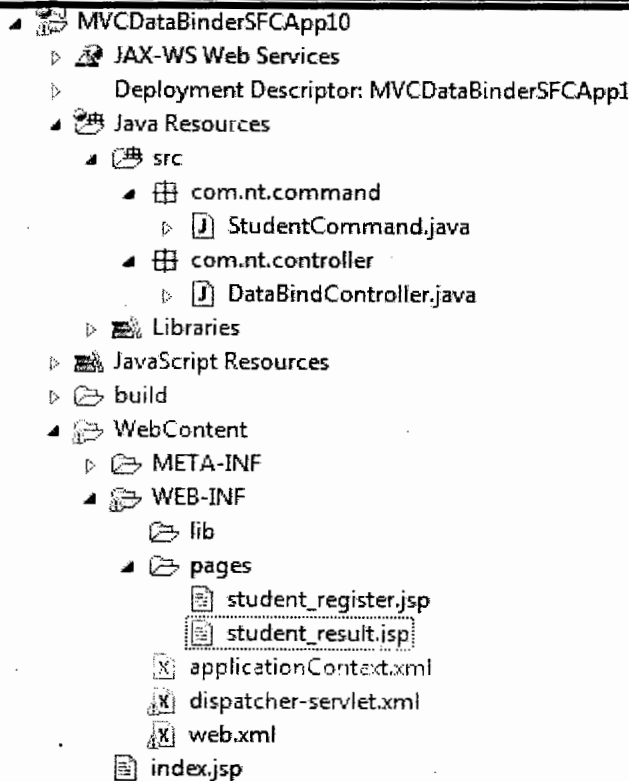
- Displaying roles in form page (Select box)
- For initial phase request and it in post back request when it generates form validation errors.

initBinder(-,-) :-

- ⇒ property editors of spring are very useful to convert given values to required type properties of Bean class or command class.
- ⇒ Spring gives built-in property editors and allows us to develop custom property editors. CustomDateEditor is given to convert spring data values into java.util.Date class object by specifying date format.
- ⇒ While working with SimpleFormController we can override initBinder(-,-) method to register property editors that are required to use while binding given form data (request param values) to command class object properties.
- ⇒ This method gives request, ServletRequestDataBinder objects. We can register property editors with servletRequestDataBinder obj because it is responsible to bind the form submitted request parameter values to command object properties.

Use case:

Storing string value of form component to command class property as java.util.Date class obj.
Internal flow of execution in handleRequest method.



Add jars: Spring 3.2.6 jars

StudentCommand.java:

```
package com.nt.command;
import java.util.Date;
public class StudentCommand {
    private String name;
    private Date dob, doj, dom;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Date getDob() {
        return dob;
    }
    public void setDob(Date dob) {
        this.dob = dob;
    }
    public Date getDoj() {
        return doj;
    }
    public void setDoj(Date doj) {
        this.doj = doj;
    }
    public Date getDom() {
```



```
        return dom;
    }
    public void setDom(Date dom) {
        this.dom = dom;
    }
    @Override
    public String toString() {
        return "StudentCommand [name=" + name + ", dob=" + dob + ", doj=" + doj
            + ", dom=" + dom + "]";
    }
}
```

DataBindController.java:

```
package com.nt.controller;
import java.text.SimpleDateFormat;
import java.util.Date;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.beans.propertyeditors.CustomDateEditor;
import org.springframework.validation.BindException;
import org.springframework.web.bind.ServletRequestDataBinder;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.SimpleFormController;
import com.nt.command.StudentCommand;
public class DataBindController extends SimpleFormController {
    @Override
    protected void initBinder(HttpServletRequest request,
        ServletRequestDataBinder binder) throws Exception {
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        binder.registerCustomEditor(Date.class,
            new CustomDateEditor(sdf,true));
    }
    @Override
    protected ModelAndView onSubmit(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        StudentCommand cmd=(StudentCommand)command;
        return new ModelAndView("student_result","cmdData",cmd);
    }
}
```

Index.jsp

```
<jsp:forward page="student.htm"/>
```

web.xml:

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/student.htm">dbc</prop>
      </props>
    </property>
  </bean>
  <bean id="dbc" class="com.nt.controller.DataBindController">
    <property name="formView" value="student_register"/>
    <property name="successView" value="student_result"/>
    <property name="commandClass" value="com.nt.command.StudentCommand"/>
    <property name="commandName" value="regCmd"/>
    <property name="sessionForm" value="true"/>
  </bean>
  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>
```

student_register.jsp:

```
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<form:form method="POST" commandName="regCmd">
<form:errors path="*" />
Name : <form:input path="name" /><br>
DOB : <form:input path="dob" /><br>
DOJ : <form:input path="doj" /><br>
DOM : <form:input path="dom" /><br>
<input type="submit" value="register" />
</form:form>
```

student_result.jsp:

The given details are :
 \${cmdData}

MultiActionController:

⇒ When form page contains multiple submit buttons, then instead of taking multiple AbstractCommandController/SimpleFormController classes take single MultiActionController having multiple user-defined methods to process request of multiple submit buttons on one per method basis.

⇒ The userdefined methods of this class can have following standard signature

```
public (C \Map\String\void) <methodname>
```

```
(HttpServletRequest request, HttpServletResponse response, [HttpSession], [anyObject]);
```

- ModelAndView(lvn+model)
- Map(only Model data), So virtual path of request uri becomes view name).
- void (no Model data , So virtual path becomes view name).
- String(no model data, only logical view name).

While configuring our MultiActionController class we must specify the MethodNameResolver obj as dependent obj that contains additional parameter name based on which the method that has to be

executed in MultiActionController class will be decided.

Conclusion

- ⇒ If source page is form submission having multiple submit buttons with optional form validation and the required destination page is static/dynamic webpage that should come after request processing then we should go after MultiActionController.

Ex:

sno::	
sname::	
sadd::	
insert	update
delete	view

```
public PerformOperationsController extends MultiActionController
{
    public MAV add(-,-,-){
        -----
    }

    public MAV update(-,-,-){
        -----
    }

    public MAV delete(-,-,-){
        -----
    }

    public MAV view(-,-,-){
        -----
    }
}
```

Flow for insert submit button:

- ⇒ Insert submit button---->DispatcherServlet---->HandlerMapping---->controller (MAController)-
----->handleRequest(-,-)----->create command obj and performs request wrapping.
- ⇒ Reads paramName(opt), defaultMethodName(invalid) from the injected MethodNamesResolver obj.

Reads the values of "opt" req param (insert)----->Calls insert (req,res, command) method.

SourcePage

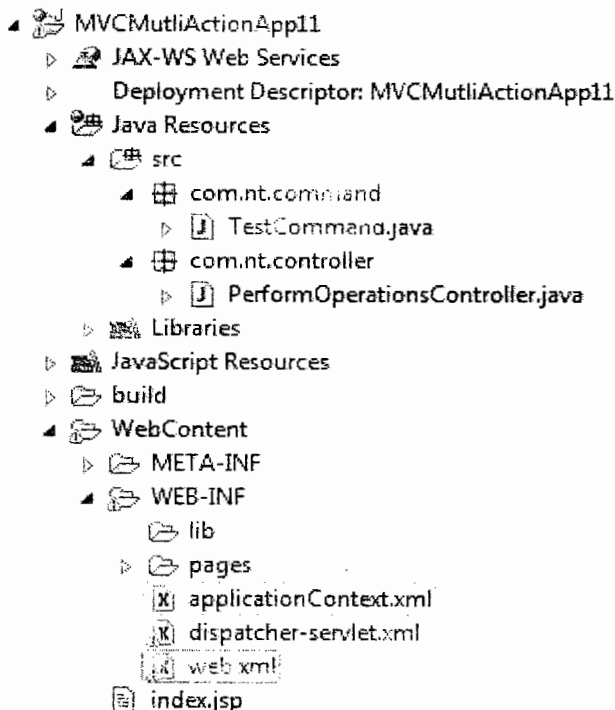
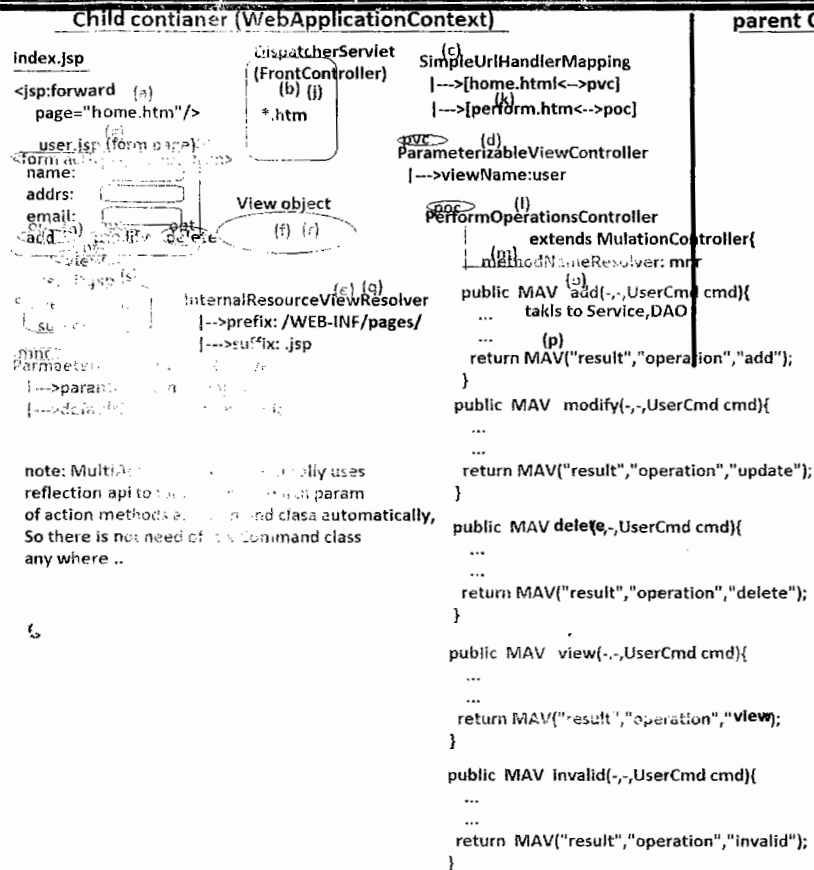
Multiple hyperlinks
(or)
multiple submitbuttons.
->optional requestwrapping
->optional form handling

MultiActionController

->MethodNameResolver
->CommandName(optional)
->commandclass(optional)
we can take multiple usermethods
for multiple hyperlinks(or)
submitbuttons.

Destination page

Simple(or)multiple
static/dynamic webpages
(After request processing)



Add Jars: Spring 4.x, JUnit 4, Mockito

Ex:

TestCommand.java:

```
package com.nt.command;
public class TestCommand {
    private int sno;
    private String sname;
    private String sadd;
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

PerformOperationsController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.multiaction.MultiActionController;
import com.nt.command.TestCommand;
public class PerformOperationsController extends MultiActionController {

    public ModelAndView insert(HttpServletRequest req,HttpServletResponse res,TestCommand
cmd){
        System.out.println(cmd.getSno());
        return new ModelAndView("user","operation","Insert Operation");
    }

    public ModelAndView update(HttpServletRequest req,HttpServletResponse
res,TestCommand cmd){
        return new ModelAndView("user","operation","Update Operation");
    }
    public ModelAndView delete(HttpServletRequest req,HttpServletResponse res,TestCommand
cmd){
```

```
return new ModelAndView("user","operation","delete Operation");
}

public ModelAndView invalid(HttpServletRequest req, HttpServletResponse res, TestCommand
cmd){
return new ModelAndView("user","operation","Invalid Operation");
}
}
```

index.jsp:

```
<jsp:forward page="form.htm"/>
```

web.xml:

```
<web-app>
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
  <property name="mappings">
    <props>
      <prop key="/form.htm">pvc1</prop>
      <prop key="/perform.htm">poc</prop>
    </props>
  </property>
</bean>
```

```

</property>
</bean>
<bean id="pvc1"
      class="org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="user" />
</bean>
<bean id="mnr"
      class="org.springframework.web.servlet.mvc.multiaction.ParameterMethodName
Resolver">
  <property name="paramName" value="opt"/>
  <property name="defaultMethodName" value="invalid"/>
</bean>
<bean id="poc" class="com.nt.controller.PerformOperationsController">
  <property name="methodNameResolver" ref="mnr"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>

```

applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>

```

user.jsp:

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<form action="perform.htm" method="post">
  Sno : <input type="text" name="sno"/><br>
  Sname : <input type="text" name="sname"/><br>
  Saddr : <input type="text" name="saddr"/><br>
  <input type="submit" name="opt" value="insert"/>
  <input type="submit" name="opt" value="update"/>
  <input type="submit" name="opt" value="delete"/>
  <input type="submit" name="opt1" value="view"/>
</form>
<c:if test="${operation ne null}">
<c:out value="${operation}"/> Successspringframeworkul
</c:if>

```


Note:

Only SimpleFormController, AbstractCommandController classes are removed from spring 4.0

While working with MultiActionController there is no need of configuring command class anywhere because the superclass handlerRequest (-,-) internally uses Reflection API to take specified 3rd parameter type or 4th parameter type (if 3rd parameter is HttpSession) as command class name...

AbstractWizardFormController:

As far as we worked with single form page based controller classes like SimpleFormController, MultiActionController and etc... In order to work with multiple form pages that comes in a sequence having wizard style we need to use AbstractWizardFormController by specifying list of form pages to display in a sequence. In this controller the data of all the form pages will come and store in single command class object. Based on the name of additional request parameter that comes along with the request this will display the next form page or process the request.

- ⇒ `_cancel`: leaves the wizard without performing final action (cancel button)
- ⇒ `_finish`: leaves the wizard by performing final Action (finish button)
- ⇒ `_target<n>`: displays the nth page from the list of form pages that are configured (for previous, next buttons)

Flow:

Welcomepage-->formpage1---->formpage2----->formpage3----->Result page.

We can override the following methods in our AbstractWizardFormController.

`formBackingObject(-)`: creates and returns commands object

`processFinish(..)`: takes final action of request processing for finish button.

`processCancel(-)`: executes the logic related to "Cancel button".

SourcePage

-> Multiple form pages in sequence having wizard style.
-> Request wrapping of multiple forms data to Single command object

AbstractWizardFormController

-> Pages: page1form, page2form, page3form
-> CommandName: reqcmd
-> CommandClass: RegisterCommand
-> MAVProcessFinish(-,-,-){...}
-> MAVProcessCancel(-,-,-){...}

DestinationPage

Static/dynamic After request Processing

Example:

Naukri.com, Shadi.com registrations.

-> `_target<n>`----->displays `nth` page(give-target<n> for previous,next buttons)

-> `_finish` ----->processFinish(-,-,-)(give-finish as name of the finish button)

-> `_cancel` ----->processCancel(-,-,-)(give-cancel as the name of cancel).

```

index.jsp (a)
<jsp:forward page="home.htm">

welcome.jsp (g)
<a href="register.htm">
  register (h)
</a>

page1Form.jsp (p)
<form:form method="POST">
...
<input type="submit"
  name="_target1"
  value="next">(q)
</form>

page2Form.jsp (x)
<form method="POST">
...
<input type="submit" name="_target2" value="next">(y)
<input type="submit" name="_target0" value="previous">
<input type="submit" name="_cancel" value="cancel">
</form>

page3Form.jsp (a6)
<form:form method="POST">
...
<input type="submit" name="_finish" value="finish">(a7)
<input type="submit" name="_target1" value="previous">
<input type="submit" name="_cancel" value="cancel">
</form:form>
  
```

DispatcherServlet

```

(b)
*.htm
(i) (z) (a8)
  
```

SimpleUrlHandlerMapping

```

|-->[home.htm<-->pvc1]
|-->[register.htm<-->rc]
  
```

ParameterizableViewController

```

|--> viewName:welcome (d)
  
```

RegisterController (AWFC)

```

|-->pages (m)
  |-->page1Form.
  |-->page2Form. (U)
  |-->page3Form. (a3)
  |-->commandName: regCmd

formBackingObject(-){
(l) return new RegisterCommand();
}

processFinish(req, req, cmd, BE){
... (a11)
... new
return MAV(-,-,-); (a12)
}

processCancel(req, res, cmd, BE){
...
...
return new MAV(-,-,-);
}
  
```

InternalResourceViewResolver

```

(e) (n) (v) (a4) (a13)
|-->prefix: /WEB-INF/pages/
|-->suffix: .jsp
  
```

View obj (a5)

```

(f) (o) (w)
(a14)
  
```

result.jsp (a15)

```

....
.....
.....
  
```

```

index.jsp (a)
<jsp:forward page="home.htm">

welcome.jsp (g)
<a href="register.htm">
  register (h)
</a>

page1Form.jsp (p)
-----
<form:form method="POST">
...
...
<input type="submit"
  name="_target1"
  value="next">(q)
</form>

page2Form.jsp (x)
-----
<form method="POST">
...
...
<input type="submit" name="_target2" value="next">
<input type="submit" name="_target0" value="previous">
<input type="submit" name="_cancel" value="cancel">
</form>

page3Form.jsp (a6)
<form:form method="POST">
...
...
<input type="submit" name="_finish" value="finish">(a7)
<input type="submit" name="_target1" value="previous">
<input type="submit" name="_cancel" value="cancel">
</form:form>

```

DispatcherServlet

(b)
 *.htm
 (i)
 (r) (z) (a8)

SimpleUrlHandlerMapping

```

|-->[home.htm<-->pvc1]
|-->[register.htm<-->rc]
      (j) (s) (a1) (a9)

```

ParameterizableViewController

```

|---> viewName:welcome (d)

```

View obj (a5)

(f) (o) (w)

(a14)

InternalResourceViewResolver

```

(e) (n) (v) (a4) (a13)
|--->prefix: /WEB-INF/pages/
|--->suffix: .jsp

```

RegisterController (AWFC)

```

rc (k) (t) (a2) (a10)
|--->pages (m)
    |-->page1Form.
    |-->page2Form. (U)
    |-->page3Form. (a3)
|--->commandName: regCmd

formBackingObject(-){
(l) return new RegisterCommand();
}

processFinish(req, req, cmd, BE){
... (a11)
... new
return MAV(-, -, -); (a12)
}

processCancel(req, res, cmd, BE){
...
...
return new MAV(-, -, -);
}

```

result.jsp (a15)

- ▲ MVCWizardFormApp12
 - ▷ JAX-WS Web Services
 - ▷ Deployment Descriptor: MVCWizardFormApp12
 - ▲ Java Resources
 - ▲ src
 - ▲ com.nt.command
 - ▷ RegisterCommand.java
 - ▲ com.nt.controller
 - ▷ RegisterController.java
 - ▷ Libraries
 - ▷ JavaScript Resources
 - ▷ build
 - ▲ WebContent
 - ▷ META-INF
 - ▲ WEB-INF
 - lib
 - ▲ pages
 - Page1Form.jsp
 - Page2Form.jsp
 - Page3Form.jsp
 - result.jsp
 - welcome.jsp
 - applicationContext.xml
 - dispatcher-servlet.xml
 - web.xml
 - index.jsp

Add Jars: add 3.x jar files

RegisterCommand.java:

```
package com.nt.command;
public class RegisterCommand{
    //Login Credentials
    private String userName;
    private String password;
    //Contact Details
    private String emailId;
    private String phone;
    //Address Details
    private String city;
    private String pincode;
    public RegisterCommand(){
        System.out.println("RegisterCommand:0-param constructor");
    }
    public String getUserName() {
        return userName;
    }
    public void setUserName(String userName) {
        this.userName = userName;
    }
}
```

```
public String getPassword() {  
    return password;  
}  
public void setPassword(String password) {  
    this.password = password;  
}  
public String getEmailId() {  
    return emailId;  
}  
public void setEmailId(String emailId) {  
    this.emailId = emailId;  
}  
public String getPhone() {  
    return phone;  
}  
public void setPhone(String phone) {  
    this.phone = phone;  
}  
public String getCity() {  
    return city;  
}  
public void setCity(String city) {  
    this.city = city;  
}  
public String getPincode() {  
    return pincode;  
}  
public void setPincode(String pincode) {  
    this.pincode = pincode;  
}  
@Override  
public String toString() {  
    return "RegisterCommand [userName=" + userName + ", password=" +  
        password + ", emailId=" + emailId + ", phone=" + phone  
        + ", city=" + city + ", pincode=" + pincode + "];"  
}  
}
```

RegisterController.java:

```
package com.nt.controller;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.validation.BindException;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractWizardFormController;  
import com.nt.command.RegisterCommand;
```

```
public class RegisterController extends AbstractWizardFormController{
    /*public RegisterController(){
        setCommandName("userForm");
    }*/
    }
    @Override
    protected ModelAndView processfinish(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        System.out.println("RegisterController.processspringframeworkinish(-,-,-)");
        //Get the data from command object
        RegisterCommand cmd = (RegisterCommand)command;

        //where is the finish page?
        return new ModelAndView("result", "res", cmd);
    }
    @Override
    protected ModelAndView processCancel(HttpServletRequest request,
        HttpServletResponse response, Object command, BindException errors)
        throws Exception {
        System.out.println("RegisterController.processCancel(-,-,-)");
        //where is the cancel page?
        return new ModelAndView("welcome");
    }
}
```

index.jsp:

```
<jsp:forward page="home.htm"/>
```

web.xml:

```
<web-app>
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
```

```
</context-param>
<welcome-file-list>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
</web-app>
```

dispatcher-servlet.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">

  <bean id="surl" class="org.springframework.web.servlet.handler.SimpleUrlHandler
Mapping">
    <property name="mappings">
      <props>
        <prop key="/home.htm">pvc1</prop>
        <prop key="/register.htm">rc</prop>
      </props>
    </property>
  </bean>

  <bean id="pvc1" class="
    org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="welcome"/>
  </bean>

  <bean id="rc" class="com.nt.controller.RegisterController" >
    <property name="commandName" value="regCmd"/>
    <property name="pages">
      <list>
        <value>Page1Form</value>
        <value>Page2Form</value>
        <value>Page3Form</value>
      </list>
    </property>
  </bean>

  <bean id="viewResolver"
    class="org.springframework.web.servlet.view.InternalResourceViewResolver" >
    <property name="prefix" value="/WEB-INF/pages/">
    <property name="suffix" value=".jsp"/>
  </bean>
</beans>
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>
```

Page1Form.jsp:

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<form:form method="POST" commandName="regCmd">
Username : <form:input path="userName"/><br>
Password: <form:password path="password"/><br>
<input type="submit" value="Next" name="_target1" />
<input type="submit" value="Cancel" name="_cancel" />
</form:form>
```

Page2Form.jsp:

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<form:form method="POST" commandName="regCmd">
Email :<form:input path="emailId"/><br>
Phone:<form:input path="phone" /><br>
<input type="submit" value="Previous" name="_target0" />
<input type="submit" value="Next" name="_target2" />
<input type="submit" value="Cancel" name="_cancel" />
</form:form>
```

Page3Form.jsp:

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<form:form method="POST" commandName="regCmd">
City : <form:input path="city" /><br>
Pincode:<form:input path="pincode" /><br>
<input type="submit" value="Previous" name="_target1" />
<input type="submit" value="Finish" name="_finish" />
<input type="submit" value="Cancel" name="_cancel" />
</form:form>
```

result.jsp:

```
<h1><center> Result page</center></h1>
${res}
```


welcome.jsp:

```
<H1><center> HOME PAGE</center></H1>
<br>
<br>
<a href="register.htm"> Register</a>
```

Note:-

- ⇒ When we give initial phase request to *AbstractWizardFormController* it internally calls *FormBackingObject()* through *handleRequest(-, -)* method to create command object and to keep that object in session scope.
- ⇒ When postback request is given having *_target<n>* request parameter name it displays next or previous page postback request having the request param name *_cancel* calls *processCancel(-)* through *handleRequest(-)* method.
- ⇒ Similarly calls *processFinish(-)* for postback request that contains *_finish* request param.

=====0000000=====

HandlerMappings:(Different types of HandlerMappings)

- ⇒ Handler Mapping is a component that maps incoming request to controller/handler class. All handlermapping classes are the implementation classes of "HandlerMapping" (i).

The important HandlerMapping classes are:

- 1) *BeanNameUrlHanlderMapping* (default for xml configuration)
- 2) *SimpleUrlHandlerMapping*
- 3) *ControllerClassNameHandlerMapping*
- 4) *DefaultAnnotationHandlerMapping* (default for annotation configuration)

1) BeanNameUrlHandlerMapping:

- ⇒ This maps incoming request to controller class based on the servlet path or virtual path of incoming request url. i.e the virtual path of incoming request url must be taken as bean name of controller class.

Eg:

```
<bean class="org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping"/>
<bean name="/home.htm" class="com.nt.controller.WelcomeController"/>
```

- ⇒ If the incoming request url contains *"/home.htm"* then it is maps the request to *WelcomeController* class.

2) SimpleUrlHandlerMapping:

⇒ This is useful to map the incoming requests with controller classes based on virtual paths of request urls and bean ids/names of controller class.

We need to supply the details as values of "mappings" property of type (java.util.Properties)

eg:

```
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
  <props>
    <prop key="/home.htm">wc</prop>
  </props>
</property>
</bean>
<bean name="wc" class="com.nt.controller.WelcomeController"/>
```

If request url contains "/home.htm" then it passes the request to WelcomeController class whose bean id /name as "wc".

3) ControllerClassNameHandlerMapping:

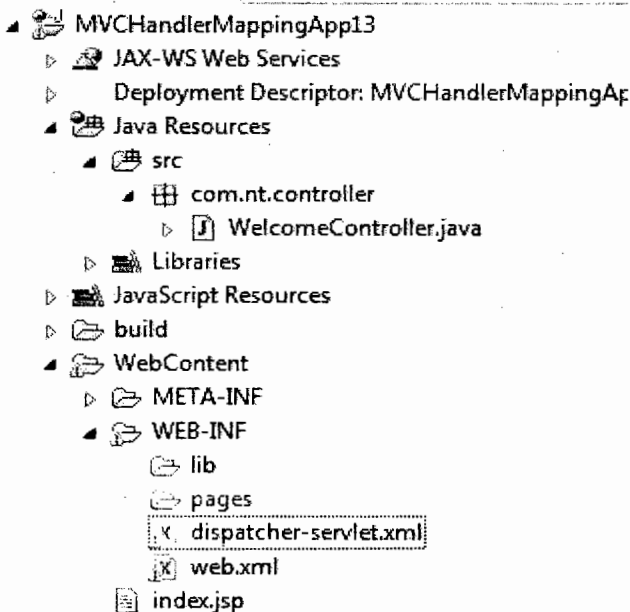
⇒ Maps incoming request with controller classes based on virtual paths and controller class names.

⇒ It takes virtual path, removes extension, makes firstLetter as uppercase letter and appends Controller word to it and looks for controller class whose name is that name.

Note:

If request url contains "welcome.htm" then it maps the request to a controller class whose name is "WelcomeController".

Ex:



WelcomeController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;
public class WelcomeController implements Controller {
    @Override
    public ModelAndView handleRequest(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        return new ModelAndView("welcome","msg","Welcome To Spring MVC");
    }
}
```

index.jsp

```
<jsp:forward page="welcome.htm"/>
```

web.xml:

```
<web-app>
<servlet>
<servlet-name>dispatcher</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>dispatcher</servlet-name>
<url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="wc" class="com.nt.controller.WelcomeController"/>
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
        <props>
            <prop key="welcome.htm">wc</prop>
        </props>
    </property>
    <property name="order" value="1"/>
</bean>
<bean class="org.springframework.web.servlet.mvc.support.ControllerClassNameHandler
```

```

Mapping">
  <property name="order" value="2"/>
</bean>
<bean id="vr" class="org.springframework.web.servlet.view.InternalResourceView
Resolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>

```

welcome.jsp:

```

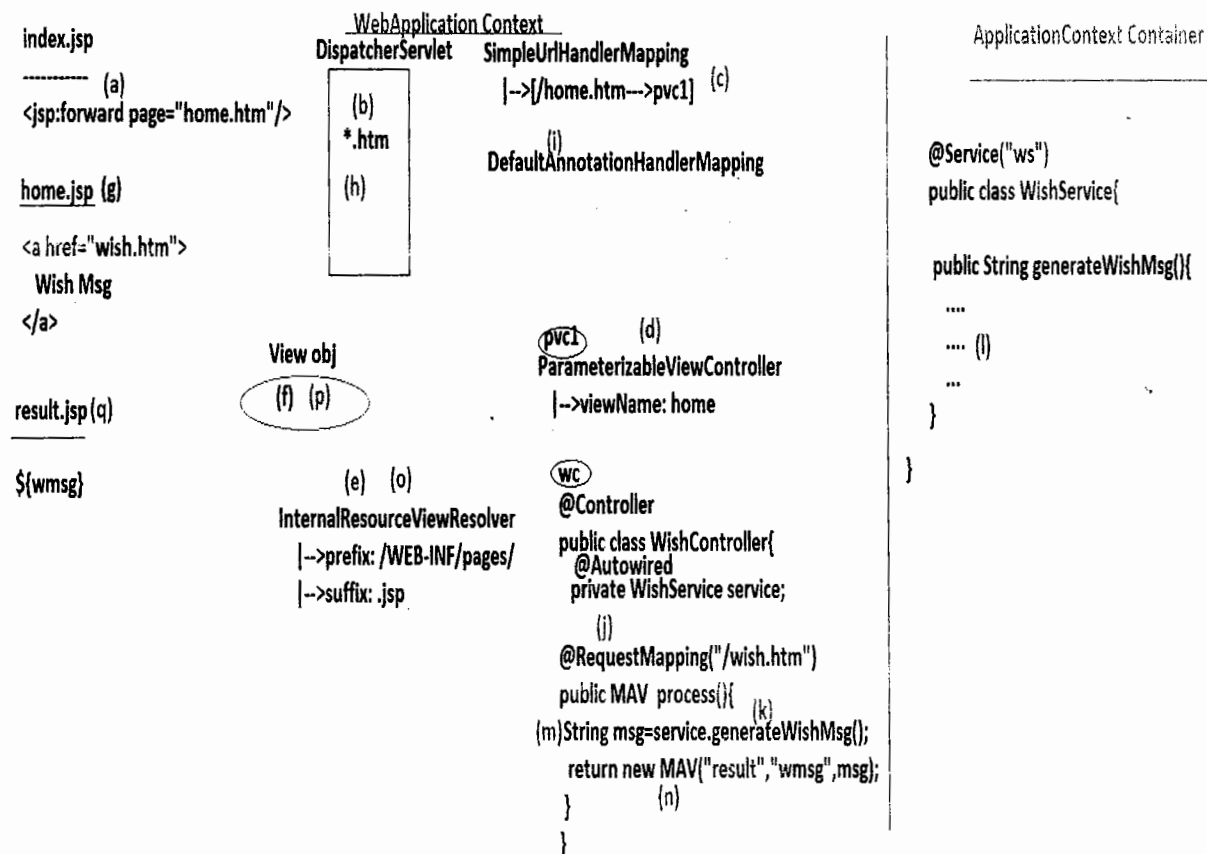
<h1><center> HOME PAGE</center></h1>
${requestScope.msg}

```

DefaultAnnotationHandlerMapping:

Maps the incoming request url with controller class methods based on @RequestMapping annotation info.

Diagram:



- MVC-AnnoWishApp18-AC
 - JAX-WS Web Services
 - Deployment Descriptor: MVC-AnnoWishApp18-AC
 - Java Resources
 - src
 - com.nt.controller
 - WishController.java
 - com.nt.service
 - WishService.java
 - Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - pages
 - dispatcher-servlet.xml
 - web.xml
 - index.jsp

WishService.java

```
package com.nt.service;
import java.util.Calendar;
import javax.inject.Named;
@Named("ws")
public class WishService {
    public String generateWishMsg(){
        // generate wish Msg based on current hour the day
        Calendar cl=Calendar.getInstance();
        int hour=cl.get(Calendar.HOUR_OF_DAY);
        if(hour<=12)
            return "Good morning";
        else if(hour<=16)
            return "Good After noon";
        else if (hour<=20)
            return "Good Evening";
        else
            return "Good Night";
    }
}
//method
}
//class
```

WishController.java

```
package com.nt.controller;
import javax.annotation.Resource;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import com.nt.service.WishService;
@Controller
public class WishController {
    @Resource
    private WishService service;
    /* @RequestMapping("/wish.htm")
    public ModelAndView process(){
        //use service class
        String msg=service.generateWishMsg();

        return new ModelAndView("result","wmsg",msg);
    } */
    /* @RequestMapping("/wish.htm")
    public String process(Model model){

        //use service class
        String msg=service.generateWishMsg();

        // add data to mode
        model.addAttribute("wmsg",msg);
        return "result";
    } */
    /* @RequestMapping("/wish.htm")
    public String process(ModelMap map){
        //use service class
        String msg=service.generateWishMsg();
        map.addAttribute("wmsg", msg);
        return "result";
    } */
    /* @RequestMapping("/wish.htm")
    public String process(Map<String,Object> map){
        //use service class
        String msg=service.generateWishMsg();
        map.put("wmsg",msg);
        return "result";
    } */
    @RequestMapping("/wish.htm")
    public void process(Model model){
        //Use service class
        String msg=service.generateWishMsg();
        model.addAttribute("wmsg",msg);
    }
}
} //class
```

index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/home.htm">pvc1</prop>
      </props>
    </property>
  </bean>
  <bean id="pvc1"
    class="org.springframework.web.servlet.mvc.ParameterizableViewController">
    <property name="viewName" value="home"/>
  </bean>
  <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping"/>
  <!-- <bean class="org.springframework.web.servlet.view.DefaultRequest
```

```

ToViewNameTranslator"/> -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
<context:component-scan base-package="com.nt.controller" />
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.nt.service" />
</beans>

```

Home.jsp

```

<h1><center> Welcome to Annotations Based Spring MVC</center></h1>
<a href="wish.htm"> wish Msg</a>

```

Result.jsp

```

<center><h1> Result Page</h1></center>
<br>
Wish Message : ${wmsg}
<a href="home.htm">home</a>

```

Wish.jsp

```

<b> wish.jsp</b>
Result is : ${wmsg}

```

- ⇒ When we configure multiple handler mapping in which order they should be executed will be decided based on "order" priority value high value indicates low priority and low value indicates high priority.

Request: welcome.htm

⇒ Eg:

```

<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
</property>
<property name="order" value="1" />
</bean>
<bean
class="org.springframework.web.servlet.mvc.support.ControllerClassNameHandlerMapping">
<property name="order" value="2" />

```


</bean>

While configuring handler mapping we can also specify the order and handler interceptor.

HandlerInterceptor:

- ⇒ It act as hook (additional) for controller/handler class to have pre and post processing logics. It is like `SrvletFilter` of J2EE web application development, But it is different from filter in 1 way. Filter can have only pre request processing and post response processing logics. Where as handler interceptor can have pre-request processing, before rendering view, after rendering view logics with respect to controller classes.
- ⇒ To develop interceptor we need to take a class that implements "HandlerInterceptor" (i), But there is adaptor class for this whose name is "HandlerInterceptorAdapter", so we can extend from this class to override our choice methods.

`preHandle(--)`----->executes before control goes to controllerclass

`postHandle(--)`----->executes once controller class finished the execution but before rendering the view.

`afterCompletion()` : executes after rendering the view.

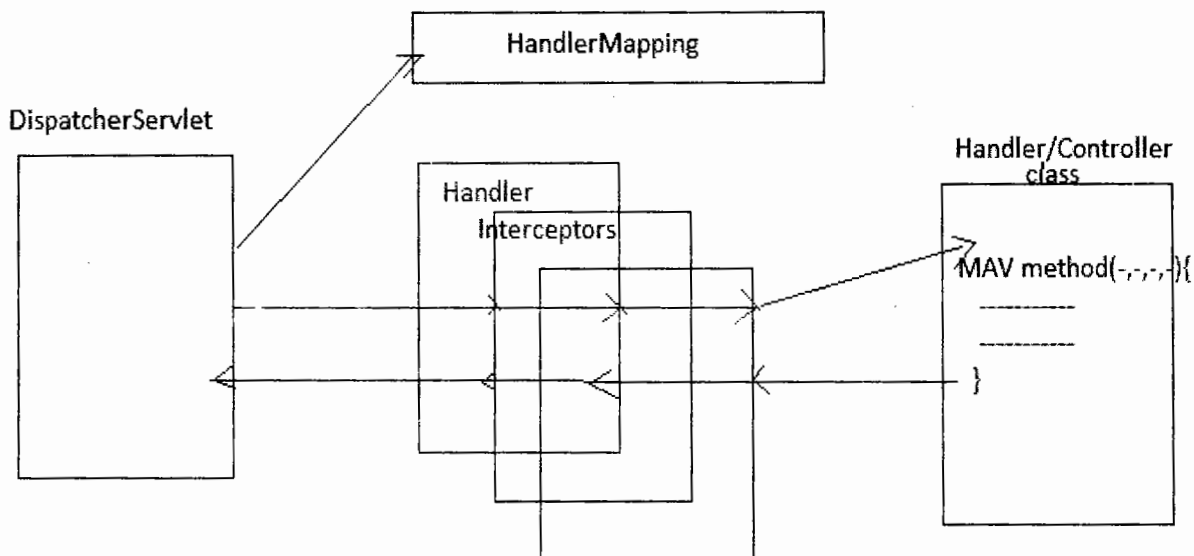
Useases of HandlerInterceptor:

We can develop HandlerInterceptor class for controller only to take the request between 9am to 5pm.

We can develop HandlerInterceptor class for controller only to allow the request that are coming from certain browser and etc....

Steps:-

- 1) Keep any spring application ready.
- 2) Develop HandlerInterceptor class.
- 3) Specifying HandlerInterceptor has the dependent value to HandlerMapping.



- Project Explorer
 - MVC-AnnoWishApp18-AC
 - JAX-WS Web Services
 - Deployment Descriptor: MVC-AnnoWishApp18-
 - Java Resources
 - src
 - com.nt.controller
 - ViewHomeController.java
 - WishController.java
 - com.nt.interceptor
 - TimeCheckInterceptor.java
 - TimeCheckInterceptor
 - Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - pages
 - dispatcher-servlet.xml
 - web.xml
 - index.jsp

TimeCheckInterceptor.java:

```
package com.nt.interceptor;
import java.util.Calendar;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.handler.HandlerInterceptorAdapter;
public class TimeCheckInterceptor extends HandlerInterceptorAdapter {
    @Override
    public boolean preHandle(HttpServletRequest request,
        HttpServletResponse response, Object handler) throws Exception {
        Calendar cl=Calendar.getInstance(); //sys date and time
        int hour=cl.get(Calendar.HOUR_OF_DAY); //current hour in 24hr format
        if(hour<9 || hour>=17){
            response.sendRedirect("timeout.jsp");
            return false;
        }
        return true;
    }
    //preHandle
}
//class
```

ViewHomeController.java:

```
package com.nt.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
```

```
public class ViewHomeController extends AbstractController {  
    @Override  
    protected ModelAndView handleRequestInternal(HttpServletRequest req,  
        HttpServletResponse res) throws Exception {  
  
        return new ModelAndView("home");  
    }  
}
```

WishController.java

```
package com.nt.controller;  
import java.util.Calendar;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import org.springframework.web.servlet.ModelAndView;  
import org.springframework.web.servlet.mvc.AbstractController;  
public class WishController extends AbstractController {  
    @Override  
    protected ModelAndView handleRequestInternal(HttpServletRequest req,  
        HttpServletResponse res) throws Exception {  
        // get System date and time  
        Calendar cl=Calendar.getInstance();  
        //get current hour of the day  
        int hour=cl.get(Calendar.HOUR_OF_DAY);  
        String msg=null;  
        if(hour<12)  
            msg="Good morning";  
        else if(hour<16)  
            msg="Good afternoon";  
        else if(hour<20)  
            msg="Good evening";  
        else  
            msg="Good Night";  
        return new ModelAndView("result","wmsg",msg);  
    }  
}
```

Index.jsp:

```
<jsp:forward page="home.htm"/>
```

Tmeout.jsp

```
<span style="color:red">Request must be given in between 9am to 5 pm</span>
```

NIT, the pioneer in IT Training, got started 13 years back in the year 2003.

NIT,
opp Satyam threater
Ammerpet

```
<b><center> Home page</center></b>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<a
href="AboutUs.htm">AboutUs</a>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<a
href="ContactUs.htm">ContactUs</a>
<br>
<a href="wish.htm">Wish Message</a>
```

```
<b><center> Result Page</center></b><br>
Wish Message : ${wmsg}.
<br>
<a href="home.htm">home</a>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="home.htm">vhc</prop>
<prop key="AboutUs.htm">ufv</prop>
<prop key="ContactUs.htm">ufv</prop>
</props>
</property>
</bean>
<bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
<props>
<prop key="wish.htm">wc</prop>
</props>
</property>
<property name="interceptors">
<list>
```

```

    <ref bean="timeCheckInterceptor"/>
  </list>
</property>
</bean>
<bean id="timeCheckInterceptor" class="com.nt.interceptor.TimeCheckInterceptor"/>
<bean id="vhc" class="com.nt.controller.ViewHomeController"/>
<bean id="wc" class="com.nt.controller.WishController"/>
<bean id="ufv" class="org.springframework.web.servlet.mvc.UrlFilenameViewController"/>
<!-- <bean id="pvc1" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="AboutUs"/>
</bean>
<bean id="pvc2" class="
org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="ContactUs"/>
</bean> -->
<bean id="vr" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>

```

web.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
</web-app>

```

ViewResolver:

- ⇒ This component takes logical view name given by controller through DispatcherServlet and returns "ViewObject" (The implementation class object of View (i)) pointing to physical view. DispatcherServlet calls render (-,-) method on this View object to render the control to view resource.
- ⇒ Like html file, jspfile, bean class . all ViewResolvers implements "ViewResolver" interface. Spring has supplied few predefined viewResolver classes. They are

WelcomeController:

```
package com.nt.controller;
import java.util.Date;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.Controller;

public class WelcomeController implements Controller {

    @Override
    public ModelAndView handleRequest(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
        return new ModelAndView("home", "sysDate", new Date());
    }
}
```

home.jsp:

```
<%@page isELIgnored="false"%>
<h1 style='color:red;text-align:center'>Spring MVC from home.jsp</h1>
System Date ::: ${sysDate}
```

namaste.jsp:

```
<%@page isELIgnored="false" %>
<h1 style='color:red;text-align:center'>Spring MVC from namaste.jsp</h1>
System Date ::: ${sysDate}
```

dispatcher-servlet.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Handler mapping -->
```

```
<bean id="suhm" class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
  <property name="mappings">
    <props>
      <prop key="home.htm">wc</prop>
    </props>
  </property>
</bean>
<!-- Controller class -->
<bean id="wc" class="com.nt.controller.WelcomeController"/>
<!-- ViewResolver -->

<!-- <bean id="ivr" class="org.springframework.web.servlet.view.UrlBasedViewResolver">
  <property name="viewClass" value="org.springframework.web.servlet.view.JstlView"/>
  <property name="viewClass"
value="org.springframework.web.servlet.view.InternalResourceView"/>
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp"/>
</bean>
-->
<bean class="org.springframework.web.servlet.view.ResourceBundleViewResolver">
  <property name="basename" value="com/nt/commons/views"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp"/>
</bean>
<!-- <bean class="org.springframework.web.servlet.view.XmlViewResolver"/> -->
</beans>
```

views.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="home" class="org.springframework.web.servlet.view.InternalResourceView">
    <property name="url" value="/WEB-INF/pages/salam.jsp"/>
  </bean></beans>
```

Web.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

  xmlns="http://xmlns.jcp.org/xml/ns/javaee"

  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"

  id="WebApp_ID" version="3.1">

  <servlet>

    <servlet-name>dispatcher</servlet-name>

    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>

    <!-- <init-param>

      <param-name>namespace</param-name>

      <param-value>myconfig</param-value>

    </init-param> -->

    <load-on-startup>2</load-on-startup>

  </servlet>

  <servlet-mapping>

    <servlet-name>dispatcher</servlet-name>

    <url-pattern>*.htm</url-pattern>

  </servlet-mapping>

</web-app>
```

- ⇒ Does not take default view class but allows to configure any viewclass to render viewObject
- ⇒ having any technology view resource.
- ⇒ The configurable view classes are Jstlview, InternalResourceView, TilesView and etc...
- ⇒ Use InternalResourceView when Jsp is our view with out using Jstl tags and use Jstlview

when Jsp is our view using Jstl tags (also we need to add Jstl.jar, standard.jar files).

ResourceBundleViewResolver:

- ⇒ The problem with above 2 view Resolver is logical view name and jsp file name must match. If jsp filename is changed later then modifying logical viewname by opening the source code of controller class is very difficult.
- ⇒ In order to overcome this problem we can use ResourceBundleViewResolver where we can configure views in the properties file.

views.properties:

- ⇒ Default name is also views.properties

(src folder)

welcome.(class)=org.springframework.web.servlet.view.JstlView

(logical view name.(class))

welcome.url=/WEB-INF/pages/namaskar.jsp

(physical view name)

In dispatcher-servlet:

- ⇒

```
<bean id="vr" class="org.springframework.web.servlet.view.ResourceBundleViewResolver">
  <property name="basename" value="views"/>
</bean>
```
- ⇒ We can use ResourceBundleViewResolver to enable internationalization on our spring MVC applications.
- ⇒ We can take multiple jsps for multiple locales and we can configure them in multiple locale specific properties files.

XmlViewResolver:

- ⇒ Similar to ResourceBundleViewResolver, but allows to take xml file to specify view configurations. Takes "view.xml" as the default name. we need to specify this file name as the value of "location" property of XmlViewResolver in configuration file.

⇒ eg:

views.xml (WEB-INF folder)

```
<bean id="welcome" class="org.springframework.web.servlet.view.JstlView">
  <property name="url" value="/WEB-INF/pages/salam.jsp"/>
</bean>
```

In dispatcher-servlet.xml

```
<bean class="org.springframework.web.servlet.view.XmlViewResolver"/>
  <property name="location" value="/WEB-INF/pages/views.jsp"/>
</bean>
```

BeanNameViewResolver:

- ⇒ Resolves java class/spring bean as the viewResource, for this the logical view name gathered from controller class must be taken as bean id or name for the class/bean that is acting as view.

dispatcher-servlet.xml

- ⇒

```
<!--Java class/Spring class acting as view-->
```

```
<bean name="welcome" class="com.nt.view.MyReportExcel"/>
```

```
<bean id="vr" class="org.springframework.web.servlet.view.BeanNameviewResolver"/>
```

⇒ **When we configure multiple viewResolvers they will execute in a chain.**

InternalResourceViewResolver always gets high priority when compare to other *ViewResolvers* in chain so it is always recommended to configure *InternalResourceViewResolver* as the last *ViewResolver* in the chain.

Working with Views:-

⇒ Spring allows to take different technologies in the view layer like jsp,html,servlet,velocity, freemarker and etc... In spring we can take java classes/spring beans as view by extending them from *Abstractxxxview* classes (like *AbstractPdfView*, *AbstractExcelView*.....) To render response as pdf,excel and etc docs.

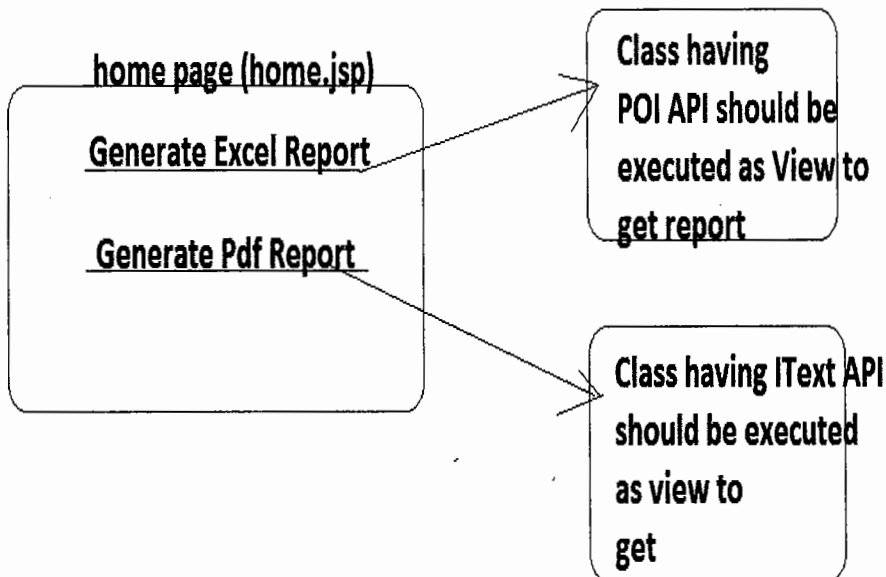
In spring view is an abstract entity so we can take anything as view.

ViewResolver gives "View obj" (*View(i)* implementation class object) and *DispatcherServlet* calls *render(-)* on the object. This *render(-)* method passes the control to physical view Files/calss (like the jsp or above discussed classes).

To develop view we should take class implementing *View(i)*. but some predefined Abstract and Concrete view classes are supplied to simplify the process. They are *JstlView*, *AbstractPdfView*, *AbstractExcelView* etc...

Example app on *BeanNameViewResolver* and java classes as views:

BeanNameViewResolver:-



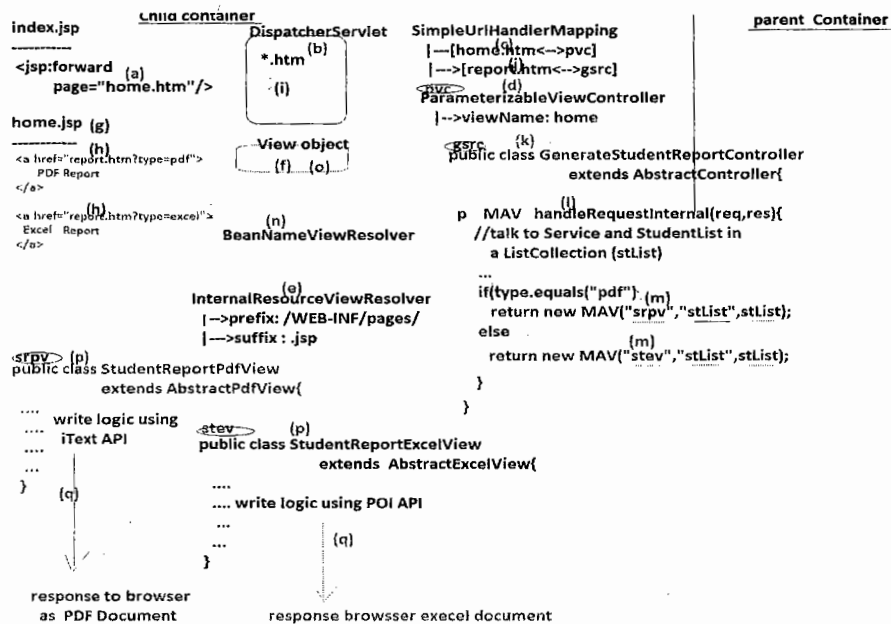
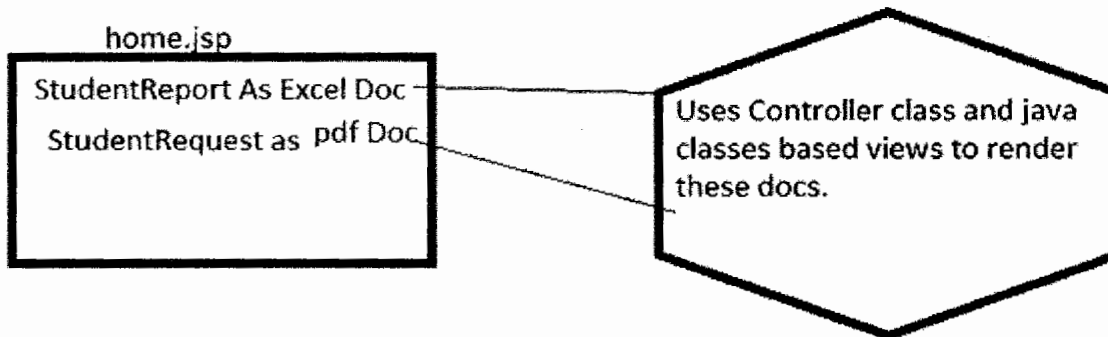


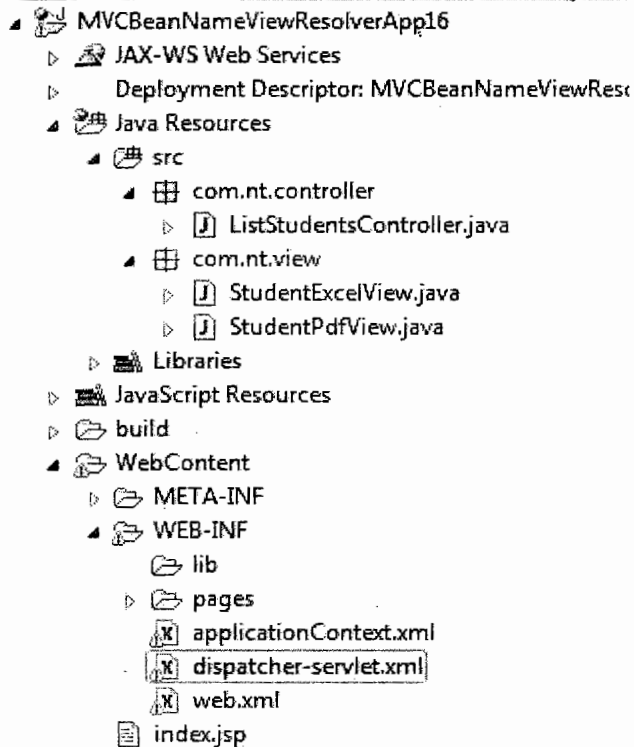
Diagram:

**NOTE:-**

We can use poi api Support to render response as **Excel doc**. We can use itext api support to render response as **PDF doc**
 => Poi is a 3rd party api given by apache to render the response as Excel sheet...

Jars:-

MVC library
 Standard.jar
 Jstl.jar
 iText-2.0.7.jar(-->Spring 2.5)---> higher version for spring 4 }
 Poi-3.0.1.jar }Gather them from spring 2.5



ListStudentsController.java

```

package com.nt.controller;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class ListStudentsController extends AbstractController {
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest req,
        HttpServletResponse res) throws Exception {
        List<String> list=new ArrayList<String>();
        list.add("raja"); list.add("ravi"); list.add("jani");
        if(req.getParameter("report").equals("pdf"))
            return new ModelAndView("pdfView","stList",list);
        else
            return new ModelAndView("excelView","stList",list);
    }
}

```

StudentExcelView.java

```

package com.nt.view;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

```

```
import org.apache.poi.hsspringframework.usermodel.HSSPRINGFRAMEWORKRichTextString;
import org.apache.poi.hsspringframework.usermodel.HSSPRINGFRAMEWORKSheet;
import org.apache.poi.hsspringframework.usermodel.HSSPRINGFRAMEWORKWorkbook;
import org.springframework.web.servlet.view.document.AbstractExcelView;
public class StudentExcelView extends AbstractExcelView{
    @Override
    protected void buildExcelDocument(Map<String, Object> map,
        HSSFWorkbook book, HttpServletRequest req, HttpServletResponse res)
        throws Exception {
        List list =(ArrayList)map.get("stList");
        HSSFSheet sheet=book.createSheet("Sheet1");
        getCell(sheet,0,0).setCellValue(new HSSFRichTextString("Students Names"));
        getCell(sheet,1,0).setCellValue(new HSSFRichTextString(list.get(0)+" ");
        getCell(sheet,2,0).setCellValue(new HSSFRichTextString(list.get(1)+" ");
        getCell(sheet,3,0).setCellValue(new HSSFRichTextString(list.get(2)+" ");
    }
}
```

StudentPdfView.java

```
package com.nt.view;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.view.document.AbstractPdfView;
import com.lowagie.text.Document;
import com.lowagie.text.Paragraph;
import com.lowagie.text.Table;
import com.lowagie.text.pdf.PdfWriter;
public class StudentPdfView extends AbstractPdfView {
    @Override
    protected void buildPdfDocument(Map<String, Object> map, Document doc,
        PdfWriter writer, HttpServletRequest req, HttpServletResponse res)
        throws Exception {
        //read Model obj attribute value
        List list =(ArrayList)map.get("stList");
        Paragraph p = new Paragraph("Student details");
        p.setAlignment("center");
        doc.add(p);
        Table t = new Table(1);
        t.addCell(list.get(0)+" ");
        t.addCell(list.get(1)+" ");
        t.addCell(list.get(2)+" ");
        doc.add(t);
    }
}
```

index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<web-app>
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener
</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="surl"
  class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
<property name="mappings">
  <props>
    <prop key="home.htm">pvc1</prop>
    <prop key="listStudents.htm">lsc</prop>
  </props>
</property>
</bean>
<bean id="lsc" class="com.nt.controller.ListStudentsController"/>
<bean id="pvc1" class="
  org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="home"/>
</bean>
<bean id="excelView" class="com.nt.view.StudentExcelView"/>
<bean id="pdfView" class="com.nt.view.StudentPdfView"/>
<bean id="bnvr" class="org.springframework.web.servlet.view.BeanNameViewResolver"/>
```

```

<bean id="vr" class="org.springframework.web.servlet.view.InternalResource
                                ViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
</bean>
</beans>

```

applicationContext.xml

```

<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:util="http://www.springframework.org/schema/util"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>

```

home.jsp

```

<b><center> Home page</center></b>
<br>
<a href="listStudents.htm?report=pdf">Student Report as Pdf</a>
<br>
<br>
<a href="listStudents.htm?report=excel">Student Report as Excel</a>

```

i18n (Internationalization)

- ⇒ Making our application working for different locales by rendering labels in different languages is called enabling i18n on the application. This improves no. of customer/visitors to the application. For this we need to take multiple properties files for multiple locales on 1 per local basis like welcome.properties, welcome_fr_FR.properties, welcome_zh_cn.properties and etc...
- ⇒ To enable i18n on spring mvc application we need to configure LocaleResolver, LocaleChangeInterceptor classes as beans and we need to pass interceptor reference to link interceptor with controller class.
- ⇒ SessionLocaleResolver is implementation class of LocaleResolver interface that allows to configure locale explicitly otherwise fallbacks to default locale configured or accept-language header value. This also enables i18n on the application

```

<bean id="localeResolver" class="org.springframework.web.servlet.i18n.SessionLocaleResolver">
    <property name="defaultLocale" value="en" />
</bean>

```

LocaleChangeInterceptor allows to change locale for every request with respect to controller based on the additional request param that is configured.

```

<bean id="lci" class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
    <property name="paramName" value="language" />
</bean>

```

- ⇒ Based on the value of **language request parameter** the locale will be changed for every request

by overriding default locale.

- ⇒ Based on this locale properties file will be picked up to render the label. While preparing multiple properties files in the application of internationalization, all properties files must have same keys and different labels specific to each locale.

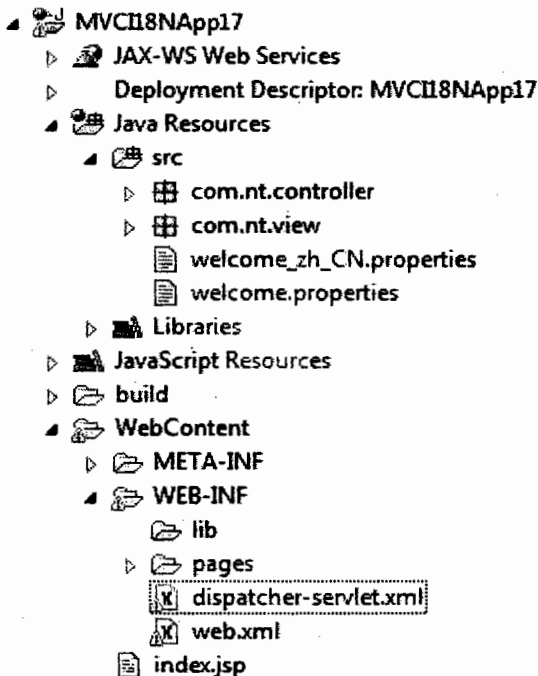
Steps to develop i18N based spring app:

- 1) Prepare multiple properties files for multiple locales including one base properties file.

Note: make sure that all properties files the keys are same.

- 2) Configure properties file in dispatcher-servlet.xml file using "ResourceBundleMessageSource" having fixed beanid (messageSource).
- 3) Configure SessionLocaleResolver specifying the default locale name.
- 4) Configure LocaleChangeInterceptor as spring bean and link the above interceptor with controller class that renders the view.
- 5) Add <spring:message> tag in jsp to get the labels from the appropriate properties file.

Ex:



welcome.properties

my.un= Enter UserName

my.pwd=Enter Password

welcome_zh_CN.properties

my.un=\u0917\u094a\u092a\u093f\u0905\u0928\u094d

my.pwd=PICHACHU

WelcomeController.java

```
package com.nat.controller;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;
public class WelcomeController extends AbstractController{
    @Override
    protected ModelAndView handleRequestInternal(HttpServletRequest request,
        HttpServletResponse response) throws Exception {

        ModelAndView model = new ModelAndView("Register");

        return model;
    }
}
```

redirect.jsp

```
<jsp:forward page="welcome.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
    <servlet>
        <servlet-name>dispatcher</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>dispatcher</servlet-name>
        <url-pattern>*.htm</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>redirect.jsp</welcome-file>
    </welcome-file-list>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
    <bean id="localeResolver"
        class="org.springframework.web.servlet.i18n.SessionLocaleResolver">
        <property name="defaultLocale" value="en" />
    </bean>
```

```

<bean id="localeChangeInterceptor"
class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
    <property name="paramName" value="language" />
</bean>
<!-- if url=/welcome,it will call WelcomeController -->
<bean class="org.springframework.web.servlet.mvc.support.Controller
                                           ClassNameHandlerMapping" >
    <property name="interceptors">
        <list>
            <ref bean="localeChangeInterceptor" />
        </list>
    </property>
</bean>
<!-- Register the bean -->
<bean class="com.nat.controller.WelcomeController" />
<!-- Register the welcome.properties -->
<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
    <property name="basename" value="welcome" />
</bean>
<bean id="viewResolver"
class="org.springframework.web.servlet.view.InternalResourceViewResolver" >
    <property name="prefix">
        <value>/WEB-INF/page</value>
    </property>
    <property name="suffix">
        <value>.jsp</value>
    </property>
</bean>
</beans>

```

register.jsp

```

<%@taglib prefix="spring" uri="http://www.springframework.org/tags" %>
<%@page isELIgnored="false" %>
Registration Page<br>
Language : <a href="?language=en">English</a>|
<a href="?language=zh_CN">Chinese</a>
<h3>
    <spring:message code="my.un" text="default text" />
    <input type="text" name="userName"><br>
    <spring:message code="my.pwd" text="default text" />
    <input type="text" name="password"><br>
</h3>
Selected Locale: ${pageContext.response.locale}
</body>
</html>

```

Annotation Driven Spring MVC:

- ⇒ In xml configuration based programmatic approach of spring MVC we develop controller class by implementing spring specific interfaces or by extending from spring specific classes. This kills non invasive behavior of spring.
- ⇒ It also makes the programmers to implement or override fixed methods with fixed signatures where programmer will be forced to place unnecessarily parameters like req, res params even though it is not using them same thing can be said about return type of methods.
- ⇒ To develop controller classes as pojo classes having flexibility of placing user-defined methods with user-defined signatures we need to go for Annotations driven spring mvc. These annotations are introduced from spring 2.5. All new projects and in the migration projects annotations driven spring mvc programming is required.

Two important annotations are:-

- 1) **@Controller**----->configure java class as spring mvc controller class and spring bean.
 - 2) **@RequestMapping**---->To map incoming request urls with the methods of controller class based on the virtual path of request url.
- ⇒ While working with annotations based spring mvc controllers, we can place flexible methods having flexible parameter types or flexible return types.
 - ⇒ Eg:

```
@Controller
public class MyController
{
@RequestMapping ("/home.htm")
public <any return type> <method name>()
{
.....
.....
}
}
```

Important possible param types:

- i) `HttpServletRequest`
- ii) `HttpServletResponse`
- iii) `HttpSession`
- iv) `Command class`
- v) `@ModelAttribute`
- vi) `BindingResult/Errors, Map, Model, ModelMap and etc.....`
(lots of param supporting web services)

Important possible return types:

- ⇒ `void, String, ModelAndView, Model, Map, View and etc.....`
(lots of other return types supporting webservices)

Note:

- ⇒ In spring mvc only controller classes can be configured with annotations and the remaining components like `HandlerMappings`, `ViewResolver` and etc must be configured through xmlfile.

AbstractController (Using annotation)

Source page: No form submission

Dest page: Static/dynamic form page with request processing

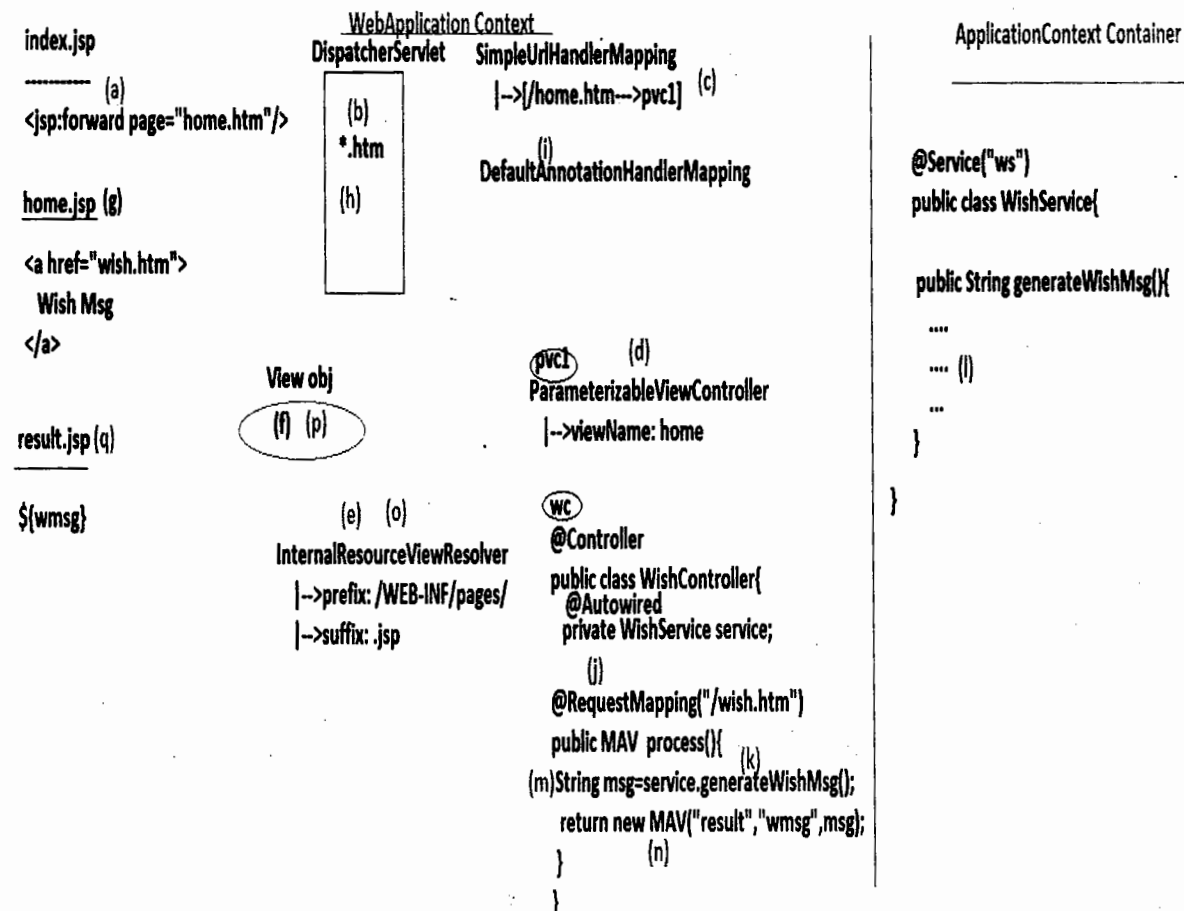
Note:

- ⇒ we still use built-in predefined controller like *ParameterizableViewController* using xml configuration"
- ⇒ Upto spring 2.5 we can work with only xml based spring MVC from spring 2.5 we can develop spring MVC app by mixing xml with annotations.(spring 2.5 to spring 3 all versions and spring 4.x initial versions).
- ⇒ From spring 3.x we can develop spring MVC application by avoiding all xml including web xml by using 100% java code approach(Every configuration through annotations).

Spring 3.x, 4.x supports 3 models of MVC application development

- 1) Xml configurations based programmatic approach.
- 2) Annotations approach (also uses xml files).
- 3) 100% java code approach (java config approach)

Diagram:



- MVC-AnnoWishApp18-AC
 - JAX-WS Web Services
 - Deployment Descriptor: MVC-AnnoWishApp18
 - Java Resources
 - src
 - com.nt.controller
 - Wishcontroller.java
 - com.nt.view
 - WishService.java
 - Libraries
 - JavaScript Resources
 - build
 - WebContent
 - META-INF
 - WEB-INF
 - lib
 - pages
 - applicationContext.xml
 - dispatcher-servlet.xml
 - web.xml
 - index.jsp

WishService.java

```
package com.nt.service;
import java.util.Calendar;
import javax.inject.Named;
@Named("ws")
public class WishService {
    public String generateWishMsg(){
        // generate wish Msg based on current hour the day
        Calendar cl=Calendar.getInstance();
        int hour=cl.get(Calendar.HOUR_OF_DAY);
        if(hour<=12)
            return "Good morning";
        else if(hour<=16)
            return "Good After noon";
        else if (hour<=20)
            return "Good Evening";
        else
            return "Good Night";
    }
}
//method
}
//class
```

WishController.java

```
package com.nt.controller;
import javax.annotation.Resource;
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.service.WishService;
@Controller
public class WishController {

    @Resource
    private WishService service;

    /* @RequestMapping("/wish.htm")
    public ModelAndView process(){
        //use service class
        String msg=service.generateWishMsg();

        return new ModelAndView("result","wmsg",msg);
    } */

    /* @RequestMapping("/wish.htm")
    public String process(Model model){

        //use service class
        String msg=service.generateWishMsg();

        // add data to mode
        model.addAttribute("wmsg",msg);

        return "result";
    } */

    /* @RequestMapping("/wish.htm")
    public String process(ModelMap map){

        //use service class
        String msg=service.generateWishMsg();

        map.addAttribute("wmsg", msg);
        return "result";
    } */

    /* @RequestMapping("/wish.htm")
    public String process(Map<String,Object> map){

        //use service class
        String msg=service.generateWishMsg();

        map.put("wmsg",msg);
        return "result";
    } */
```

```
@RequestMapping("/wish.htm")
public void process(Model model){
    //Use service class
    String msg=service.generateWishMsg();

    model.addAttribute("wmsg",msg);
}
} //class
```

index.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
    <servlet>
        <servlet-name>dispatcher</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>2</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>dispatcher</servlet-name>
        <url-pattern>*.htm</url-pattern>
    </servlet-mapping>
    <listener>
        <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
    </listener>
    <context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/applicationContext.xml</param-value>
    </context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
                <prop key="/home.htm">pvc1</prop>
            </props>
        </property>
    </bean>
</beans>
```

```
</props>
</property>
</bean>
<bean id="pvc1"
      class="org.springframework.web.servlet.mvc.ParameterizableViewController">
  <property name="viewName" value="home"/>
</bean>
<bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping"/>

<!-- <bean class="org.springframework.web.servlet.view.DefaultRequestTo
ViewNameTranslator"/> -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="prefix" value="/WEB-INF/pages/">
  <property name="suffix" value=".jsp"/>
</bean>
<context:component-scan base-package="com.nt.controller"/>
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
  <context:component-scan base-package="com.nt.service"/>
</beans>
```

home.jsp

```
<h1><center> Welcome to Annotations Based Spring MVC</center></h1>
<a href="wish.htm"> wish Msg</a>
```

result.jsp

```
<center><h1> Result Page</h1></center>
<br>
Wish Message : ${wmsg}
<br><a href="home.htm">home</a>
```

wish.jsp

```
<b> wish.jsp</b>
Result is : ${wmsg}
```


DefaultRequestToViewNameTranslator:

- ⇒ If controller class handler() (annotated with @RequestMapping) return type is void (i.e not returning logical view name) then DispatcherServlet takes request uri (virtual path) of current request and translates into view name by trimming slashes and extension by using "org.springframework.servlet.view.RequestToViewNameTranslator"

In controller class:

```
@RequestMapping ("/wish.htm")
public void process (Model model)
{
.....
.....
}
```

- ⇒ if the request url is http://localhost:3030/WishApp/wish.htm then "wish" will be taken as logical view name and wish.jsp will be taken as view Resource if the resolver is InternalResourceViewResolver. For this DispatcherServlet internally uses. DefaultRequestToViewNameTranslator". This translator strips/removes leading and trailing slashes and also remove extension from Request uri (makes "/home.htm" as "home") to use the result as logical viewname.

Annotation based AbstractCommandController:**Source Page:**

- ⇒ Form page with form submission (command class required)
- ⇒ No serverside form validation
- ⇒ No form handling. (displays same page with old values when form errors will come)

Destination Page:

- ⇒ Static/dynamic page through Request processing.
- ⇒ Use @ModelAttribute based param in handler method of controller class to specify the command class and its object to perform request wrapping (writing form data into command class object)
- ⇒ @ModelAttribute is multipurpose annotation. It can be used to expose any data to webview, to make parameter of handler method as Command class to place the return value of handler method as ModelAttribute and etc...

Diagram:

Annotation based AbstractCommandController

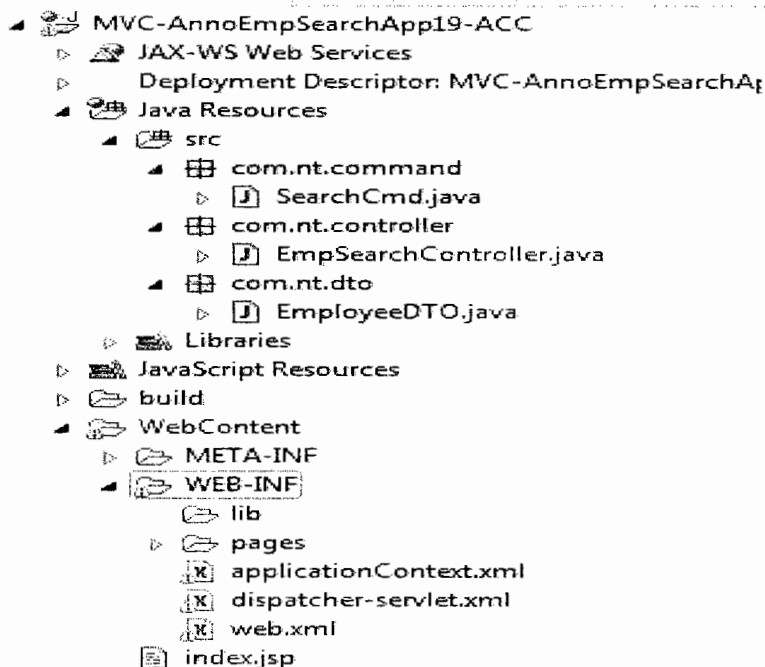
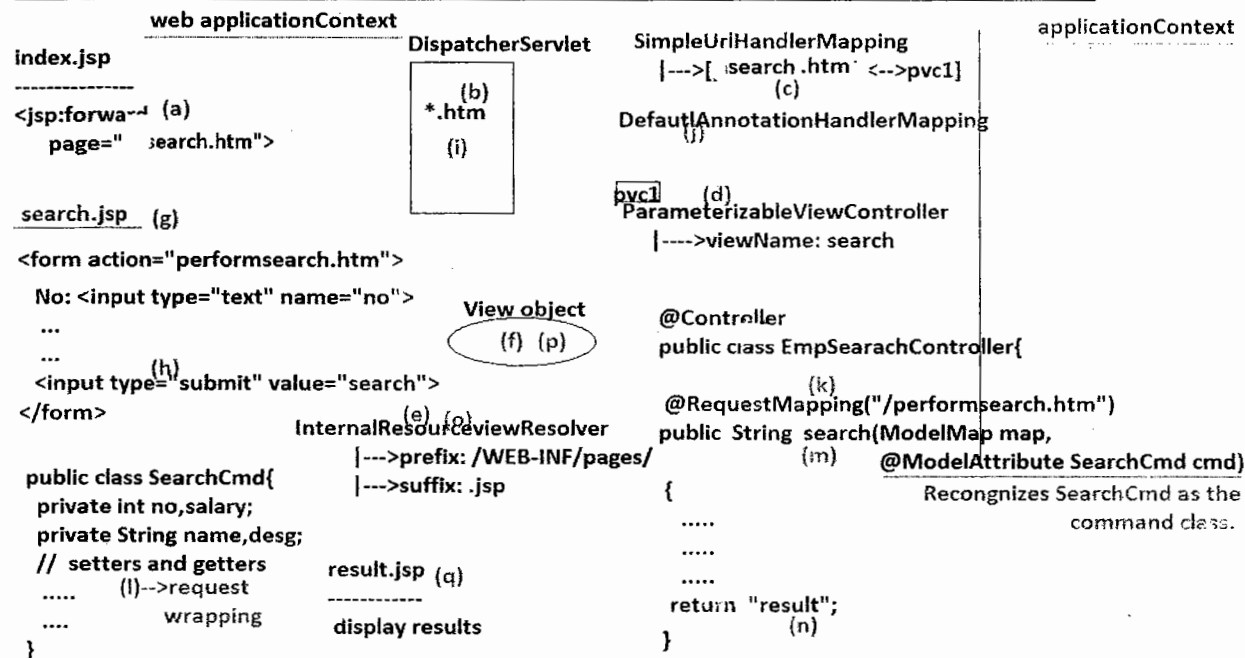
Source Page:

- > Form page with form submission (Command class is required)
- > No Server Side form validations
- > No Form handling

Dest page:

- > Static/Dynamic page through request processing

Use @ModelAttribute based param in handler method of controller class to specify the Command class and its obj to perform request wrapping (Writing form data into Command class obj)



SearchCmd.java

```
package com.nt.command;
public class SearchCmd {
    private int no,salary;
    private String name,desg;
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDesg() {
        return desg;
    }
    public void setDesg(String desg) {
        this.desg = desg;
    }
}
```

EmployeeDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int no,salary;
    private String name,desg;
    public EmployeeDTO() {
        // TODO Auto-generated constructor stub
    }
    public EmployeeDTO(int no, int salary, String name, String desg) {
        this.no = no;
        this.salary = salary;
        this.name = name;
        this.desg = desg;
    }
    public int getNo() {
        return no;
    }
}
```

```
}  
public void setNo(int no) {  
    this.no = no;  
}  
public int getSalary() {  
    return salary;  
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
public String getName() {  
    return name;  
}  
public void setName(String name) {  
    this.name = name;  
}  
public String getDesg() {  
    return desg;  
}  
public void setDesg(String desg) {  
    this.desg = desg;  
}  
}
```

EmpSearchController.java

```
package com.nt.controller;  
import java.util.ArrayList;  
import java.util.List;  
import org.springframework.stereotype.Controller;  
import org.springframework.ui.ModelMap;  
import org.springframework.web.bind.annotation.ModelAttribute;  
import org.springframework.web.bind.annotation.RequestMapping;  
import com.nt.command.SearchCmd;  
import com.nt.dto.EmployeeDTO;  
@Controller  
public class EmpSearchController {  
    @RequestMapping("/performsearch.htm")  
    public String search(ModelMap map,@ModelAttribute SearchCmd cmd){  
        System.out.println(cmd.getNo()+" "+cmd.getName());  
        List<EmployeeDTO> emplist=new ArrayList<EmployeeDTO>();  
        EmployeeDTO dto1=new EmployeeDTO(101,5000,"raja","clerk");  
        EmployeeDTO dto2=new EmployeeDTO(102,6000,"ravi","clerk");  
        emplist.add(dto1); emplist.add(dto2);  
        // Add result to ModelMap  
        map.addAttribute("searchResultsList",emplist);  
        return "emplist";  
    }  
}
```

index.jsp

```
<jsp:forward page="search.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
  <bean class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
    <property name="mappings">
      <props>
        <prop key="/search.htm">pvc1</prop>
      </props>
    </property>
  </bean>
  <bean id="pvc1" class="org.springframework.web.servlet.mvc.Parameterizable
                                                                    ViewController">
    <property name="viewName" value="search"/>
  </bean>
  <bean class="org.springframework.web.servlet.mvc.annotation.
                                                                    DefaultAnnotationHandlerMapping"/>
```

```
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
</bean>
<context:component-scan base-package="com.nt.controller" />
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
</beans>
```

emplist.jsp

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<center><h1>Search Results</h1></center>
<c:if test="${not empty searchResultsList}">
<table>
<tr>
    <th>Emp No</th>
    <th>Emp Name</th>
    <th>Salary</th>
    <th>Desg</th>
</tr>
<c:forEach items="${searchResultsList}" var="result">
<tr>
    <td><c:out value="${result.no}" /></td>
    <td><c:out value="${result.name}" /></td>
    <td><c:out value="${result.salary}" /></td>
    <td><c:out value="${result.desg}" /></td>
</tr>
</c:forEach>
</table>
</c:if>
```

search.jsp

```
<h1><center> Emp Search Page</center></h1>
<br>
<form action="performsearch.htm" method="post">
    No : <input type="text" name="no"><br>
    Name : <input type="text" name="name"><br>
    Salary : <input type="text" name="salary"><br>
    Desg : <input type="text" name="desg"><br>
    <input type="submit" value="search">
</form>
```

Annotation Based SimpleFormController (or) AnnotationDrivenSimpleFormController:**Source Page:**

Form page-->form submission

Form validation (server side)

Form handling

Destination Page:

Static/Dynamic page that comes after request process

InitialPhase Request (reqmethod=GET)(with respect to xml)

Create command object

init Binder ()

add command obj to model attribute

referencedata() and etc...

PostBack request (req method=POST)(with respect to xml)

Create/locate command obj

initBindier ()

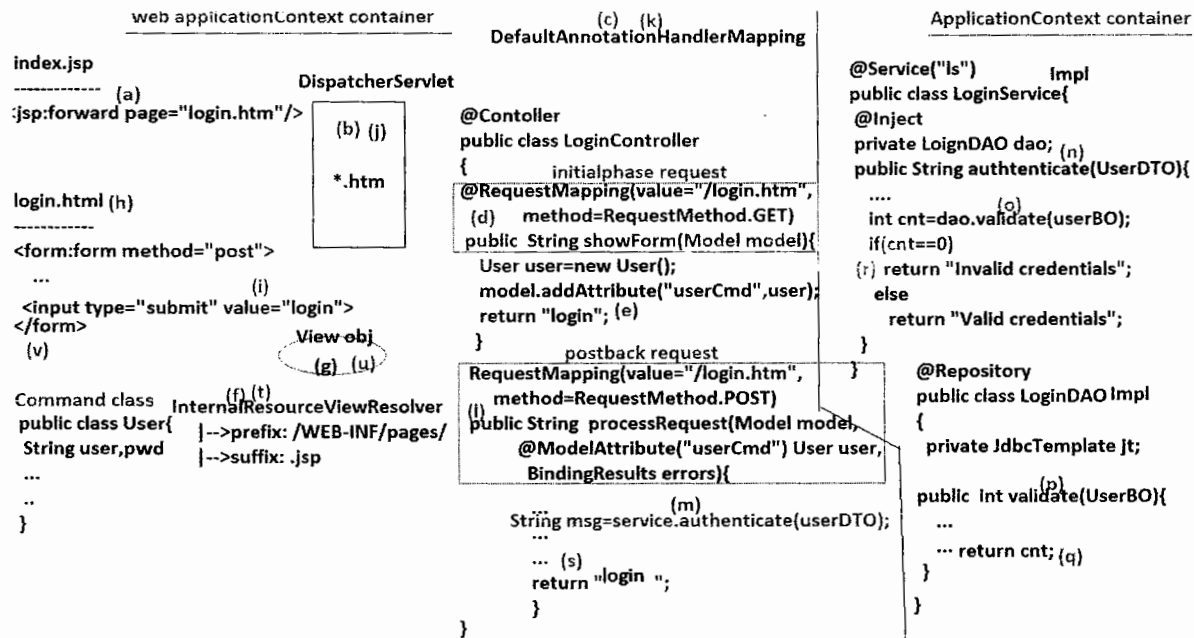
request wrapping

BindException obj creation

Validation support (-), validate (-)

Referencedata

onSubmit() and etc....

Diagram**Sample AnnotationBasedSimpleFormController Class**

```

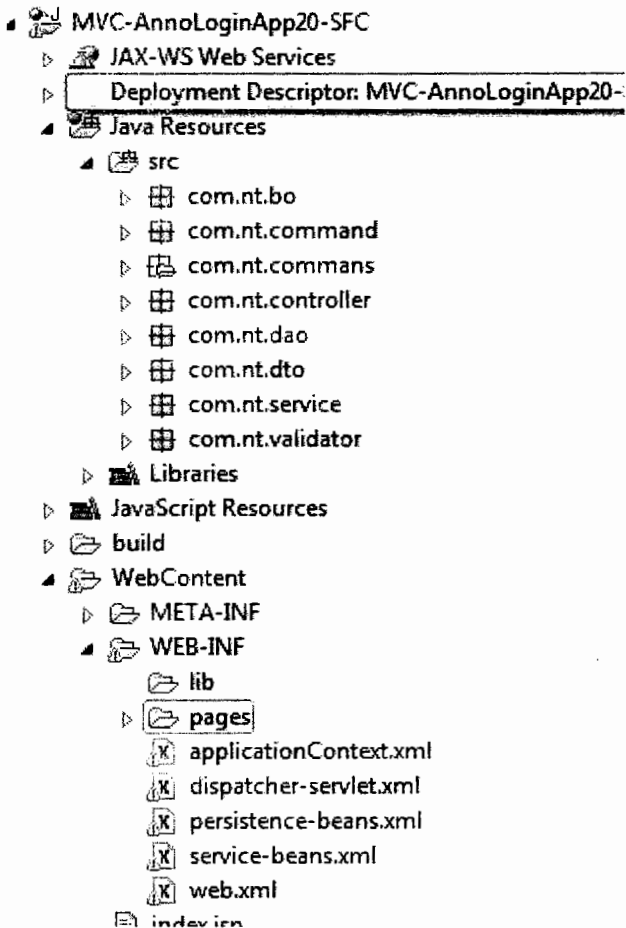
@Controller
public class LoginController
{
    //Initial phase request
    @RequestMapping (value="/login.htm", method=RequestMethod.GET)

```

```

public String showForm(Model model){
    User user=new User();
    model.addAttribute("usercmd", user);
    return "login";
}
@RequestMapping (value="/login.htm", method=RequestMethod.POST)
public String processRequest(Model model,@ModelAttribute ("usercmd") User user,
    BindingResult errors){
    String msg=service.authenticateconfigure:userDTO);
    .....
    .....
    return "login";
}
}

```



UserBO.java

```

package com.nt.bo;
public class UserBO {
    private String user,pwd;

    public String getUser() {

```



```
        return user;
    }
    public void setUser(String user) {
        this.user = user;
    }
    public String getPwd() {
        return pwd;
    }
    public void setPwd(String pwd) {
        this.pwd = pwd;
    }
}
```

LoginDao.java

```
package com.nt.dao;
import com.nt.bo.UserBO;
public interface LoginDao {
    public int validate(UserBO ubo);
}
```

LoginDaoImpl.java

```
package com.nt.dao;
import javax.inject.Inject;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import com.nt.bo.UserBO;
@Repository
public class LoginDaoImpl implements LoginDao {
    private static final String AUTH_QRY="SELECT COUNT(*) FROM USERLIST WHERE
UNAME=? AND PWD=? ";
    @Inject
    private JdbcTemplate jt;
    @Override
    public int validate(UserBO ubo) {
        int cnt=jt.queryForInt(AUTH_QRY, ubo.getUser(),ubo.getPwd());
        return cnt;
    }
}
```

UserDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class UserDTO implements Serializable {
    private String user,pwd;
    public String getUser() {
        return user;
    }
    public void setUser(String user) {
```

```
        this.user = user;
    }
    public String getPwd() {
        return pwd;
    }
    public void setPwd(String pwd) {
        this.pwd = pwd;
    }
}
```

LoginService.java

```
package com.nt.service;
import com.nt.dto.UserDTO;
public interface LoginService {
    public String authenticate(UserDTO udto);
}
```

LoginServiceImpl.java

```
package com.nt.service;
import javax.inject.Inject;
import org.springframework.stereotype.Service;
import com.nt.bo.UserBO;
import com.nt.dao.LoginDao;
import com.nt.dto.UserDTO;
@Service
public class LoginServiceImpl implements LoginService {
    @Inject
    private LoginDao dao;
    @Override
    public String authenticate(UserDTO udto) {
        //convert DTO obj to BO obj
        UserBO ubo=new UserBO();
        ubo.setUser(udto.getUser());
        ubo.setPwd(udto.getPwd());
        //Use DAO class
        int cnt=dao.validate(ubo);
        if(cnt==0)
            return "Invalid Credentials";
        else
            return "Valid Credentials";
    }
}
```

LoginValidator.java

```
package com.nt.validator;
import org.springframework.stereotype.Component;
import org.springframework.validation.Errors;
import org.springframework.validation.Validator;
```

```
import com.nt.command.User;
@Component
public class LoginValidator implements Validator {
    @Override
    public boolean supports(Class<?> clazz) {
        return clazz.isAssignableFrom(User.class);
    }
    @Override
    public void validate(Object cmd, Errors errors) {
        //type casting
        User ucmd=(User)cmd;
        if(ucmd.getUser()==null || ucmd.getUser().equals("") ||
        ucmd.getUser().length()==0){
            errors.rejectValue("user","user.required");
        }

        if(ucmd.getPwd()==null || ucmd.getPwd().equals("") ||
        ucmd.getPwd().length()==0){
            errors.rejectValue("pwd","pwd.required");
        }
    }
}
```

Errors.properties

user.required=Username is mandatory
pwd.required=Password is mandatory

User.java

```
package com.nt.command;
import java.util.Date;
public class User {
    private String user,pwd,dmn;
    private Date dor;
    public String getUser() {
        return user;
    }
    public void setUser(String user) {
        this.user = user;
    }
    public String getPwd() {
        return pwd;
    }
    public void setPwd(String pwd) {
        this.pwd = pwd;
    }
    public String getDmn() {
        return dmn;
    }
}
```

```
public void setDmn(String dmn) {
    this.dmn = dmn;
}
public Date getDor() {
    return dor;
}
public void setDor(Date dor) {
    this.dor = dor;
}
}
```

LoginController.java

```
package com.nt.controller;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import javax.inject.Inject;
import org.springframework.beans.propertyeditors.CustomDateEditor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.WebDataBinder;
import org.springframework.web.bind.annotation.InitBinder;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import com.nt.command.User;
import com.nt.dto.UserDTO;
import com.nt.service.LoginService;
import com.nt.validator.LoginValidator;
@Controller
public class LoginController {
    @Inject
    private LoginService service;
    @Inject
    private LoginValidator validator;
    @RequestMapping(value="/login.htm",method=RequestMethod.GET)
    public String showForm(Model model){
        //create command obj and keep it as model attribute
        User user=new User();
        model.addAttribute("userCmd",user);
        //return lvn
        return "login";
    }
    @RequestMapping(value="/login.htm",method=RequestMethod.POST)
    public String processForm(Model model,@ModelAttribute("userCmd")User
    cmd,BindingResult errors){
```

```
//convert Command obj to DTO obj
UserDTO udto=new UserDTO();
udto.setUser(cmd.getUser());
udto.setPwd(cmd.getPwd());

//use validator
if(validator.supports(User.class)){
    validator.validate(cmd, errors);
    if(errors.hasErrors()){
        return "login";
    }
}

//use service class
String result=service.authenticate(udto);
model.addAttribute("result",result);
//return lvn
return "login";
}

@ModelAttribute("domains") // alternate for referenceData
public List<String> populateDomains(){
    List <String>domainsList =new ArrayList<String>();
    domainsList.add("Gmail.com");
    domainsList.add("yahoo.com");
    domainsList.add("rediff.com");
    return domainsList;
}

@InitBinder
public void myInitBinder(WebDataBinder binder){
    SimpleDateFormat sdf=new SimpleDateFormat("dd-MM-yyyy");
    binder.registerCustomEditor(Date.class,
                                new CustomDateEditor(sdf,true));
}
}
```

index.jsp

```
<jsp:forward page="login.htm"/>
```

web.xml

```
<web-app>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
```

```
<servlet-name>dispatcher</servlet-name>
<url-pattern>*.htm</url-pattern>
</servlet-mapping>
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
</web-app>
```

service-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.nt.service"/>
</beans>
```

persistence-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="scott"/>
  <property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>
<context:component-scan base-package="com.nt.dao"/>
</beans>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```

http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean class="org.springframework.web.servlet.mvc.annotation.Default
AnnotationHandlerMapping"/>

<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
    <property name="basename" value="com/nt/commons/errors"/>
</bean>
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/pages/" />
    <property name="suffix" value=".jsp" />
</bean>
    <context:component-scan base-package="com.nt.controller,com.nt.validator"/>
</beans>

```

applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<import resource="service-beans.xml"/>
<import resource="persistence-beans.xml"/>
</beans>

```

login.jsp

```

<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<h1><center> Login Page</center></h1>
<form:form method="post" commandName="userCmd">
<span style='color:red'><form:errors path="*" /></span><br>
    Username: <form:input path="user" /><br>
    Password : <form:password path="pwd" /><br>
    Domains: <form:checkboxes items="${domains}" path="dmn" /><br>
    Date of Registration: <form:input path="dor" />(dd-MM-yyyy) <br>
<input type="submit" value="Login" /><br>
</form:form>
<br>
<c:if test="${result ne null}">
    <c:out value="${result}" />
</c:if>
<c:out value="${userCmd.dor}" />

```

-> Difference between Xml based SimpleFormController and Annotation based SimpleFormController?

Xml based SimpleFormController

```
1.onSubmit(-,-,-)

2.formBackingObject(-,-)

3.Map referenceData(-,-)

4.void initBinder(servlet Request
Binder){
-----
}

5.handleInvalidSubmit(-,-)
```

Annotation based SimpleFormController

```
@RequestMapping(...,method=POST)
public String ProcessForm(Map map,@ModelAttribute)

public String showForm(Map map){
-----
-----
}

@ModelAttribute(...)
public List<...> populateDomains(){---}

@InitBinder
public void myInitBinder(WebDataBinder,binder){----}
```

No direct replace method place extra logic in handlerInterceptor

⇒ To perform form validation while working with SimpleFormController with annotations:-
(programatic Approach)

- 1) Prepare properties file having form validation error messages.
- 2) Configure properties file.
- 3) Develop validator class implementing Validator (i) and overriding supports(-), validate(-) methods.
- 4) Inject validator class in controller class and call supports(-), validate(-) methods in the handler method, that can process postback request.
- 5) Place <form:errors> tag in form page to display form validation errors.

While working with annotation based SimpleFormController we can use @ModelAttribute annotation method as alternate to referenceData() of xml based SimpleFormController. This holds data that has to be displayed in the form page as additional data as part of initial phase request and postback request with validation error messages.

In controller class:

```
@ModelAttribute ("domains")
public List<String> populateDomains()
{
List<String> domainList=new ArrayList <String> ();
domainList.add("Gmail.com");
domainList.add("Yahoo.com");
domainList.add("rediff.com");
return domainList();
}
```


In form Page:

⇒ Domains :<form:checkboxes items="{domains}" path="dmn"/>

In order to register property editors that are required to convert form supplied String values to various types of command class properties we override `InitBinder()` in our controller class while working with xml configuration based `SimpleFormController`. We can bring the same affect by using

@InitBinder while using annotations driven SimpleFormController.

⇒ eg :

In Controller Class

@InitBinder

```
public void myInitBinder(WebDataBinder binder)
```

```
{
```

```
    SimpleDateFormat sdf=new SimpleDateFormat("dd-MM-yyyy");
```

```
    binder.registerCustomEditor(Date.class, new CustomDateEditor(sdf, true));
```

```
}
```

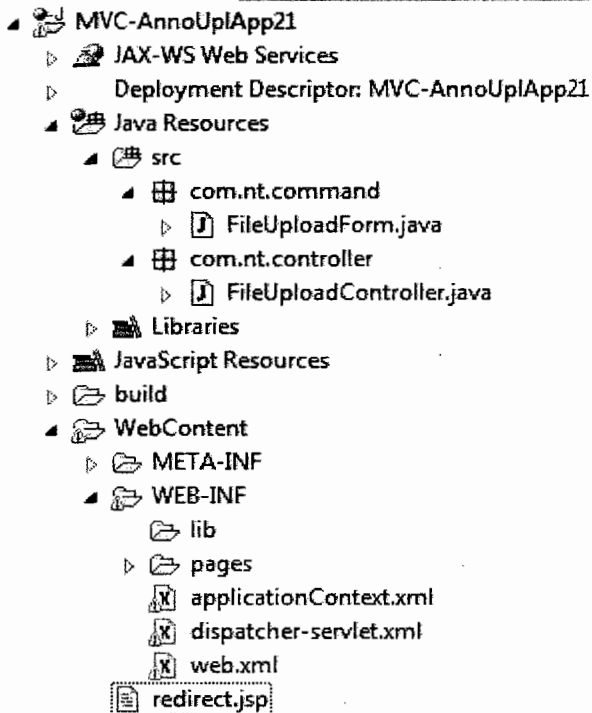
File Uploading and file downloading:

- ⇒ The process of selecting file from client machine file system and sending that file to server system file system is called file uploading and reverse is called File downloading.
- ⇒ While developing matrimony application, job portal, video sharing apps the file uploading and downloading are quietly required.
- ⇒ Spring MVC gives built-in support for fileuploading. We need to configure "CommonsMultipartResolver" in `dispatcher-servlet.xml` to make controller class ready to resolve multi MIME TYPES/parts based form data (form data having uploading files)
Using this class we can also configure properties like "maxUploadSize", "maxInMemorySize", "defaultEncoding" properties.

Perform the following operations for file uploading:

- 1) Take file uploading component in form page and also specify "enctype" and request method as "POST".
- 2) Takes command class having property of type "MultipartFile" to receive and hold uploaded file.
- 3) Configure "CommonsMultipartResolver" class as bean in `dispatcher-servlet.xml`
- 4) Write the stream based logic in controller to save the receive in server machine file system.

-> If "CommonsMultipartResolver" is not configured the controller that receives request from `formSubmission` does not recognize other than text data send by form, So request wrapping into command object having uploaded files do not take place, to overcome this problem we must configure "CommonsMultipartResolver" having fixed bean id="multipartResolver".



Jar: MVC jar, AOP jars,
commons-fileupload-1.1.1.jar,
commons-io-1.3.1.jar.

redirect.jsp

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
  <welcome-file-list>
    <welcome-file>redirect.jsp</welcome-file>
  </welcome-file-list>
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
</web-app>
```

dispatcher-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotation
HandlerMapping"/>
<bean id="multipartResolver"
class="org.springframework.web.multipart.commons.CommonsMultipartResolver"/>

<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/pages/" />
<property name="suffix" value=".jsp" />
</bean>
<context:component-scan base-package="com.nt.controller"/>
</beans>
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
</beans>
```

file_upload_form.jsp

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"%>
<h2>File Upload App</h2>
<form:form method="post" action="save.htm" modelAttribute="uploadForm"
enctype="multipart/form-data">
Please select a file to upload : <input type="file" name="file" /><br/>
<input type="submit" value="upload" />
</form:form>
</body>
</html>
```

file_upload_success.jsp

```
{files} uploaded successfully.
```

FileUploadController.java

```
package com.nt.controller;
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
```

```
import java.io.InputStream;
import java.io.OutputStream;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.multipart.MultipartFile;
import com.nt.command.FileUploadForm;
@Controller
public class FileUploadController {
    @RequestMapping(value = "/home.htm",
                    method = RequestMethod.GET)
    public String displayForm(Model model) {
        return "file_upload_form"; //returns the page name to ViewResolver ultimately
    }
    @RequestMapping(value = "/save.htm", method = RequestMethod.POST)
    public String upload(Model map, @ModelAttribute("uploadForm")
FileUploadForm cmd) {
        File dir=new File("E:/springfileuploads/");
        //get Uploaded file
        MultipartFile file = cmd.getFile();
        String fileName = "";
        if (file != null) {
            fileName = file.getOriginalFilename();
            System.out.println("orginal file name="+fileName);
            //Handle file content - multipartFile.getInputStream()
            InputStream inputStream=null;
            OutputStream outputStream;
            try {
                inputStream = file.getInputStream();
                //If file is not present then only create a file and write the data into      them
                if(!dir.exists()){
                    dir.mkdir();
                }
                File newFile = new File("E:/springfileUploads/"+fileName);
                outputStream = new FileOutputStream(newFile);
                int read = 0;
                byte[] buffer = new byte[1024];
                while ((read = inputStream.read(buffer)) != -1) {

                    outputStream.write(buffer, 0, read);
                }
                outputStream.close();
                inputStream.close();
            } catch (IOException e) {
                System.out.println("Exception "+e);
            }
        }
    }
}
```

```
    }  
    map.addAttribute("files", fileName);  
    return "file_upload_success";  
}  
}
```

FileUploadForm.java

```
package com.nt.command;  
import org.springframework.web.multipart.MultipartFile;  
public class FileUploadForm {  
    //Can be used to hold the uploaded file  
    private MultipartFile file;  
    public MultipartFile getFile() {  
        return file;  
    }  
    public void setFile(MultipartFile file) {  
        this.file = file;  
    }  
}
```

File Downloading:

- ⇒ The process of getting the file belonging to server machine file system to client machine file system is called file downloading. By using "Content-Disposition" header we can give instruction to browser to display the generated output as downloadable or attachable file.

For example application on file uploading.

We can use @RequestParam annotation to store the received request parameter value into handler method parameter.

To download file we need to do following activities in the controller class handler method:

- 1) Locate the file.
- 2) Get the length of the file and set it as response content length.
- 3) Get the MIME type of the file and set it as response content type.
- 4) Get input stream pointing to the file.
- 5) Get output stream pointing to the response object.
- 6) Use buffer based logic to read file content and to write to response object.
- 7) Specify content-disposition header having attachment of the value to make browser to display the received content as the downloadable file.

- MVC-AnnoFileDownloadApp22
 - ▷ JAX-WS Web Services
 - ▷ Deployment Descriptor: MVC-AnnoFileDownload
 - ▲ Java Resources
 - ▲ src
 - ▲ com.nt.controller
 - ▷ FileController.java
 - ▷ Libraries
 - ▷ JavaScript Resources
 - ▷ build
 - ▲ WebContent
 - ▷ META-INF
 - ▲ WEB-INF
 - ▷ lib
 - ▷ pages
 - applicationContext.xml
 - dispatcher-servlet.xml
 - web.xml
 - redirect.jsp

MVC library Jars:-**commons-fileupload-1.1.1.jar****commons-io-1.3.1.jar****index.jsp**

```
<jsp:forward page="home.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
  <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
</web-app>
```

dispatcher-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:mvc="http://www.springframework.org/schema/mvc"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc.xsd">

```

Since No handler mapping is configured the default AnnotationHandlerMapping will take place.

```

<context:component-scan base-package="com.nat.controller"/>
<context:annotation-config />

```

<!-- View Resolver, pointing to WEB-INF/jsp folder -->

```

<bean id="jspViewResolver"
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <property name="viewClass" value="org.springframework.web.servlet.view.JstlView" />
  <property name="prefix" value="/WEB-INF/pages/" />
  <property name="suffix" value=".jsp" />
</bean>
</beans>

```

file_download_form.jsp

```

<%@page import="java.io.File" %>
<table border=1>
<tr><td>FileName</td><td>Download</td></tr>
<% String path = "E:/springfileuploads";
String files;
File folder = new File(path);
File[] listOfFiles = folder.listFiles();

for (int i = 0; i < listOfFiles.length; i++)
{
  if (listOfFiles[i].isFile())
  {
    files = listOfFiles[i].getName();
  }
}
%>
<tr><td><%= files%></td>
<td><a href="download.htm?file_name=<%= files%>">Download</a></td></tr>
<% }
}
%>
</table>
</html>

```

FileDownloadController.java

```
package com.nat.controller;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import javax.servlet.http.HttpServletResponse;
import org.apache.commons.io.IOUtils;
import org.springframework.core.io.FileSystemResource;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
@Controller
public class FileDownloadController {
    @RequestMapping(value = "/home.htm")
    public String displayForm() {
        return "file_download_form"; //returns the page name to
                                     ViewResolver ultimately
    }
    @RequestMapping(value = "/download.htm")
    public void download(@RequestParam("file_name") String fileName,
                        HttpServletResponse response) {
        InputStream is=null;
        try {
            System.out.println("Requested File"+fileName+" response="+response);
            String filePath="E:/springfileuploads/"+fileName;
            // get your file as InputStream
            is=new FileInputStream(filePath);

            //Remove the below code,will dipatch the file in view mode like txt,bat file
            response.setHeader("Content-Disposition", "attachment; filename="+
            fileName);
            IOUtils.copy(is, response.getOutputStream());
            response.flushBuffer();

        } catch (IOException ex) {
            //log.info("Error writing file to output stream. Filename was " + fileName + "");
            throw new RuntimeException("IOError writing file to output stream");
        }
    }
}
```

SpringMVC with Tiles:

A tile is a logical portion / part in a webpage like header tile, footer tile, body tile and etc....
Apache tiles framework/plugin is given to construct webpages based on layout having layout control. This is like working with compositeView design pattern.

Layout control means all the web pages will be constructed based on single layout page for uniform look and any modification in layout page will be reflected to all webpages popular layouts are:-

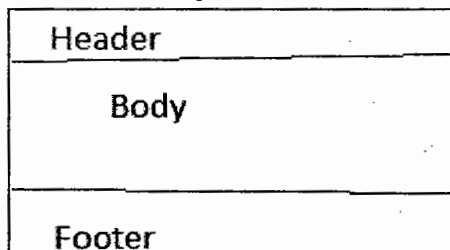
- a) Classic layout
- b) Two-Column layout
- c) Three-Column layout
- d) Circular layout

- ⇒ Spring gives built-in support to work with tiles. For this we need to configure "TileConfigurer" specifying the tile definition file name (xml). The default file name is /WEB-INF/tiles.xml.
- ⇒ Every tile definition specifies the tile values i.e. each webpage content based on layout page. We create tile definition based on 1 per webpage basis by having logical name. We can use "FilesViewResolver" to display webpage based on each tile definition.

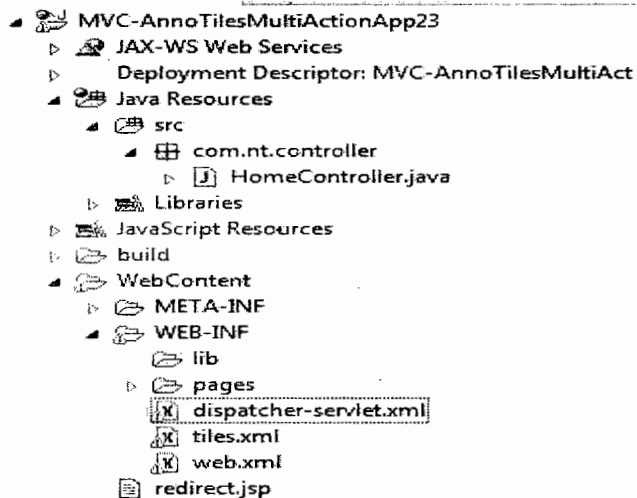
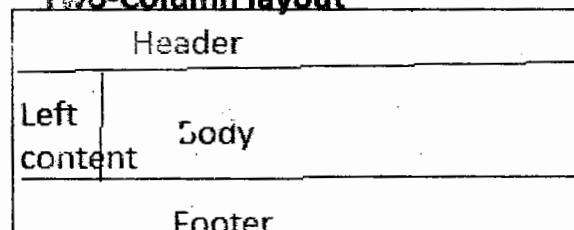
Procedure to develop spring MVC with tiles application:

- 1) Design layout page for templating webpages and to maintain webpages having layout control.
- 2) Develop files xml in WEB-INF folder having tile definitions based on no. of webpages in our website on tile definition for webpage basis. We can use tile definition inheritance concept here.
- 3) Configure "TilesConfigurer" class in dispatcher-servlet.xml file as spring bean specifying "tile definition configuration" filename.
- 4) Configure "TilesViewResolver" as ViewResolver.
- 5) Make a controller class handler methods to return "tile definition name" as logical view name.

Classic layout



Two-Column layout



Jar:- MVC jar, AOP jar, Tiles-template-2.2.2.jar, Tiles-servlet, tiles-core, tiles-jsp, tiles-api, sl4j-api, SL4j-log4j12, JSTL, antlr.jar

Commons-beanutils, commons-collections, commons-deigester, commons-fileupload, commons-10-jar.

HomeController.java

```
package com.nat.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class HomeController {
    @RequestMapping("/demo")
    public ModelAndView hello() {
        return new ModelAndView("page1", "message", "Body Content goes Here !");
    }

    @RequestMapping("/link1")
    public ModelAndView register() {
        return new ModelAndView("page2", "message", "Registration page");
    }

    @RequestMapping("/link2")
    public ModelAndView login() {
        return new ModelAndView("page3", "message", "Login Page");
    }

    @RequestMapping("/link3")
    public ModelAndView reset() {
        return new ModelAndView("page4", "message", "Login Page");
    }
}
```

redirect.jsp

```
<jsp:forward page="demo.htm"/>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
    <welcome-file-list>
        <welcome-file>redirect.jsp</welcome-file>
    </welcome-file-list>
    <servlet>
        <servlet-name>dispatcher</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
```

```

</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>*.htm</url-pattern>
</servlet-mapping>
</web-app>

```

tiles.xml

```

<!DOCTYPE tiles-definitions PUBLIC
  "-//Apache Software Foundation//DTD Tiles Configuration 2.0//EN"
  "http://tiles.apache.org/dtds/tiles-config_2_0.dtd">
<tiles-definitions>
  <definition name="baseDef" template="/WEB-INF/pages/Layout.jsp">
    <put-attribute name="title" value="" />
    <put-attribute name="header" value="/WEB-INF/pages/Header.jsp" />
    <put-attribute name="menu" value="/WEB-INF/pages/Menu.jsp" />
    <put-attribute name="body" value="" />
    <put-attribute name="footer" value="/WEB-INF/pages/Footer.jsp" />
  </definition>
  <definition name="page1" extends="baseDef">
    <put-attribute name="title" value="Home Page" />
    <put-attribute name="body" value="/WEB-INF/pages/Hello.jsp" />
  </definition>
  <definition name="page2" extends="baseDef">
    <put-attribute name="title" value="course page" />
    <put-attribute name="body" value="/WEB-INF/pages/courses.jsp" />
  </definition>
  <definition name="page3" extends="baseDef">
    <put-attribute name="title" value="faculty page" />
    <put-attribute name="body" value="/WEB-INF/pages/faculties.jsp" />
  </definition>
  <definition name="page4" extends="base.definition">
    <put-attribute name="title" value="address page" />
    <put-attribute name="body" value="/WEB-INF/pages/address.jsp" />
  </definition>
</tiles-definitions>

```

dispatcher-servlet.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:p="http://www.springframework.org/schema/p"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context-3.0.xsd">
  <!-- since No handler Mapping is configured DefaultAnnotationHandlerMapping will
  be taken.-->

```

```
<context:component-scan base-package="com.nat.controller" />
<bean id="viewResolver"
      class="org.springframework.web.servlet.view.UrlBasedViewResolver">
  <property name="viewClass"
    value="org.springframework.web.servlet.view.tiles2.TilesView"/>
</bean>
<bean id="tilesConfigurer"
      class="org.springframework.web.servlet.view.tiles2.TilesConfigurer">
  <property name="definitions">
    <list>
      <value>/WEB-INF/tiles.xml</value>
    </list>
  </property>
</bean>
<!-- <bean id="tilesviewResolver"
      class="org.springframework.web.servlet.view.tiles3.TilesViewResolver"/>
<bean id="tilesConfigurer"
      class="org.springframework.web.servlet.view.tiles3.TilesConfigurer">
  <property name="definitions">
    <list>
      <value>/WEB-INF/tiles.xml</value>
    </list>
  </property>
</bean>
-->
</beans>
```

Address.sjp

Naresh IT
Satyam Theater,
Ameerpet

Courses.jsp

Java
.net
Oracle

faculty.jsp

Hari Krishna
Satish Gupta
Vijay kumar

Footer.jsp

<center>
<p>Copyright © www.nataraj.in</p>
</center>

Header.jsp

```
<marquee><b>Welcome to Tiles Example</b></marquee>
```

Hello.jsp

```
<%@page isELIgnored="false" %>
<center>${message}</center>
```

layout.jsp

```
<%@ taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles"%>
<html>
<head>
<title><tiles:insertAttribute name="title" ignore="true" />
</title>
</head>
<body>
<table border="1" cellspacing="2" align="center" width="100%" height="100%">
  <tr bgcolor="cyan">
    <td height="50" colspan="2"><tiles:insertAttribute name="header" />
    </td>
  </tr>
  <tr>
    <td height="250" width="150"><tiles:insertAttribute name="menu" />
    </td>
    <td width="400"><tiles:insertAttribute name="body" />
    </td>
  </tr>
  <tr bgcolor="red">
    <td height="30" colspan="2"><tiles:insertAttribute name="footer" />
    </td>
  </tr>
</table>
</body>
</html>
```

menu.jsp

```
<a href="link1.htm">Courses</a><br>
<a href="link2.htm">faculties</a><br>
<a href="link3.htm">addresses</a>
```

Handling exceptions

HandlerExceptionResolver

Spring `HandlerExceptionResolver` implementations deal with unexpected exceptions that occur during controller execution. A `HandlerExceptionResolver` somewhat resembles the exception mappings you can define in the web application descriptor `web.xml`. However, they provide a more flexible way to do so. For example they provide information about which handler was executing when the exception was thrown. Furthermore, a programmatic way of handling exceptions gives you more options for responding appropriately before the request is forwarded to another URL (the same end result as when you use the Servlet specific exception mappings).

⇒ Besides implementing the `HandlerExceptionResolver` interface, which is only a matter of implementing the `resolveException(Exception, Handler)` method and returning a `ModelAndView`, you may also use the provided `SimpleMappingExceptionResolver` or create `@ExceptionHandler` methods. The `SimpleMappingExceptionResolver` enables you to take the class name of any exception that might be thrown and map it to a view name. This is functionally equivalent to the exception mapping feature from the Servlet API, but it is also possible to implement more finely grained mappings of exceptions from different handlers. The `@ExceptionHandler` annotation on the other hand can be used on methods that should be invoked to handle an exception. Such methods may be defined locally within an `@Controller` or may apply to many `@Controller` classes when defined within an `@ControllerAdvice` class. The following sections explain this in more detail.

@ExceptionHandler

⇒ The `HandlerExceptionResolver` interface and the `SimpleMappingExceptionResolver` implementations allow you to map Exceptions to specific views declaratively along with some optional Java logic before forwarding to those views. However, in some cases, especially when relying on `@ResponseBody` methods rather than on view resolution, it may be more convenient to directly set the status of the response and optionally write error content to the body of the response.

⇒ You can do that with `@ExceptionHandler` methods. When declared within a controller such methods apply to exceptions raised by `@RequestMapping` methods of that controller (or any of its sub-classes). You can also declare an `@ExceptionHandler` method within an `@ControllerAdvice` class in which case it handles exceptions from `@RequestMapping` methods from many controllers. Below is an example of a controller-local `@ExceptionHandler` method:

@Controller

```
public class SimpleController {
```

```
// @RequestMapping methods omitted ...
```

```
@ExceptionHandler(IOException.class)
```

```
public ResponseEntity<String> handleIOException(IOException ex) {
```

```
// prepare responseEntity  
return responseEntity;  
}}
```

- ⇒ The `@ExceptionHandler` value can be set to an array of Exception types. If an exception is thrown that matches one of the types in the list, then the method annotated with the matching `@ExceptionHandler` will be invoked. If the annotation value is not set then the exception types listed as method arguments are used.
- ⇒ Much like standard controller methods annotated with a `@RequestMapping` annotation, the method arguments and return values of `@ExceptionHandler` methods can be flexible. For example, the `HttpServletRequest` can be accessed in Servlet environments and the `PortletRequest` in Portlet environments. The return type can be a `String`, which is interpreted as a view name, a `ModelAndView` object, a `ResponseEntity`, or you can also add the `@ResponseBody` to have the method return value converted with message converters and written to the response stream.

Handling Standard Spring MVC Exceptions

- ⇒ Spring MVC may raise a number of exceptions while processing a request. The `SimpleMappingExceptionHandler` can easily map any exception to a default error view as needed. However, when working with clients that interpret responses in an automated way you will want to set specific status code on the response. Depending on the exception raised the status code may indicate a client error (4xx) or a server error (5xx). The `DefaultHandlerExceptionHandler` works transparently by setting the status of the response. However, it stops short of writing any error content to the body of the response while your application may need to add developer-friendly content to every error response for example when providing a REST API. You can prepare a `ModelAndView` and render error content through view resolution — i.e. by configuring a `ContentNegotiatingViewResolver`, `MappingJackson2JsonView`, and so on. However, you may prefer to use `@ExceptionHandler` methods instead.
- ⇒ If you prefer to write error content via `@ExceptionHandler` methods you can extend `ResponseEntityExceptionHandler` instead. This is a convenient base for `@ControllerAdvice` classes providing an `@ExceptionHandler` method to handle standard Spring MVC exceptions and return `ResponseEntity`. That allows you to customize the response and write error content with message converters. See the `ResponseEntityExceptionHandler` javadocs for more details.

Customizing the Default Servlet Container Error Page

- ⇒ When the status of the response is set to an error status code and the body of the response is empty, Servlet containers commonly render an HTML formatted error page. To customize the default error page of the container, you can declare an `<error-page>` element in `web.xml`. Up until Servlet 3, that element had to be mapped to a specific status code or exception type. Starting with Servlet 3 an error page does not need to be mapped, which effectively means the specified

location customizes the default Servlet container error page.

```
<error-page>

  <location>/error</location>

</error-page>
```

Note that the actual location for the error page can be a JSP page or some other URL within the container including one handled through an `@Controller` method:

When writing error information, the status code and the error message set on the `HttpServletResponse` can be accessed through request attributes in a controller:

```
@Controller
publicclass ErrorController {

  @RequestMapping(path = "/error", produces = MediaType.APPLICATION_JSON_UTF8_VALUE)
  @ResponseBody
  public Map<String, Object> handle(HttpServletRequest request) {

    Map<String, Object> map = new HashMap<String, Object>();
    map.put("status", request.getAttribute("javax.servlet.error.status_code"));
    map.put("reason", request.getAttribute("javax.servlet.error.message"));

    return map;
  }
}
```

or in a JSP:

```
<%@pagecontentType="application/json"pageEncoding="UTF-8"%>
{
  status:<%=request.getAttribute("javax.servlet.error.status_code")%>,
  reason:<%=request.getAttribute("javax.servlet.error.message")%>
}
```

100% Java Code Based Spring MVC

Few months ago I started playing with Spring Framework, which I really enjoy. Looking for the tutorials I found some really great resources on the web as well as books. Unfortunately many of them were written for Spring 2.x, which lacks some recent features, mainly ability to configure everything by Java annotations (a.k.a. JavaConfig) rather than XML. That was something that kept me away from Spring at the first place - doing it programmatically gives much more sense of control and is more readable. So, that's what I'm trying to achieve here, I'll try to show you how to do it in more 'modern' way, showing integrations with various software packages along the way.

Let's start with a simple skeleton Spring MVC application. Since version 3.1 it has Servlet 3 API support, I won't use web.xml for configuring DispatcherServlet, rather I'll configure it programmatically.

Maven configuration

Most important dependencies for this project will be spring-webmvc, which provides DispatcherServlet and pulls other Spring artifacts together with it and Servlet 3.0 API.

```
<dependency>

    <groupId>org.springframework</groupId>

    <artifactId>spring-webmvc</artifactId>

    <version>3.2.4</version>

</dependency>

<dependency>

    <groupId>javax.servlet</groupId>

    <artifactId>javax.servlet-api</artifactId>

    <version>3.0.1</version>

</dependency>
```

Initializer

AppInitializer is a class that implements WebApplicationInitializer interface. We hook up to onStartup() method, to add DispatcherServlet to ServletContext.

```
public class AppInitializer implements WebApplicationInitializer {

    @Override

    public void onStartup(ServletContext servletContext) throws ServletException {

        WebApplicationContext context = getContext();

        servletContext.addListener(new ContextLoaderListener(context));

        ServletRegistration.Dynamic dispatcher =

        servletContext.addServlet("DispatcherServlet", new

        DispatcherServlet(context));

        dispatcher.setLoadOnStartup(1);

        dispatcher.addMapping("/");

    }

}
```

```
private AnnotationConfigWebApplicationContext getContext() {  
    AnnotationConfigWebApplicationContext context = new  
    AnnotationConfigWebApplicationContext();  
    context.setConfigLocation("eu.kielczewski.example.config");  
    return context;  
}  
  
}
```

Several things happen here:

1. *AnnotationConfigWebApplicationContext is created. It's WebApplicationContext implementation that looks for Spring configuration in classes annotated with @Configuration annotation. setConfigLocation() method gets hint in which package(s) to look for them.*
2. *ContextLoaderListener is added to ServletContext – the purpose of this is to 'glue' WebApplicationContext to the lifecycle of ServletContext.*
3. *DispatcherServlet is created and initialized with WebApplicationContext we have created, and it's mapped to "/*" URLs and set to eagerly load on application startup.*

Configuration classes

The main AppConfig configuration class doesn't do anything but hits Spring on where to look for its components through @ComponentScan annotation.

```
@Configuration  
  
@ComponentScan(basePackages = "eu.kielczewski.example")  
  
public class AppConfig {  
    }  
}
```

WebMvcConfig class enables Spring MVC with @EnableWebMvc annotation. It extends WebMvcConfigurerAdapter, which provides empty methods that can be overridden to customize default configuration of Spring MVC. We will stick to default configuration at this time, but it's advised for you to see what the possibilities are.

```
@EnableWebMvc  
  
@Configuration  
  
public class WebMvcConfig extends WebMvcConfigurerAdapter {  
    }  
}
```

Controller

Controllers are annotated with `@Controller`. It is found by Spring because of `@ComponentScan` annotation in `AppConfig`. The method will intercept GET request to "/" to which the response will be sent. `@ResponseBody` indicates that whatever this method returns will be response body, and in this case it's just a "Hello world" String.

```
@Controller

public class IndexController {

    @RequestMapping(value = "/", method = RequestMethod.GET)
    @ResponseBody
    public String showIndex() {

        return "Hello world";

    }

}
```

Example to show web page

index.jsp

```
<jsp:forward page="home.htm"/>
```

welcome.jsp

```
<h1><center> Welcome page</center></h1>
<br><br>
${sysDate }
```

RootConfig.java

```
package com.nt.cfgs;

import org.springframework.context.annotation.Configuration;
@Configuration
public class RootAppConfig {

}
```

WebMvcAppConfig.java

```
package com.nt.cfgs;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
```

```
@EnableWebMvc
@Configuration
@ComponentScan("com.nt.*")
public class WebMvcAppConfig {
    @Bean
    public ViewResolver createViewResolver(){
        InternalResourceViewResolver ivr=null;
        ivr=new InternalResourceViewResolver();
        ivr.setPrefix("/WEB-INF/pages/");
        ivr.setSuffix(".jsp");
        return ivr;
    }
}
```

ShowHomePageController.java

```
package com.nt.controller;
import java.util.Date;
import java.util.Map;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class ShowHomePageController {

    @RequestMapping("/home.htm")
    public String showHomePage(Map map){
        map.put("sysDate",new Date());

        return "welcome";
    }
}
```

WebAppInitializer.java

```
package com.nt.webinit;

import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.ServletRegistration;

import org.springframework.web.WebApplicationInitializer;
import org.springframework.web.context.ContextLoaderListener;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

import com.nt.cfgs.RootAppConfig;
import com.nt.cfgs.WebMvcAppConfig;
```

```
public class WebAppInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext sc) throws ServletException {

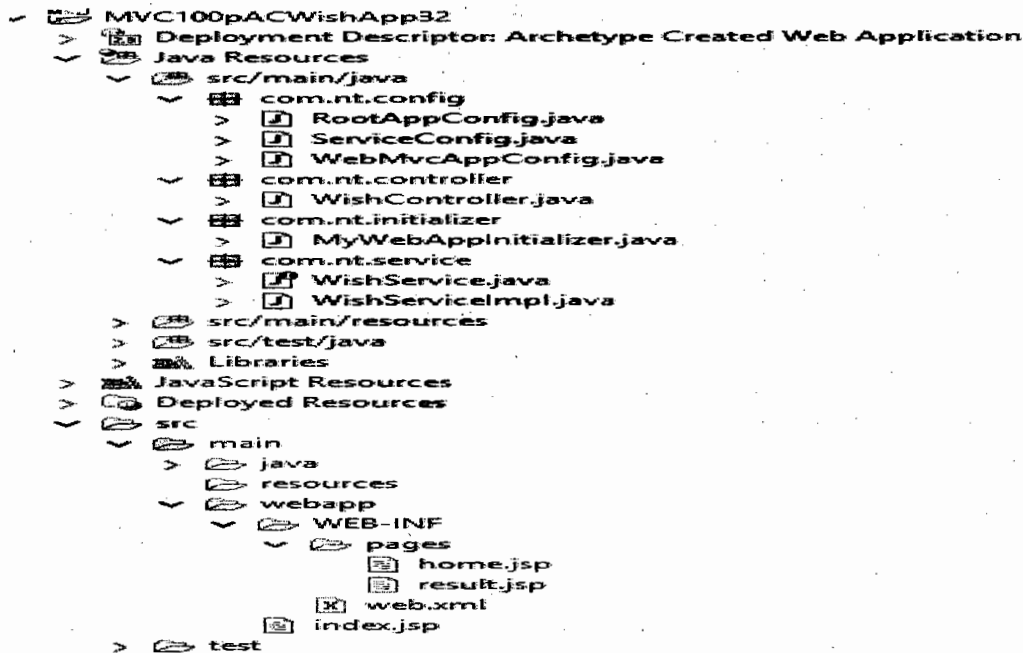
        //create Root Application context Container
        AnnotationConfigWebApplicationContext rootCtx=new
        AnnotationConfigWebApplicationContext();
        rootCtx.register(RootAppConfig.class);
        //create Web Application context container
        AnnotationConfigWebApplicationContext webCtx=new
        AnnotationConfigWebApplicationContext();
        webCtx.register(WebMvcAppConfig.class);

        //create ContextLoaderListener
        ContextLoaderListener listener=new ContextLoaderListener(rootCtx);
        //add Listener to Servletcontxt obj
        sc.addListener(listener);

        //Create DispatcherServlet
        DispatcherServlet servlet=new DispatcherServlet(webCtx);
        ServletRegistration.Dynamic register=sc.addServlet("dispatcher",servlet);
        register.setLoadOnStartup(2);
        register.addMapping("*.htm");
    }
}
```

Deployment related initial activities:

- a) Programmer deploys the application in the server -->ServletContext object will be created-->servlet container Spring ServletContainerInitializer class object by collecting it from jars added to WEB/lib folder-->onTrartup() method will be called on the above created object--> this method checks is their any class that implements WebAppInitializer- --> creates that class object and calls onStartup() method having ServletContext object in this method we place the logic of both RootContext, WEbAppContext containers having RootConfig, WebMvcAppConfig classes and these containers will be linked with ContextLoaderListener, DispatcherServlet class objects respectively and same listener, servlet will be registered with container dynamically.



Example App that acts as AbstractController to process the request generated by hyperlink

> target
pom.xml

index.jsp

```
<jsp:forward page="home.htm"/>
```

welcome.jsp

```
<h1><center> Welcome page</center></h1><br>
<a href="wish.htm">wish message</a>
```

result.jsp

```
<h1><center>Result Page</center></h1>
${resultMsg}
```

RootAppConfig.java

```
package com.nt.cfigs;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.nt.service.*")
public class RootAppConfig {

}
```

WebMvcAppInitializer.java

```
package com.nt.cfgs;
```

```
import java.util.Properties;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.web.servlet.HandlerMapping;
```

```
import org.springframework.web.servlet.ViewResolver;
```

```
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
```

```
import org.springframework.web.servlet.handler.SimpleUrlHandlerMapping;
```

```
import org.springframework.web.servlet.mvc.Controller;
```

```
import org.springframework.web.servlet.mvc.ParameterizableViewController;
```

```
import org.springframework.web.servlet.view.InternalResourceViewResolver;
```

```
@EnableWebMvc
```

```
@Configuration
```

```
@ComponentScan("com.nt.*")
```

```
public class WebMvcAppConfig {
```

```
    @Bean
```

```
    public HandlerMapping createHandlerMapping(){
```

```
        SimpleUrlHandlerMapping handlerMapping=new SimpleUrlHandlerMapping();
```

```
        Properties mappings=new Properties();
```

```
        mappings.put("/home.htm","pvc1");
```

```
        handlerMapping.setMappings(mappings);
```

```
        return handlerMapping;
```

```
    }
```

```
    @Bean(name="pvc1")
```

```
    public Controller createController(){
```

```
        ParameterizableViewController pvc=new ParameterizableViewController();
```

```
        pvc.setViewName("welcome");
```

```
        return pvc;
```

```
    }
```

```
    @Bean
```

```
    public ViewResolver createViewResolver(){
```

```
        InternalResourceViewResolver ivr=null;
```

```
        ivr=new InternalResourceViewResolver();
```

```
        ivr.setPrefix("/WEB-INF/pages/");
```

```
        ivr.setSuffix(".jsp");
```

```
        return ivr;
```

```
}  
  
}
```

WebApplInitializer.java

```
package com.nt.webinit;  
  
import javax.servlet.ServletContext;  
import javax.servlet.ServletException;  
import javax.servlet.ServletRegistration;  
  
import org.springframework.web.WebApplicationInitializer;  
import org.springframework.web.context.ContextLoaderListener;  
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;  
import org.springframework.web.servlet.DispatcherServlet;  
  
import com.nt.cfgs.RootAppConfig;  
import com.nt.cfgs.WebMvcAppConfig;  
  
public class WebApplInitializer implements WebApplicationInitializer {  
  
    @Override  
    public void onStartup(ServletContext sc) throws ServletException {  
  
        //create Root Application context Container  
        AnnotationConfigWebApplicationContext rootCtx=new  
AnnotationConfigWebApplicationContext();  
        rootCtx.register(RootAppConfig.class);  
        //create Web Application context container  
        AnnotationConfigWebApplicationContext webCtx=new  
AnnotationConfigWebApplicationContext();  
        webCtx.register(WebMvcAppConfig.class);  
  
        //create ContextLoaderListener  
        ContextLoaderListener listener=new ContextLoaderListener(rootCtx);  
        //add Listener to Servletcontxt obj  
        sc.addListener(listener);  
  
        //Create DispatcherServlet  
        DispatcherServlet servlet=new DispatcherServlet(webCtx);  
        ServletRegistration.Dynamic register=sc.addServlet("dispatcher",servlet);  
        register.setLoadOnStartup(2);  
        register.addMapping("*.htm");  
    }  
}
```


WishController.java

```
package com.nt.controller;

import java.util.Map;

import javax.annotation.Resource;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.service.WishService;

@Controller
public class WishController {

    @Resource
    private WishService service;

    @RequestMapping("/wish.htm")
    public String processRequest(Map map){
        //use Service class
        String wishMsg=service.generateWishMsg();
        //add Model Data
        map.put("resultMsg",wishMsg);

        return "result";
    }
}
```

WishService.java

```
package com.nt.service;

public interface WishService {

    public String generateWishMsg();
}
```

WishServiceImpl.java

```
package com.nt.service;

import org.springframework.stereotype.Service;

@Service("ws")
public class WishServiceImpl implements WishService{

    @Override
    public String generateWishMsg() {
        return "Good Morning";
    }
}
```

Most people know that Spring's long supported Servlet 3 Java-based initializer classes. This facility builds on Servlet 3's `javax.servlet.ServletContainerInitializer` facility. Basically, you declare a class that implements `org.springframework.web.WebApplicationInitializer` and this class will be scanned by Spring on application startup and bootstrapped. This class has one responsibility: assemble the web application's moving parts, like you would in a `web.xml`, but in code. Here's an example:

```
public class MyWebAppInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext container) {

        // Create the 'root' Spring application context
        AnnotationConfigWebApplicationContext rootContext = new
        AnnotationConfigWebApplicationContext();
        rootContext.register(ServiceConfiguration.class);

        // Manage the lifecycle of the root application context
        container.addListener(new ContextLoaderListener(rootContext));

        // Create the dispatcher servlet's Spring application context
        AnnotationConfigWebApplicationContext dispatcherServlet = new
        AnnotationConfigWebApplicationContext();
        dispatcherServlet.register(WebMvcConfiguration.class);

        // Register and map the dispatcher servlet
        ServletRegistration.Dynamic dispatcher =
        container.addServlet("dispatcher", new DispatcherServlet(dispatcherServlet));
        dispatcher.setLoadOnStartup(1);
        dispatcher.addMapping("/");

    }

}
```

In the above example, `WebConfiguration` and `ServiceConfiguration` are Java configuration classes. `ServiceConfiguration` presumably describes how the service tier - with its data sources and transaction managers and thread pools and ORM mapping - are assembled. The `WebMvcConfiguration` class describes how the web tier - supported by Spring MVC - is put together. We use the `ServletContext` to build up the web application. But, a lot of this is procedural. We will have basically the same arrangement each time, simply plugging in different values as required.

The point of this post isn't to describe how Java configuration works, rather to introduce the `WebApplicationInitializer` implementation, called `org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer`.

To use it, you simply extend it and satisfy fill in the blanks (in the form of template methods). Here's an example:

```
import com.jl.crm.services.ServiceConfiguration;
import org.springframework.context.annotation.*;
import org.springframework.data.repository.support.DomainClassConverter;
import org.springframework.data.rest.webmvc.config.RepositoryRestMvcConfiguration;
import org.springframework.format.support.FormattingConversionService;
```

```
import org.springframework.hateoas.config.EnableHypermediaSupport;
import org.springframework.orm.jpa.support.OpenEntityManagerInViewFilter;
import org.springframework.security.web.session.HttpSessionEventPublisher;
import org.springframework.web.filter.HiddenHttpMethodFilter;
import org.springframework.web.multipart.MultipartResolver;
import org.springframework.web.multipart.support.*;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.*;
import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

import javax.servlet.*;
import java.io.File;

public class CrmWebApplicationInitializer extends
AbstractAnnotationConfigDispatcherServletInitializer {

    private int maxUploadSizeInMb = 5 * 1024 * 1024; // 5 MB

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class<?>[]{ServiceConfiguration.class};
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class<?>[]{RepositoryRestMvcConfiguration.class, WebMvcConfiguration.class};
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{"/*"};
    }

    @Override
    protected Filter[] getServletFilters() {
        return new Filter[]{new HiddenHttpMethodFilter(), new MultipartFilter(), new
OpenEntityManagerInViewFilter()};
    }

    @Override
    protected void registerDispatcherServlet(ServletContext servletContext) {
        super.registerDispatcherServlet(servletContext);

        servletContext.addListener(new HttpSessionEventPublisher());
    }
}
```

```
@Override
protected void customizeRegistration(ServletRegistration.Dynamic registration) {

    File uploadDirectory = ServiceConfiguration.CRM_STORAGE_UPLOADS_DIRECTORY;

    MultipartConfigElement multipartConfigElement =
        new MultipartConfigElement(uploadDirectory.getAbsolutePath(),
            maxUploadSizeInMb, maxUploadSizeInMb * 2, maxUploadSizeInMb / 2);

    registration.setMultipartConfig(multipartConfigElement);

}
```

In typical Spring fashion, we have callbacks where Spring can't possibly devine our intents (it can't possibly know what the names of our configuration classes are, for example) but otherwise it's all taken care of for us. I had to override the default `ServletRegistration.Dynamic` instance so I could tailor where file uploads are stored by the Servlet 3 container, which I do in the `customizeRegistration` method. The rest of the code is fairly straightforward: Spring registers the Filters provided by the `getServletFilters` method, it registers the configuration classes returned from the `getRootConfigurationClasses` as part of a global `ContextLoaderListener`, and it registers the configuration classes returned from the `getServletFilters` as part of the `DispatcherServlet`. This reflects the typical layering in most Spring applications: global beans and services that are potentially to be shared among multiple `DispatcherServlet` implementations are stored in the `ContextLoaderListener`, and `DispatcherServlet`-local beans are configured there. My web configuration class is shown below. I show this mainly for completeness, but you can see I've registered and configured .jsp support, as well as multipart file upload support (taking advantage of the Servlet 3-powered `StandardServletMultipartResolver`).

Example App on AbstractAnnotationConfigDispatcherServletInitializer

WebAppInitializer.java

```
package com.nt.webinit;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.ServletRegistration;

import org.springframework.web.WebApplicationInitializer;
import org.springframework.web.context.ContextLoaderListener;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;
import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

import com.nt.cfgs.RootAppConfig;
import com.nt.cfgs.WebMvcAppConfig;
```

```
public class WebAppInitializer extends AbstractAnnotationConfigDispatcherServletInitializer{

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[]{RootAppConfig.class};
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[]{WebMvcAppConfig.class};
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{"*.htm"};
    }

}
```

MVC100pFirstApp:

- =====
- ▼  MVC100pFirstApp30
 - ▼  Java Resources
 - ▼  src/main/java
 - ▼  com.nt.config
 - >  RootAppConfig.java
 - >  WebMvcAppConfig.java
 - ▼  com.nt.controller
 - >  HomeController.java
 - ▼  com.nt.webinitializer
 - >  MyWebInitializer.java
 - >  src/main/resources
 - >  src/test/java
 - >  Libraries
 - >  JavaScript Resources
 - >  Deployed Resources
 - ▼  src
 - ▼  main
 - >  java
 - >  resources
 - ▼  webapp
 - ▼  WEB-INF
 - ▼  pages
 - >  welcome.jsp
 - >  index.jsp
 - >  test
 - >  target
 - >  pom.xml

MyWebInitializer.java

```
package com.nt.webinitializer;

import javax.servlet.ServletContext;
import javax.servlet.ServletRegistration;
import org.springframework.web.WebApplicationInitializer;
import org.springframework.web.context.ContextLoaderListener;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

import com.nt.config.RootAppConfig;
import com.nt.config.WebMvcAppConfig;

public class MyWebInitializer implements WebApplicationInitializer {

    public void onStartup(ServletContext sc){
        AnnotationConfigWebApplicationContext rootCtx=null,webCtx=null;
        ContextLoaderListener listener=null;
        DispatcherServlet servlet=null;
        //create RootAppContext Container for Businer tier comps
        rootCtx=new AnnotationConfigWebApplicationContext();
        rootCtx.register(RootAppConfig.class);
        //create webAppContext Container for presentation tier comps
        webCtx=new AnnotationConfigWebApplicationContext();
        webCtx.register(WebMvcAppConfig.class);

        //Create ContextLoaderListener having RootAppContext container
        listener=new ContextLoaderListener(rootCtx);
        // add/register Listener
        sc.addListener(listener);

        //Create DispatcherServlet having WebAppContext container
        servlet=new DispatcherServlet(webCtx);
        //register Servlet
        ServletRegistration.Dynamic registration=sc.addServlet("dispatcher",servlet);
        registration.setLoadOnStartup(1);
        registration.addMapping("*.htm");
    }
}
```

HomeController.java

```
package com.nt.controller;

import java.util.Date;
import java.util.Map;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
@Controller
public class HomeController {

    @RequestMapping("/home.htm")
    public String viewHomePage(Map<String, Object> map){

        //add model attribute
        map.put("sysDate", new Date());
        return "welcome";
    }
}
```

RootAppConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RootAppConfig {

}
```

WebMvcAppConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.ViewResolver;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

@Configuration
@ComponentScan(basePackages="com.nt.controller")
@EnableWebMvc
public class WebMvcAppConfig {

    @Bean
    public ViewResolver createViewResolver(){
        InternalResourceViewResolver viewResolver=null;
        //create and return ViewResolver
        viewResolver=new InternalResourceViewResolver();
        viewResolver.setPrefix("/WEB-INF/pages/");
        viewResolver.setSuffix(".jsp");
        return viewResolver;
    }
}
//method
}
//class
```

welcome.jsp

```
<%@page isELIgnored="false" %>
<h1 style="color:red;text-align:center"> Welcome to the world Spring MVC 100%Code
Approach</h1>
<br>
System Date:: ${sysDate }
```

index.jsp

```
<jsp:forward page="home.htm"/>
```

MVC100pWishApp-AbstractController:

=====

- ▼ MVC100pWishApp31-AbstractController
 - > Deployment Descriptor: Archetype Created Web Application
 - ▼ Java Resources
 - ▼ src/main/java
 - ▼ com.nt.config
 - > RootAppConfig.java
 - > ServiceConfig.java
 - > WebMvcAppConfig.java
 - ▼ com.nt.controller
 - > WishController.java
 - ▼ com.nt.service
 - > WishService.java
 - > WishServiceImpl.java
 - ▼ com.nt.webinitializer
 - > MyWebAppInitializer.java
 - > src/main/resources
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ▼ src
 - ▼ main
 - > java
 - > resources
 - ▼ webapp
 - ▼ WEB-INF
 - ▼ pages
 - result.jsp
 - welcome.jsp
 - web.xml
 - index.jsp
 - > test
 - > target
 - pom.xml

MyWebAppInitializer.java

```
package com.nt.webinitializer;
```



```
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.ServletRegistration;

import org.springframework.web.WebApplicationInitializer;
import org.springframework.web.context.ContextLoaderListener;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

import com.nt.config.RootAppConfig;
import com.nt.config.WebMvcAppConfig;

public class MyWebAppInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext sc) throws ServletException {
        AnnotationConfigWebApplicationContext rootCtx=null,webCtx=null;
        ContextLoaderListener listener=null;
        DispatcherServlet servlet=null;
        ServletRegistration.Dynamic registration=null;
        //create root Container , web application context container
        rootCtx=new AnnotationConfigWebApplicationContext();
        rootCtx.register(RootAppConfig.class);
        webCtx=new AnnotationConfigWebApplicationContext();
        webCtx.register(WebMvcAppConfig.class);

        //create ContextLoaderListener having Root ApplicationContext container
        listener=new ContextLoaderListener(rootCtx);
        sc.addListener(listener);
        //create DispatcherServlet having WebApplicationContext container
        servlet=new DispatcherServlet(webCtx);
        registration=sc.addServlet("dispatcher",servlet);
        registration.setLoadOnStartup(1);
        registration.addMapping("*.htm");
    }
}
}
}
```

WishController.java

```
package com.nt.controller;

import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

import com.nt.service.WishService;

@Controller
```

```
public class WishController {
    @Autowired
    private WishService service;

    @RequestMapping("/wish.htm")
    public String generateWishMessage(Map<String, Object> model) {
        String msg = null;
        //use Service class
        msg = service.generateWishMsg();
        //put result in model attribute
        model.put("wishMsg", msg);
        return "result";
    }
}
```

WishService.java

```
package com.nt.service;

public interface WishService {
    public String generateWishMsg();
}
```

WishServiceImpl.java

```
package com.nt.service;

import java.util.Calendar;

import org.springframework.stereotype.Service;

@Service("wishService")
public class WishServiceImpl implements WishService {

    @Override
    public String generateWishMsg() {
        Calendar calendar = null;
        int hour = 0;
        //get System Date
        calendar = Calendar.getInstance();
        hour = calendar.get(Calendar.HOUR_OF_DAY);
        if (hour <= 12)
            return "Good Morning";
        else if (hour <= 16)
            return "Good Afternoon";
        else if (hour <= 20)
            return "Good Evening";
        else
            return "Good Night";
    }
}

//method
//class
```

RootAppConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(value=ServiceConfig.class)
public class RootAppConfig {

}
```

ServiceConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

result.jsp

```
<%@page isELIgnored="false" %>

<h1 style="color:red;text-align:center">Result Page</h1>

wish Message::: ${wishMsg} <br><br>

<a href="home.htm">home</a>
```

welcome.jsp

```
<h1 style="color:red;text-align:center">Home Page</h1>

<a href="wish.htm">get Wish Message</a>
```

Index.jsp

```
<jsp:forward page="home.htm"/>
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>nit</groupId>
    <artifactId>MVCFirstApp</artifactId>
    <packaging>war</packaging>
```

```
<version>0.0.1-SNAPSHOT</version>
<name>MVCFirstApp Maven Webapp</name>
<url>http://maven.apache.org</url>
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
<!--
https://mvnrepository.com/artifact/org.springframework/spring-context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework/spring-context-support -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context-support</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.apache.commons/commons-dbcp2 -->
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-dbcp2</artifactId>
    <version>2.1.1</version>
  </dependency>
  <dependency>
```

```
<groupId>com.oracle</groupId>
<artifactId>ojdbc6</artifactId>
<version>11.2.0</version>
</dependency>
<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>

<!-- https://mvnrepository.com/artifact/taglibs/standard -->
<dependency>
  <groupId>taglibs</groupId>
  <artifactId>standard</artifactId>
  <version>1.1.2</version>
</dependency>

<!--
https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>3.1.0</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.aspectj/aspectjweaver -->
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.8.10</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.aspectj/aspectjrt
-->
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjrt</artifactId>
  <version>1.8.10</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io
-->
<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>2.4</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-
fileupload/commons-fileupload -->
```

```

<dependency>
  <groupId>commons-fileupload</groupId>
  <artifactId>commons-fileupload</artifactId>
  <version>1.3.1</version>
</dependency>

</dependencies>
<build>
  <finalName>MVCFirstApp</finalName>
</build>
</project>

```

- ✓  MVC100pSearchApp33-ACC
 -  Deployment Descriptor: Archetype Created Web Application
 - ✓  Java Resources
 - ✓  src/main/java
 - ✓  com.nt.bo
 - >  SearchBO.java
 - >  SearchResultBO.java
 - ✓  com.nt.command
 - >  SearchCommand.java
 - ✓  com.nt.config
 - >  PersistenceConfig.java
 - >  RootAppConfig.java
 - >  ServiceConfig.java
 - >  WebMvcAppConfig.java
 - ✓  com.nt.controller
 - >  EmployeeSearchController.java
 - ✓  com.nt.dao
 - >  EmployeeSearchDAO.java
 - >  EmployeeSearchDAOImpl.java
 - ✓  com.nt.dto
 - >  SearchDTO.java
 - >  SearchResultDTO.java
 - ✓  com.nt.initializer
 - >  MyWebAppInitializer.java
 - ✓  com.nt.service
 - >  EmployeeSearchService.java
 - >  EmployeeSearchServiceImpl.java
 -  src/main/resources
 -  src/test/java
 - Libraries
 - JavaScript Resources
 - Referenced Libraries

> Deployed Resourcesv src

v main

java

resources

v webappv WEB-INFv pages list_emps.jsp search.jsp web.xml index.jsp> test> target pom.xml**SearchBO.java:-**

```
package com.nt.bo;

public class SearchBO {
    private int eno;
    private String ename;
    private String job;
    private int salary;
    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public String getName() {
        return ename;
    }
    public void setName(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

SearchResultBO.java:-

```
package com.nt.bo;

public class SearchResultBO extends SearchBO {
    private int deptno;
    private int mgrNo;
    public int getDeptno() {
        return deptno;
    }
    public void setDeptno(int deptno) {
        this.deptno = deptno;
    }
    public int getMgrNo() {
        return mgrNo;
    }
    public void setMgrNo(int mgrNo) {
        this.mgrNo = mgrNo;
    }
}
```

SearchCommand.java:-

```
package com.nt.command;

public class SearchCommand {
    private int eno;
    private String ename;
    private String job;
    private int salary;

    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public String getEname() {
        return ename;
    }
    public void setName(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
}
```



```
}  
public void setSalary(int salary) {  
    this.salary = salary;  
}  
  
}
```

PersistenceConfig.java:-

```
package com.nt.config;  
  
import javax.sql.DataSource;  
import org.apache.commons.dbcp2.BasicDataSource;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.ComponentScan;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.jdbc.core.JdbcTemplate;  
  
@Configuration  
@ComponentScan(basePackages="com.nt.dao")  
public class PersistenceConfig {  
  
    @Bean  
    public DataSource createDataSource(){  
        BasicDataSource bds=null;  
        bds=new BasicDataSource();  
        bds.setDriverClassName("oracle.jdbc.driver.OracleDriver");  
        bds.setUrl("jdbc:oracle:thin:@localhost:1521:xe");  
        bds.setUsername("system");  
        bds.setPassword("manager");  
        return bds;  
    }  
  
    @Bean  
    public JdbcTemplate createJdbcTemplate(){  
        return new JdbcTemplate(createDataSource());  
    }  
  
}
```

RootAppConfig.java:-

```
package com.nt.config;  
  
import org.springframework.context.annotation.Configuration;  
import org.springframework.context.annotation.Import;  
  
@Configuration  
@Import(value={ServiceConfig.class,PersistenceConfig.class})  
public class RootAppConfig {  
  
}
```

ServiceConfig.java:-

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

WebMvcAppConfig.java:-

```
package com.nt.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages="com.nt.controller")
public class WebMvcAppConfig {

    @Bean
    public InternalResourceViewResolver createIVR(){
        InternalResourceViewResolver ivr=null;
        ivr=new InternalResourceViewResolver();
        ivr.setPrefix("/WEB-INF/pages/");
        ivr.setSuffix(".jsp");
        return ivr;
    }

}
```

EmployeeSearchController.java:-

```
package com.nt.controller;

import java.util.List;
import java.util.Map;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.servlet.ModelAndView;
import com.nt.command.SearchCommand;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;
import com.nt.service.EmployeeSearchService;

@Controller
public class EmployeeSearchController {

    @Autowired
    private EmployeeSearchService service;

    @RequestMapping("/search.htm")
    public String showHomePage(){
        return "search";
    }

    @RequestMapping("/searchEmps.htm")
    public String search(Map<String,Object>
map,@ModelAttribute("searchCmd")SearchCommand cmd)
        throws Exception {
        SearchCommand command=null;
        SearchDTO dto=null;
        List<SearchResultDTO> listRDTO=null;
        ModelAndView mav=null;
        //type casting
        command=(SearchCommand)cmd;
        //Convert Command class obj to DTO class object
        dto=new SearchDTO();
        BeanUtils.copyProperties(command,dto);
        // use Service class
        listRDTO=service.findEmployees(dto);
        //create and return MAV
        map.put("searchList",listRDTO);
        return "list_emps";
    }

}
```

EmployeeSearchDAO.java:-

```
package com.nt.dao;

import java.util.List;
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;

public interface EmployeeSearchDAO {

    public List<SearchResultBO> searchEmps(SearchBO bo);

}
```

EmployeeSearchDAOImpl.java:-

```
package com.nt.dao;

import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataAccessException;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.ResultSetExtractor;
import org.springframework.stereotype.Repository;
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;

@Repository
public class EmployeeSearchDAOImpl implements EmployeeSearchDAO {
    private static final String EMP_SEARCH_QUERY=" SELECT
EMPNO,ENAME,JOB,SAL,DEPTNO,MGR FROM EMP WHERE (EMPNO IS NOT NULL AND
EMPNO=?) OR"+ "(ENAME IS NOT NULL AND ENAME=?) "+
OR (JOB IS NOT NULL AND  JOB=?)"+ "
OR (SAL IS NOT NULL AND SAL=?)";

    @Autowired
    private JdbcTemplate jt;

    public List<SearchResultBO> searchEmps(SearchBO bo) {
        List<SearchResultBO> listRBO=null;
        listRBO=jt.query(EMP_SEARCH_QUERY,new SearchExtractor(),
bo.getEno(),bo.getEname(),bo.getJob(),bo.getSalary());
        return listRBO;
    } //method

    private class SearchExtractor implements
ResultSetExtractor<List<SearchResultBO>>{

        public List<SearchResultBO> extractData(ResultSet rs) throws
SQLException, DataAccessException {
            List<SearchResultBO> listRBO=null;
            SearchResultBO RBO=null;
            listRBO=new ArrayList();
            while(rs.next()){
                RBO=new SearchResultBO();
                RBO.setEno(rs.getInt(1));
                RBO.setEname(rs.getString(2));
                RBO.setJob(rs.getString(3));
                RBO.setSalary(rs.getInt(4));
            }
        }
    }
}
```

```
        RBO.setDeptno(rs.getInt(5));
        RBO.setMgrNo(rs.getInt(6));
        listRBO.add(RBO);
    } //while
    return listRBO;
} //extractData(-)
} //inner class
} //outer class
```

SearchDTO.java:-

```
package com.nt.dto;

import java.io.Serializable;

public class SearchDTO implements Serializable {
    private int eno;
    private String ename;
    private String job;
    private int salary;

    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
}
```

SearchResultDTO.java:-

```
package com.nt.dto;

import java.io.Serializable;

public class SearchResultDTO extends SearchDTO implements Serializable{
    private int deptno;
    private int mgrNo;
    public int getDeptno() {
        return deptno;
    }
    public void setDeptno(int deptno) {
        this.deptno = deptno;
    }
    public int getMgrNo() {
        return mgrNo;
    }
    public void setMgrNo(int mgrNo) {
        this.mgrNo = mgrNo;
    }
}
```

MyWebAppInitializer.java:-

```
package com.nt.initializer;

import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.ServletRegistration;
import org.springframework.web.WebApplicationInitializer;
import org.springframework.web.context.ContextLoaderListener;
import org.springframework.web.context.support.AnnotationConfigWebApplicationCon
text;
import org.springframework.web.servlet.DispatcherServlet;
import com.nt.config.RootAppConfig;
import com.nt.config.WebMvcAppConfig;

public class MyWebAppInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext sc) throws ServletException {

        AnnotationConfigWebApplicationContext
rootCtx=null,webCtx=null;
        ContextLoaderListener listener=null;
        DispatcherServlet servlet=null;
        ServletRegistration.Dynamic registration=null;
        //create root Container , web application context container
```

```
        rootCtx=new AnnotationConfigWebApplicationContext();
        rootCtx.register(RootAppConfig.class);
        webCtx=new AnnotationConfigWebApplicationContext();
        webCtx.register(WebMvcAppConfig.class);

        //create ContextLoaderListener having Root ApplicationContext
container
        listener=new ContextLoaderListener(rootCtx);
        sc.addListener(listener);
        //create DispatcherServlet having WebApplicationContext
container
        servlet=new DispatcherServlet(webCtx);
        registration=sc.addServlet("dispatcher",servlet);
        registration.setLoadOnStartup(1);
        registration.addMapping("*.htm");
    }//onStartup(-,-)
} //class
```

EmployeeSearchService.java:-

```
package com.nt.service;

import java.util.List;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;

public interface EmployeeSearchService {
    public List<SearchResultDTO> findEmployees(SearchDTO dto);
}
```

EmployeeSearchServiceImpl.java:-

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.nt.bo.SearchBO;
import com.nt.bo.SearchResultBO;
import com.nt.dao.EmployeeSearchDAO;
import com.nt.dto.SearchDTO;
import com.nt.dto.SearchResultDTO;

@Service
public class EmployeeSearchServiceImpl implements EmployeeSearchService {
    @Autowired
    private EmployeeSearchDAO dao;
```

```
public List<SearchResultDTO> findEmployees(SearchDTO dto) {
    SearchBO bo=null;
    List<SearchResultBO> listRBO=null;
    List<SearchResultDTO> listRDTO=null;
    SearchResultDTO RDTO=null;
    //Convert DTO class obj BO class object
    bo=new SearchBO();
    BeanUtils.copyProperties(dto,bo);
    //use DAO
    listRBO=dao.searchEmps(bo);
    //Covert listRBO to listRDTO
    listRDTO=new ArrayList();
    for(SearchResultBO RBO:listRBO){
        RDTO=new SearchResultDTO();
        BeanUtils.copyProperties(RBO,RDTO);
        listRDTO.add(RDTO);
    }//for
    return listRDTO;
} //findEmployees()
} //class
```

list_emps.jsp:-

```
<%@page isELIgnored="false"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>

<c:choose>
    <c:when test="${searchList ne null}">
        <h1 style="color:red;text-align:center">Search Results </h1>
        <table border="1">

<tr><th>Empno</th><th>Empname</th><th>EmpJob</th><th>EmpSalary</th><th>Dept no</th><th>Manager no</th></tr>
        <c:forEach var="dto" items="${searchList}">
            <tr>
                <td>${dto.eno}</td>
                <td>${dto.ename}</td>
                <td>${dto.job}</td>
                <td>${dto.salary}</td>
                <td>${dto.deptno}</td>
                <td>${dto.mgrNo}</td>
            </tr>
        </c:forEach>
    </table>
    </c:when>
    <c:otherwise>
        <p style='color:red'>Records not found...</p>
    </c:otherwise>
</c:choose>
```



```
<br>
<a href='home.htm'>home</a>
```

search.jsp:-

```
<h1 style="color:red">Search Employee Details--
AbstractCommandController</h1>

<form action="searchEmps.htm" method="post">
  Employee no: <input type="text" name="eno"> <br>
  Employee name: <input type="text" name="ename"> <br>
  Employee job: <input type="text" name="job"> <br>
  Employee salary: <input type="text" name="salary"> <br>
  <input type="submit" value="Search">
</form>
```

index.jsp:-

```
<jsp:forward page="home.htm"/>
```

SimpleFormController:-

- ▼  MVC100pLoginApp34-SFC
 -  Deployment Descriptor: Archetype Created Web Application
 - ▼  Java Resources
 - ▼  src/main/java
 - ▼  com.nt.bo
 -  UserBO.java
 - ▼  com.nt.command
 -  UserCommand.java
 - ▼  com.nt.commons
 -  jdbc.properties
 - ▼  com.nt.config
 -  AOPConfig.java
 -  PersistenceConfig.java
 -  RootAppConfig.java
 -  ServiceConfig.java
 -  WebMvcAppConfig.java
 - ▼  com.nt.controller
 -  LoginController.java
 - ▼  com.nt.dao
 -  LoginDAO.java
 -  LoginDAOImpl.java
 - ▼  com.nt.dto
 -  UserDTO.java
 - ▼  com.nt.initializer
 -  MyWebApplInitializer.java
 - ▼  com.nt.service
 -  LoginService.java
 -  LoginServiceImpl.java
 -  src/main/resources
 -  src/test/java

- > Libraries
- > JavaScript Resources
- > Referenced Libraries
- > Deployed Resources
- ▼ src
 - ▼ main
 - > java
 - resources
 - ▼ webapp
 - ▼ WEB-INF
 - ▼ pages
 - login.jsp
 - web.xml
 - index.jsp
- > test
- > target
- pom.xml

MyWebInitializer.java

```
package com.nt.webinitializer;
```

```
import
```

```
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;
```

```
import com.nt.config.RootAppConfig;
```

```
import com.nt.config.WebMvcAppConfig;
```

```
public class MyWebInitializer extends AbstractAnnotationConfigDispatcherServletInitializer{
```

```
    @Override
```

```
    public Class<?>[] getRootConfigClasses() {  
        return new Class[]{RootAppConfig.class};  
    }
```

```
    @Override
```

```
    public Class<?>[] getServletConfigClasses() {  
        return new Class[]{WebMvcAppConfig.class};  
    }
```

```
    @Override
```

```
    public String[] getServletMappings() {  
        return new String[]{"*.htm"};  
    }
```

```
}//class
```

UserCommand.java

```
package com.nt.command;

import org.hibernate.validator.constraints.NotEmpty;

public class UserCommand {
    @NotEmpty(message="user name must not be empty")
    private String uname;
    @NotEmpty(message="password must not be empty")
    private String pwd;

    public String getUname() {
        return uname;
    }

    public void setUname(String uname) {
        this.uname = uname;
    }

    public String getPwd() {
        return pwd;
    }

    public void setPwd(String pwd) {
        this.pwd = pwd;
    }
}
```

LoginController.java

```
package com.nt.controller;

import java.util.Map;

import javax.validation.Valid;

import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

import com.nt.command.UserCommand;
import com.nt.dto.UserDTO;
import com.nt.service.LoginService;
```

```
@Controller
public class LoginController {

    @Autowired
    private LoginService service;

    @RequestMapping(value="/login.htm",method=RequestMethod.GET)
    public String showFormPage(Map<String,Object> map){
        map.put("userCmd",new UserCommand());
        return "login";
    }

    @RequestMapping(value="/login.htm",method=RequestMethod.POST)
    public String processFormPage(Map<String,Object> map,
                                   @Valid @ModelAttribute("userCmd")
                                   UserCommand cmd,
                                   BindingResult errors){

        UserDTO dto=null;
        dto=new UserDTO();
        String result=null;
        if(errors.hasErrors()){
            return "login";
        }
        //Convert Command obj to DTO
        BeanUtils.copyProperties(cmd,dto);
        //use service
        result=service.authenticate(dto);
        map.put("result", result);
        return "login";
    }
}
//method
}
//class
```

UserBO.java

```
package com.nt.bo;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="userlist")
public class UserBO {
    @Id
    private String uname;
    private String pwd;
}
```

```
public String getUsername() {  
    return uname;  
}  
  
public void setUsername(String uname) {  
    this.uname = uname;  
}  
  
public String getPwd() {  
    return pwd;  
}  
  
public void setPwd(String pwd) {  
    this.pwd = pwd;  
}  
}
```

UserDTO.java

```
package com.nt.dto;  
  
import java.io.Serializable;  
  
public class UserDTO implements Serializable {  
    private String uname;  
    private String pwd;  
  
    public String getUsername() {  
        return uname;  
    }  
  
    public void setUsername(String uname) {  
        this.uname = uname;  
    }  
  
    public String getPwd() {  
        return pwd;  
    }  
  
    public void setPwd(String pwd) {  
        this.pwd = pwd;  
    }  
}
```

LoginDAO.java

```
package com.nt.dao;  
  
import com.nt.bo.UserBO;
```

```
public interface LoginDAO {  
    public long validate(UserBO bo);  
}
```

LoginDAOImpl.java

```
package com.nt.dao;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.orm.hibernate5.HibernateTemplate;  
import org.springframework.stereotype.Repository;  
  
import com.nt.bo.UserBO;  
  
@Repository("loginDAO")  
public class LoginDAOImpl implements LoginDAO {  
    private static final String HQL_AUTH_QUERY="SELECT count(*) from UserBO where  
uname=? and pwd=?";  
    @Autowired  
    private HibernateTemplate ht;  
    @Override  
    public long validate(UserBO bo) {  
        long count=0;  
        List<?> list=null;  
        //execute query  
        list=ht.find(HQL_AUTH_QUERY, bo.getUsername(),bo.getPwd());  
        count=(Long)list.get(0);  
        return count;  
    }  
}
```

LoginService.java

```
package com.nt.service;  
  
import com.nt.dto.UserDTO;  
  
public interface LoginService {  
    public String authenticate(UserDTO dto);  
}
```

LoginServiceImpl.java

```
package com.nt.service;
```

```
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
```

```
import com.nt.bo.UserBO;
import com.nt.dao.LoginDAO;
import com.nt.dto.UserDTO;
```

@Service

```
public class LoginServiceImpl implements LoginService {
```

@Autowired

```
private LoginDAO dao;
```

@Override

```
@Transactional(propagation=Propagation.REQUIRED,  
readOnly=true,transactionManager="hibTxMgmr")
```

```
public String authenticate(UserDTO dto) {
```

```
    long count=0;
```

```
    UserBO bo=null;
```

```
    //convert DTO to BO
```

```
    bo=new UserBO();
```

```
    BeanUtils.copyProperties(dto,bo);
```

```
    //use DAO
```

```
    count=dao.validate(bo);
```

```
    if(count==0)
```

```
        return "Invalid Credentials";
```

```
    else
```

```
        return "Valid Credentials";
```

```
    } //method
```

```
} //class
```

AOPConfig.java

```
package com.nt.config;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.orm.hibernate5.HibernateTransactionManager;
```

@Configuration

```
public class AOPConfig {
```

@Bean(name="hibTxMgmr")

```
public HibernateTransactionManager createHTxMgmr(){
```

```
    HibernateTransactionManager hibTxMgmr=null;
```

```
    hibTxMgmr=new
```

```
HibernateTransactionManager(PersistenceConfig.createSessionFactory());
```

```
        return hbTxMgmr;
```

```
    }
```

```
}
```

PersistenceConfig.java

```
package com.nt.config;
```

```
import javax.sql.DataSource;
```

```
import org.hibernate.SessionFactory;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.jdbc.datasource.lookup.JndiDataSourceLookup;
```

```
import org.springframework.orm.hibernate5.HibernateTemplate;
```

```
import org.springframework.orm.hibernate5.LocalSessionFactoryBuilder;
```

```
import com.nt.bo.UserBO;
```

```
@Configuration
```

```
@ComponentScan(basePackages="com.nt.dao")
```

```
public class PersistenceConfig {
```

```
    @Bean(name="smds")
```

```
    public static DataSource createDataSource(){
```

```
        JndiDataSourceLookup jdsl=null;
```

```
        DataSource ds=null;
```

```
        //use server managed jdbc con pool
```

```
        jdsl=new JndiDataSourceLookup();
```

```
        ds=jdsl.getDataSource("java:/comp/env/DsJndi");
```

```
        return ds;
```

```
    }
```

```
    @Bean(name="sesfact")
```

```
    public static SessionFactory createSessionFactory(){
```

```
        LocalSessionFactoryBuilder builder=null;
```

```
        builder=new LocalSessionFactoryBuilder(createDataSource());
```

```
        builder.addAnnotatedClass(UserBO.class);
```

```
        builder.setProperty("hibernate.dialect","org.hibernate.dialect.Oracle10gDialect");
```

```
        builder.setProperty("hibernate.show_sql","true");
```

```
        return builder.buildSessionFactory();
```

```
    }
```

```
    @Bean(name="template")
```

```
    public HibernateTemplate createHibenrateTemplate(){
```

```
        HibernateTemplate ht=null;
```

```
        ht=new HibernateTemplate(createSessionFactory());
```



```
        return ht;  
    }  
  
} //class
```

ServiceConfig.java

```
package com.nt.config;  
  
import org.springframework.context.annotation.ComponentScan;  
import org.springframework.context.annotation.Configuration;  
  
@Configuration  
@ComponentScan(basePackages="com.nt.service")  
public class ServiceConfig {  
  
}
```

RootAppConfig.java

```
package com.nt.config;  
  
import org.springframework.context.annotation.Configuration;  
import org.springframework.context.annotation.Import;  
  
@Configuration  
@Import(value={ServiceConfig.class,PersistenceConfig.class,AOPConfig.class})  
public class RootAppConfig {  
  
}
```

WebMvcAppConfig.java

```
package com.nt.config;  
  
import org.springframework.context.MessageSource;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.ComponentScan;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.context.support.ResourceBundleMessageSource;  
import org.springframework.web.servlet.ViewResolver;  
import org.springframework.web.servlet.config.annotation.EnableWebMvc;  
import org.springframework.web.servlet.view.InternalResourceViewResolver;  
  
@Configuration  
@EnableWebMvc  
@ComponentScan(basePackages="com.nt.controller")  
/*@PropertySource(value="classpath:com/nt/commons/validation.properties")  
public class WebMvcAppConfig {  
  
    @Bean
```

```

public ViewResolver createViewResolver(){
    InternalResourceViewResolver resolver=null;
    resolver=new InternalResourceViewResolver();
    resolver.setPrefix("/WEB-INF/pages/");
    resolver.setSuffix(".jsp");
    return resolver;
}

@Bean(name="messageSource")
public MessageSource createResourceBundleMessageSource(){
    ResourceBundleMessageSource rbms=null;
    rbms=new ResourceBundleMessageSource();
    rbms.setBasename("com.nt.commons.validation");
    return rbms;
}
}

```

login.jsp:-

```

<%@ page isELIgnored="false" language="java" contentType="text/html;
charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>

<form:form method="post" commandName="userCmd">
    Username ::: <form:input path="user"/><br>
    Password ::: <form:input path="pwd"/><br>
    <input type="submit" value="Login">
</form:form>

<h1>${result} </h1>

```

index.jsp:-

```

<jsp:forward page="home.htm"/>

```

pom.xml:-

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>nit</groupId>
    <artifactId>MVC100pACWishApp32</artifactId>
    <packaging>war</packaging>
    <version>0.0.1-SNAPSHOT</version>
    <name>MVC100pACWishApp32 Maven Webapp</name>
    <url>http://maven.apache.org</url>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>

```

```
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!--
https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>

<!--
https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>
<!--
https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>

<!--
https://mvnrepository.com/artifact/javax.inject/javax.inject -->
<dependency>
  <groupId>javax.inject</groupId>
  <artifactId>javax.inject</artifactId>
  <version>1.0-PFD-1</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context-support</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>
<!--
https://mvnrepository.com/artifact/org.springframework/spring-web -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-web</artifactId>
```

```
<version>4.3.9.RELEASE</version>
</dependency>

<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.1.2</version>
</dependency>

<!-- https://mvnrepository.com/artifact/taglibs/standard -->
<dependency>
  <groupId>taglibs</groupId>
  <artifactId>standard</artifactId>
  <version>1.1.2</version>
</dependency>
<!--
https://mvnrepository.com/artifact/org.apache.commons/commons-dbcp2 -->
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-dbcp2</artifactId>
  <version>2.1.1</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
<dependency>

  <groupId>org.springframework</groupId>
  <artifactId>spring-jdbc</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>

</dependencies>

<repositories>
  <repository>
    <id>jbossRep</id>

    <url>https://repository.jboss.org/nexus/content/repositories/thirdparty-
releases/</url>
  </repository>
</repositories>
<build>
  <finalName>MVC100pACWishApp32</finalName>
</build>
</project>
```

Dynamic Registration of Servlet :-

Different ways of configuring Servlet component in java web application?

- Using web.xml (b) using annotations (@ web servlet) (from servlet 3.x) (c) using dynamic registration (from servlet 3.x)
- web.xml based configuration for servlet component is outdated .
- As alternate we can use @ web servlet annotation but we cannot apply annotations on predefined (or) third party supplied classes like dispatcherservlet (spring MVC) facesServlet (JSF) and etc... To overcome this problems use dynamic registration of servlet.

Example app on dynamic registration:-

- (a) Develop any normal predefined servlet in web application without xml(or) annotations.

Dateservlet.java:-

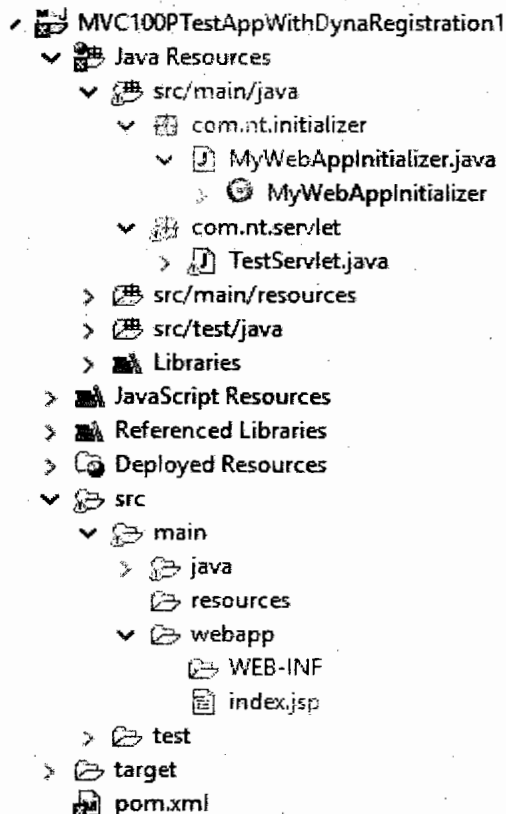
```
public class Dateservlet {  
  
    public static void doGet (-,-) throws SE, IOE {  
  
        .....  
  
        .....  
  
        pw.println ("<b> Date and Time </b>" + new java.util.date ());  
  
        .....  
    }  
  
    public static void doPost (-,-) throws SE, IOE {  
        doGet(res, req);  
    }  
}
```

- (b) Develop web application initializer class implementing javax.servlet.ServletContainerInitializer.
- (c) Prepare jar file specifying the above class name in javax.servlet.ServletContainerInitializer file of META-INF/ services folder.
- (d) Place the jar file in WEB-INF\lib folder.

->Always develop these kind of applications through **MAVEN** .

Folw of execution:-

- We deploy web application -> container creates servlet context object – >The special class loaders of container searches in the jar's added to WEB-INF /lib folder for META-INF /services/javax.servlet.ServletContainerIntilizer file loads - >and instantiates com.nt.initializer. MyWebAppInitializer class and also calls onStartup () having servletContext object as the argument. This method registers the Servlet dynamically – >remaining follow is regular.



MyWebAppInitializer.java:-

```
package com.nt.initializer;

import java.util.Set;
import javax.servlet.ServletContainerInitializer;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.ServletRegistration;

import com.nt.servlet.TestServlet;

public class MyWebAppInitializer implements ServletContainerInitializer {

    static{
        System.out.println("static block : MyWebInitializer");
    }
}
```

```
}  
public MyWebAppInitializer() {  
    System.out.println("0-param constructor::MyWebInitializer");  
}  
  
@Override  
public void onStartUp(Set<Class<?>> set, ServletContext sc) throws  
ServletException {  
    System.out.println("MyWebInitializer::onStartup(-,-)");  
    TestServlet ts=null;  
    ServletRegistration.Dynamic registration=null;  
    //register Servlet  
    ts=new TestServlet();  
    registration=sc.addServlet("test",ts);  
    registration.addMapping("/testurl");  
}  
}
```

TestServlet.java:-

```
package com.nt.servlet;  
  
import java.io.IOException;  
import java.io.PrintWriter;  
import javax.servlet.ServletException;  
import javax.servlet.http.HttpServlet;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
  
public class TestServlet extends HttpServlet{  
  
    @Override  
    public void doGet(HttpServletRequest req, HttpServletResponse res)  
throws ServletException, IOException {  
        PrintWriter pw=null;  
        //general settings  
        pw=res.getWriter();  
        res.setContentType("text/html");  
        //Display out  
        pw.println("<marquee><h1 style='color:red'>100% Code Based Web  
Applicaition(Servlet 3.0)</marquee></h1>");  
        //close stream  
        pw.close();  
    }//doGet(-,-)  
    @Override  
    public void doPost(HttpServletRequest req, HttpServletResponse res)  
throws ServletException, IOException {  
        doGet(req,res);  
    }  
}
```

index.jsp:-

```
<a href="testurl">get Message</a>
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>nit</groupId>
  <artifactId>MVC100PTestAppWithDynaRegistration</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>MVC100PTestAppWithDynaRegistration Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>3.0.1</version>
    </dependency>
  <!--
https://mvnrepository.com/artifact/javax.servlet.jsp.jstl/jstl -->

</dependencies>
<build>
  <finalName>MVC100PTestAppWithDynaRegistration</finalName>
</build>
</project>
```

SpringBootMVC

- Built in container(tomcat server containers) (Embedded container) default port no:8080
- Powerful annotations
- 100% java code is promised
- Regular Configurations or pre-defined classes will be take care internally or through properties file
- Overall takes less time for development and gives good performance
- Hot-cake in the market

Thumb rule:

Configure user defined classes through annotations pre-defined classes through @Bean method (or) through properties file.

- To develop web application initialiser in spring boot application takes a class extending from Spring Boot ServletInitializer which takes care of dispatcher Servlet registration.
- Spring boot servlet initializer
-> Internally it is a web application initializer

**Procedure to develop first spring boot application using STS plug-in for eclipse:-
[Spring boot version-14.7 release](Lancing home page)**

Step-1:- Make sure that STS plug-in properly installed in or not.

Step-2:- create spring starter project as shown below

- File menu -> spring starter project -> fill up all details -> select boot version - 14.7 -> search and select web -> next -> finish (uses two default classes (i) web Initializer (ii) config class).

Step-3:- Add following <dependency> tag **tomcat-embed-jasper.jar**.

Step-4:- configure view resolver in application .properties as shown below.

Spring.mvc.view.prefix = /WEB-WF/pages/

Spring.mvc.view.suffix = .jsp

Step-5:- Develop index.jsp and Controller class in regular manner

Step-6:- Add @component scan in spring boot configuration class specifying the package.

Step-7:- Run the application as spring boot application.

- For changing port no we have to write in application .properties file

server.port=4041

Note:

Parent/root container also completes pre-instantiation and Injections.

- While working with spring boot data sources it looks to use data sources in the following order
 - 1) Tomcat data source (if not available next one it uses the following)
 - 2) Hikari
 - 3) Apache dbcp
 - 4) Apache dbcp2

Note:-

- To break above algorithm & to make Spring boot giving certain data source object of our choice (belongs to above list directly) along with regular datasource JDBC properties configure the following type property.

spring.datasource.type=org.apache.commons.dbcp2.BasicDataSource.

- **While working with server managed connection pool:-**

Generally it is recommended to use server managed jdbc connection pool in the application

that are deployable in the server. In spring boot application we can mention data source Jndi in application. properties as shown below.

```
spring.datasource.jndi-name =java:/comp/env/DsJndi
```

- While performing non-select operations there is compatibility between spring-4.x and Hibernate-5.x through ORM module. To overcome this problem use Spring-4.x with Hibernate-4.x & also call

```
ht.setCheckWriteOperations(false);
```

MvcBootApplication:

- ✓  MvcBootApplication [boot]
 - >  Spring Elements
 - ✓  Java Resources
 - ✓  src/main/java
 - ✓  com.nt.controller
 - >  HomeController.java
 - ✓  com.nt.initializer
 - >  WebAppInitializer.java
 - ✓  src/main/resources
 -  application.properties
 - >  src/test/java
 - >  Libraries
 - >  JavaScript Resources
 - >  Deployed Resources
 - ✓  src
 - ✓  main
 - >  java
 - >  resources
 - ✓  webapp
 - ✓  WEB-INF
 - ✓  pages
 -  welcome.jsp
 -  index.jsp
 - >  test
 - >  target
 -  pom.xml

WebAppInitializer.java

```
package com.nt.initializer;  
  
import org.springframework.boot.SpringApplication;  
  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
import org.springframework.boot.builder.SpringApplicationBuilder;
```

```
import org.springframework.boot.context.web.SpringBootServletInitializer;

import org.springframework.context.annotation.ComponentScan;

@SpringBootApplication

@ComponentScan(basePackages="com.nt.controller")

public class WebAppInitializer extends SpringBootServletInitializer {

    @Override

    protected SpringApplicationBuilder configure(

        SpringApplicationBuilder application) {

        return application.sources(WebAppInitializer.class);

    }

    public static void main(String[] args)

        throws Exception {

        SpringApplication.run(WebAppInitializer.class, args);

    }

}
```

HomeController:

```
package com.nt.controller;

import org.springframework.stereotype.Controller;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RequestMethod;

import org.springframework.web.bind.annotation.ResponseBody;

@Controller

public class HomeController {
```

```
@RequestMapping(value="/home1.htm",method={RequestMethod.GET,RequestMethod.POST})

public String showHome(){

    return "welcome";

}

@RequestMapping(value="/direct.htm")

@ResponseBody

public String helloHome(){

    return "Welcome to jsp";

}

}
```

application.properties

```
spring.mvc.view.prefix: /WEB-INF/pages/

spring.mvc.view.suffix: .jsp
```

welcome.jsp

```
<h1 style="color:red">Welcome page</h1>
```

index.jsp

```
<jsp:forward page="home.htm"/>
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.brightjavatutorial.spring.boot</groupId>
    <artifactId>SpringBootMvcJsp</artifactId>
    <packaging>war</packaging>
    <version> 0.0.1-SNAPSHOT</version>
    <name>
        SpringBootMvcJsp Maven Webapp
    </name>
```

```
</name>
<url>
  http://maven.apache.org
</url>

<parent>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-parent
  </artifactId>
  <version>
    1.3.5.RELEASE
  </version>
</parent>

<properties>
  <java.version>
    1.8
  </java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>
      org.springframework.boot
    </groupId>
    <artifactId>
      spring-boot-starter-web
    </artifactId>
  </dependency>

  <dependency>
    <groupId>
      org.springframework.boot
    </groupId>
    <artifactId>
      spring-boot-starter-tomcat
    </artifactId>
    <scope>
      provided
    </scope>
  </dependency>

  <dependency>
    <groupId>
      org.apache.tomcat.embed
    </groupId>
    <artifactId>
      tomcat-embed-jasper
    </artifactId>
    <scope>
      provided
    </scope>
  </dependency>
</dependencies>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>
        org.springframework.boot
      </groupId>
      <artifactId>
        spring-boot-maven-plugin
      </artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

MVCBootApplication:

- ▼ MVCBootApplication2 [boot]
 - > Deployment Descriptor: MVCBootApplication2
 - > Spring Elements
 - > JAX-WS Web Services
 - ▼ Java Resources
 - ▼ src/main/java
 - ▼ com.nt.controller
 - > HomeController.java
 - ▼ com.nt.initializer
 - > MvcBootApplication2Application.java
 - > ServletInitializer.java
 - > src/main/resources
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ▼ src
 - ▼ main
 - > java
 - > resources
 - ▼ webapp
 - ▼ WEB-INF
 - welcome.jsp
 - > test
 - > target
 - > mvnw
 - > mvnw.cmd
 - > pom.xml

ServletInitializer.java

```
package com.nt.initializer;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.support.SpringBootServletInitializer;

public class ServletInitializer extends SpringBootServletInitializer {

    @Override

    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {

        return application.sources(MvcBootApp2Application.class);

    }

}
```

MvcBootApp2Application.java

```
package com.nt.initializer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;

@SpringBootApplication
@ComponentScan(basePackages="com.nt.controller")

public class MvcBootApp2Application {

    public static void main(String[] args) {

        SpringApplication.run(MvcBootApp2Application.class, args);

    }

}
```

HomeController.java

```
package com.nt.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller

public class HomeController {

    @RequestMapping(value="/home.htm",method={RequestMethod.GET,RequestMethod.POST})

    public String showHome(){

        return "welcome";

    }

    @RequestMapping(value="/direct.htm")

    @ResponseBody

    public String helloHome(){

        return "Welcome to jsp";

    }

}
```

application.properties

```
spring.mvc.view.prefix= /
spring.mvc.view.suffix= .jsp
```

welcome.jsp

```
<h1 style="color:red;text-align:center">Welcome to Home page</h1>
```


MVCBootFirstApp:

- ▼  MVCBootFirstApp35 [boot]
 - >  Deployment Descriptor: MVCBootFirstApp35
 - >  Spring Elements
 - >  JAX-WS Web Services
 - ▼  Java Resources
 - ▼  src/main/java
 - ▼  com.nt.config
 - >  MvcBootFirstApp35Application.java
 - >  ServletInitializer.java
 - >  WebMvcAppConfig.java
 - ▼  com.nt.controller
 - >  ShowHomeController.java
 - ▼  src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - Libraries
 - >  JavaScript Resources
 - >  Deployed Resources
 - ▼  src
 - ▼  main
 - >  java
 - >  resources
 - ▼  webapp
 - ▼  WEB-INF
 - >  pages
 -  welcome.jsp
 -  index.jsp
 - >  test
 - >  target
 -  mvnw

 mvnw.cmd pom.xml**WebMvcAppConfig.java:**

```
package com.nt.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.controller")
public class WebMvcAppConfig {
}
```

MvcBootFirstApp35Application:-

```
package com.nt.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;
@SpringBootApplication
public class MvcBootFirstApp35Application {

    public static void main(String[] args) {
        System.out.println("MBFAA::main(-)");
        SpringApplication.run(MvcBootFirstApp35Application.class, args);
    }
}
```

ServletInitializer:-

```
package com.nt.config;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.support.SpringBootServletInitializer;

public class ServletInitializer extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        System.out.println("ServletInitializer::configure(-)");
        return application.sources(WebMvcAppConfig.class);
    }
}
```

ShowHomeController:-

```
package com.nt.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
@Controller
public class ShowHomeController {
    @RequestMapping("/home.htm")
    public String showHomePage(){
        return "welcome";
    }
}
```

welcome.jsp:

```
<h1 style="color:red;text-align:center">Welcome Spring Boot MVC</h1>
```

application.properties:-

```
#View Resolver Configuration
spring.mvc.view.prefix=/WEB-INF/pages/
spring.mvc.view.suffix=.jsp

#For changing server port
server.port=4041
```

index.jsp:-

```
<jsp:forward page="home.htm"/>
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>nit</groupId>
  <artifactId>MVCBootFirstApp35</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

  <name>MVCBootFirstApp35</name>
  <description>Demo project for Spring Boot</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>1.4.7.RELEASE</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<!--
https://mvnrepository.com/artifact/org.apache.tomcat.embed/tomcat-embed-
jasper -->
    <dependency>
        <groupId>org.apache.tomcat.embed</groupId>
        <artifactId>tomcat-embed-jasper</artifactId>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>
```

MVCBootApplicationApp:-

- ▼  MVCBootApplicationApp36 [boot]
 - >  Deployment Descriptor: MVCBootApplicationApp36
 - >  Spring Elements
 - >  JAX-WS Web Services
 - ▼  Java Resources
 - ▼  src/main/java
 - ▼  com.nt.config
 - >  RootAppConfig.java
 - >  ServiceConfig.java
 - >  ServletInitializer.java
 - >  WebMvcAppConfig.java
 - ▼  com.nt.controller
 - >  ShowLanguagesController.java
 - ▼  com.nt.service
 - >  LanguageService.java
 - >  LanguageServiceImpl.java
 - ▼  src/main/resources
 -  static
 -  templates
 -  application.properties
 - >  src/test/java
 - >  Libraries
 - >  JavaScript Resources
 - >  Deployed Resources
 - ▼  src
 - ▼  main
 - >  java
 - >  resources
 - ▼  webapp
 - ▼  WEB-INF
 - ▼  pages
 -  home.jsp
 -  languages.jsp
 -  index.jsp
 - >  test
 - >  target
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

ServiceConfig.java:-

```
package com.nt.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {
}
```

RootAppConfig.java:-

```
package com.nt.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Import;

@SpringBootApplication
@Import(value={ServiceConfig.class})
public class RootAppConfig {

    public static void main(String[] args) {
        SpringApplication.run(RootAppConfig.class, args);
    }
}
```

ServletInitializer.java:-

```
package com.nt.config;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.support.SpringBootServletInitializer;

public class ServletInitializer extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(
        SpringApplicationBuilder application) {
        return application.sources(new
            Class[]{WebMvcAppConfig.class, RootAppConfig.class});
    }
}
```

WebMvcAppConfig.java:-

```
package com.nt.config;
import java.util.Properties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.HandlerMapping;
import org.springframework.web.servlet.handler.SimpleUrlHandlerMapping;
import org.springframework.web.servlet.mvc.Controller;
import org.springframework.web.servlet.mvc.ParameterizableViewController;

@Configuration
@ComponentScan(basePackages="com.nt.controller")
public class WebMvcAppConfig {

    @Bean
    public HandlerMapping createSUHMapping(){
        SimpleUrlHandlerMapping suh=null;
        Properties props=null;

        suh=new SimpleUrlHandlerMapping();
        suh.setOrder(Integer.MAX_VALUE-2);
        props=new Properties();
        props.setProperty("/home.htm", "pvc");
        suh.setMappings(props);
        return suh;
    }

    @Bean(name="pvc")
    public Controller createPVController(){
        ParameterizableViewController pvc=null;
        pvc=new ParameterizableViewController();
        pvc.setViewName("home");
        return pvc;
    }
}
```

ShowLanguagesController.java:-

```
package com.nt.controller;

import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;

import org.springframework.web.bind.annotation.RequestMapping;
```

```
import com.nt.service.LanguageService;

@Controller
public class ShowLanguagesController {
    @Autowired
    private LanguageService service;

    /*@RequestMapping("/home.htm")
    public String showHomePage(){
        return "home";
    }
    */

    @RequestMapping("/languages.htm")
    public String getLanguages(Map<String,Object> map){
        List<String> langList;
        //user Service
        langList=service.getLanguages();
        //add model attribute
        map.put("speakingLangs",langList);
        return "languages";
    }
}
```

LanguageService.java:-

```
package com.nt.service;

import java.util.List;

public interface LanguageService {
    public List<String> getLanguages();
}
```


LanguageServiceImpl.java:-

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import org.springframework.stereotype.Service;
@Service
public class LanguageServiceImpl implements LanguageService {
    @Override
    public List<String> getLanguages() {
        List<String> langList=null;
        Locale locales[]=null;
        //get all available Locales
        locales=Locale.getAvailableLocales();
        langList=new ArrayList();
        for(Locale loc:locales){
            langList.add(loc.getDisplayLanguage());
        }
        return langList;
    }
}
```

application.properties:-

```
#change Embbeded Server port no
server.port=4041
#View Resolver
spring.mvc.view.prefix=/WEB-INF/pages/
spring.mvc.view.suffix=.jsp
```

home.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<h1><a href="/languages.htm">Get All languages</a></h1>
```

languages.jsp:-

```
<%@ page isElIgnored="false" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<c:choose>
  <c:when test="${!empty speakingLangs}">
    <c:forEach var="lang" items="${speakingLangs}">
      <b>${lang}</b> <br>
    </c:forEach>
  </c:when>
  <c:otherwise>
    <b>No data found</b>
  </c:otherwise></c:choose>
<br>
<a href="home.htm">home</a>
```

index.jsp:-

```
<jsp:forward page="home.htm"/>
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>nit</groupId>
    <artifactId>MVCBootLanguageApp36</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>war</packaging>

    <name>MVCBootLanguageApp36</name>
    <description>Demo project for Spring Boot</description>
    <parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>1.4.7.RELEASE</version>
      <relativePath /> <!-- lookup parent from repository -->
    </parent>
    <properties>
```

```
<project.build.sourceEncoding>UTF8
</project.build.sourceEncoding>
<project.reporting.outputEncoding>UTF8</project.reporting.outputEncoding>
<java.version>1.8</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/taglibs/standard -->
  <dependency>
    <groupId>taglibs</groupId>
    <artifactId>standard</artifactId>
    <version>1.1.2</version>
  </dependency>

</dependencies>

<build>
  <plugins>
```

```

<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
</plugins>
</build>
</project>

```

MVCBootListProfilesApp-AC:-

- ✓ MVCBootListProfilesApp37-AC [boot]
 - > Deployment Descriptor: MVCBootListProfilesApp37-AC
 - > Spring Elements
 - > JAX-WS Web Services
 - ✓ Java Resources
 - ✓ src/main/java
 - ✓ com.nt.bo
 - > ProfileBO.java
 - ✓ com.nt.config
 - > AOPConfig.java
 - > PersistenceConfig.java
 - > RootAppConfig.java
 - > ServiceConfig.java
 - > ServletInitializer.java
 - > WebMVCConfig.java
 - ✓ com.nt.controller
 - > ShowProfilesController.java
 - ✓ com.nt.dao
 - > FBProfileDAO.java
 - > FBProfileDAOImpl.java
 - ✓ com.nt.dto
 - > ProfileDTO.java
 - ✓ com.nt.service
 - > FBProfileService.java
 - > FBProfileServiceImpl.java
 - ✓ src/main/resources
 - static
 - templates
 - application.properties
 - > src/test/java
 - static
 - templates
 - application.properties
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ✓ src
 - ✓ main
 - > java
 - > resources
 - ✓ webapp
 - ✓ WEB-INF
 - ✓ pages
 - home.jsp
 - show_profiles.jsp
 - index.jsp
 - > test
 - > target
 - mvnw
 - mvnw.cmd
 - pom.xml

ProfileBO.java:-

```
package com.nt.bo;

public class ProfileBO {
    private int profileId;
    private String profileName;
    private String email;
    private int followers;
    private int friends;
    public int getProfileId() {
        return profileId;
    }
    public void setProfileId(int profileId) {
        this.profileId = profileId;
    }
    public String getProfileName() {
        return profileName;
    }
    public void setProfileName(String profileName) {
        this.profileName = profileName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public int getFollowers() {
        return followers;
    }
    public void setFollowers(int followers) {
        this.followers = followers;
    }
    public int getFriends() {
        return friends;
    }
    public void setFriends(int friends) {
        this.friends = friends;
    }
}
```

RootAppConfig.java:-

```
package com.nt.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Import;

@SpringBootApplication
@Import(value={PersistenceConfig.class,ServiceConfig.class,AOPConfig.class})
public class RootAppConfig {

    public static void main(String[] args) {
        SpringApplication.run(RootAppConfig.class, args);
    }
}
```

ProfileDTO.java:-

```
package com.nt.dto;

import java.io.Serializable;

public class ProfileDTO implements Serializable {
    private int profileId;
    private String profileName;
    private String email;
    private int followers;
    private int friends;
    public int getProfileId() {
        return profileId;
    }
    public void setProfileId(int profileId) {
        this.profileId = profileId;
    }
    public String getProfileName() {
        return profileName;
    }
}
```

```
public void setProfileName(String profileName) {
    this.profileName = profileName;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
public int getFollowers() {
    return followers;
}
public void setFollowers(int followers) {
    this.followers = followers;
}
public int getFriends() {
    return friends;
}
public void setFriends(int friends) {
    this.friends = friends;
}
}
```

AOPConfig.java:-

```
package com.nt.config;

import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;

@Configuration
public class AOPConfig {
    @Autowired
    private DataSource ds;
    /*
    @Bean(name="dsTxMgmr",autowire=Autowire.BY_TYPE)
    public PlatformTransactionManager createDsTxMgmr(){
        return new DataSourceTransactionManager();
    }

    @Bean(name="dsTxMgmr")
    public PlatformTransactionManager createDsTxMgmr(){
        return new DataSourceTransactionManager(ds);
    }
    */
}
```

PersistenceConfig.java:-

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.dao")
public class PersistenceConfig {

}
```

ServiceConfig.java:-

```
package com.nt.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

ServletInitializer.java:-

```
package com.nt.config;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.support.SpringBootServletInitializer;

public class ServletInitializer extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(WebMVCConfig.class, RootAppConfig.class);
    }

}
```

WebMVCConfig.java:-

```
package com.nt.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.controller")
public class WebMVCConfig {

}
```


FBProfileDAO.java:-

```
package com.nt.dao;
import java.util.List;
import com.nt.bo.ProfileBO;
public interface FBProfileDAO {
    public List<ProfileBO> getAllProfiles();
}
```

FBProfileDAOImpl.java:-

```
package com.nt.dao;
import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import com.nt.bo.ProfileBO;
@Repository("profileDAO")
public class FBProfileDAOImpl implements FBProfileDAO {
    private static final String GET_FB_PROFILES="SELECT
PROFILE_ID,PROFILE_NAME,EMAIL,FOLLOWERS,NUMBER_FRIENDS FROM FACEBOOK_PROFILE";
    @Autowired
    private JdbcTemplate jt;
    @Override
    public List<ProfileBO> getAllProfiles() {
List<ProfileBO> listBO=null;
listBO=jt.query(GET_FB_PROFILES,rs->{
    List<ProfileBO> listBOs=null;
    ProfileBO bo=null;
    listBOs=new ArrayList();
    while(rs.next()){
        bo=new ProfileBO();
        bo.setProfileId(rs.getInt(1));
        bo.setProfileName(rs.getString(2));
        bo.setEmail(rs.getString(3));
        bo.setFollowers(rs.getInt(4));
        bo.setFriends(rs.getInt(5));
        listBOs.add(bo);
    }
    System.out.println("FBDAO:"+jt.getDataSource().getClass());
    return listBOs;
});
return listBO;
}
}
```

FBProfileService.java:-

```
package com.nt.service;

import java.util.List;
import com.nt.dto.ProfileDTO;
public interface FBProfileService {
    public List<ProfileDTO> fetchAllProfiles();
}
```

FBProfileServiceImpl.java:-

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.bo.ProfileBO;
import com.nt.dao.FBProfileDAO;
import com.nt.dto.ProfileDTO;
@Service("profileService")
public class FBProfileServiceImpl implements FBProfileService {
    @Autowired
    private FBProfileDAO dao;
    @Override
    @Transactional(propagation=Propagation.REQUIRED,
transactionManager="dsTxMgmr")
    public List<ProfileDTO> fetchAllProfiles() {
        List<ProfileBO> listBO=null;
        final List<ProfileDTO> listDTO=new ArrayList();

        //get All Profiles
        listBO=dao.getAllProfiles();
        //copy ListBO to ListDTO
        listBO.forEach(bo->{
            ProfileDTO dto=null;
            dto=new ProfileDTO();
            BeanUtils.copyProperties(bo,dto);
            listDTO.add(dto);
        });
        return listDTO;
    } //method

} //class
```

application.properties:-

```
#Server Port
server.port=4041
#Configure DataSource
#spring.datasource.type=org.apache.commons.dbcp2.BasicDataSource
#spring.datasource.driver-class-name=com.mysql.jdbc.Driver
#spring.datasource.url=jdbc:mysql://ntsp86db
#spring.datasource.username=root
#spring.datasource.password=root
spring.datasource.jndi-name=java:/comp/env/DsJndi
#Configure ViewResolver
spring.mvc.view.prefix=/WEB-INF/pages/
spring.mvc.view.suffix=.jsp
```

home.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<h1><a href="list_profiles.htm">List Profiles</a></h1>
```

ShowProfilesController.java:-

```
package com.nt.controller;
import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import com.nt.dto.ProfileDTO;
import com.nt.service.FBProfileService;
@Controller
public class ShowProfilesController {
    @Autowired
    private FBProfileService service;
    @RequestMapping("/home.htm")
    public String showHomePage(){
        return "home";
    }
    @RequestMapping("/list_profiles.htm")
    public String getAllProfiles(Map<String, Object> map){
        List<ProfileDTO> listDTO=null;
        //get All Profiles
        listDTO=service.fetchAllProfiles();
        //add model attribute
        map.put("listProfiles", listDTO);
        return "show_profiles";
    }
}
```

show_profiles.jsp:-

```
<%@page isELIgnored="false"%>
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<c:choose>
  <c:when test="${!empty listProfiles}">
    <h1 style="color:red">Profile Details </h1>
    <table border="1" width="100%" height="100%">
      <tr><th>profile_id</th><th>profile_name</th><th>email</th><th>
followers</th><th>friends</th></tr>
      <c:forEach var="dto" items="${listProfiles}">
        <tr>
          <td>${dto.profileId}</td>
          <td>${dto.profileName}</td>
          <td>${dto.email}</td>
          <td>${dto.followers}</td>
          <td>${dto.friends}</td>
        </tr>
      </c:forEach>
    </table>
  </c:when>
  <c:otherwise>
    <h1 style="color:red"><b> No records found</b></h1>
  </c:otherwise>
</c:choose>
<a href="home.htm">home</a>
```

index.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<jsp:forward page="home.htm"/>
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>nit</groupId>
  <artifactId>MVCBootListProfilesApp-AC</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
```

```
<name>MVCBootListProfilesApp-AC</name>
<description>Demo project for Spring Boot</description>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>1.4.7.RELEASE</version>
  <relativePath /> <!-- lookup parent from repository -->
</parent>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
  <java.version>1.8</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
    <exclusions>
      <exclusion>
        <groupId>org.apache.tomcat</groupId>
        <artifactId>tomcat-jdbc</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- https://mvnrepository.com/artifact/taglibs/standard -->
<dependency>
  <groupId>taglibs</groupId>
  <artifactId>standard</artifactId>
  <version>1.1.2</version>
</dependency>

  <!-- https://mvnrepository.com/artifact/org.apache.commons/commons-dbcp2 -->
-->
<dependency>
  <groupId>org.apache.commons</groupId>
```

```
<artifactId>commons-dbcp2</artifactId>
<version>2.1.1</version>
</dependency>

<!-- https://mvnrepository.com/artifact/javax.servlet/jstl -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.tomcat.embed/tomcat-
embed-jasper -->
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <version>9.0.0.M22</version>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>
```

MVCBootInsertApp38-SFC-ORM:-

- ▼ MVCBootInsertApp38-SFC-ORM [boot]
 - > Deployment Descriptor: MVCBootInsertApp38-SFC-ORM
 - > Spring Elements
 - > JAX-WS Web Services
 - ▼ Java Resources
 - ▼ src/main/java
 - ▼ com.nt.command
 - > StudentCommand.java
 - ▼ com.nt.config
 - > AOPConfig.java
 - > PersistenceConfig.java
 - > RootAppConfig.java
 - > ServiceConfig.java
 - > ServletInitializer.java
 - > WebMvcAppConfig.java
 - ▼ com.nt.controller
 - > StudentInsertController.java
 - ▼ com.nt.dao
 - > StudentInsertDAO.java
 - > StudentInsertDAOImpl.java
 - ▼ com.nt.domain
 - > StudentHLO.java
 - ▼ com.nt.dto
 - > StudentDTO.java
 - ▼ com.nt.service
 - > StudentInsertService.java
 - > StudentInsertServiceImpl.java
 - ▼ src/main/resources
 - static
 - templates
 - application.properties
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ▼ src
 - ▼ main
 - > java
 - > resources
 - ▼ webapp
 - ▼ WEB-INF
 - ▼ pages
 - register.jsp
 - index.jsp
 - > test
 - > target
 - mvnw
 - mvnw.cmd
 - pom.xml

StudentCommand.java:-

```
package com.nt.command;
public class StudentCommand {
    private String sname;
    private String sadd;
    public String getSname() {
        return sname;
    }
    public void setName(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

StudentDTO.java:-

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private String sname;
    private String sadd;
    public String getSname() {
        return sname;
    }
    public void setName(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
}
```


StudentHLO.java:-

```
package com.nt.domain;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
```

ServiceConfig.java:-

```
package com.nt.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

AOPConfig.java:-

```
package com.nt.config;

import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate4.HibernateTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;
@Configuration
@EnableTransactionManagement
public class AOPConfig {
    @Autowired
    private SessionFactory sesfact;

    @Bean(name="hbTxMgmr")
    public PlatformTransactionManager createHibernateTransactionManager(){
        return new HibernateTransactionManager(sesfact);
    }
}
```

PersistenceConfig.java:-

```
package com.nt.config;

import javax.sql.DataSource;
import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.orm.hibernate4.HibernateTemplate;
import org.springframework.orm.hibernate4.LocalSessionFactoryBean;
import com.nt.domain.StudentHLO;

@Configuration
@ComponentScan(basePackages="com.nt.dao")
public class PersistenceConfig {
    @Autowired
    private DataSource ds;

    @Bean(autowire=Autowire.BY_TYPE)
    public LocalSessionFactoryBean createLocalSessionFactoryBean(){
        LocalSessionFactoryBean localSesFact=null;
        localSesFact=new LocalSessionFactoryBean();
        //localSesFact.setDataSource(ds);
        localSesFact.setAnnotatedClasses(StudentHLO.class);
        localSesFact.setAnnotatedPackages("com.nt.domain");
        return localSesFact;
    }

    @Bean
    @Primary
    public SessionFactory createSessionFactory(){
        return createLocalSessionFactoryBean().getObject();
    }

    @Bean(autowire=Autowire.BY_TYPE)
    public HibernateTemplate createHibernateTemplate(){
        return new HibernateTemplate();
    }
    /*@Bean
    public LocalSessionFactoryBean createLocalSessionFactoryBean(){
        LocalSessionFactoryBean localSesFact=null;
        localSesFact=new LocalSessionFactoryBean();
        localSesFact.setDataSource(ds);
        localSesFact.setAnnotatedClasses(StudentHLO.class);
        localSesFact.setAnnotatedPackages("com.nt.domain");
        return localSesFact;
    }
    @Bean(autowire=Autowire.BY_TYPE)
```

```
public HibernateTemplate createHibernateTemplate(){
    return new HibernateTemplate();
}*/
}
```

RootAppConfig.java:-

```
package com.nt.config;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Import;

@SpringBootApplication
@Import(value={ServiceConfig.class,PersistenceConfig.class})
public class RootAppConfig {
    public static void main(String[] args) {
        SpringApplication.run(RootAppConfig.class, args);
    }
}
```

ServletInitializer.java:-

```
package com.nt.config;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.support.SpringBootServletInitializer;
public class ServletInitializer extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(new
Class[]{WebMvcAppConfig.class,RootAppConfig.class});
    }
}
```

WebMvcAppConfig.java:-

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
@Configuration
@ComponentScan(basePackages="com.nt.controller")
public class WebMvcAppConfig {
}
}
```

StudentInsertController.java:-

```
package com.nt.controller;

import java.util.Map;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import com.nt.command.StudentCommand;
import com.nt.dto.StudentDTO;
import com.nt.service.StudentInsertService;

@Controller
public class StudentInsertController {
    @Autowired
    private StudentInsertService service;

    @RequestMapping(value="/register.htm",method=RequestMethod.GET)
    public String showHomePage(@ModelAttribute("stCmd") StudentCommand cmd){
        return "register";
    }

    @RequestMapping(value="/register.htm",method=RequestMethod.POST)
    public String processForm(Map<String,Object> map,@ModelAttribute("stCmd")
StudentCommand cmd){
        StudentDTO dto=null;
        String status=null;
        //Convert Cmd to DTO
        dto=new StudentDTO();
        BeanUtils.copyProperties(cmd, dto);
        //use service
        status=service.registerStudent(dto);
        //put result in Model
        map.put("result",status);
        return "register";
    }
}
```

StudentInsertDAO:-

```
package com.nt.dao;

import com.nt.domain.StudentHLO;
public interface StudentInsertDAO {
    public int insert(StudentHLO hlo);
}
```

StudentInsertDAOImpl.java:-

```
package com.nt.dao;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.orm.hibernate4.HibernateTemplate;
import org.springframework.stereotype.Repository;
import com.nt.domain.StudentHLO;

@Repository
public class StudentInsertDAOImpl implements StudentInsertDAO {
    @Autowired
    private HibernateTemplate ht;
    @Override
    public int insert(StudentHLO hlo) {
        int sno=0;
        ht.setCheckWriteOperations(false);
        sno=(int) ht.save(hlo);
        ht.flush();

        return sno;
    }
}
```

StudentInsertService.java:-

```
package com.nt.service;

import com.nt.dto.StudentDTO;

public interface StudentInsertService {
    public String registerStudent(StudentDTO dto);
}
```

StudentInsertServiceImpl:-

```
package com.nt.service;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.nt.dao.StudentInsertDAO;
import com.nt.domain.StudentHLO;
import com.nt.dto.StudentDTO;

@Service
public class StudentInsertServiceImpl implements StudentInsertService {
    @Autowired
    private StudentInsertDAO dao;
    @Override
    //@Transactional(propagation=Propagation.REQUIRED,transactionManager
    ="hbTxMgmr",readOnly=false)
    public String registerStudent(StudentDTO dto) {
        StudentHLO hlo=null;
        int sno=0;
        //convert DTO to HLO
        hlo=new StudentHLO();
        BeanUtils.copyProperties(dto, hlo);
        //use DAO
        sno=dao.insert(hlo);
        return "Student is registred with id ::"+sno;
    }
}
```

application.properties:-

```
#View Resolver Cfg...
spring.mvc.view.prefix=/WEB-INF/pages/
spring.mvc.view.suffix=.jsp

#Server Managed jdbc con pool
spring.datasource.jndi-name=java:/comp/env/DsJndi
```

register.jsp:-

```
<%@ page language="java" isELIgnored="false" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<%@taglib uri="http://www.springframework.org/tags/form" prefix="form"%>
<form:form method="post" commandName="stCmd">
    name :: <form:input path="sname"/><br>
    address :: <form:input path="sadd"/><br>
    <input type="submit" value="register"/>
</form:form>
${result}
```

index.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<jsp:forward page="register.htm"/>
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>MVCBootInsertApp-SFC-ORM</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

  <name>MVCBootInsertApp-SFC-ORM</name>
  <description>Demo project for Spring Boot</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>1.4.7.RELEASE</version>
    <relativePath /> <!-- lookup parent from repository -->
  </parent>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>
```

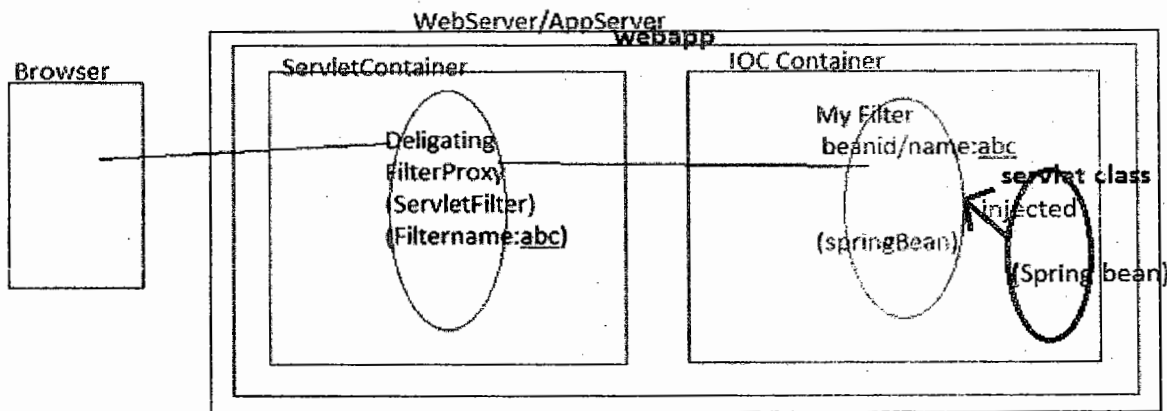
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<!-- https://mvnrepository.com/artifact/org.springframework/spring-orm -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>4.3.11.Final</version>
</dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```


Spring Security

To perform dependency injection on servletFilter component its class must be maintained by IOC container as spring bean having configuration in spring bean configuration file. Due to this it can't take request from client as filter is not configured as web component in web.xml file. To overcome this problem we configure, special proxy filter in web.xml file to take the request from client to pass the request to filter comp that is there, in IOC container as spring bean. The special filter is "org.springframework.web.filter.DelegatingFilterProxy". But we need to match logical name (Given in <filter-name>) of DelegatingFilterProxy with bean name of filter comp managed by IOC container. Diagram:-



Note:- My filter is spring bean here so we can inject other object to MyFilter...

Security (authentication + authorization)

- ⇒ Checking the identity of the user is called authentication (verifying username, password)
- ⇒ Checking the Access permission of the user to utilize the service is called authorization.
- Provide authorization/access permission not based on username, provide them based on roles.
- ⇒ Roles are like designations.
- Checking the identity of user to use banking application is called authentication.

Checking access permission of a user based on his roles to the various services is called authorization.

While implementing security we need the following services.

a) Authentication provider/Authentication info provider

- ⇒ It is the place where we can maintain set of username and passwords that are required for authentication. We can take....
- File system file (Text file, xml file etc....)
- DB software
- LDAP server (light weight Directory Access protocol)
- LDAP server is dedicatedly given to maintain username and password.

- ⇒ The speciality of this server is once you forgot the password you can't get the password but you can reset the password. It is also to maintain username, password that required in single sign

on operations (login once use them for multiple apps)

Eg: Login to gmail and use in youtube.com, googleDrive.com, google+.com, gmail.com and etc..

b) Authentication Manager:

⇒ It is separate comp that is linked servletfilter to verify to given user, password with username, password that are available in authentication provider. If valid allows the request going to the actually requested resource, If not valid shows 401, 403 error pages.

This has to ignore authentication when request is given some resource like home page and has to enable authentication for other resources.

In non spring environment we can implement security in two ways.(Different Security Models)

a) Programatic/Custom security model

Here we need to write code as Authentication manager that takes the request, render dialog boxes asking username, password and Talks to Authentication provider for verification and either allows the request to go to required resource or blocks the request by throwing 403 or 401 error page. Doing this work manually is very complex. So it is not recommended model.

b) Server Managed declarative Security model:

Here authentication manager work, authentication provider work will be taken care by server/container based on the configuration done in web.xml to reduce burden on the programmer but few limitations are there in this approach.

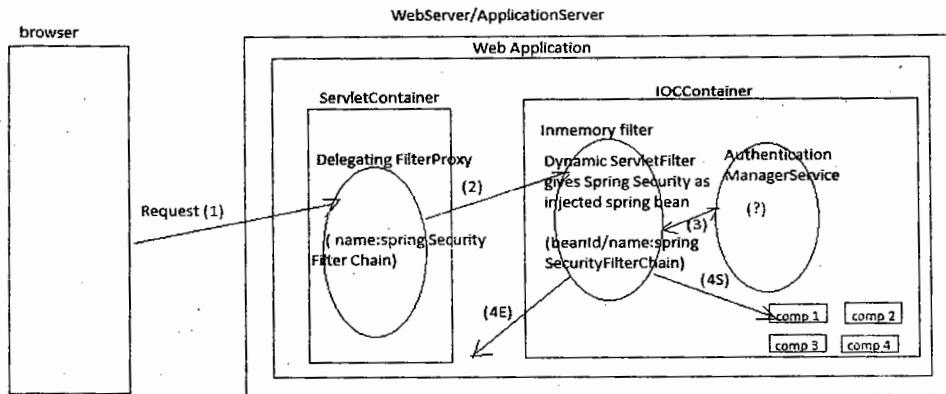
- 1) web.xml entries are not portable accross the multiple servers.
- 2) Some web servers don't supports in this model.
- 3) Going to commercial Application server only for security model is cost effective.
- 4) Some server donot support LDAP server as authentication provider.

⇒ **To overcome this problem use spring security whose earlier name is (Spring acegi security)(**

- 1) It runs in all servers and works irrespective they support security or not.
- 2) Supports all models of authentication providers.
- 3) It is also declarative security model.
- 4) Can be applied on normal web application and spring mvc web application and etc...

Note:

We can configure spring security definitions in springbean configuration file having security namespace, based on this configuration internally one ServletFilter will be generated as spring bean in IOC container having bean name, "springSecurityFilterChain" so to take request from client and to pass to request the above generated filter we need to configure "DelegatingFilterProxy" in web.xml with logical name "springSecurityFilterChain".



comp x -> web comps in normal applications they are Servlet, Jsp components in spring MVC App they are controllers.

- ✓ MVCSpringSecurityApp39
 - > Deployment Descriptor: <web app>
 - ✓ Java Resources
 - src/main/java
 - src/main/resources
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ✓ src
 - main
 - java
 - resources
 - webapp
 - secure
 - admin.jsp
 - faculty.jsp
 - WEB-INF
 - applicationContext.xml
 - web.xml
 - index.jsp
 - logout.jsp
 - test
 - target
 - pom.xml

Jars:- MVC library+spring-security-config-3.0.5 RELEASE.jar, spring-security-core-3.0.5RELEASE.jar, Spring-security-web-3.0.5RELEASE.jar (Gather from spring security separate installer and the ZIP name is:- spring-security-3.0.5RELEASE).

home.jsp

```
<p><a href="secure/Admin.jsp">Admin Page</a></p>
<p><a href="secure/Faculty.jsp">Faculty Page</a></p>
```

logout.jsp

```
You have successfully LoggedOut<br>
<a href="home.jsp">start again</a>
```

admin.jsp

```
<h1>Admin Page</h1>
Welcome Mr:<%=request.getUserPrincipal().getName()%><br><br><br>
Wanna goto Faculty Page <a href="Faculty.jsp">click here</a>
<p><a href="..j_spring_security_logout">Logout</a>
```

faculty.jsp

```
<h1> Faculty Page </h1>
Welcome Mr:<%=request.getUserPrincipal().getName()%><br><br><br>
Wanna goto Admin Page <a href="Admin.jsp">click here</a>
<p><a href="..j_spring_security_logout">Logout</a>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:web="http://xmlns.jcp.org/xml/ns/javaee">
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
  </listener>
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
  </context-param>
  <welcome-file-list>
    <welcome-file>home.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
  </filter>
  <filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>
```

applicationContext.xml

```
<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/security
    http://www.springframework.org/schema/security/spring-security-3.0.3.xsd">
  <http use-expressions="true">
    <intercept-url pattern="/home.jsp" access="permitAll" />
```

```

<intercept-url pattern="/secure/admin.jsp" access="hasRole('admin')" />
<intercept-url pattern="/secure/faculty.jsp" access="hasRole('faculty')" />
<form-login />
<logout />
<remember-me />
<session-management invalid-session-url="/logout.jsp">
  <concurrency-control max-sessions="3" error-if-maximum-exceeded="true" />
</session-management>
</http>
<authentication-manager>
  <authentication-provider>
    <user-service>
      <user name="raja" password="rani" authorities="admin,faculty" />
      <user name="ravi" password="ravi" authorities="faculty" />
    </user-service>
  </authentication-provider>
</authentication-manager>
</beans>

```

- ▼  MVCSpringSecurityApp40
 - >  Deployment Descriptor: <web app>
 - ▼  Java Resources
 - ▼  src/main/java
 - ▼  com.nt.controller
 - >  InboxController.java
 - ▼  src/main/resources
 - ▼  com
 - ▼  nt
 - ▼  commons
 -  security.properties
 - >  src/test/java
 - >  Libraries
 - >  JavaScript Resources
 - >  Deployed Resources
 - ▼  src
 - ▼  main
 - >  java
 - >  resources
 - ▼  webapp
 - ▼  WEB-INF
 - ▼  pages
 -  home.jsp
 -  inbox.jsp
 -  applicationContext.xml
 -  dispatcher-servlet.xml
 -  persistence-config.xml
 -  security-config.xml
 -  web.xml
 -  error.jsp
 -  index.jsp
 - >  test

> target
DbTables.txt
pom.xml

InboxController.java:-

```
package com.nt.controller;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.AbstractController;

public class InboxController extends AbstractController {
    @Override
    public ModelAndView handleRequestInternal(HttpServletRequest arg0,
        HttpServletResponse arg1) throws Exception {
        return new ModelAndView("inbox");
    }
}
```

security.properties:-

```
#username=password,role,status
raja=rani,ROLE_OWNER,enabled
ravi=ravi,ROLE_OWNER,ROLE_ADMIN,enabled
kiran=kiran,ROLE_ADMIN,enabled
ramesh=chari,ROLE_CITIZEN,disabled
```

home.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">

<h1><a href="inbox.htm">InBox</a></h1>
```

inbox.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">

<h1> This is InBox Page Check u r mails slowly.....</h1>

<a href="home.htm">home</a>
<br><br>
<a href="j spring security logout">Logout</a>

</body>
</html>
```

applicationContext.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <import resource="security-config.xml"/>
    <import resource="persistence-config.xml"/>
</beans>
```

dispatcher-servlet.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <bean
class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
            <props>
                <prop key="/home.htm">pvc</prop>
                <prop key="/inbox.htm">ic</prop>
            </props>
        </property>
    </bean>
    <bean id="pvc"
class="org.springframework.web.servlet.mvc.ParameterizableViewController">
        <property name="viewName" value="home"/>
    </bean>

    <bean id="ic" class="com.nt.controller.InboxController"/>

    <!-- View Resolver cfg -->
    <bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/pages/" />
        <property name="suffix" value=".jsp" />
    </bean>
</beans>
```

persistence-config.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="jofb"
class="org.springframework.jndi.JndiObjectFactoryBean">
    <property name="jndiName" value="java:/comp/env/DsJndi"/>
    </bean>

</beans>

```

security-config.xml:-

```

<beans:beans xmlns="http://www.springframework.org/schema/security"
xmlns:beans="http://www.springframework.org/schema/beans"
xmlns:security="http://www.springframework.org/schema/security"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-
beans.xsd
http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security-3.2.xsd">

<security:http use-expressions="true">
    <security:intercept-url pattern="/home.htm"
        access="permitAll" />
    <security:intercept-url pattern="/inbox.htm"
        access="hasRole('ROLE_OWNER')" />
    <security:form-login />
    <security:access-denied-handler
        error-page="/error.jsp" />
    <security:logout logout-success-url="/home.htm" />

<security:authentication-manager>
    <security:authentication-provider>

<!-- <security:user-service>
    <security:user name="raja" password="rani"
authorities="ROLE_OWNER" />
    <security:user name="kiran" password="kiran"
authorities="ROLE_ADMIN" />

    <security:user name="ravi" password="ravi"
authorities="ROLE_OWNER,ROLE_ADMIN"/>
    <security:user name="ramesh" password="chari"
authorities="ROLE_OWNER" disabled="true"/>

```



```
<security:user name="nana" password="military"
authorities="ROLE_OWNER" locked="true"/>

</security:user-service>
-->
<!-- For Properties file Auth Provider -->
<!--
    <security:user-service
properties="classpath:com/nt/commons/security.properties"/>
-->

    <jdbc-user-service
        data-source-ref="jofb"
        users-by-username-query="select uname,pwd,status from users
where uname=?"
        authorities-by-username-query="select uname,role from
users_roles where uname=?" />
    </security:authentication-provider>
</security:authentication-manager>
</beans:beans>
```

web.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">

    <listener>
        <listener-
class>org.springframework.web.context.ContextLoaderListener</listener-
class>
        </listener>
    <context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/applicationContext.xml</param-value>
    </context-param>

    <servlet>
        <servlet-name>dispatcher</servlet-name>
        <servlet-
```

```
class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>2</load-on-startup>
</servlet>
  <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
  </servlet-mapping>

  <filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-
class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
  </filter>
  <filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
</web-app>
```

index.jsp:-

```
<jsp:forward page="home.htm"/>
```

error.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<h1 style="color:red">Check U r Roles</h1>

<a href="home.htm">home</a>
<br><br>
<a href="j_spring_security_logout">Logout</a>
```

DbTables.txt:-

db tables

SQL> select * from users;

UNAME(PK)	PWD	STATUS
<u>ramesh</u>	<u>chari</u>	1
<u>raja</u>	<u>rani</u>	1
<u>ravi</u>	<u>hyd</u>	1
<u>jani</u>	<u>jani</u>	1

SQL> select * from users_roles;

ROLE_ID	UNAME(FK)	ROLE
5	<u>ramesh</u>	ROLE_FAMILY_MEMBER
1	<u>raja</u>	ROLE_OWNER
2	<u>ravi</u>	ROLE_ADMIN
3	<u>jani</u>	ROLE_ADMIN
4	<u>jani</u>	ROLE_OWNER

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>nit</groupId>
  <artifactId>MVCWishApp-PVC3</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>MVCWishApp-PVC3 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>

<groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
  <!--
```

```
https://mvnrepository.com/artifact/org.springframework/spring-context -
->
```

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>4.2.2.RELEASE</version>
</dependency>
<!--
https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context-support</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework/spring-webmvc --
>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-config -->
  <dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-config</artifactId>
    <version>3.2.3.RELEASE</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <dependency>
    <groupId>nit.oracle</groupId>
    <artifactId>ojdbc6</artifactId>
    <version>11.2</version>
  </dependency>

  <!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-web -->
  <dependency>
```

```

        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-web</artifactId>
        <version>3.2.3.RELEASE</version>
    </dependency>
    <!--
https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
    <dependency>
        <groupId>javax.servlet</groupId>
        <artifactId>javax.servlet-api</artifactId>
        <version>3.1.0</version>
    </dependency>
    <!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-core -->
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-core</artifactId>
        <version>3.2.3.RELEASE</version>
    </dependency>
</dependencies>
<build>
    <finalName>MVCWishApp-PVC3</finalName>
</build>
</project>

```

MVCSpringSecurityApp41-XML+Annotations
 Deployment Descriptor: <web app>
 Java Resources
 src/main/java
 com.nt.controller
 src/main/resources
 com
 nt
 commons
 security.properties
 src/test/java
 Libraries
 JavaScript Resources
 Deployed Resources
 src
 main
 java
 resources
 webapp
 WEB-INF
 pages
 home.jsp
 inbox.jsp
 applicationContext.xml
 dispatcher-servlet.xml
 persistence-config.xml
 security-config.xml
 web.xml
 error.jsp
 index.jsp
 test
 target
 DbTables.txt
 pom.xml

InboxController.java:-

```
package com.nt.controller;

import java.util.Map;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class InboxController {

    @RequestMapping("/home.htm")
    public String showHomePage() {
        return "home";
    }

    @RequestMapping("/inbox.htm")
    public String showInbox(Map<String, Object> map){
        return "inbox";
    }

}
```

security.properties:-

```
#username=password,role,status
raja=rani,ROLE_OWNER,enabled
ravi=ravi,ROLE_OWNER,ROLE_ADMIN,enabled
kiran=kiran,ROLE_ADMIN,enabled
ramesh=chari,ROLE_CITIZEN,disabled
```

home.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<h1><a href="inbox.htm">InBox</a></h1>
```

inbox.jsp:-

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<h1> This is InBox Page Check u r mails slowly.....</h1>

<a href="home.htm">home</a>
<br><br>
<a href="j_spring_security_logout">Logout</a>
```

applicationContext.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <import resource="security-config.xml"/>
    <import resource="persistence-config.xml"/>
</beans>
```

dispatcher-servlet.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <!-- View Resolver cfg -->
    <bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver"
>
        <property name="prefix" value="/WEB-INF/pages/" />
        <property name="suffix" value=".jsp" />
    </bean>
    <context:component-scan base-package="com.nt.controller" />

</beans>
```

persistence-config.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="jofb"
class="org.springframework.jndi.JndiObjectFactoryBean">
        <property name="jndiName" value="java:/comp/env/DsJndi" />
    </bean>

</beans>
```

security-config.xml:-

```

<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:security="http://www.springframework.org/schema/security"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-
beans.xsd
    http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security-3.2.xsd">

  <security:http use-expressions="true">
    <security:intercept-url pattern="/home.htm"
      access="permitALL" />
    <security:intercept-url pattern="/inbox.htm"
      access="hasRole('ROLE_OWNER')" />
    <security:form-login />
    <security:access-denied-handler
      error-page="/error.jsp" />
    <security:logout logout-success-url="/home.htm" />
  </security:http>

  <security:authentication-manager>
    <security:authentication-provider>

<!--    <security:user-service>
      <security:user name="raja" password="rani"
authorities="ROLE_OWNER" />
      <security:user name="kiran" password="kiran"
authorities="ROLE_ADMIN" />
      <security:user name="ravi" password="ravi"
authorities="ROLE_OWNER,ROLE_ADMIN"/>
      <security:user name="ramesh" password="chari"
authorities="ROLE_OWNER" disabled="true"/>
      <security:user name="nana" password="milatary"
authorities="ROLE_OWNER" locked="true"/>

    </security:user-service>
  -->

  <!-- For Properties file Auth Provider -->
<!--    <security:user-service
properties="classpath:com/nt/commons/security.properties"/>
  -->

  <jdbc-user-service
    data-source-ref="jofb"
    users-by-username-query="select uname,pwd,status from users
where uname=?"
    authorities-by-username-query="select uname,role from

```



```

users_roles where uname=?" />
    </security:authentication-provider>
</security:authentication-manager>
</beans:beans>

```

web.xml:-

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:web="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
    <listener>
    <listener-
class>org.springframework.web.context.ContextLoaderListener</listener-
class>
    </listener>
    <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
    </context-param>
    <servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-
class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>2</load-on-startup>
    </servlet>
    <servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
    </servlet-mapping>
    <filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-
class>org.springframework.web.filter.DelegatingFilterProxy</filter-
class>
    </filter>
    <filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
    </filter-mapping>

```

error.jsp:-

```

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<h1 style="color:red">Check U r Roles</h1>
<a href="home.htm">home</a>
<br><br>
<a href="j_spring_security_logout">Logout</a>

```

index.jsp:-

```
<jsp:forward page="home.htm"/>
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>nit</groupId>
  <artifactId>MVCWishApp-PVC3</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>MVCWishApp-PVC3 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!--
https://mvnrepository.com/artifact/org.springframework/spring-context -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>4.2.2.RELEASE</version>
    </dependency>
    <!--
https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context-support</artifactId>
      <version>4.2.2.RELEASE</version>
    </dependency>
    <!--
https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>4.2.2.RELEASE</version>
    </dependency>
    <!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-config -->
```

```
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-config</artifactId>
            <version>3.2.3.RELEASE</version>
        </dependency>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>4.2.2.RELEASE</version>
</dependency>

<dependency>
    <groupId>nit.oracle</groupId>
    <artifactId>ojdbc6</artifactId>
    <version>11.2</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-web -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-web</artifactId>
    <version>3.2.3.RELEASE</version>
</dependency>
<!--
https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
</dependency>
<!--
https://mvnrepository.com/artifact/org.springframework.security/spring-
security-core -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-core</artifactId>
    <version>3.2.3.RELEASE</version>
</dependency>

</dependencies>
<build>
    <finalName>MVCWishApp-PVC3</finalName>
</build>
</project>
```

Spring Security Example Using Spring MVC

While working with 100% java code based spring MVC application we can write method in configuration class to get DataSource object from underlying servers IndiRegistry as shown below.

```
@Bean
public static DataSource create IndiDataSource(){
    IndiDataSourceLookup dsLookup=null;
    DataSource ds=null;
    dsLookup=new IndiDataSourceLookup();
    ds=dsLookup.getDataSource("java:/comp/env/DsIndi");
    return ds;
}
```

100% Code Based Spring Security:

Note:

While developing 100% code based spring security

- (a) Remaining 100% configurations are same as any 100% code SpringMVCApplcation.
- (b) Develop additional web application initializer class that registers delegating filter proxy with servlet container as shown below.

public class MysecurityIntializer extends

Abstract Security WebApplicationIntializer {

}

- (c) Develop security config class & link with RootConfigclass

eg:

@Configuration

@EnableWebMVCSecurity

public class SecurityConfig extends WebSecurityConfigurerAdapter {

@Resource (name="jofb")

private DataSource ds;

MVCSecurityApp42-100pCode

- ▼ MVCSpringSecurityApp42-100PCode
 - ▼ Java Resources
 - ▼ src/main/java
 - ▼ com.nt.config
 - > PersistenceConfig.java
 - > RootAppConfig.java
 - > SecurityConfig.java
 - > WebMvcAppConfig.java
 - ▼ com.nt.controller
 - > InboxController.java
 - ▼ com.nt.initializer
 - > MySecurityInitializer.java
 - > MyWebApplicationInitializer.java
 - > src/main/resources
 - > src/test/java
 - > Libraries
 - > JavaScript Resources
 - > Deployed Resources
 - ▼ src
 - ▼ main
 - > java
 - > resources
 - ▼ webapp
 - ▼ WEB-INF
 - ▼ pages
 - home.jsp
 - inbox.jsp
 - error.jsp
 - index.jsp
 - > test
 - > target
 - DbTables.txt
 - pom.xml

MyWebApplicationInitializer.java:-

```
package com.nt.initializer;

import
org.springframework.web.servlet.support.AbstractAnnotationConfigDispatche
rServletInitializer;
import com.nt.config.RootAppConfig;
import com.nt.config.WebMvcAppConfig;

public class MyWebApplicationInitializer extends
AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[]{RootAppConfig.class};
    }

    @Override
```

```
protected Class<?>[] getServletConfigClasses() {  
    return new Class[]{WebMvcAppConfig.class};  
}  
  
@Override  
protected String[] getServletMappings() {  
    return new String[]{"*.htm"};  
}  
}
```

MySecurityInitializer.java

```
package com.nt.initializers;  
import org.springframework.security.web.context.AbstractSecurityWebApplicationInitializer;  
public class WebAppSecurityInitializer extends AbstractSecurityWebApplicationInitializer {  
  
}
```

InboxController.java:-

```
package com.nt.controller;  
  
import java.util.Map;  
import org.springframework.stereotype.Controller;  
import org.springframework.web.bind.annotation.RequestMapping;  
  
@Controller  
public class InboxController {  
  
    @RequestMapping("/home.htm")  
    public String showHomePage() {  
        return "home";  
    }  
  
    @RequestMapping("/inbox.htm")  
    public String showInbox(Map<String, Object> map){  
        return "inbox";  
    }  
  
}
```

PersistenceConfig.java

```
package com.nt.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.jndi.JndiObjectFactoryBean;  
  
@Configuration  
public class PersistenceConfig {  
  
    @Bean(name="jofb")
```

7. `Connection#hasExpired()` would return `false`.
8. `Connection#refresh()` would not do anything since connections to Twitter do not expire.
9. `Connection#fetchUserProfile()` would make a remote API call to Twitter to get @jbauer's profile data and normalize it into a `UserProfile` model.
10. `Connection#updateStatus(String)` would post a status update to @jbauer's timeline.
11. `Connection#getApi()` would return a Twitter giving the client application access to the full capabilities of Twitter's native API.
12. `Connection#createData()` would return `ConnectionData` that could be serialized and used to restore the connection at a later time.

Establishing connections

So far we have discussed how existing connections are modeled, but we have not yet discussed how new connections are established. The manner in which connections between local users and provider users are established varies based on the authorization protocol used by the `ServiceProvider`. Some service providers use OAuth, others use HTTP Basic Auth, others may use something else. Spring Social currently provides native support for OAuth-based service providers, including support for OAuth 1 and OAuth 2. This covers the leading social networks, such as Facebook and Twitter, all of which use OAuth to secure their APIs. Support for other authorization protocols can be added by extending the framework.

Each authorization protocol is treated as an implementation detail where protocol-specifics are kept out of the core Connection API. A `ConnectionFactory` abstraction encapsulates the construction of connections that use a specific authorization protocol. In the following sections, we will discuss the major `ConnectionFactory` classes provided by the framework. Each section will also describe the protocol-specific flow required to establish a new connection.

After signing in, the flow will redirect back to the application to complete the sign in process.

Signing up after a failed sign in

If `ProviderSignInController` can't find a local user associated with a provider user attempting to sign in, there may be an opportunity to have the user sign up with the application. Leveraging the information about the user received from the provider, the user may be presented with a pre-filled sign up form to explicitly sign up with the application. It's also possible to use the user's provider data to implicitly create a new local application user without presenting a sign up form.

Signing up with a sign up form

By default, the sign up URL is `"/signup"`, relative to the application root. You can override that default by setting the `signUpUrl` property on the controller. For example, the following configuration of `ProviderSignInController` sets the sign up URL to `"/register"`:

@Bean

```
publicProviderSignInControllerproviderSignInController(  
    ConnectionFactoryLocator connectionFactoryLocator,  
    UsersConnectionRepository usersConnectionRepository){  
    ProviderSignInController controller =newProviderSignInController(  
        connectionFactoryLocator,  
        usersConnectionRepository(),  
        newSimpleSignInAdapter(newHttpSessionRequestCache()));  
    controller.setSignUpUrl("/register");  
    return controller;  
}
```

Or to set the sign up URL using XML configuration:

```
<beanclass="org.springframework.social.connect.web.ProviderSignInController">  
    <propertyname="signUpUrl"value="/register"/>  
</bean>
```

Before redirecting to the sign up page, `ProviderSignInController` collects some information about the authentication attempt. This information can be used to prepopulate the sign up form and then, after successful sign up, to establish a connection between the new account and the provider account.

To prepopulate the sign up form, you can fetch the user profile data from a connection retrieved from `ProviderSignInUtils#getConnection()`. For example, consider this Spring MVC controller method that setups up the sign up form with a `SignUpForm` to bind to the sign up form:

@Autowired

ProviderSignInUtils providerSignInUtils;

@RequestMapping(value="/signup", method=RequestMethod.GET)

```
publicSignUpFormsignupForm(WebRequest request){
```



```
Connection<?> connection = providerSignInUtils.getConnection(request);  
if(connection !=null){  
    returnSignUpForm.fromProviderUser(connection.fetchUserProfile());  
}else{  
    returnnewSignUpForm();  
}  
}
```

If `ProviderSignInUtils#getConnection()` returns a connection, that means there was a failed provider sign in attempt that can be completed if the user registers to the application. In that case, a `SignUpForm` object is created from the user profile data obtained from the connection's `fetchUserProfile()` method. Within `fromProviderUser()`, the `SignUpForm` properties may be set like this:

```
publicstaticSignUpFormfromProviderUser(UserProfile providerUser){  
    SignUpForm form =newSignUpForm();  
    form.setFirstName(providerUser.getFirstName());  
    form.setLastName(providerUser.getLastName());  
    form.setUsername(providerUser.getUsername());  
    form.setEmail(providerUser.getEmail());  
    return form;  
}
```

Here, the `SignUpForm` is created with the user's first name, last name, username, and email from the `UserProfile`. In addition, `UserProfile` also has a `getName()` method which will return the user's full name as given by the provider.

The availability of `UserProfile`'s properties will depend on the provider. Twitter, for example, does not provide a user's email address, so the `getEmail()` method will always return null after a sign in attempt with Twitter.

After the user has successfully signed up in your application a connection can be created between the new local user account and their provider account. To complete the connection call `ProviderSignInUtils#doPostSignUp()`. For example, the following method handles the sign up form submission, creates an account and then calls `ProviderSignInUtils#doPostSignUp()` to

complete the connection:

```
@RequestMapping(value="/signup", method=RequestMethod.POST)
public String signup(@Valid SignupForm form, BindingResult formBinding, WebRequest request){
    if(formBinding.hasErrors()){
        return null;
    }
    Account account = createAccount(form, formBinding);
    if(account != null){
        SignInUtils.signin(account.getUsername());
        ProviderSignInUtils.doPostSignUp(account.getUsername(), request);
        return "redirect:/";
    }
    return null;
}
```

Implicit sign up

To enable implicit sign up, you must create an implementation of the `ConnectionSignUp` interface and inject an instance of that `ConnectionSignUp` to the connection repository.

The `ConnectionSignUp` interface is simple, with only a single method to implement:

```
public interface ConnectionSignUp{
    String execute(Connection<?> connection);
}
```

The `execute()` method is given a `Connection` that it can use to retrieve information about the user. It can then use that information to create a new local application user and return the new local user ID. For example, the following implementation fetches the user's provider profile and uses it to create a new account:

```
public class AccountConnectionSignUp implements ConnectionSignUp{
```

```
private final AccountRepository accountRepository;

public AccountConnectionSignUp(AccountRepository accountRepository){
    this.accountRepository = accountRepository;
}

public String execute(Connection<?> connection){
    UserProfile profile = connection.fetchUserProfile();
    Account account = new Account(profile.getUsername(), profile.getFirstName(),
    profile.getLastName());
    accountRepository.createAccount(account);
    return account.getUsername();
}
}
```

If there is any problem in creating the new user implicitly (for example, if the implicitly chosen username is already taken) `execute()` may return null to indicate that the user could not be created implicitly. This will ultimately result in `ProviderSignInController` redirecting the user to the signup page.

Once you've written a `ConnectionSignUp` for your application, you'll need to inject it into the `UsersConnectionRepository`. In Java-based configuration:

```
@Bean
@Scope(value="singleton", proxyMode=ScopedProxyMode.INTERFACES)
public UsersConnectionRepository usersConnectionRepository(AccountRepository
accountRepository){
    JdbcUsersConnectionRepository repository = new JdbcUsersConnectionRepository(
        dataSource, connectionFactoryLocator(), Encryptors.noOpText());
    repository.setConnectionSignUp(new AccountConnectionSignUp(accountRepository));
}
```

```
return repository;
```

```
}
```

Adding Support for a NewService Provider

Spring Social makes it easy to add support for service providers that are not already supported by the framework. If you review the existing client modules, such as `spring-social-twitter` and `spring-social-facebook`, you will discover they are implemented in a consistent manner and they apply a set of well-defined extension points. In this chapter, you will learn how to add support for new service providers you wish to integrate into your applications.

The process of adding support for a new service provider consists of several steps:

1. Create a source project for the client code e.g. `spring-social-twitter`.
2. Develop or integrate a Java binding to the provider's API e.g. `Twitter`.
3. Create a `ServiceProvider` model that allows users to authorize with the remote provider and obtain authorized API instances e.g. `TwitterServiceProvider`.
4. Create an `ApiAdapter` that maps the provider's native API onto the uniform Connection model e.g. `TwitterAdapter`.
5. Finally, create a `ConnectionFactory` that wraps the other artifacts up and provides a simple interface for establishing connections e.g. `TwitterConnectionFactory`.

The following sections of this chapter walk you through each of the steps with examples.

Creating a source project for the provider client code

A Spring Social client module is a standard Java project that builds a single jar artifact e.g. `spring-social-twitter.jar`. We recommend the code structure of a client module follow the guidelines described below.

Code structure guidelines

We recommend the code for a new Spring Social client module reside within the `org.springframework.social.{providerId}` base package, where `{providerId}` is a unique identifier you assign to the service provider you are adding support for. Consider some of the providers already supported by the framework as examples:

Table 3. Spring Social Client Modules

Provider ID	Artifact Name	Base Package
Facebook	<code>spring-social-facebook</code>	<code>org.springframework.social.facebook</code>

Table 3. Spring Social Client Modules

Provider ID	Artifact Name	Base Package
Twitter	spring-social-twitter	org.springframework.social.twitter
Linkedin	spring-social-linkedin	org.springframework.social.linkedin

Within the base package, we recommend the following subpackage structure:

Table 4. Module Structure

Subpackage	Description
Api	The public interface that defines the API binding.
api.impl	The implementation of the API binding.
Connect	The types necessary to establish connections to the service provider.

You can see this recommended structure in action by reviewing one of the other client modules such as spring-social-twitter:

Here, the central service API type, Twitter, is located in the api package along with its supporting operations types and data transfer object types. The primary implementation of that interface, TwitterTemplate, is located in the api.impl package (along with other package-private impl types have that been excluded from this view). Finally, the connect package contains the implementations of various connect SPIs that enable connections to Twitter to be established and persisted.

How to access facebook data:-

Accessing Facebook Data:

This guide walks you through the process of creating a simple web application that accesses Facebook data.

Enable Facebook

Before you can fetch a user's data from Facebook, you must specify your application's ID and secret by setting the `spring.social.facebook.appId` and `spring.social.facebook.appSecret` properties. You can set these via any means supported by Spring Boot, including setting them in an `application.properties` file:

```
src/main/resources/application.properties
```

```
spring.social.facebook.appId=233668646673605
```

```
spring.social.facebook.appSecret=33b17e044ee6a4fa383f46ec6e28ea1d
```

As shown here, the properties have fake values. The values given to these properties correspond to your application's consumer key and secret you obtain when you register the application with Facebook. For the code to work, substitute the real values given to you by Facebook in place of the fake values.

The presence of these properties and Spring Social Facebook in the classpath will trigger automatic configuration of Spring Social's `ConnectController`, `FacebookConnectionFactory`, and other components of Spring Social's connection framework.

Create connection status views

Although much of what `ConnectController` does involves redirecting to Facebook and handling a redirect from Facebook, it also shows connection status when a GET request to `/connect` is made. It defers to a view named `connect/{providerId}Connect` when no existing connection is available and to `connect/{providerId}Connected` when a connection exists for the provider. In this case, **provider ID** is "facebook".

`ConnectController` does not define its own connection views, so you need to create them. First, here's a Thymeleaf view to be shown when no connection to Facebook exists:

```
src/main/resources/templates/connect/facebookConnect.html
```

The form on this view will POST to `/connect/facebook`, which is handled by `ConnectController` and will kick off the OAuth authorization code flow.

Here's the view to be displayed when a connection exists:

```
src/main/resources/templates/connect/facebookConnected.html
```

Fetch Facebook data

With Facebook configured in your application, you now can write a Spring MVC controller that fetches data for the user who authorized the application and presents it in the browser. `HelloController` is just such a controller:

src/main/java/hello/HelloController.java

HelloController is created by injecting a Facebook object into its constructor. The Facebook object is a reference to Spring Social's Facebook API binding.

The helloFacebook() method is annotated with @RequestMapping to indicate that it should handle GET requests for the root path (/). The first thing the method does is check whether the user has authorized the application to access the user's Facebook data. If not, the user is redirected to ConnectController with the option to kick off the authorization process.

If the user has authorized the application to access Facebook data, the application fetches the user's profile as well as several of the most recent entries in the user's feed. The data is placed into the model to be displayed by the view identified as "hello".

Speaking of the "hello" view, here it is as a Thymeleaf template:

src/main/resources/templates/hello.html

Make the application executable

Although it is possible to package this service as a traditional *web application archive* or WARfile for deployment to an external application server, the simpler approach demonstrated below creates a *standalone application*. You package everything in a single, executable JAR file, driven by a good old Java main() method. And along the way, you use Spring's support for embedding the Tomcat servlet container as the HTTP runtime, instead of deploying to an external instance.

src/main/java/hello/FaceBookBootApplication.java

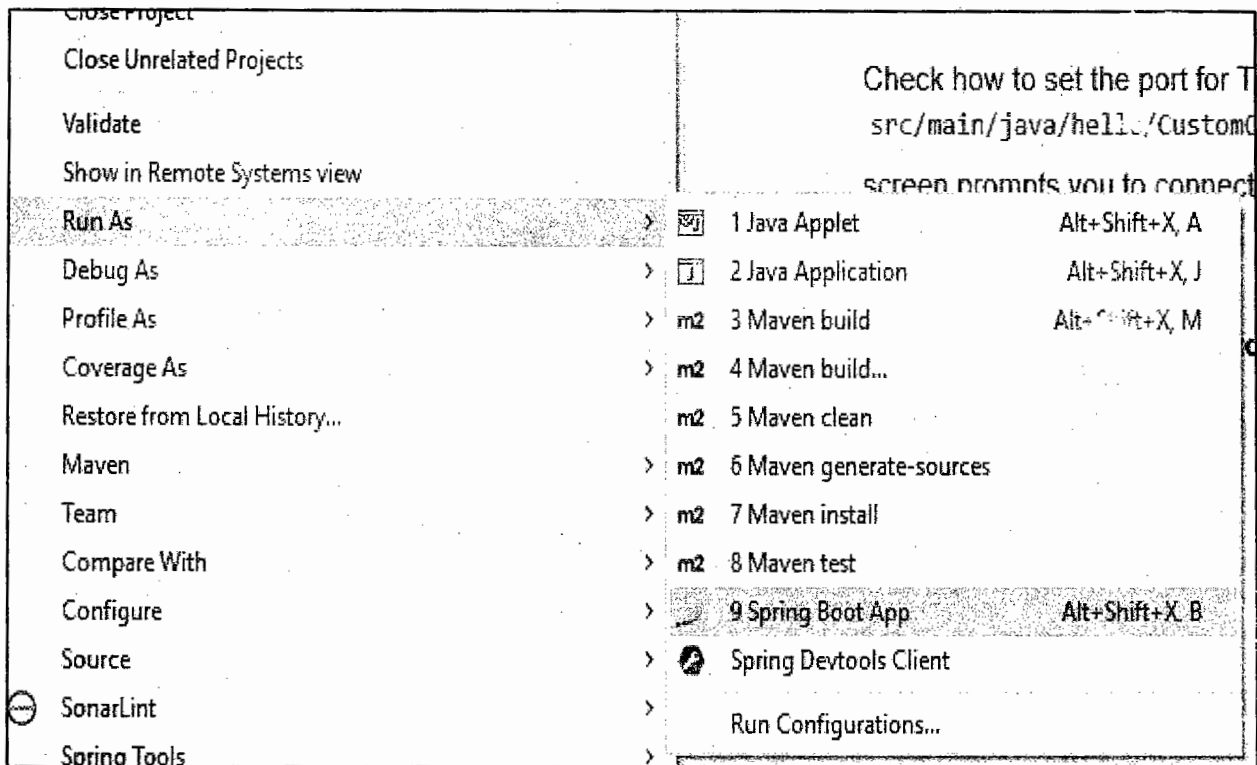
@SpringBootApplication is a convenience annotation that adds all of the following:

- @Configuration tags the class as a source of bean definitions for the application context.
- @EnableAutoConfiguration tells Spring Boot to start adding beans based on classpath settings, other beans, and various property settings.
- Normally you would add @EnableWebMvc for a Spring MVC app, but Spring Boot adds it automatically when it sees **spring-webmvc** on the classpath. This flags the application as a web application and activates key behaviors such as setting up a DispatcherServlet.
- @ComponentScan tells Spring to look for other components, configurations, and services in the hello package, allowing it to find the controllers.

The `main()` method uses Spring Boot's `SpringApplication.run()` method to launch an application. Did you notice that there wasn't a single line of XML? No `web.xml` file either. This web application is 100% pure Java and you didn't have to deal with configuring any plumbing or infrastructure.

Check how to set the port for Tomcat in Spring boot app in
`src/main/java/hello/CustomContainer.java`

Run the App from Eclipse (Run As → Spring Boot App)



Once the application starts up,

```
: Mapped "{[/connect],methods=[GET]}" onto public java.lang.String org.spring
: Mapped "{[/connect/{providerId}],methods=[GET]}" onto public java.lang.Str
: Mapped "{[/connect/{providerId}],methods=[DELETE]}" onto public org.spring
: Mapped "{[/connect/{providerId}],methods=[GET],params=[error]}" onto publi
: Mapped URL path [/webjars/**] onto handler of type [class org.springframework
: Mapped URL path [/**] onto handler of type [class org.springframework.web..
: Mapped URL path [/**/favicon.ico] onto handler of type [class org.springfr
: Registering beans for JMX exposure on startup
: Tomcat started on port(s): 9191 (http)
: Started FaceBookBootApplication in 5.479 seconds (JVM running for 7.039)
```

point your web browser to <http://localhost:9191> .No connection is established yet, so this screen

prompts you to connect with Facebook:



Connect to Facebook

You aren't connected to Facebook yet. Click the button to connect this application with your Facebook account.

[Connect to Facebook](#)

When you click the **Connect to Facebook** button, the browser is redirected to Facebook for authorization:



Log in to Facebook

Email address or phone number

Password

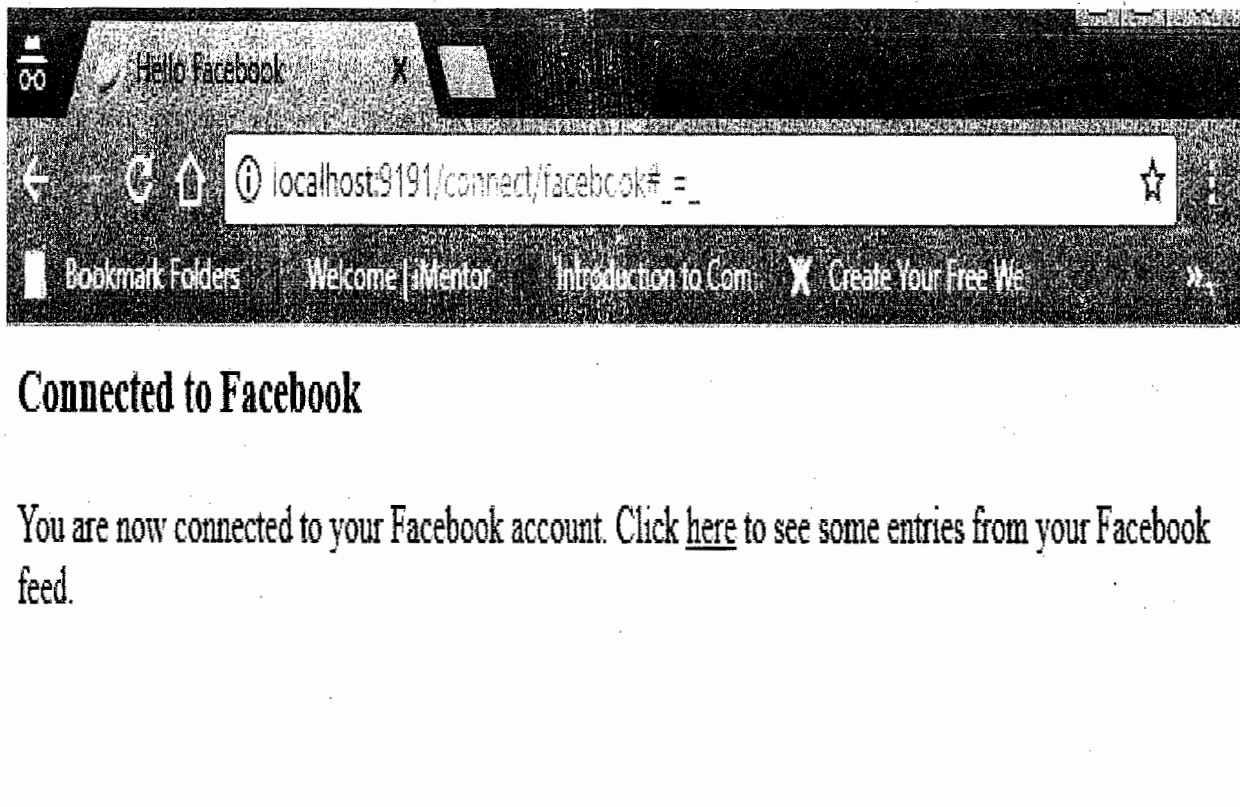
[Log In](#)

[Forgotten account?](#) · [Sign up for Facebook](#)

[Not now](#)

Click "Okay" to grant permission for the sample application to access your public profile and your feed.

Once permission is granted, Facebook redirects the browser back to the application. A connection is created and stored in the connection repository. You should see this page indicating that a connection was successful:



Click the link on the connection status page, and you are taken to the home page. This time, now that a connection has been created, you see your name on Facebook and a list of some recent entries in your home feed. Included for each post in the feed list is the name of the sender, the post's message, and the post's picture (if available).



Hello, Anurag Kumar!

Here is your feed:

Anurag Kumar wrote:

Santosh Sundar Ray & Umesh Yadav ...Lets Go 😊😊



Anurag Kumar wrote:

#boycott_Chinese_products



Anurag Kumar wrote:

Still, Remember This Day Tried Multiple Image Capturing from Dell Cam



Summary

Congratulations! You have developed a simple web application that obtains user authorization to fetch data from Facebook. The application connects the user to Facebook through Spring Social, retrieves data from the user's Facebook profile, and also fetches profile data from the user's Facebook friends.

Creating facebook login Application in facebook:-

Login to facebook account-> www.Developer.facebook.com -> create application -> Displayname:project name ,Contact Email: -----,go to dashboard setting gather AppId ,AppSecretId(to use them in application.properties) -> Add product -> facebooklogin ->others -> enableclientOAuthlogin, enablewebOAuthlogin, type following url where we are planning to run our application as valid OAuth,Callback URL's(for callback use https).

FaceBookBoot:-

- ✓  FaceBookBoot [boot]
 - >  Spring Elements
 - ✓  src/main/java
 - ✓  com.nt.controller
 - >  HelloController.java
 - ✓  com.spring.social.fb
 - >  FaceBookBootApplication.java
 -  READ_ME.docx
 - ✓  src/main/resources
 - ✓  templates
 - ✓  connect
 -  facebookConnect.html
 -  facebookConnected.html
 -  hello.html
 -  application.properties
 -  JRE System Library [JavaSE-1.8]
 -  Maven Dependencies
 - >  src
 - >  target
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

HelloController.java:-

```
package com.nt.controller;
```

```
import org.springframework.social.connect.ConnectionRepository;
import org.springframework.social.facebook.api.Facebook;
import org.springframework.social.facebook.api.PagedList;
import org.springframework.social.facebook.api.Post;
import org.springframework.social.facebook.api.User;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
```

```
@Controller
```

```
@RequestMapping("/")
```

```
public class HelloController {
```

```
    private Facebook facebook;
```

```
    private ConnectionRepository connectionRepository;
```

```
    public HelloController(Facebook facebook, ConnectionRepository connectionRepository) {
```

```
        this.facebook = facebook;
```

```
        this.connectionRepository = connectionRepository;
```

```
    }
```

```
@GetMapping
public String helloFacebook(Model model) {
    if (connectionRepository.findPrimaryConnection(Facebook.class) == null) {
        return "redirect:/connect/facebook";
    }
    /*
    model.addAttribute("facebookProfile", facebook.userOperations().getUserProfile());
    PagedList<Post> feed = facebook.feedOperations().getFeed();
    model.addAttribute("feed", feed);
    return "hello";*/
    String [] fields = {
        "id","name","birthday","email","location","hometown","gender","first_name","last_name";
    }
    User user = facebook.fetchObject("me", User.class, fields);
    String name=user.getName();
    String birthday=user.getBirthday();
    String email=user.getEmail();
    String gender=user.getGender();
    String firstname=user.getFirstName();
    String lastname=user.getLastName();
    model.addAttribute("name",name );
    model.addAttribute("birthday",birthday );
    model.addAttribute("email",email );
    model.addAttribute("gender",gender);
    model.addAttribute("firstname",firstname);
    model.addAttribute("lastname",lastname);
    model.addAttribute("facebookProfile", facebook.fetchObject("me", User.class, fields));
    PagedList<Post> feed = facebook.feedOperations().getFeed();
    model.addAttribute("feed", feed);
    return "hello";
}
}
```

FaceBookBootApplication.java:-

```
package com.spring.social.fb;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;
/**
 * fb App: Spring Test New
 */
@SpringBootApplication
@ComponentScan(basePackages="com.nt.controller")
public class FaceBookBootApplication {

    public static void main(String[] args) {
```

```
SpringApplication.run(FaceBookBootApplication.class, args);
```

```
}
```

facebookConnect.html:-

```
<html>
  <head>
    <title>Hello Facebook</title>
  </head>
  <body>
    <h3>Connect to Facebook</h3>

    <form action="/connect/facebook" method="POST">
      <input type="hidden" name="scope" value="user_posts" />
      <div class="formInfo">
        <p>You aren't connected to Facebook yet. Click the button to connect
this application with your Facebook account.</p>
      </div>
      <p><button type="submit">Connect to Facebook</button></p>
    </form>
  </body>
</html>
```

facebookConnected.html:-

```
<html>
  <head>
    <title>Hello Facebook</title>
  </head>
  <body>
    <h3>Connected to Facebook</h3>

    <p>
      You are now connected to your Facebook account.
      Click <a href="/">here</a> to see some entries from your Facebook feed.
    </p>
  </body>
</html>
```

hello.html:-

```
<html>
  <head>
    <title>Hello Facebook</title>
  </head>
  <body>
    <h3>Hello, <span th:text="${facebookProfile.name}">Some User</span>!</h3>

    <h4>Here is your feed:</h4>
```

```
<div th:each="post:${feed}">
  <b th:text="${post.from.name}">from</b> wrote:
  <p th:text="${post.message}">message text</p>
  
  <hr/>
</div>
</body>
</html>
```

application.properties:-

```
spring.social.facebook.appId=< ----give generated id ---->
spring.social.facebook.appSecret=< -----give generated secret id ---->

#Embedded server port no
server.port=4041
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.spring.social</groupId>
  <artifactId>FaceBookBoot</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>

  <name>FaceBookBoot</name>
  <description>Demo project for Spring Social Facebook</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>1.5.4.RELEASE</version>
    <relativePath /> <!-- lookup parent from repository -->
  </parent>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-social-facebook</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>
```

How to access Twitter Data ?

Accessing Twitter Data

This guide walks you through the process of creating a simple web application that fetches data from Twitter.

Enable Twitter

Before you can fetch a user's data from Twitter, you must specify your application's ID and secret by setting the `spring.social.twitter.appId` and `spring.social.twitter.appSecret` properties. You can set these via any means supported by Spring Boot, including setting them in an `application.properties` file:

```
src/main/resources/application.properties
```

```
spring.social.twitter.appId={{put app ID here}}
```

```
spring.social.twitter.appSecret={{put app secret here}}
```

As shown here, the properties have fake values. The values given to these properties correspond to your application's consumer key and secret you obtain when you register the application with Twitter. For the code to work, substitute the real values given to you by Twitter in place of the fake

values. The `appId` is Twitter's "Consumer Key", and the `appSecret` is the "Consumer Secret". Be sure to register a callback URL with Twitter (it has to be a valid URL, but it doesn't need to be the URL of your app).

The presence of these properties and Spring Social Twitter in the classpath will trigger automatic configuration of Spring Social's `ConnectController`, `TwitterConnectionFactory`, and other components of Spring Social's connection framework.

Create connection status views

Although much of what `ConnectController` does involves redirecting to Twitter and handling a redirect from Twitter, it also shows connection status when a GET request to `/connect` is made. `ConnectController` defers to a view named `connect/{provider ID}Connect` when no existing connection is available and to `connect/{providerId}Connected` when a connection exists for the provider. In this case, `{provider ID}` is "twitter".

`ConnectController` does not define its own connection views, so you need to create them. First, here's a Thymeleaf view to be shown when no connection to Twitter exists:

```
src/main/resources/templates/connect/twitterConnect.html
```

The form on this view will POST to `/connect/twitter`, which is handled by `ConnectController` and will kick off the OAuth authorization code flow.

Here's the view to be displayed when a connection exists:

```
src/main/resources/templates/connect/twitterConnected.html
```

Fetch Twitter data

With Twitter configured in your application, you now can write a Spring MVC controller that fetches data for the user who authorized the application and presents it in the browser. `HelloController` is just such a controller:

```
src/main/java/hello/HelloController.java
```

`HelloController` is created by injecting a `Twitter` object into its constructor. The `Twitter` object is a reference to Spring Social's Twitter API binding.

The `helloTwitter()` method is annotated with `@RequestMapping` to indicate that it should handle

GET requests for the root path (/). The first thing it does is check to see if the user has **authorized** the application to access the user's Twitter data. If not, then the user is redirected to `ConnectController` with the option to begin the authorization process.

If the user authorizes the application to access the data, the application can fetch almost any data pertaining to the authorizing user. For the purposes of this guide, the application only fetches the user's profile as well as a list of profiles belonging to Twitter users whom the user follows (but not those who follow the user). Both are placed into the model to be displayed by the view identified as "hello".

Speaking of the "hello" view, here it is as a Thymeleaf template:

```
src/main/resources/templates/hello.html
```

This template simply displays a greeting to the user and a list of the user's friends. Note that even though the full user profiles were fetched, only the names from those profiles are used in this template.

Make the application executable

Although it is possible to package this service as a traditional WAR file for deployment to an external application server, the simpler approach demonstrated below creates a *standalone application*. You package everything in a single, executable JAR file, driven by a good old Java `main()` method. And along the way, you use Spring's support for embedding the Tomcatservlet container as the HTTP runtime, instead of deploying to an external instance.

```
src/main/java/hello/TwitterBootApplication.java
```

`@SpringBootApplication` is a convenience annotation that adds all of the following:

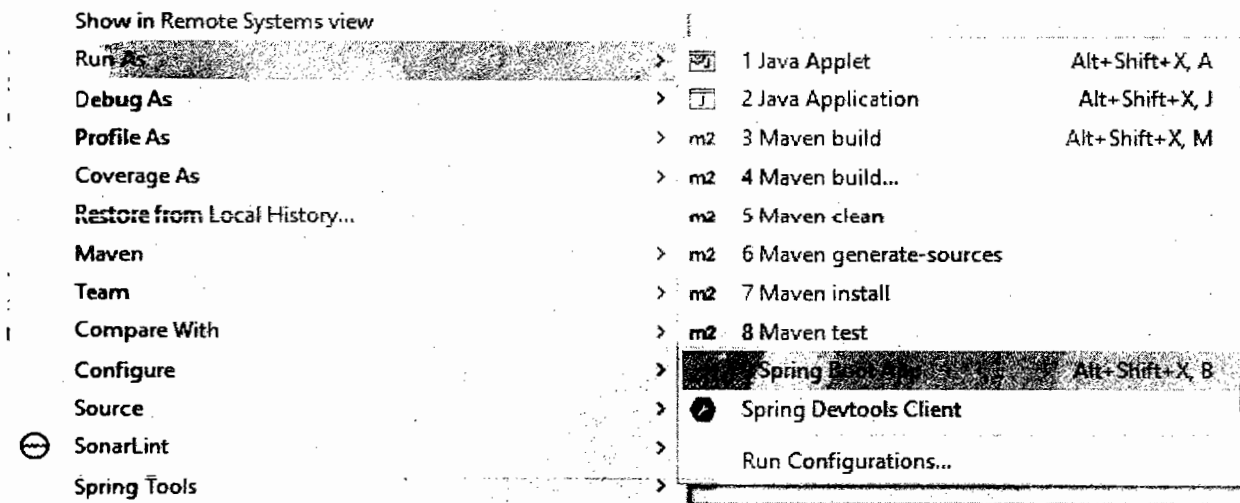
- `@Configuration` tags the class as a source of bean definitions for the application context.
- `@EnableAutoConfiguration` tells Spring Boot to start adding beans based on classpath settings, other beans, and various property settings.
- Normally you would add `@EnableWebMvc` for a Spring MVC app, but Spring Boot adds it automatically when it sees `spring-webmvc` on the classpath. This flags the application as a web application and activates key behaviors such as setting up a `DispatcherServlet`.
- `@ComponentScan` tells Spring to look for other components, configurations, and services in the `hello` package, allowing it to find the controllers.

The `main()` method uses Spring Boot's `SpringApplication.run()` method to launch an application. Did you notice that there wasn't a single line of XML? No `web.xml` file either. This web application is 100% pure Java and you didn't have to deal with configuring any plumbing or infrastructure.

Change the Tomcat Port

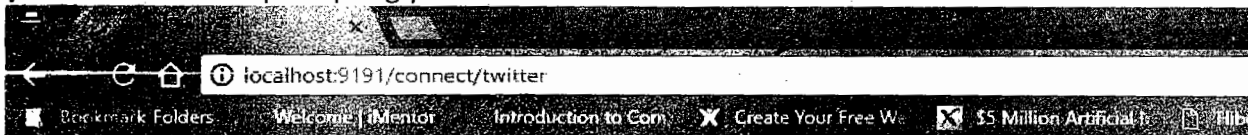
See the Prog → `src/main/java/hello/CustomContainer.java`

Run the Prog



Once the application starts up,
: Mapped URL path [/**/favicon.ico] onto handler c
: Registering beans for JMX exposure on startup
: Tomcat started on port(s): 9191 (http)
: Started TwitterBootApplication in 9.172 seconds

you can point your web browser to <http://localhost:9191> Because no connection is established yet, you see this screen prompting you to connect with Twitter:

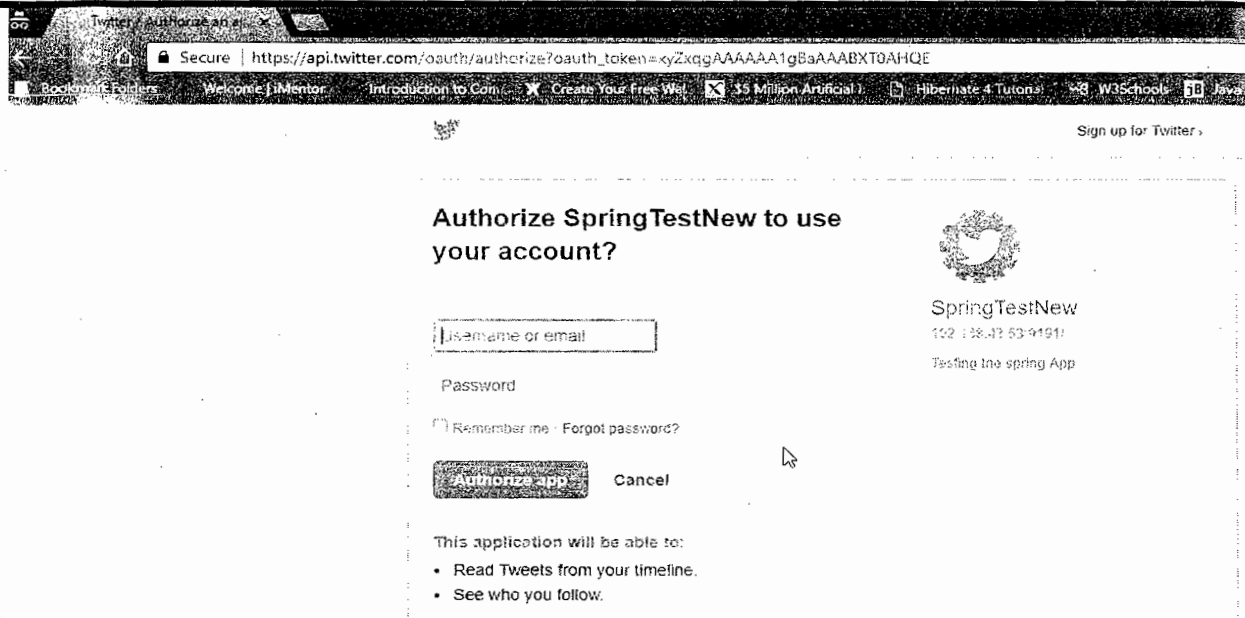


Connect to Twitter

You aren't connected to Twitter yet. Click the button to connect this application with your Twitter account.

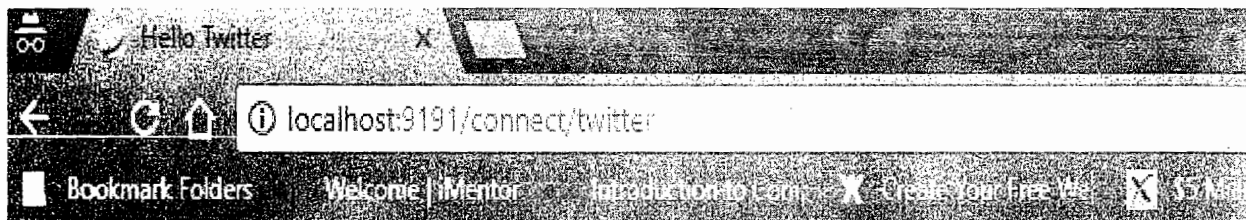
[Connect to Twitter](#)

When you click **Connect to Twitter**, the browser is redirected to Twitter for authorization:



At this point, Twitter asks if you'd like to allow the sample application to read tweets from your profile and see who you follow. Here the screen is misleading, because the application in this case will only read your profile details and the profile details of the people you follow. Click **Authorize app** to grant permission.

Once permission is granted, Twitter redirects the browser to the application. A connection is created and stored in the connection repository. You should see this page indicating that a connection was successful:



Connected to Twitter

You are now connected to your Twitter account. Click [here](#) to see your Twitter friends.

If you click on the link on the connection status page, you are taken to the home page. This time, now that a connection exists, you see your name on Twitter and a list of your friends:



Hello, Anurag Kumar!

These are your friends:

- Jagadeesh Pathi
- Amit Shah
- Smriti Z Irani
- Subramanian Swamy
- Panda Venkatesh
- Mahendra Singh Dhoni
- State Bank of India
- Arun Jaitley
- Make in India
- HDFC Bank
- Parineeti Chopra
- YES BANK
- Alia Bhatt
- Jacqueline Fernandez
- NDTV
- TIMES NOW
- A.R. Rahman
- Hrithik Roshan

Summary

Congratulations! You've developed a simple web application that uses Spring Social to obtain user authorization to fetch data from the user's Twitter profile and from the profiles of people whom the user follows.

Procedure to create Twitter Application in twitter website:-

Get in to <https://dev.twitter.com> ->myApps -> sign in -> create new Application :project name, Desc:-Testing twitter application here
Website -><http://192.168.0.35:4041>, Callback URL -><http://192.168.0.35:4041>
-> keys and AccessToken ,collect api key, apisecret key to place in application.properties
->Modify Application permissions to read,write and accessPermissions -> update settings.

TwitterBoot:-

- ▼  TwitterBoot [boot]
 - >  Spring Elements
 - ▼  src/main/java
 - ▼  com.spring.social.tw
 - >  HelloController.java
 - >  TwitterBootApplication.java
 -  **READ ME.docx**
 - ▼  src/main/resources
 - ▼  templates
 - ▼  connect
 -  twitterConnect.html
 -  twitterConnected.html
 -  hello.html
 -  application.properties
 -  src/test/java
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 - >  src
 - >  target
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

HelloController.java:-

```
package com.spring.social.tw;

import javax.inject.Inject;
import org.springframework.social.connect.ConnectionRepository;
import org.springframework.social.twitter.api.CursoredList;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.social.twitter.api.TwitterProfile;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/")
public class HelloController {

    private Twitter twitter;

    private ConnectionRepository connectionRepository;

    @Inject
    public HelloController(Twitter twitter, ConnectionRepository
connectionRepository) {
```

```

        this.twitter = twitter;
        this.connectionRepository = connectionRepository;
    }

    @GetMapping
    public String helloTwitter(Model model) {
        if (connectionRepository.findPrimaryConnection(Twitter.class) ==
null) {
            return "redirect:/connect/twitter";
        }

        model.addAttribute(twitter.userOperations().getUserProfile());
        CursoredList<TwitterProfile> friends =
twitter.friendOperations().getFriends();
        model.addAttribute("friends", friends);
        return "hello";
    }
}

```

TwitterBootApplication.java:-

```

package com.spring.social.tw;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TwitterBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(TwitterBootApplication.class, args);
    }

}

```

twitterConnect.html:-

```

<div>
<head>
    <title>Hello Twitter</title>
</head>
<body>
    <h3>Connect to Twitter</h3>
    <span th:if="${social_provider_error}">Provider error (maybe
you need to configure the app id and secret?)</span>
    <form action="/connect/twitter" method="POST">
        <div class="formInfo">
            <p>You aren't connected to Twitter yet. Click the
button to connect this application with your Twitter account.</p>
        </div>
        <p><button type="submit">Connect to Twitter</button></p>
    </form>

```

```
</body>
</html>
```

twitterConnected.html:-

```
<html>
<head>
  <title>Hello Twitter</title>
</head>
<body>
  <h3>Connected to Twitter</h3>

  <p>
    You are now connected to your Twitter account.
    Click <a href="/">here</a> to see your Twitter friends.
  </p>
</body>
</html>
```

hello.html:-

```
<html>
<head>
  <title>Hello Twitter</title>
</head>
<body>
  <h3>Hello, <span th:text="${twitterProfile.name}">Some
User</span>!</h3>

  <h4>These are your friends:</h4>

  <ul>
    <li th:each="fr:${friends}"
        th:text="${fr.name}">Friend
    </li>
  </ul>
</body>
</html>
```

application.properties:-

```
spring.social.twitter.appId=< ----give generated id ---->
spring.social.twitter.appSecret=< ----give generated secret id ---->

#Embedded server port
server.port=4041
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```



```
<modelVersion>4.0.0</modelVersion>

<groupId>com.spring.social</groupId>
<artifactId>TwitterBoot</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>TwitterBoot</name>
<description>Demo project for Spring Social Twitter</description>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>1.5.4.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>

<properties>
  <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-
8</project.reporting.outputEncoding>
  <java.version>1.8</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-social-
twitter</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

</project>
```

SpringSocialAll:-

- ▼  SpringSocialAll [boot] [devtools]
 -  Spring Elements
 - ▼  src/main/java
 - ▼  com.spring.social
 -  CustomContainer.java
 -  ExceptionHandlingControllerAdvice.java
 -  HomeController.java
 - ▼  com.spring.social.account
 -  Account.java
 -  AccountRepository.java
 -  JdbcAccountRepository.java
 -  UsernameAlreadyInUseException.java
 - ▼  com.spring.social.config
 -  SecurityConfig.java
 -  SpringSocialAllApplication.java
 - ▼  com.spring.social.facebook
 -  FacebookExpiredToken.java
 -  FacebookFeedController.java
 -  FacebookFriendsController.java
 -  FacebookHandlerInterceptor.java
 -  FacebookPhotosController.java
 -  FacebookProfileController.java
 -  PostToWallAfterConnectInterceptor.java
 - ▼  com.spring.social.message
 -  Message.java
 -  MessageType.java
 - ▼  com.spring.social.signin
 -  SigninController.java
 -  SignInUtils.java
 -  SimpleSignInAdapter.java
 - ▼  com.spring.social.signup
 -  SignupController.java

- > SignupForm.java
- ▼ com.spring.social.twitter
 - > MessageForm.java
 - > TweetAfterConnectInterceptor.java
 - > TwitterFriendsController.java
 - > TwitterProfileController.java
 - > TwitterRevokedToken.java
 - > TwitterSearchController.java
 - > TwitterTimelineController.java
 - > TwitterTrendsController.java
- ▼ src/main/resources
 - > static
 - ▼ templates
 - ▼ connect
 - facebookConnect.html
 - facebookConnected.html
 - status.html
 - twitterConnect.html
 - twitterConnected.html
 - ▼ facebook
 - album.html
 - albums.html
 - feed.html
 - friends.html
 - profile.html
 - > twitter
 - home.html
 - layout.html
 - ratelimit.html
 - signin.html
 - signup.html
 - application.properties
 - data.sql
 - messages.properties
 - schema.sql
- > src/test/java
- > JRE System Library [JavaSE-1.8]
- > Maven Dependencies
- > src
- > target
 - mvnw
 - mvnw.cmd
 - pom.xml

CustomContainer.java:-

```
package com.spring.social;

import
org.springframework.boot.context.embedded.ConfigurableEmbeddedServletCont
ainer;
import
org.springframework.boot.context.embedded.EmbeddedServletContainerCustomi
zer;
```

```
import org.springframework.stereotype.Component;

@Component
public class CustomContainer implements
EmbeddedServletContainerCustomizer {

    @Override
    public void customize(ConfigurableEmbeddedServletContainer
container) {

        container.setPort(9191);

    }

}
```

ExceptionHandlerControllerAdvice.java:-

```
package com.spring.social;

import org.springframework.social.RateLimitExceededException;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class ExceptionHandlingControllerAdvice {

    @ExceptionHandler(RateLimitExceededException.class)
    public String rateLimitExceeded(RateLimitExceededException e, Model
model) {

        model.addAttribute("providerId", e.getProviderId());
        return "rateLimit";

    }

}
```

HomeController.java:-

```
package com.spring.social;

import java.security.Principal;
import javax.inject.Inject;
import javax.inject.Provider;
import org.springframework.social.connect.ConnectionRepository;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

import com.spring.social.account.AccountRepository;
```

```
@Controller
public class HomeController {

    private final Provider<ConnectionRepository>
connectionRepositoryProvider;

    private final AccountRepository accountRepository;

    @Inject
    public HomeController(Provider<ConnectionRepository>
connectionRepositoryProvider, AccountRepository accountRepository) {
        this.connectionRepositoryProvider =
connectionRepositoryProvider;
        this.accountRepository = accountRepository;
    }

    @RequestMapping("/")
    public String home(Principal currentUser, Model model) {
        model.addAttribute("connectionsToProviders",
getConnectionRepository().findAllConnections());
        if (currentUser != null) {
            model.addAttribute(accountRepository.findAccountByUsername(currentUser.getName()));
        }
        return "home";
    }

    private ConnectionRepository getConnectionRepository() {
        return connectionRepositoryProvider.get();
    }
}
```

Account.java:-

```
package com.spring.social.account;

public class Account {

    private final String username;

    private final String password;

    private final String firstName;

    private final String lastName;

    public Account(String username, String password, String firstName,
String lastName) {
        this.username = username;
        this.password = password;
    }
}
```

```
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String getUsername() {
        return username;
    }

    public String getPassword() {
        return password;
    }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }
}
```

AccountRepository.java:-

```
package com.spring.social.account;

public interface AccountRepository {

    void createAccount(Account account) throws
    UsernameAlreadyInUseException;

    Account findAccountByUsername(String username);
}
```

JdbcAccountRepository.java:-

```
package com.spring.social.account;

import java.sql.ResultSet;
import java.sql.SQLException;
import javax.inject.Inject;
import org.springframework.dao.DuplicateKeyException;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

@Repository
public class JdbcAccountRepository implements AccountRepository {

    private final JdbcTemplate jdbcTemplate;
```

```
private final PasswordEncoder passwordEncoder;

@Inject
public JdbcAccountRepository(JdbcTemplate jdbcTemplate,
    PasswordEncoder passwordEncoder) {
    this.jdbcTemplate = jdbcTemplate;
    this.passwordEncoder = passwordEncoder;
}

@Transactional
public void createAccount(Account user) throws
    UsernameAlreadyInUseException {
    try {
        jdbcTemplate.update(
            "insert into Account (firstName, lastName,
            username, password) values (?, ?, ?, ?)",
            user.getFirstName(), user.getLastName(),
            user.getUsername(),
            passwordEncoder.encode(user.getPassword()));
    } catch (DuplicateKeyException e) {
        throw new
            UsernameAlreadyInUseException(user.getUsername());
    }
}

public Account findAccountByUsername(String username) {
    return jdbcTemplate.queryForObject("select username,
    firstName, lastName from Account where username = ?",
        new RowMapper<Account>() {
            public Account mapRow(ResultSet rs, int
            rowNum) throws SQLException {
                return new
                    Account(rs.getString("username"), null, rs.getString("firstName"), rs
                    .getString("lastName"));
            }
        }, username);
}
}
```

UsernameAlreadyInUseException.java:-

```
package com.spring.social.account;

@SuppressWarnings("serial")
public class UsernameAlreadyInUseException extends Exception {
    public UsernameAlreadyInUseException(String username) {
        super("The username '" + username + "' is already in use.");
    }
}
```

SecurityConfig.java:-

```
package com.spring.social.config;

import javax.inject.Inject;
import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableWebSecurity;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.builders.WebSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.security.crypto.password.NoOpPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.thymeleaf.extras.springsecurity4.dialect.SpringSecurityDialect;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter{

    @SuppressWarnings("unused")
    @Autowired
    private ApplicationContext context;

    @Inject
    private DataSource dataSource;

    @Autowired
    public void registerAuthentication(AuthenticationManagerBuilder auth) throws Exception {
        auth.jdbcAuthentication()
            .dataSource(dataSource)
            .usersByUsernameQuery("select username, password, true from Account where username = ?")
            .authoritiesByUsernameQuery("select username, 'ROLE_USER' from Account where username = ?")
            .passwordEncoder(passwordEncoder());
    }
}
```



```
}

@Override
public void configure(WebSecurity web) throws Exception {
    web
        .ignoring()
            .antMatchers("/**/*.css", "/**/*.png", "/**/*.gif",
"/**/*.jpg");
}

@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .formLogin()
            .loginPage("/signin")
            .loginProcessingUrl("/signin/authenticate")
            .defaultSuccessUrl("/connect")
            .failureUrl("/signin?param.error=bad_credentials")
        .and()
            .logout()
                .logoutUrl("/signout")
                .deleteCookies("JSESSIONID")
        .and()
            .authorizeRequests()
                .antMatchers("/", "/webjars/**", "/admin/**",
"/favicon.ico", "/resources/**", "/auth/**", "/signin/**", "/signup/**",
"/disconnect/facebook").permitAll()
                .antMatchers("/**").authenticated()
        .and()
            .rememberMe();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
}

@Bean
public TextEncryptor textEncryptor() {
    return Encryptors.noOpText();
}

@Bean
public SpringSecurityDialect springSecurityDialect() {
    return new SpringSecurityDialect();
}
}
```

SpringSocialAllApplication.java:-

```
package com.spring.social.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.security.web.savedrequest.HttpSessionRequestCache;
import org.springframework.social.connect.web.SignInAdapter;
import com.spring.social.signin.SimpleSignInAdapter;

@ComponentScan(basePackages="com.spring.social")
@EnableConfigurationProperties
@EnableAutoConfiguration
public class SpringSocialAllApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringSocialAllApplication.class, args);
    }

    @Bean
    public SignInAdapter signInAdapter() {
        return new SimpleSignInAdapter(new HttpSessionRequestCache());
    }
}
```

SpringSocialAllApplication.java:-

```
package com.spring.social.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.security.web.savedrequest.HttpSessionRequestCache;
import org.springframework.social.connect.web.SignInAdapter;
import com.spring.social.signin.SimpleSignInAdapter;

@ComponentScan(basePackages="com.spring.social")
@EnableConfigurationProperties
@EnableAutoConfiguration
public class SpringSocialAllApplication {
    public static void main(String[] args) {
```

```
        SpringApplication.run(SpringSocialAllApplication.class, args);
    }

    @Bean
    public SignInAdapter signInAdapter() {
        return new SimpleSignInAdapter(new HttpSessionRequestCache());
    }
}
```

FacebookExpiredToken.java:-

```
package com.spring.social.facebook;

import org.springframework.social.ExpiredAuthorizationException;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class FacebookExpiredToken {

    @RequestMapping("/facebook/expired")
    public void simulateExpiredToken() {
        throw new ExpiredAuthorizationException("facebook");
    }
}
```

FacebookFeedController.java:-

```
package com.spring.social.facebook;

import javax.inject.Inject;
import org.springframework.social.facebook.api.Facebook;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class FacebookFeedController {

    private final Facebook facebook;

    @Inject
    public FacebookFeedController(Facebook facebook) {
        this.facebook = facebook;
    }

    @RequestMapping(value="/facebook/feed", method=RequestMethod.GET)
    public String showFeed(Model model) {
```

```
        model.addAttribute("feed",
facebook.feedOperations().getFeed());
        return "facebook/feed";
    }

    @RequestMapping(value="/facebook/feed", method=RequestMethod.POST)
    public String postUpdate(String message) {
        facebook.feedOperations().updateStatus(message);
        return "redirect:/facebook/feed";
    }
}
```

FacebookFriendsController.java:-

```
package com.spring.social.facebook;

import javax.inject.Inject;
import org.springframework.social.facebook.api.Facebook;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class FacebookFriendsController {

    private final Facebook facebook;

    @Inject
    public FacebookFriendsController(Facebook facebook) {
        this.facebook = facebook;
    }

    @RequestMapping(value = "/facebook/friends", method =
RequestMethod.GET)
    public String showFeed(Model model) {
        model.addAttribute("friends",
facebook.friendOperations().getFriendProfiles());
        return "facebook/friends";
    }
}
```

FacebookHandlerInterceptor.java:-

```
package com.spring.social.facebook;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

@Component
public class FacebookHandlerInterceptor implements HandlerInterceptor {

    @Override
    public void afterCompletion(HttpServletRequest arg0,
        HttpServletResponse arg1, Object arg2, Exception arg3)
        throws Exception {

    }

    @Override
    public void postHandle(HttpServletRequest arg0, HttpServletResponse
arg1, Object arg2, ModelAndView arg3)
        throws Exception {

    }

    @Override
    public boolean preHandle(HttpServletRequest request,
        HttpServletResponse response, Object obj) throws Exception {
        System.out.println("-----> " +
request.getPathInfo());
        return true;
    }
}
```

FacebookPhotosController.java:-

```
package com.spring.social.facebook;

import javax.inject.Inject;
import org.springframework.social.facebook.api.Facebook;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class FacebookPhotosController {

    private final Facebook facebook;

    @Inject
    public FacebookPhotosController(Facebook facebook) {
        this.facebook = facebook;
    }
}
```

```
}

@RequestMapping(value="/facebook/albums", method=RequestMethod.GET)
public String showAlbums(Model model) {
    model.addAttribute("albums",
facebook.mediaOperations().getAlbums());
    return "facebook/albums";
}

@RequestMapping(value="/facebook/album/{albumId}",
method=RequestMethod.GET)
public String showAlbum(@PathVariable("albumId") String albumId,
Model model) {
    model.addAttribute("album",
facebook.mediaOperations().getAlbum(albumId));
    model.addAttribute("photos",
facebook.mediaOperations().getPhotos(albumId));
    return "facebook/album";
}
}
```

FacebookProfileController.java:-

```
package com.spring.social.facebook;

import javax.inject.Inject;
import org.springframework.social.connect.Connection;
import org.springframework.social.connect.ConnectionRepository;
import org.springframework.social.facebook.api.Facebook;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class FacebookProfileController {

    @Inject
    private ConnectionRepository connectionRepository;

    @RequestMapping(value="/facebook", method=RequestMethod.GET)
    public String home(Model model) {
        Connection<Facebook> connection =
connectionRepository.findPrimaryConnection(Facebook.class);
        if (connection == null) {
            return "redirect:/connect/facebook";
        }
        model.addAttribute("profile",
connection.getApi().userOperations().getUserProfile());
        return "facebook/profile";
    }
}
```

```
}  
  
}
```

PostToWallAfterConnectInterceptor.java:-

```
package com.spring.social.facebook;  
  
import org.springframework.social.ApiException;  
import org.springframework.social.connect.Connection;  
import org.springframework.social.connect.ConnectionFactory;  
import org.springframework.social.connect.web.ConnectInterceptor;  
import org.springframework.social.facebook.api.Facebook;  
import org.springframework.stereotype.Component;  
import org.springframework.util.MultiValueMap;  
import org.springframework.util.StringUtils;  
import org.springframework.web.context.request.WebRequest;  
  
@Component  
public class PostToWallAfterConnectInterceptor implements  
ConnectInterceptor<Facebook> {  
  
    public void preConnect(ConnectionFactory<Facebook>  
connectionFactory, MultiValueMap<String, String> parameters, WebRequest  
request) {  
        if  
(StringUtils.hasText(request.getParameter(POST_TO_WALL_PARAMETER))) {  
            request.setAttribute(POST_TO_WALL_ATTRIBUTE,  
Boolean.TRUE, WebRequest.SCOPE_SESSION);  
        }  
    }  
  
    public void postConnect(Connection<Facebook> connection, WebRequest  
request) {  
        if (request.getAttribute(POST_TO_WALL_ATTRIBUTE,  
WebRequest.SCOPE_SESSION) != null) {  
            try {  
                connection.updateStatus("I've connected with the  
Spring Social Showcase!");  
            } catch (ApiException e) {  
                // Do nothing: No need to break down if the post-  
connect post can't be made.  
            }  
            request.removeAttribute(POST_TO_WALL_ATTRIBUTE,  
WebRequest.SCOPE_SESSION);  
        }  
    }  
  
    private static final String POST_TO_WALL_PARAMETER = "postToWall";  
    private static final String POST_TO_WALL_ATTRIBUTE =  
"facebookConnect." + POST_TO_WALL_PARAMETER;  
}
```

Message.java:-

```
package com.spring.social.message;

public final class Message {

    private final MessageType type;

    private final String text;

    /**
     * Creates a new Message of a certain type consisting of the text
     provided.
     */
    public Message(MessageType type, String text) {
        this.type = type;
        this.text = text;
    }

    /**
     * The type of message; such as info, warning, error, or success.
     */
    public MessageType getType() {
        return type;
    }

    /**
     * The info text.
     */
    public String getText() {
        return text;
    }

    public String toString() {
        return type + ": " + text;
    }
}
```

MessageType.java:-

```
package com.spring.social.message;

public enum MessageType {

    /**
     * The message is informative in nature, like a note or notice.
     */
    INFO,
```



```
/**
 * The message indicates that an action initiated by the user was
 * performed successfully.
 */
SUCCESS,

/**
 * The message warns the user something is not quite right.
 * Corrective action is generally recommended but not required.
 */
WARNING,

/**
 * The message reports an error condition that needs to be addressed
 * by the user.
 */
ERROR;

private final String cssClass;

private MessageType() {
    cssClass = name().toLowerCase();
}

/**
 * The css class for styling messages of this type.
 */
public String getCssClass() {
    return cssClass;
}
}
```

SignInController.java:-

```
package com.spring.social.signin;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class SignInController {

    @RequestMapping(value="/signin", method=RequestMethod.GET)
    public void signin() {
    }

}
```

SignInUtils.java:-

```
package com.spring.social.signin;
```

```
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;

public class SignInUtils {

    /**
     * Programmatically signs in the user with the given the user ID.
     */
    public static void signin(String userId) {
        SecurityContextHolder.getContext().setAuthentication(new
        UsernamePasswordAuthenticationToken(userId, null, null));
    }
}
```

SimpleSignInAdapter.java:-

```
package com.spring.social.signin;

import javax.inject.Inject;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import org.springframework.security.web.WebAttributes;
import org.springframework.security.web.savedrequest.RequestCache;
import org.springframework.security.web.savedrequest.SavedRequest;
import org.springframework.social.connect.Connection;
import org.springframework.social.connect.web.SignInAdapter;
import org.springframework.web.context.request.NativeWebRequest;

public class SimpleSignInAdapter implements SignInAdapter {

    private final RequestCache requestCache;

    @Inject
    public SimpleSignInAdapter(RequestCache requestCache) {
        this.requestCache = requestCache;
    }

    @Override
    public String signIn(String localUserId, Connection<?> connection,
        NativeWebRequest request) {
        SignInUtils.signin(localUserId);
        return extractOriginalUrl(request);
    }

    private String extractOriginalUrl(NativeWebRequest request) {
        HttpServletRequest nativeReq =
```

```
request.getNativeRequest(HttpServletRequest.class);
    HttpServletResponse nativeRes =
request.getNativeResponse(HttpServletResponse.class);
    SavedRequest saved = requestCache.getRequest(nativeReq,
nativeRes);
    if (saved == null) {
        return null;
    }
    requestCache.removeRequest(nativeReq, nativeRes);
    removeAutheticationAttributes(nativeReq.getSession(false));
    return saved.getRedirectUrl();
}

private void removeAutheticationAttributes(HttpSession session) {
    if (session == null) {
        return;
    }

    session.removeAttribute(WebAttributes.AUTHENTICATION_EXCEPTION);
}
}
```

SignupController.java:-

```
package com.spring.social.signup;

import javax.inject.Inject;
import javax.validation.Valid;
import org.springframework.social.connect.Connection;
import org.springframework.social.connect.ConnectionFactoryLocator;
import org.springframework.social.connect.UsersConnectionRepository;
import org.springframework.social.connect.web.ProviderSignInUtils;
import org.springframework.stereotype.Controller;
import org.springframework.util.StringUtils;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.context.request.WebRequest;
import com.spring.social.account.Account;
import com.spring.social.account.AccountRepository;
import com.spring.social.account.UsernameAlreadyInUseException;
import com.spring.social.message.Message;
import com.spring.social.message.MessageType;
import com.spring.social.signin.SignInUtils;

@Controller
public class SignupController {

    private final AccountRepository accountRepository;
    private final ProviderSignInUtils providerSignInUtils;
```

```
@Inject
public SignupController(AccountRepository accountRepository,
                        ConnectionFactoryLocator
connectionFactoryLocator,
                        UsersConnectionRepository
connectionRepository) {
    this.accountRepository = accountRepository;
    this.providerSignInUtils = new
ProviderSignInUtils(connectionFactoryLocator, connectionRepository);
}

@RequestMapping(value="/signup", method=RequestMethod.GET)
public SignupForm signupForm(WebRequest request) {
    Connection<?> connection =
providerSignInUtils.getConnectionFromSession(request);
    if (connection != null) {
        request.setAttribute("message", new
Message(MessageType.INFO, "Your " +
StringUtils.capitalize(connection.getKey().getProviderId()) + " account
is not associated with a Spring Social Showcase account. If you're new,
please sign up."), WebRequest.SCOPE_REQUEST);
        return
SignupForm.fromProviderUser(connection.fetchUserProfile());
    } else {
        return new SignupForm();
    }
}

@RequestMapping(value="/signup", method=RequestMethod.POST)
public String signup(@Valid SignupForm form, BindingResult
formBinding, WebRequest request) {
    if (formBinding.hasErrors()) {
        return null;
    }
    Account account = createAccount(form, formBinding);
    if (account != null) {
        SignInUtils.signin(account.getUsername());
        providerSignInUtils.doPostSignUp(account.getUsername(),
request);
        return "redirect:/";
    }
    return null;
}

// internal helpers

private Account createAccount(SignupForm form, BindingResult
formBinding) {
    try {
```

```
        Account account = new Account(form.getUsername(),
form.getPassword(), form.getFirstName(), form.getLastName());
        accountRepository.createAccount(account);
        return account;
    } catch (UsernameAlreadyInUseException e) {
        formBinding.rejectValue("username",
"user.duplicateUsername", "already in use");
        return null;
    }
}
```

SignupForm.java:-

```
package com.spring.social.signup;

import javax.validation.constraints.Size;
import org.hibernate.validator.constraints.NotEmpty;
import org.springframework.social.connect.UserProfile;

public class SignupForm {

    @NotEmpty
    private String username;

    @Size(min = 6, message = "must be at least 6 characters")
    private String password;

    @NotEmpty
    private String firstName;

    @NotEmpty
    private String lastName;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

```
public String getFirstName() {
    return firstName;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public String getLastName() {
    return lastName;
}

public void setLastName(String lastName) {
    this.lastName = lastName;
}

public static SignupForm fromProviderUser(UserProfile providerUser)
{
    SignupForm form = new SignupForm();
    form.setFirstName(providerUser.getFirstName());
    form.setLastName(providerUser.getLastName());
    form.setUsername(providerUser.getUsername());
    return form;
}
}
```

MessageForm.java:-

```
package com.spring.social.twitter;

public class MessageForm {

    private String to;

    private String text;

    public String getTo() {
        return to;
    }

    public void setTo(String to) {
        this.to = to;
    }

    public String getText() {
        return text;
    }

    public void setText(String text) {
        this.text = text;
    }
}
```

```
}
```

TweetAfterConnectInterceptor.java:-

```
package com.spring.social.twitter;

import org.springframework.social.DuplicateStatusException;
import org.springframework.social.connect.Connection;
import org.springframework.social.connect.ConnectionFactory;
import org.springframework.social.connect.web.ConnectInterceptor;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Component;
import org.springframework.util.MultiValueMap;
import org.springframework.util.StringUtils;
import org.springframework.web.context.request.WebRequest;

@Component
public class TweetAfterConnectInterceptor implements
ConnectInterceptor<Twitter> {

    public void preConnect(ConnectionFactory<Twitter> provider,
MultiValueMap<String, String> parameters, WebRequest request) {
        if
        (StringUtils.hasText(request.getParameter(POST_TWEET_PARAMETER))) {
            request.setAttribute(POST_TWEET_ATTRIBUTE, Boolean.TRUE,
WebRequest.SCOPE_SESSION);
        }
    }

    public void postConnect(Connection<Twitter> connection, WebRequest
request) {
        if (request.getAttribute(POST_TWEET_ATTRIBUTE,
WebRequest.SCOPE_SESSION) != null) {
            try {
                connection.updateStatus("I've connected with the
Spring Social Showcase!");
            } catch (DuplicateStatusException e) {
            }
            request.removeAttribute(POST_TWEET_ATTRIBUTE,
WebRequest.SCOPE_SESSION);
        }
    }

    private static final String POST_TWEET_PARAMETER = "postTweet";

    private static final String POST_TWEET_ATTRIBUTE = "twitterConnect."
+ POST_TWEET_PARAMETER;
}
```

TwitterFriendsController.java:-

```
package com.spring.social.twitter;

import javax.inject.Inject;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class TwitterFriendsController {

    private final Twitter twitter;

    @Inject
    public TwitterFriendsController(Twitter twitter) {
        this.twitter = twitter;
    }

    @RequestMapping(value="/twitter/friends", method=RequestMethod.GET)
    public String friends(Model model) {
        model.addAttribute("profiles",
            twitter.friendOperations().getFriends());
        return "twitter/friends";
    }

    @RequestMapping(value="/twitter/followers",
        method=RequestMethod.GET)
    public String followers(Model model) {
        model.addAttribute("profiles",
            twitter.friendOperations().getFollowers());
        return "twitter/friends";
    }
}
```

TwitterProfileController.java:-

```
package com.spring.social.twitter;

import java.security.Principal;
import javax.inject.Inject;
import org.springframework.social.connect.Connection;
import org.springframework.social.connect.ConnectionRepository;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
```



```
public class TwitterProfileController {

    @Inject
    private ConnectionRepository connectionRepository;

    @RequestMapping(value="/twitter", method=RequestMethod.GET)
    public String home(Principal currentUser, Model model) {
        Connection<Twitter> connection =
connectionRepository.findPrimaryConnection(Twitter.class);
        if (connection == null) {
            return "redirect:/connect/twitter";
        }
        model.addAttribute("profile",
connection.getApi().userOperations().getUserProfile());
        return "twitter/profile";
    }

}
```

TwitterRevokedToken.java:-

```
package com.spring.social.twitter;

import org.springframework.social.ExpiredAuthorizationException;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class TwitterRevokedToken {

    @RequestMapping("/twitter/revoked")
    public void simulateExpiredToken() {
        throw new ExpiredAuthorizationException("twitter");
    }

}
```

TwitterSearchController.java:-

```
package com.spring.social.twitter;

import javax.inject.Inject;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class TwitterSearchController {

    private final Twitter twitter;
```

```
@Inject
public TwitterSearchController(Twitter twitter) {
    this.twitter = twitter;
}

@RequestMapping(value="/twitter/search", method=RequestMethod.GET)
public String showTrends(@RequestParam("query") String query, Model
model) {
    model.addAttribute("timeline",
twitter.searchOperations().search(query).getTweets());
    return "twitter/timeline";
}
}
```

TwitterTimelineController.java:-

```
package com.spring.social.twitter;

import javax.inject.Inject;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class TwitterTimelineController {

    private final Twitter twitter;

    @Inject
    public TwitterTimelineController(Twitter twitter) {
        this.twitter = twitter;
    }

    @RequestMapping(value="/twitter/timeline", method=RequestMethod.GET)
    public String showTimeline(Model model) {
        return showTimeline("Home", model);
    }

    @RequestMapping(value="/twitter/timeline/{timelineType}",
method=RequestMethod.GET)
    public String showTimeline(@PathVariable("timelineType") String
timelineType, Model model) {
        if (timelineType.equals("Home")) {
            model.addAttribute("timeline",
twitter.timelineOperations().getHomeTimeline());
        } else if (timelineType.equals("User")) {

```

```
        model.addAttribute("timeline",
twitter.timelineOperations().getUserTimeline());
        } else if (timelineType.equals("Mentions")) {
            model.addAttribute("timeline",
twitter.timelineOperations().getMentions());
        } else if (timelineType.equals("Favorites")) {
            model.addAttribute("timeline",
twitter.timelineOperations().getFavorites());
        }
        model.addAttribute("timelineName", timelineType);
        return "twitter/timeline";
    }

    @RequestMapping(value="/twitter/tweet", method=RequestMethod.POST)
    public String postTweet(String message) {
        twitter.timelineOperations().updateStatus(message);
        return "redirect:/twitter";
    }
}
```

TwitterTrendsController.java:-

```
package com.spring.social.twitter;

import javax.inject.Inject;
import org.springframework.social.twitter.api.Twitter;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

@Controller
public class TwitterTrendsController {

    // Yahoo Where On Earth ID representing the entire world
    private static final long WORLDWIDE_WOE = 1L;

    private final Twitter twitter;

    @Inject
    public TwitterTrendsController(Twitter twitter) {
        this.twitter = twitter;
    }

    @RequestMapping(value="/twitter/trends", method=RequestMethod.GET)
    public String showTrends(Model model) {
        model.addAttribute("trends",
twitter.searchOperations().getLocalTrends(WORLDWIDE_WOE));
        return "twitter/trends";
    }
}
```

```

    }
}

```

facebookConnect.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
  <head>
    <title>Spring Social NTZ Classroom</title>
  </head>
  <body>
    <div id="header">
      <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
    </div>

    <div id="content" layout:fragment="content">
      <h3>Connect to Facebook</h3>

      <form th:action="@{/connect/facebook}" method="POST">
        <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
        <input type="hidden" name="scope"
value="user_posts,user_photos" />
        <div class="formInfo">
          <p>You aren't connected to Facebook yet. Click
the button to connect Spring Social NTZ Classroom with your Facebook
account.</p>
        </div>
        <p><button type="submit"></button></p>
        <label for="postToWall"><input id="postToWall"
type="checkbox" name="postToWall" /> Tell your friends about Spring
Social NTZ Classroom on your Facebook wall</label>
      </form>
    </div>
  </body>
</html>

```

facebookConnected.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
  <head>

```

```

<title>Spring Social NTZ Classroom</title>
</head>
<body>
    <div id="header">
        <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
    </div>

    <div id="content" layout:fragment="content">
        <h3>Connected to Facebook</h3>

        <form id="disconnect" method="post">
            <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
            <div class="formInfo">
                <p>
                    Spring Social NTZ Classroom is connected
to your Facebook account.
                    Click the button if you wish to
disconnect.
                </p>
            </div>
            <button type="submit">Disconnect</button>
            <input type="hidden" name="_method" value="delete"
/>
        </form>

        <a th:href="@{/facebook}">View your Facebook profile</a>
    </div>
</body>
</html>

```

status.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    xmlns:social="http://spring.io/springsocial"
    xmlns:layout="http://www.ultraq.net.nz/thymeleaf/Layout"
    layout:decorator="layout">
<head>
    <title>Spring Social NTZ Classroom</title>
</head>
<body>
    <div id="header">
        <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
    </div>

    <div id="content" layout:fragment="content">
        <h3>Your Connections</h3>
        <!-- TODO: Thymeleaf seems to be tacking local/language

```

on the end of properties..."facebook.displayName_en_US"...BECAUSE it's looking up a property and can't find it!!! -->

```

    <div class="accountConnection" th:each="providerId :
    ${providerIds}">
        <h4><span th:text="#{${providerId} +
'.displayName'}">provider name</span></h4>
        <p>
            <span
th:if="${!#lists.isEmpty(connectionMap[__${providerId}__])}">
                You are connected to <span
th:text="#{${providerId} + '.displayName'}">provider name</span> as <span
th:text="${connectionMap[__${providerId}__][0].displayName}">user display
name</span>.
            </span>
            <span
th:if="${#lists.isEmpty(connectionMap[__${providerId}__])}">
                You are not yet connected to <span
th:text="#{${providerId} + '.displayName'}">provider name</span>.
            </span>
            Click <a th:href="@{'/connect/' +
${providerId}}">here</a> to manage your <span th:text="#{${providerId} +
'.displayName'}">provider name</span> connection.
        </p>
    </div>
</div>
</body>
</html>

```

twitterConnect.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    xmlns:social="http://spring.io/springsocial"
    xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
    layout:decorator="layout">
    <head>
        <title>Spring Social NTZ ClassRoom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
        </div>

        <div id="content" layout:fragment="content">
            <h3>Connect to Twitter</h3>

            <form th:action="@{/connect/twitter}" method="POST">
                <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
            </form>
        </div>
    </body>
</html>

```

```

        <div class="formInfo">
            <p>
                You haven't created any connections with
                Twitter yet. Click the button to connect Spring Social NTZ Classroom with
                your Twitter account.
                (You'll be redirected to Twitter where
                you'll be asked to authorize the connection.)
            </p>
        </div>
        <p><button type="submit"></button></p>
        <label for="postTweet"><input id="postTweet"
type="checkbox" name="postTweet" /> Post a tweet about connecting with
Spring Social NTZ Classroom</label>
    </form>
</div>
</body>
</html>

```

twitterConnected.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
    <head>
        <title>Spring Social NTZ Classroom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
        </div>

        <div id="content" layout:fragment="content">
            <h3>Connected to Twitter</h3>

            <form id="disconnect" method="post">
                <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
                <div class="formInfo">
                    <p>The Spring Social NTZ Classroom sample
application is already connected to your Twitter account.
                    Click the button if you wish to
disconnect.</p>
                </div>

                <button type="submit">Disconnect</button>
                <input type="hidden" name="_method" value="delete"
/>

```

```

        </form>

        <p><a th:href="@{/twitter}">View your Twitter
profile</a></p>
    </div>
</body>
</html>

```

album.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
    <head>
        <title>Spring Social NTZ Classroom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
Classroom</a></h1>
        </div>

        <div id="content" layout:fragment="content">
            <h3>Your Facebook Photo Album: <span
th:text="${album.name}">album name</span></h3>

            <span th:each="photo : ${photos}">
                
            </span>
        </div>
    </body>
</html>

```

albums.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
    <head>
        <title>Spring Social NTZ Classroom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
Classroom</a></h1>
        </div>

```



```

<div id="content" layout:fragment="content">
    <h3>Your Facebook Photo Albums</h3>

    <ul class="albums">
        <li th:each="album : ${albums}"><a
th:href="@{'/facebook/album/' + ${album.id}}"
th:text="${album.name}"></a></li>
    </ul>
</div>
</body>
</html>

```

feed.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
    <head>
        <title>Spring Social NTZ Classroom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
        </div>

        <div id="content" layout:fragment="content">
            <h3>Your Facebook Feed</h3>

            <form method="POST" th:action="@{/facebook/feed}">
                <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
                <p>Post to your Facebook wall:</p>
                <textarea id="message" name="message" rows="2"
cols="60"></textarea><br/>
                <input type="submit" value="Post" />
            </form>

            <div class="feed">
                <ul class="feedList">
                    <li class="post" th:each="post : ${feed}">
                        <p>
                            
                            <span
th:text="${post.message}">message</span> - <span
th:text="${post.name}">name</span>
                        </p>

```

```

        </li>
    </ul>
</div>
</div>
</body>
</html>

```

friends.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
<head>
    <title>Spring Social NTZ Classroom</title>
</head>
<body>
    <div id="header">
        <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
    </div>

    <div id="content" layout:fragment="content">
        <h3>Your Facebook Friends</h3>
        <p><i>NOTE: Applications are limited to only seeing
friends of the authorizing user
that are also users of the application. If your friend
list is empty or seems
smaller than you expect, it may be that most/all of your
Facebook friends are
not users of this application.</i></p>

        <ul class="friends">
            <li th:each="friend : ${friends}"><span th:text="${friend.name}">name</span></li>
        </ul>
    </div>
</body>
</html>

```

profile.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
<head>
    <title>Spring Social NTZ Classroom</title>
</head>

```

```

<body>
    <div id="header">
        <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
    </div>

    <div id="content" layout:fragment="content">
        <h3>Your Facebook Profile</h3>
        <p>Hello, <span th:text="${profile.firstName}">first
name</span>!</p>
        <dl>
            <dt>Facebook ID:</dt>
            <dd th:text="${profile.id}"></dd>
            <dt>Name:</dt>
            <dd th:text="${profile.name}"></dd>
            <dt>Email:</dt>
            <dd th:text="${email}"></dd>
        </dl>

        <form id="disconnect" th:action="@{/connect/facebook}"
method="post">
            <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
            <button type="submit">Disconnect from
Facebook</button>
            <input type="hidden" name="_method" value="delete"
/>
        </form>
    </div>
</body>
</html>

```

home.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
    xmlns:social="http://spring.io/springsocial"
    xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
    layout:decorator="layout">
    <head>
        <title>Spring Social NTZ ClassRoom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
        </div>

        <div id="content" layout:fragment="content">
            <div class="jumbotron">
                

    <div class="project-title">
    Welcome to Spring Social Learning World
    </div>

    <h1>Spring Social Learning World </h1>
    <p>
    This application demonstrates how an application may use
    Spring Social to connect to and consume data from Facebook and Twitter.
    </p>
    </div>
    </div>
  </body>
</html>

```

layout.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-
springsecurity3">
  <head>
    <meta charset="utf-8"/>
    <meta http-equiv="X-UA-Compatible" content="IE=edge"/>
    <meta name="viewport" content="width=device-width, initial-scale=1"/>
    <!-- The above 3 meta tags *must* come first in the head; any other
head content must come *after* these tags -->
    <meta name="description" content=""/>
    <meta name="author" content=""/>
    <link rel="icon" href="../../favicon.ico"/>

    <title>Navbar Template for Bootstrap</title>

    <!-- Bootstrap core CSS -->
    <link th:href="@{/webjars/bootstrap/3.3.6/css/bootstrap.min.css}"
rel="stylesheet"/>

    <!-- IE10 viewport hack for Surface/desktop Windows 8 bug -->
    <link th:href="@{/webjars/bootstrap/3.3.6/css/ie10-viewport-bug-
workaround.css}" rel="stylesheet"/>

    <!-- Custom styles for this template -->
    <link th:href="@{/showcase.css}" rel="stylesheet"/>

    <!-- Just for debugging purposes. Don't actually copy these 2 lines!
-->
    <!--[if lt IE 9]><script src="../../assets/js/ie8-responsive-file-

```

```

warning.js"></script><![endif]-->
<script th:src="@{/webjars/bootstrap/3.3.6/js/ie-emulation-modes-
warning.js}"></script>

<!-- HTML5 shim and Respond.js for IE8 support of HTML5 elements and
media queries -->
<!--[if lt IE 9]>
<script
src="https://oss.maxcdn.com/html5shiv/3.7.2/html5shiv.min.js"></script>
<script
src="https://oss.maxcdn.com/respond/1.4.2/respond.min.js"></script>
<![endif]-->
</head>

<body>

<div class="container">

<!-- Static navbar -->
<nav class="navbar navbar-default navbar-fixed-top">
<div class="container-fluid">
<div class="navbar-header">
<button type="button" class="navbar-toggle collapsed" data-
toggle="collapse" data-target="#navbar" aria-expanded="false" aria-
controls="navbar">
<span class="sr-only">Toggle navigation</span>
<span class="icon-bar"></span>
<span class="icon-bar"></span>
<span class="icon-bar"></span>
</button>
<a class="navbar-brand" th:href="@{/}">Spring Social NTZ
ClassRoom</a>
</div>
<div id="navbar" class="navbar-collapse collapse">
<ul class="nav navbar-nav">
<li><a th:href="@{/}">Home</a></li>
<li class="dropdown" sec:authorize="isAuthenticated()"><a
th:href="@{/connect}">Connections</a></li>
<li class="dropdown" sec:authorize="isAuthenticated()">
<a href="#" class="dropdown-toggle" data-
toggle="dropdown" role="button" aria-haspopup="true" aria-
expanded="false">Facebook <span class="caret"></span></a>
<ul class="dropdown-menu">
<li><a th:href="@{/connect/facebook}">Connection</a></li>
<li social:connected="facebook"><a
th:href="@{/facebook}">Facebook Profile</a></li>
<li social:connected="facebook"><a
th:href="@{/facebook/feed}">Feed</a></li>
<li social:connected="facebook"><a
th:href="@{/facebook/friends}">Friends</a></li>

```

```

        <li social:connected="facebook"><a
th:href="@{/facebook/albums}">Albums</a></li>
        </ul>
    </li>
    <li class="dropdown" sec:authorize="isAuthenticated()">
        <a href="#" class="dropdown-toggle" data-
toggle="dropdown" role="button" aria-haspopup="true" aria-
expanded="false">Twitter <span class="caret"></span></a>
        <ul class="dropdown-menu">
            <li><a th:href="@{/connect/twitter}">Connection</a></li>
            <li social:connected="twitter"><a
th:href="@{/twitter}">Twitter Profile</a></li>
            <li social:connected="twitter"><a
th:href="@{/twitter/timeline}">Timeline</a></li>
            <li social:connected="twitter"><a
th:href="@{/twitter/friends}">Friends</a></li>
            <li social:connected="twitter"><a
th:href="@{/twitter/followers}">Followers</a></li>
            <li social:connected="twitter"><a
th:href="@{/twitter/trends}">Trends</a></li>
        </ul>
    </li>
</ul>
<ul class="nav navbar-nav navbar-right">
    <li sec:authorize="!isAuthenticated()"><a
th:href="@{/signin}">Sign In</a></li>
    <li sec:authorize="isAuthenticated()"><a
href="javascript:document.LogoutForm.submit();">Sign out <span
sec:authentication="name">Bob</span></a></li>
</ul>
<form method="POST" th:action="@{/signout}"
name="LogoutForm"></form>
</div><!-- /.nav-collapse -->
</div><!-- /.container-fluid -->
</nav>

<div id="content" layout:fragment="content">
    Content goes here
</div>

</div> <!-- /container -->

<!-- Bootstrap core JavaScript
===== -->
<!-- Placed at the end of the document so the pages load faster -->
<script th:src="@{/webjars/jquery/1.11.1/jquery.min.js}"></script>
<!-- <script>window.jQuery || document.write('<script
src="../../assets/js/vendor/jquery.min.js"></script>')</script> -->
<script>

```

```
th:src="@{/webjars/bootstrap/3.3.6/js/bootstrap.min.js}"></script>
<!-- IE10 viewport hack for Surface/desktop Windows 8 bug -->
<script th:src="@{/webjars/bootstrap/3.3.6/js/ie10-viewport-bug-
workaround.js}"></script>
</body>
</html>
```

ratelimit.html:-

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
<head>
  <title>Spring Social NTZ Classroom</title>
</head>
<body>
  <div id="header">
    <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
  </div>

  <div id="content" layout:fragment="content">
    <h2>Rate limit exceeded</h2>
    <p>Sorry, but the rate limit for accessing <span
th:text="${providerId}"/> has been exceeded. Please try again later.</p>
  </div>
</body>
</html>
```

signin.html:-

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="layout">
<head>
  <title>Spring Social NTZ Classroom</title>
</head>
<body>
  <div id="header">
    <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
  </div>

  <div id="LeftNav">
    Left nav menu
  </div>

  <div id="content" layout:fragment="content">
```

```

<div class="jumbotron">
  

  <div class="project-title">
    Spring Social Samples
  </div>

  <h1>Sign In</h1>

  <form id="signin" th:action="@{/signin/authenticate}"
method="post">
    <input type="hidden" name="_csrf"
th:value="${_csrf.token}"></input>
    <div class="formInfo">
      <div class="error" th:if="${param.error eq
'bad_credentials'}">
        Your sign in information was incorrect.
        Please try again or <a
th:href="@{/signup}">sign up</a>.
      </div>
      <div class="error" th:if="${param.error eq
'multiple_users'}">
        Multiple local accounts are connected to
the provider account.
        Try again with a different provider or
with your username and password.
      </div>
    </div>
    <fieldset class="LoginForm">
      <label for="login">Username</label>
      <input id="login" name="username" type="text"
size="25"></input><br/>
      <label for="password">Password</label>
      <input id="password" name="password"
type="password" size="25"></input>
      <br/><br/>
      <button type="submit" class="btn btn-
form">Sign In</button>
    </fieldset><br/>

    <p>Some test user/password pairs you may use
are:</p>
    <ul>
      <li>habuma/tacos</li>
      <li>rclarkson/atlanta</li>
    </ul>

    <p>Or you can <a th:href="@{/signup}">signup</a>
with a new account.</p>

```



```

        </form>

        <!-- TWITTER SIGNIN -->
        <form id="tw_signin" th:action="@{/signin/twitter}"
method="POST">
            <input type="hidden" name="_csrf"
th:value="${_csrf.token}"></input>
            <button type="submit"></img></button>
        </form>

        <!-- FACEBOOK SIGNIN -->
        <form name="fb_signin" id="fb_signin"
th:action="@{/signin/facebook}" method="POST">
            <input type="hidden" name="_csrf"
th:value="${_csrf.token}"></input>
            <input type="hidden" name="scope"
value="read_stream,user_posts,user_photos"></input>
            <button type="submit"></img></button>
        </form>
    </div>
</div>
</body>
</html>

```

signup.html:-

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:social="http://spring.io/springsocial"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorator="Layout">
    <head>
        <title>Spring Social NTZ Classroom</title>
    </head>
    <body>
        <div id="header">
            <h1><a th:href="@{/}">Spring Social NTZ
ClassRoom</a></h1>
        </div>

        <div id="content" layout:fragment="content">

            <div class="jumbotron">
                

                <div class="project-title">
                    Spring Social Samples
                </div>
            </div>

```

```

    <h1>Sign Up</h1>
    <div th:if="${!#strings.isEmpty(message)}"
        th:class="${message.type.cssClass}"
        th:text="${message.text}">error message text</div>

    <form id="signup" th:action="@{/signup}" method="post"
th:object="${signupForm}">
        <input type="hidden" name="_csrf"
th:value="${_csrf.token}" />
        <div class="formInfo">
            <div class="error"
th:if="${#fields.hasErrors('*')}">Unable to sign up. Please fix the
errors below and resubmit.</div>
            </div>

            <fieldset>
                <label for="firstName">First Name <span
th:if="${#fields.hasErrors('firstName')}" class="error"
th:text="${#fields.errors('firstName')[0]}">Error</span></label>
                <input type="text" th:field="*{firstName}"
/><br/>
                <label for="lastName">Last Name <span
th:if="${#fields.hasErrors('lastName')}" class="error"
th:text="${#fields.errors('lastName')[0]}">Error</span></label>
                <input type="text" th:field="*{lastName}"
/><br/>
                <label for="username">Username <span
th:if="${#fields.hasErrors('username')}" class="error"
th:text="${#fields.errors('username')[0]}">Error</span></label>
                <input type="text" th:field="*{username}"
/><br/>
                <label for="password">Password<br/><span
th:if="${#fields.hasErrors('password')}" class="error"
th:text="${#fields.errors('password')[0]}">Error</span></label>
                <input type="password" th:field="*{password}"
/> <small>(at least 6 characters)</small>
            </fieldset>
            <br/>
            <button type="submit" class="btn btn-form">Sign
Up</button>
        </form>

    </div>

</div>

</body>

</html>

```

application.properties:-

```
spring.social.facebook.appId=< --- give generated facebook id --->
spring.social.facebook.appSecret=< --- give generated facebook
                                secret id --->
spring.social.twitter.appId=< --- give generated twitter id --->
spring.social.twitter.appSecret=< --- give generated twitter secret id---
>
```

messages.properties:-

```
twitter.icon=/social/twitter/t_logo-a.png
twitter.displayName=Twitter
facebook.icon=/social/facebook/f_logo.jpg
facebook.displayName=Facebook
linkedin.icon=/social/linkedin/connect_with_linkedin-logo-150x150.jpg
linkedin.displayName=LinkedIn
```

pom.xml:-

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.spring.social</groupId>
  <artifactId>SpringSocialAll</artifactId>
  <version>0.1.0</version>
  <packaging>jar</packaging>

  <name>SpringSocialAll</name>
  <description>Complete project for Spring Social Facebook & Twitter</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>1.5.4.RELEASE</version>
    <relativePath /> <!-- lookup parent from repository -->
  </parent>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <java.version>1.8</java.version>
    <spring-cloud.version>Dalston.SR1</spring-cloud.version>
  </properties>

  <dependencies>
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-social-
facebook</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-social-
twitter</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!--
https://mvnrepository.com/artifact/org.thymeleaf.extras/thymeleaf-extras-
springsecurity4 -->
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity4</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
</dependencies>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

</project>
```

SPRING ORM

Introduction:-

Jdbc Technology allows the programmers to develop DB software dependent persistence logic by using SQL queries due to this jdbc code is not portable across the multiple DB softwares.

ORM allows to develop objects based DB software independent persistence logic due to this ORM Mapping persistence logic is portable across the multiple DB softwares.

- ⇒ The process of mapping java class with DB table java class properties with DB table cols and making java class objects representing DB table records having synchronization between them is called o-r mapping.
- ⇒ Hibernate, ibatis, jdo, Toplink and etc... ORM softwares.
- ⇒ Spring ORM is not direct ORM software. It is spring module that provides abstraction layer on multiple ORM software like Hibernate, iBatis, JDO, JPA etc.... and simplifies o-r mapping persistence logic development by providing template classes to avoid boilerplate code problem.

Spring ORM Advantages:

- 1) **Common Transaction management** : instead of performing each ORM software programmatic Transaction, management we can go for common spring style declarative Trans management.
- 2) **Nested transaction support**: Most of the ORM s/w don't support Nested Transaction management directly. But spring supports Nested Transaction management . By using spring ORM we can add nested Transaction management support on ORM persistence logic.
- 3) **Portability** : gives portability to change one ORM software to another ORM software to develop persistence logic with minimum changes.
- 4) **Avoids Boilerplate code**: with the support of multiple Template classes we can avoid boiler plate code (Repeated code)
- 5) **CommonException handling**: Instead of Handling each ORM specific checked and unchecked Exceptions spring ORM throws them as common "DataAccessException" hierarchy classes based exceptions and etc....

Spring Hibernate Integration (Spring with Hibernate):

- ⇒ Spring with Hibernate application means developing Hibernate style object based OR mapping persistence logic by using HibernateTemplate class in the DAO class of spring application.
- ⇒ To create HibernateTemplate object we need HB sessionFactory object. We can use FactoryBeans like "LocalSessionFactory" or "AnnotationSessionFactory" classes as beans to create sessionFactory object by supplying HB configure properties/HB configure file and mapping annotation based Domain class/HLO (Hibernate Language class) and also a DataSource object. This HibernateTemplate object will be injected to DAO class to perform persistence operation.
- ⇒ **HibernateTemplate class important methods are:**
 - i) save(-), persist(-), update(-), load(-), get etc for single row operations.

- ii) `bulkUpdate(-)` ----> to perform HQL based non-select operations.
- iii) `find(-)` or `findXxx(-)` : To perform HQL/NativeSQL/criteria based bulk select operations.

Plain hibernate application

- 1) create configuration object and call `configure()` method. |
- 2) Create service registry object by using `serviceRegistryBuilder` | //Common logics
- 3) Create `SessionFactory` object |
- 4) Open session with DB software |
- 5) Begin Transaction. |
- 6) Perform persistence operation -----> //Application specific logics
- 7) Commit or Rollback Transaction management
- 8) Close session, `sessionfactory` object

NOTE:

In plain HB application we need to write both common and application specific logics.

Spring with Hibernate application (Using spring ORM)

- 1) Inject `HibernateTemplate` object
- 2) Perform persistence operation.

⇒ Executing HQL queries with named parameters:

```
List<EmployeeHLO> list=(List<EmployeeHLO>)ht. findByNameParam ("from EmployeeHLO where salary>=:min and salary<=:max", new String [] {"min","max"}, new Object[] {9000, 10000});
```

⇒ Executing HQL query with positional parameters:

```
List<EmployeeHLO> list=(List<EmployeeHLO>)h.find("from EmpolyeeHLO where salary>=? and salary<=?", 9000.0f, 10000.0f);
```

While working with Hibernate we place `Nativesql/HQL` query in mapping file or domain class as named queries to make them visible in multiple session objects of one or more DAO class.

⇒ Executing named HQLQuery with positional parameters:

In `EmployeeHLO.java` (Domain Class)

```
@NamedQuery(name="GET_EMPS_BY_SAL_RANGE", QUERY="from EmployeeHLO where salary>=? and salary<=?")
```

In DAO Class Method

```
List<EmployeeHLO>List=(ListEmployeeHLO)ht.findByNamedQuery("GET_EMPS_BY_SAL_RANGE", 9000.0f,10000.0f);
```

⇒ Executing Named HQLQuery with Named parameters:

In Domain class (`EmployeeHLO.java`)

```
@NamedQuery(name="GET_EMPS_BY_SAL_RANG", Query="from EmployeeHLO where salary>=:min")
```

In DAO class

```
List<EmployeeHLO>list=(List<EmployeeHLO>)ht.findNamedQueryAndNmaedParam("GETEMPS_BY_SAL_RNG", new String[]{"min"} new Object[] {9000.0f});
```

If `HibernateTemplate` class doesn't provide environment to use certain feature of plain Hibernate, then take the support of `hibernate Callback` interface, that gives `doInHibernate(Session session)`

method. Using this object we can finish that operation.

For example

We can't use native Sql query using HibernateTemplate, to make it possible we can use this Hibernate Callback Interface support.

Ex:

```
List<EmployeeHLO> list=ht.execute(new HibernateCallback<List<EmployeeHLO>>(){
{
@Override
public List<EmployeeHLO> doInHibernate(Session ses) throws HibernateException, SQLException
{
//Plain Hb code
SQLQuery query=ses.createSQLQuery("Select * from spemployee");
query.addEntity(EmployeeHLO.class);
List<EmployeeHLO>list=query.list();
return list;
}
}
```

NOTE:-

To perform TransactionManagement on Spring ORM based Hibernate persistence logics we should use HibernateTransactionManager which can be configured by injecting HibernateSessionFactoryObject.

Executing Named HQL queries with NamedParameters.(for more visibility of query)

Place NamedQuery in the domain class

```
@NamedQuery(name="AUTH_QUERY",query="SELECT COUNT(*) FROM UserHLO WHERE
UNAME=:un and pwd=:pass")
public class UserHLO{
// logic
}
```

In DAO class (write following code)

```
List<?> list=ht.findNamedQueryAndNamedParam("AUTH_QUERY", new String[]{"un","pass"} new
String[] {bo.getUname(),bo.getPwd()});
```

Executing HQL non select query with positional parameters:

```
Int cnt=ht.bulkUpdate("delete from UserHLO where unmae=?", "raja");
System.out.println("records effected:"+ cnt);
return cnt
```

DB Table:

SPEmployee

|---->eno number pk
|---->ename varchar
|---->salary float

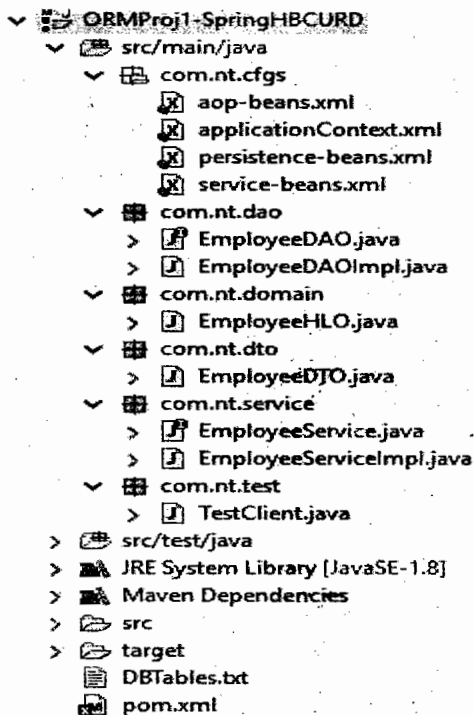
⇒ while importing certain framework/technology main jar file if you want to exclude certain unnecessary dependent jar files using maven then you can write <exclusions>, <exclusion>

tags in pom.xml file under dependency tag as shown below.

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.1.0.Final</version>
  <exclusions>
    <exclusion>
      <groupId>org.jboss</groupId>
      <artifactId>jandex</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Jar IOC: jars + Hibernate 3.6.5 jars+ AOP jar+ ojdbc 14.jar+Apache DPCP jar+Spring-ORM-4+SpjDBC jar+aspects.jar+aopalliance+cglib
(Hibernate 3.6.5 jars= Hibernate 3.6.5 Home/Hibernate 3.0,lib folder all jar files,lib -JPA folder 1 jar file).

ORMProj1-Xml-CURDOperations Example



EmployeeHLO.java:-

```
package com.nt.domain;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;

@Entity
@Table(name="Employee")
@NamedQuery(name="GET_EMP_DETAILS_BY_LNAME",
    query="from EmployeeHLO where lastName in(:surname1,:surname2,:surname3)")
public class EmployeeHLO {
    @Id
    @GenericGenerator(name="gen1",strategy="increment")
    @GeneratedValue(generator="gen1")
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

EmployeeDTO.java:-

```
package com.nt.dto;

import java.io.Serializable;

public class EmployeeDTO implements Serializable {
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [eid=" + eid + ", firstName=" + firstName + ", lastName=" +
        lastName + ", email=" + email
        + "]\n";
    }
}
```

EmployeeDAO.java:-

```
package com.nt.dao;

import java.util.List;
import com.nt.domain.EmployeeHLO;

public interface EmployeeDAO {

    public int insert(EmployeeHLO hlo);
    public List<EmployeeHLO> getAllEmployees();
    public int updateEmployeeEmail(int eid,String newEmail);
    public int deleteEmployeeByLastName(String surName);
    public List<EmployeeHLO> getEmployeesByEmailDomain(String domain);
    public List<EmployeeHLO> getEmployeesByLastName(String surName1,String
SurName2,String surName3);

}
```

EmployeeDAOImpl.java:-

```
package com.nt.dao;

import java.util.List;
import org.hibernate.query.Query;
import org.springframework.orm.hibernate5.HibernateTemplate;
import com.nt.domain.EmployeeHLO;

public class EmployeeDAOImpl implements EmployeeDAO {
    private static final String HQL_GET_EMPS="from EmployeeHLO";
    private static final String HQL_UPDATE_EMAIL_BY_NO="update EmployeeHLO set email=?
where eid=?";
    private static final String HQL_DELETE_EMPS_BY_LASTNAME="delete from EmployeeHLO
where lastName=:lname";
    private static final String HQL_GET_EMPS_BY_EMAIL_DOMAIN="from EmployeeHLO
where email like :dmn";
    private HibernateTemplate ht;

    public void setHt(HibernateTemplate ht) {
        this.ht = ht;
    }
    @Override
    public int insert(EmployeeHLO hlo) {
        int idVal=0;
        //save object
        idVal=(int) ht.save(hlo);
    }
}
```

```
        return idVal;
    }

    @Override
    public List<EmployeeHLO> getAllEmployees() {
        List<EmployeeHLO> listHLO;
        listHLO= (List<EmployeeHLO>) ht.find(HQL_GET_EMPS);
        return listHLO;
    }

    @Override
    public int updateEmployeeEmail(int eid, String newEmail) {
        return ht.bulkUpdate(HQL_UPDATE_EMAIL_BY_NO, newEmail,eid);
    }

    @Override
    public int deleteEmployeeByLastName(String surName) {
        int count=0;
        //count=ht.execute(new MyHibernateCallback(surName));
        /*
        count=ht.execute(new HibernateCallback<Integer>() {
            @Override
            public Integer doInHibernate(Session ses) throws HibernateException {
                Query query=null;
                int count=0;
                //create Query object
                query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
                //set param values
                query.setParameter("lname",surName);
                count=query.executeUpdate();
                return count;
            }
        }); */
        count=ht.execute(ses->{
            Query query=null;
            int cnt=0;
            //create Query object
            query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
            //set param values
            query.setParameter("lname",surName);
            cnt=query.executeUpdate();
            return cnt;
        });

        return count;
    }
} //method
```



```
public interface EmployeeService {  
    public String registerEmployee(EmployeeDTO dto);  
    public List<EmployeeDTO> fetchAllEmployees();  
    public String modifyEmailByNo(int no,String newEmail);  
    public String removeEmployeeByLastName(String surName);  
    public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain);  
    public List<EmployeeDTO> fetchEmployeesByLastName(String surname1,String  
surname2,String surname3);  
}
```

EmployeeServiceImpl.java:-

```
package com.nt.service;  
  
import java.util.ArrayList;  
import java.util.List;  
import org.springframework.beans.BeanUtils;  
import com.nt.dao.EmployeeDAO;  
import com.nt.domain.EmployeeHLO;  
import com.nt.dto.EmployeeDTO;  
  
public class EmployeeServiceImpl implements EmployeeService {  
    private EmployeeDAO dao;  
  
    public void setDao(EmployeeDAO dao) {  
        this.dao = dao;  
    }  
  
    @Override  
    public String registerEmployee(EmployeeDTO dto) {  
        EmployeeHLO hlo=null;  
        int result=0;  
        //Convert DTO To BO  
        hlo=new EmployeeHLO();  
        BeanUtils.copyProperties(dto,hlo);  
        //use DAO  
        result=dao.insert(hlo);  
        if(result==0)  
            return "Registration failed";  
        else  
            return "Employee Registered with Number:"+result;  
    }  
}
```

```
@Override
public List<EmployeeDTO> fetchAllEmployees() {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getAllEmployees();
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method

@Override
public String modifyEmailByNo(int no, String newEmail) {
    int count=0;
    //use DAO
    count=dao.updateEmployeeEmail(no,newEmail);
    if(count==0)
        return "Record not updated";
    else
        return "Record updated";
} //method

@Override
public String removeEmployeeByLastName(String surName) {
    int count=0;
    //use dAO
    count=dao.deleteEmployeeByLastName(surName);
    if(count==0)
        return "Employee(s) Not Deleted";
    else
        return "Employee(s) Deleted";
} //method

@Override
public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain) {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByEmailDomain(domain);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
```



```
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
}

@Override
public List<EmployeeDTO> fetchEmployeesByLastName(String surname1, String
surname2, String surname3) {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByLastName(surname1, surname2, surname3);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method

} //class
```

TestClient.java:-

```
package com.nt.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

public class TestClient {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;
        //create IOC container
        ctx=new
        FileSystemXmlApplicationContext("src/main/java/com/nt/cfgs/applicationContext.xml");
        //get Bean
        service=ctx.getBean("empService",EmployeeService.class);
        //prepare DTO
        dto=new EmployeeDTO();
        dto.setFirstName("faja");
    }
}
```

```

dto.setLastName("Iraja");dto.setEmail("mail.raja@gmail.com");
    //invoke B.method
    //System.out.println(service.registerEmployee(dto));
    //System.out.println("All Emp Details:"+service.fetchAllEmployees());
    //System.out.println(service.modifyEmailByNo(1, "raja@King.com"));
    //System.out.println(service.removeEmployeeByLastName("Iraja"));
    //System.out.println(service.fetchEmployeesByEmailDomain("gmail.com"));
    System.out.println(service.fetchEmployeesByLastName("rao","chari","reddy"));
    //close container
    ((AbstractApplicationContext) ctx).close();
}
}

```

aop-beans.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/tx
        http://www.springframework.org/schema/tx/spring-tx.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="hbTxMgmr"
        class="org.springframework.orm.hibernate5.HibernateTransactionManager">
        <property name="sessionFactory" ref="sesfact"/>
    </bean>

    <!-- Cfg TxMgmr AspectJAOP Advice -->
    <tx:advice id="txAdvice" transaction-manager="hbTxMgmr">
    <tx:attributes>
        <tx:method name="*" read-only="false" propagation="REQUIRED"/>
    </tx:attributes>
    </tx:advice>
    <aop:config>
        <aop:pointcut id="ptc" expression="execution(* com.nt.service.EmployeeService.*(..))"/>
        <aop:advisor pointcut-ref="ptc" advice-ref="txAdvice"/>
    </aop:config>
</beans>

```

Persistence-beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans

```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!-- Configure DataSource -->
    <bean id="dbcp" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="system"/>
        <property name="password" value="manager"/>
        <property name="initialSize" value="10"/>
        <property name="maxTotal" value="200"/>
    </bean>
    <bean id="sesfact" class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
        <property name="dataSource" ref="dbcp"/>
        <property name="annotatedClasses">
            <list>
                <value>com.nt.domain.EmployeeHLO</value>
            </list>
        </property>
        <property name="hibernateProperties">
            <props>
                <prop key="hibernate.show_sql">true</prop>
                <prop key="hibernate.format_sql">true</prop>
                <prop key="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prop>
                <prop key="connection.autocommit">true</prop>
            </props>
        </property>
    </bean>
    <bean id="template" class="org.springframework.orm.hibernate5.HibernateTemplate">
        <property name="sessionFactory" ref="sesfact"/>
    </bean>
    <bean id="empDAO" class="com.nt.dao.EmployeeDAOImpl">
        <property name="ht" ref="template"/>
    </bean>
</beans>
```

service-beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="empService" class="com.nt.service.EmployeeServiceImpl">
        <property name="dao" ref="empDAO"/>
    </bean>
</beans>
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<import resource="service-beans.xml"/>
<import resource="persistence-beans.xml"/>
<import resource="aop-beans.xml"/>
</beans>
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>nit</groupId>
    <artifactId>ORMProj1-SpringHBCURD</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>ORMProj1-SpringHBCURD</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>
        <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
        <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>5.2.10.Final</version>
        </dependency>

        <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>4.3.8.RELEASE</version>
        </dependency>
```

```
support -->
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context-support</artifactId>
        <version>4.3.8.RELEASE</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-orm -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-orm</artifactId>
        <version>4.3.8.RELEASE</version>
    </dependency>

    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-dbcp2</artifactId>
        <version>2.1.1</version>
    </dependency>

    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.8.10</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.aspectj/aspectjrt -->
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjrt</artifactId>
        <version>1.8.10</version>
    </dependency>
    <dependency>
        <groupId>com.oracle</groupId>
        <artifactId>ojdbc6</artifactId>
        <version>11.2.0</version>
    </dependency>

</dependencies>
</project>
```

ORMProj2-xml+Annotations-CURDOperations Example

```
=====
v [icon] ORMPProj2-SpringHBCURD-XML+Annotations
v [icon] src/main/java
  v [icon] com.nt.cfgs
    [icon] aop-beans.xml
    [icon] applicationContext.xml
    [icon] persistence-beans.xml
    [icon] service-beans.xml
  v [icon] com.nt.dao
    > [icon] EmployeeDAO.java
    > [icon] EmployeeDAOImpl.java
  v [icon] com.nt.domain
    > [icon] EmployeeHLO.java
  v [icon] com.nt.dto
    > [icon] EmployeeDTO.java
  v [icon] com.nt.service
    > [icon] EmployeeService.java
    > [icon] EmployeeServiceImpl.java
  v [icon] com.nt.test
    > [icon] TestClient.java
> [icon] src/test/java
> [icon] JRE System Library [JavaSE-1.8]
> [icon] Maven Dependencies
> [icon] src
> [icon] target
  [icon] DBTables.txt
  [icon] pom.xml
```

EmployeeHLO.java

```
package com.nt.domain;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;

@Entity
@Table(name="Employee")
@NamedQuery(name="GET_EMP_DETAILS_BY_LNAME",
    query="from EmployeeHLO where lastName in(:surname1,:surname2,:surname3)")
public class EmployeeHLO {
    @Id
    @GenericGenerator(name="gen1",strategy="increment")
    @GeneratedValue(generator="gen1")
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
}
```

```
}  
public void setEid(int eid) {  
    this.eid = eid;  
}  
public String getFirstName() {  
    return firstName;  
}  
public void setFirstName(String firstName) {  
    this.firstName = firstName;  
}  
public String getLastName() {  
    return lastName;  
}  
public void setLastName(String lastName) {  
    this.lastName = lastName;  
}  
public String getEmail() {  
    return email;  
}  
public void setEmail(String email) {  
    this.email = email;  
}  
}
```

EmployeeDTO.java:-

```
package com.nt.dto;  
  
import java.io.Serializable;  
  
public class EmployeeDTO implements Serializable {  
    private int eid;  
    private String firstName;  
    private String lastName;  
    private String email;  
  
    public int getEid() {  
        return eid;  
    }  
    public void setEid(int eid) {  
        this.eid = eid;  
    }  
    public String getFirstName() {  
        return firstName;  
    }  
    public void setFirstName(String firstName) {  
        this.firstName = firstName;  
    }  
    public String getLastName() {
```

```
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [eid=" + eid + ", firstName=" + firstName + ", lastName=" +
lastName + ", email=" + email
        + "]";
    }
}
```

EmployeeDAO.java

```
package com.nt.dao;

import java.util.List;
import com.nt.domain.EmployeeHLO;

public interface EmployeeDAO {

    public int insert(EmployeeHLO hlo);
    public List<EmployeeHLO> getAllEmployees();
    public int updateEmployeeEmail(int eid,String newEmail);
    public int deleteEmployeeByLastName(String surName);
    public List<EmployeeHLO> getEmployeesByEmailDomain(String domain);
    public List<EmployeeHLO> getEmployeesByLastName(String surName1,String
SurName2,String surName3);
}
```

EmployeeDAOImpl.java

```
package com.nt.dao;

import java.util.List;
import javax.annotation.Resource;
import org.hibernate.query.Query;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.stereotype.Repository;

import com.nt.domain.EmployeeHLO;
```



```
@Repository("empDAO")
public class EmployeeDAOImpl implements EmployeeDAO {
    private static final String HQL_GET_EMPS="from EmployeeHLO";
    private static final String HQL_UPDATE_EMAIL_BY_NO="update EmployeeHLO set email=?
where eid=?";
    private static final String HQL_DELETE_EMPS_BY_LASTNAME="delete from EmployeeHLO
where lastName=:lname";
    private static final String HQL_GET_EMPS_BY_EMAIL_DOMAIN="from EmployeeHLO
where email like :dmn";

    @Resource
    private HibernateTemplate ht;

    @Override
    public int insert(EmployeeHLO hlo) {
        int idVal=0;
        //save object
        idVal=(int) ht.save(hlo);
        return idVal;
    }

    @Override
    public List<EmployeeHLO> getAllEmployees() {
        List<EmployeeHLO> listHLO;
        listHLO= (List<EmployeeHLO>) ht.find(HQL_GET_EMPS);
        return listHLO;
    }

    @Override
    public int updateEmployeeEmail(int eid, String newEmail) {
        return ht.bulkUpdate(HQL_UPDATE_EMAIL_BY_NO, newEmail,eid);
    }

    @Override
    public int deleteEmployeeByLastName(String surName) {
        int count=0;
        //count=ht.execute(new MyHibernateCallback(surName));
        /* count=ht.execute(new HibernateCallback<Integer>() {
            @Override
            public Integer doInHibernate(Session ses) throws HibernateException {
                Query query=null;
                int count=0;
                //create Query object
                query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
                //set param values
                query.setParameter("lname",surName);
                count=query.executeUpdate();
            }
        });
    }
}
```

```
        return count;
    }
    }); */
    count=ht.execute(ses->{
        Query query=null;
        int cnt=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname",surName);
        cnt=query.executeUpdate();
        return cnt;
    });

    return count;
} //method

@Override
public List<EmployeeHLO> getEmployeesByEmailDomain(String domain) {
    List<EmployeeHLO> listHLO=null;
    listHLO=(List<EmployeeHLO>)
ht.findByNameParam(HQL_GET_EMPS_BY_EMAIL_DOMAIN, "dmn","%" +domain);
    return listHLO;
}

@Override
public List<EmployeeHLO> getEmployeesByLastName(String surName1, String surName2,
String surName3) {
    List<EmployeeHLO> listHLO=null;
    //get Access to Named HQL Query
    listHLO=(List<EmployeeHLO>)
ht.findByNameQueryAndNamedParam("GET_EMP_DETAILS_BY_LNAME",
                                new String[]{"surname1","surname2","surname3"},
                                new String[]{surName1,surName2,surName3});
    return listHLO;
}

/* private class MyHibernateCallback implements HibernateCallback<Integer>{
    private String surname;

    public MyHibernateCallback(String surName){
        this.surname=surName;
    }

    @Override
    public Integer doInHibernate(Session ses) throws HibernateException {
        Query query=null;
```

```
        int count=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname",surname);
        count=query.executeUpdate();
        return count;
    } //method

} //inner class */

}
```

EmployeeService.java:-

```
package com.nt.service;

import java.util.List;
import com.nt.dto.EmployeeDTO;

public interface EmployeeService {
    public String registerEmployee(EmployeeDTO dto);
    public List<EmployeeDTO> fetchAllEmployees();
    public String modifyEmailByNo(int no,String newEmail);
    public String removeEmployeeByLastName(String surName);
    public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain);
    public List<EmployeeDTO> fetchEmployeesByLastName(String surname1,String
surname2,String surname3);
}
```

EmployeeServiceImpl.java

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import javax.annotation.Resource;
import org.springframework.beans.BeanUtils;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.EmployeeDAO;
import com.nt.domain.EmployeeHLO;
import com.nt.dto.EmployeeDTO;

@Service("empService")
@Transactional(propagation=Propagation.REQUIRED,readOnly=false)
public class EmployeeServiceImpl implements EmployeeService {
    @Resource
    private EmployeeDAO dao;
```

@Override

```
public String registerEmployee(EmployeeDTO dto) {
    EmployeeHLO hlo=null;
    int result=0;
    //Convert DTO To BO
    hlo=new EmployeeHLO();
    BeanUtils.copyProperties(dto,hlo);
    //use DAO
    result=dao.insert(hlo);
    if(result==0)
        return "Registration failed";
    else
        return "Employee Registered with Number:"+result;
} //method
```

@Override

```
public List<EmployeeDTO> fetchAllEmployees() {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getAllEmployees();
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method
```

@Override

```
public String modifyEmailByNo(int no, String newEmail) {
    int count=0;
    //use DAO
    count=dao.updateEmployeeEmail(no,newEmail);
    if(count==0)
        return "Record not updated";
    else
        return "Record updated";
} //method
```

@Override

```
public String removeEmployeeByLastName(String surName) {
    int count=0;
```

```
//use dao
count=dao.deleteEmployeeByLastName(surName);
if(count==0)
    return "Employee(s) Not Deleted";
else
    return "Employee(s) Deleted";
} //method

@Override
public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain) {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByEmailDomain(domain);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
}

@Override
public List<EmployeeDTO> fetchEmployeesByLastName(String surname1, String surname2,
String surname3) {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByLastName(surname1, surname2, surname3);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method

} //class
```

TestClient.java

```
package com.nt.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
```

```
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

public class TestClient {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;
        //create IOC container
        ctx=new
        FileSystemXmlApplicationContext("src/main/java/com/nt/cfgs/applicationContext.xml");
        //get Bean
        service=ctx.getBean("empService",EmployeeService.class);
        //prepare DTO
        dto=new EmployeeDTO();
        dto.setFirstName("faja");
        dto.setLastName("Iraja");dto.setEmail("mail.raja@gmail.com");
        //invoke B.method
        System.out.println(service.registerEmployee(dto));
        //System.out.println("All Emp Details::"+service.fetchAllEmployees());
        //System.out.println(service.modifyEmailByNo(1, "raja@King.com"));
        //System.out.println(service.removeEmployeeByLastName("Iraja"));
        //System.out.println(service.fetchEmployeesByEmailDomain("gmail.com"));
        //System.out.println(service.fetchEmployeesByLastName("rao", "chari", "reddy"));

        //close container
        ((AbstractApplicationContext) ctx).close();
    }
}
```

persistence-beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <!-- Configure DataSource -->
    <bean id="dbcp" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
        <property name="username" value="system" />
        <property name="password" value="manager" />
    </bean>
</beans>
```

```
<property name="initialSize" value="10" />
<property name="maxTotal" value="200" />
</bean>
<bean id="sesfact"
      class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
  <property name="dataSource" ref="dbcp" />
  <property name="annotatedClasses">
    <list>
      <value>com.nt.domain.EmployeeHLO</value>
    </list>
  </property>
  <property name="hibernateProperties">
    <props>
      <prop key="hibernate.show_sql">true</prop>
      <prop key="hibernate.format_sql">true</prop>
      <prop
key="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prop>
      <prop key="connection.autocommit">true</prop>
    </props>
  </property>
</bean>
<bean id="template" class="org.springframework.orm.hibernate5.HibernateTemplate">
  <property name="sessionFactory" ref="sesfact" />
</bean>
<context:component-scan base-package="com.nt.dao"/>

</beans>
```

service-beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

  <context:component-scan base-package="com.nt.service"/>
</beans>
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<import resource="service-beans.xml"/>
<import resource="persistence-beans.xml"/>
<import resource="aop-beans.xml"/>
</beans>
```

aop-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">

  <bean id="hbTxMgmr"
class="org.springframework.orm.hibernate5.HibernateTransactionManager">
    <property name="sessionFactory" ref="sesfact"/>
  </bean>

  <tx:annotation-driven transaction-manager="hbTxMgmr"/>
</beans>
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>nit</groupId>
  <artifactId>ORMProj1-SpringHBCURD</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>ORMProj1-SpringHBCURD</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
```



```
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.2.10.Final</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>4.3.8.RELEASE</version>
</dependency>

support -->
<!-- https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context-support</artifactId>
    <version>4.3.8.RELEASE</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.springframework/spring-orm -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-orm</artifactId>
    <version>4.3.8.RELEASE</version>
</dependency>

<dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-dbcp2</artifactId>
    <version>2.1.1</version>
</dependency>

<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.8.10</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.aspectj/aspectjrt -->
<dependency>
    <groupId>org.aspectj</groupId>
```

```
        <artifactId>aspectjrt</artifactId>
        <version>1.8.10</version>
    </dependency>

    <dependency>
        <groupId>com.oracle</groupId>
        <artifactId>ojdbc6</artifactId>
        <version>11.2.0</version>
    </dependency>

</dependencies>
</project>
```

ORMProj3-100pCode-CURDOperations Example

```

v [M] ORMPProj3-SpringHBCURD-100pCode
v [C] src/main/java
  v [C] com.nt.config
    > [C] AOPConfig.java
    > [C] AppConfig.java
    > [C] PersistenceConfig.java
    > [C] ServiceConfig.java
  v [C] com.nt.dao
    > [C] EmployeeDAO.java
    > [C] EmployeeDAOImpl.java
  v [C] com.nt.domain
    > [C] EmployeeHLO.java
  v [C] com.nt.dto
    > [C] EmployeeDTO.java
  v [C] com.nt.service
    > [C] EmployeeService.java
    > [C] EmployeeServiceImpl.java
  v [C] com.nt.test
    > [C] TestClient.java
> [C] src/test/java
> [C] JRE System Library [JavaSE-1.8]
> [C] Maven Dependencies
> [C] src
> [C] target
  [C] DBTables.txt
  [C] pom.xml
```

EmployeeHLO.java

```
package com.nt.domain;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
```

```
import javax.persistence.Id;
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;

@Entity
@Table(name="Employee")
@NamedQuery(name="GET_EMP_DETAILS_BY_LNAME",
    query="from EmployeeHLO where lastName in(:surname1,:surname2,:surname3)")
public class EmployeeHLO {
    @Id
    @GenericGenerator(name="gen1",strategy="increment")
    @GeneratedValue(generator="gen1")
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

EmployeeDTO.java

```
package com.nt.dto;
```

```
import java.io.Serializable;

public class EmployeeDTO implements Serializable {
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [eid=" + eid + ", firstName=" + firstName + ", lastName=" +
        lastName + ", email=" + email
            + "]";
    }
}
```

EmployeeDAO.java

```
package com.nt.dao;

import java.util.List;
import com.nt.domain.EmployeeHLO;

public interface EmployeeDAO {
```

```
public int insert(EmployeeHLO hlo);
public List<EmployeeHLO> getAllEmployees();
public int updateEmployeeEmail(int eid,String newEmail);
public int deleteEmployeeByLastName(String surName);
public List<EmployeeHLO> getEmployeesByEmailDomain(String domain);
public List<EmployeeHLO> getEmployeesByLastName(String surName1,String
SurName2,String surName3);
}
```

EmployeeDAOImpl.java

```
package com.nt.dao;

import java.util.List;
import javax.annotation.Resource;
import org.hibernate.query.Query;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.stereotype.Repository;
import com.nt.domain.EmployeeHLO;

@Repository("empDAO")
public class EmployeeDAOImpl implements EmployeeDAO {
    private static final String HQL_GET_EMPS="from EmployeeHLO";
    private static final String HQL_UPDATE_EMAIL_BY_NO="update EmployeeHLO set email=?
where eid=?";
    private static final String HQL_DELETE_EMPS_BY_LASTNAME="delete from EmployeeHLO
where lastName=:lname";
    private static final String HQL_GET_EMPS_BY_EMAIL_DOMAIN="from EmployeeHLO
where email like :dmn";

    @Resource
    private HibernateTemplate ht;

    @Override
    public int insert(EmployeeHLO hlo) {
        int idVal=0;
        //save object
        idVal=(int) ht.save(hlo);
        return idVal;
    }

    @Override
    public List<EmployeeHLO> getAllEmployees() {
        List<EmployeeHLO> listHLO;
        listHLO= (List<EmployeeHLO>) ht.find(HQL_GET_EMPS);
        return listHLO;
    }
}
```

```
@Override
public int updateEmployeeEmail(int eid, String newEmail) {
    return ht.bulkUpdate(HQL_UPDATE_EMAIL_BY_NO, newEmail, eid);
}

@Override
public int deleteEmployeeByLastName(String surName) {
    int count=0;
    //count=ht.execute(new MyHibernateCallback(surName));
    /*
    count=ht.execute(new HibernateCallback<Integer>() {
        @Override
        public Integer doInHibernate(Session ses) throws HibernateException {
            Query query=null;
            int count=0;
            //create Query object
            query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
            //set param values
            query.setParameter("lname",surName);
            count=query.executeUpdate();
            return count;
        }
    }); */
    count=ht.execute(ses->{
        Query query=null;
        int cnt=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname",surName);
        cnt=query.executeUpdate();
        return cnt;
    });

    return count;
}

@Override
public List<EmployeeHLO> getEmployeesByEmailDomain(String domain) {
    List<EmployeeHLO> listHLO=null;
    listHLO=(List<EmployeeHLO>)
ht.findByNameParam(HQL_GET_EMPS_BY_EMAIL_DOMAIN, "dmn","%" + domain);
    return listHLO;
}

@Override
public List<EmployeeHLO> getEmployeesByLastName(String surName1, String surName2,
String surName3) {
```

```

List<EmployeeHLO> listHLO=null;
//get Access to Named HQL Query
listHLO=(List<EmployeeHLO>)
ht.findByNameQueryAndNamedParam("GET_EMP_DETAILS_BY_LNAME",
                                new String[]{"surname1","surname2","surname3"},
                                new String[]{surName1,surName2,surName3});

return listHLO;
}

/* private class MyHibernateCallback implements HibernateCallback<Integer>{
    private String surname;

    public MyHibernateCallback(String surName){
        this.surname=surName;
    }

    @Override
    public Integer doInHibernate(Session ses) throws HibernateException {
        Query query=null;
        int count=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname",surname);
        count=query.executeUpdate();
        return count;
    }
}

}

```

EmployeeService.java

```
package com.nt.service;

import java.util.List;
import com.nt.dto.EmployeeDTO;

public interface EmployeeService {
    public String registerEmployee(EmployeeDTO dto);
    public List<EmployeeDTO> fetchAllEmployees();
    public String modifyEmailByNo(int no,String newEmail);
    public String removeEmployeeByLastName(String surName);
    public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain);
    public List<EmployeeDTO> fetchEmployeesByLastName(String surname1,String
surname2,String surname3);
}
```

EmployeeServiceImpl.java

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import javax.annotation.Resource;
import org.springframework.beans.BeanUtils;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.EmployeeDAO;
import com.nt.domain.EmployeeHLO;
import com.nt.dto.EmployeeDTO;

@Service("empService")
@Transactional(propagation=Propagation.REQUIRED,readOnly=false,transactionManager="hbTxMgr")
public class EmployeeServiceImpl implements EmployeeService {
    @Resource
    private EmployeeDAO dao;
    @Override
    public String registerEmployee(EmployeeDTO dto) {
        EmployeeHLO hlo=null;
        int result=0;
        //Convert DTO To BO
        hlo=new EmployeeHLO();
        BeanUtils.copyProperties(dto,hlo);
        //use DAO
        result=dao.insert(hlo);
        if(result==0)
            return "Registration failed";
        else
            return "Employee Registered with Number:"+result;
    }
    //method

    @Override
    public List<EmployeeDTO> fetchAllEmployees() {
        List<EmployeeDTO> listDTO=new ArrayList();
        List<EmployeeHLO> listHLO=null;

        //use DAO
        listHLO=dao.getAllEmployees();
        //convert listHLO to ListDTO
        listHLO.forEach(hlo->{
            EmployeeDTO dto=new EmployeeDTO();
            BeanUtils.copyProperties(hlo,dto);
            listDTO.add(dto);
        });
    }
}
```



```
        return listDTO;
    } //method

    @Override
    public String modifyEmailByNo(int no, String newEmail) {
        int count=0;
        //use DAO
        count=dao.updateEmployeeEmail(no,newEmail);
        if(count==0)
            return "Record not updated";
        else
            return "Record updated";
    } //method

    @Override
    public String removeEmployeeByLastName(String surName) {
        int count=0;
        //use dAO
        count=dao.deleteEmployeeByLastName(surName);
        if(count==0)
            return "Employee(s) Not Deleted";
        else
            return "Employee(s) Deleted";
    } //method

    @Override
    public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain) {
        List<EmployeeDTO> listDTO=new ArrayList();
        List<EmployeeHLO> listHLO=null;

        //use DAO
        listHLO=dao.getEmployeesByEmailDomain(domain);
        //convert listHLO to ListDTO
        listHLO.forEach(hlo->{
            EmployeeDTO dto=new EmployeeDTO();
            BeanUtils.copyProperties(hlo,dto);
            listDTO.add(dto);
        });
        return listDTO;
    }

    @Override
    public List<EmployeeDTO> fetchEmployeesByLastName(String surname1, String surname2,
String surname3) {
        List<EmployeeDTO> listDTO=new ArrayList();
        List<EmployeeHLO> listHLO=null;

        //use DAO
```

```
listHLO=dao.getEmployeesByLastName(surname1, surname2, surname3);
//convert listHLO to ListDTO
listHLO.forEach(hlo->{
    EmployeeDTO dto=new EmployeeDTO();
    BeanUtils.copyProperties(hlo,dto);
    listDTO.add(dto);
});
return listDTO;
} //method
} //class
```

TestClient.java

```
package com.nt.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
import com.nt.config.AppConfig;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

public class TestClient {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;
        //create IOC container
        ctx=new AnnotationConfigApplicationContext(AppConfig.class);
        //get Bean
        service=ctx.getBean("empService",EmployeeService.class);
        //prepare DTO
        dto=new EmployeeDTO();
        dto.setFirstName("fraja");
        dto.setLastName("Iraja");dto.setEmail("mail.raja@gmail.com");
        //invoke B.method
        System.out.println(service.registerEmployee(dto));
        //System.out.println("All Emp Details::"+service.fetchAllEmployees());
        //System.out.println(service.modifyEmailByNo(1, "raja@King.com"));
        //System.out.println(service.removeEmployeeByLastName("Iraja"));
        //System.out.println(service.fetchEmployeesByEmailDomain("gmail.com"));
        //System.out.println(service.fetchEmployeesByLastName("rao","chari","reddy"));

        //close container
        ((AbstractApplicationContext) ctx).close();
    }
}
```

```
}
```

AopConfig.java

```
package com.nt.config;

import org.hibernate.SessionFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableTransactionManagement
public class AOPConfig {

    @Autowired
    @Qualifier("sesfact")
    private SessionFactory factory;

    @Bean("hbTxMgmr")
    public HibernateTransactionManager createHBTxMgmr(){
        return new HibernateTransactionManager(factory);
    }

}
```

AppConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(value={PersistenceConfig.class,ServiceConfig.class,AOPConfig.class})
public class AppConfig {

}
```

PersistenceConfig.java

```
package com.nt.config;

import java.util.Properties;
import javax.sql.DataSource;
import org.apache.commons.dbcp2.BasicDataSource;
import org.hibernate.SessionFactory;
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import com.nt.domain.EmployeeHLO;

@Configuration
@ComponentScan(basePackages="com.nt.dao")
public class PersistenceConfig {

    @Bean(name="dbcpds")
    public DataSource createDataSource(){
        BasicDataSource cpds=null;
        cpds=new BasicDataSource();
        cpds.setDriverClassName("oracle.jdbc.driver.OracleDriver");
        cpds.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
        cpds.setUsername("system");
        cpds.setPassword("manager");
        return cpds;
    }

    @Bean(name="localSesFact")
    public LocalSessionFactoryBean createLocalSessionFactoryBean(){

        LocalSessionFactoryBean bean=null;
        Properties props=null;

        bean=new LocalSessionFactoryBean();
        bean.setAnnotatedClasses(EmployeeHLO.class);
        bean.setDataSource(createDataSource());

        props=new Properties();
        props.put("hibernate.show_sql","true");
        props.put("hibernate.format_sql","true");
        props.put("hibernate.dialect","org.hibernate.dialect.Oracle10gDialect");
        bean.setHibernateProperties(props);
        return bean;
    }

    @Bean(name="sesfact")
    public SessionFactory createSessionFactory(){
        return createLocalSessionFactoryBean().getObject();
    }

    @Bean("ht")
    public HibernateTemplate createHibernateTemplate(){
        return new HibernateTemplate(createSessionFactory());
    }
}
```

```
}//class
```

ServiceConfig.java

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

Spring Boot ORM:-

- ✓  BootORM-CURDOperations [boot]
 - >  Spring Elements
 - ✓  src/main/java
 - ✓  com.nt.config
 - >  AOPConfig.java
 - >  PersistenceConfig.java
 - >  ServiceConfig.java
 - ✓  com.nt.dao
 - >  EmployeeDAO.java
 - >  EmployeeDAOImpl.java
 - ✓  com.nt.domain
 - >  EmployeeHLO.java
 - ✓  com.nt.dto
 - >  EmployeeDTO.java
 - ✓  com.nt.service
 - >  EmployeeService.java
 - >  EmployeeServiceImpl.java
 - ✓  com.nt.test
 - >  BootOrmCurdOperationsApplication.java
 - ✓  src/main/resources
 -  application.properties
 - >  src/test/java
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 - >  src
 - >  target
 -  mvnw
 -  mvnw.cmd
 -  pom.xml

EmployeeHLO.java:-

```
package com.nt.domain;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
```

```
import javax.persistence.NamedQuery;
import javax.persistence.Table;
import org.hibernate.annotations.GenericGenerator;

@Entity
@Table(name="Employee")
@NamedQuery(name="GET_EMP_DETAILS_BY_LNAME",
            query="from EmployeeHLO where lastName
in(:surname1,:surname2,:surname3)")
public class EmployeeHLO {
    @Id
    @GenericGenerator(name="gen1",strategy="increment")
    @GeneratedValue(generator="gen1")
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```

EmployeeDTO.java:-

```
package com.nt.dto;

import java.io.Serializable;
```

```
public class EmployeeDTO implements Serializable {
    private int eid;
    private String firstName;
    private String lastName;
    private String email;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [eid=" + eid + ", firstName=" + firstName
+ ", lastName=" + lastName + ", email=" + email
        + "]";
    }
}
```

EmployeeDAO.java:-

```
package com.nt.dao;

import java.util.List;
import com.nt.domain.EmployeeHLO;

public interface EmployeeDAO {

    public int insert(EmployeeHLO hlo);
    public List<EmployeeHLO> getAllEmployees();
    public int updateEmployeeEmail(int eid,String newEmail);
}
```

```
public int deleteEmployeeByLastName(String surName);
public List<EmployeeHLO> getEmployeesByEmailDomain(String domain);
public List<EmployeeHLO> getEmployeesByLastName(String
surName1,String SurName2,String surName3);
}
```

EmployeeDAOImpl.java:-

```
package com.nt.dao;

import java.util.List;
import com.nt.domain.EmployeeHLO;

public interface EmployeeDAO {

    public int insert(EmployeeHLO hlo);
    public List<EmployeeHLO> getAllEmployees();
    public int updateEmployeeEmail(int eid,String newEmail);
    public int deleteEmployeeByLastName(String surName);
    public List<EmployeeHLO> getEmployeesByEmailDomain(String domain);
    public List<EmployeeHLO> getEmployeesByLastName(String
surName1,String SurName2,String surName3);
}
```

EmployeeDAOImpl.java:-

```
package com.nt.dao;

import java.util.List;
import javax.annotation.Resource;
import org.hibernate.query.Query;
import org.springframework.orm.hibernate5.HibernateTemplate;
import org.springframework.stereotype.Repository;
import com.nt.domain.EmployeeHLO;

@Repository("empDAO")
public class EmployeeDAOImpl implements EmployeeDAO {
    private static final String HQL_GET_EMPS="from EmployeeHLO";
    private static final String HQL_UPDATE_EMAIL_BY_NO="update
EmployeeHLO set email=? where eid=?";
    private static final String HQL_DELETE_EMPS_BY_LASTNAME="delete from
EmployeeHLO where lastName=:lname";
    private static final String HQL_GET_EMPS_BY_EMAIL_DOMAIN="from
EmployeeHLO where email like :dmn";

    @Resource
    private HibernateTemplate ht;

    @Override
    public int insert(EmployeeHLO hlo) {
        int idVal=0;
```



```
//save object
    idVal=(int) ht.save(hlo);
    return idVal;
}

@Override
public List<EmployeeHLO> getAllEmployees() {
    List<EmployeeHLO> listHLO;
    listHLO= (List<EmployeeHLO>) ht.find(HQL_GET_EMPS);
    return listHLO;
}

@Override
public int updateEmployeeEmail(int eid, String newEmail) {
    return ht.bulkUpdate(HQL_UPDATE_EMAIL_BY_NO, newEmail,eid);
}

@Override
public int deleteEmployeeByLastName(String surName) {
    int count=0;
    //count=ht.execute(new MyHibernateCallback(surName));
    /* count=ht.execute(new HibernateCallback<Integer>() {
        @Override
        public Integer doInHibernate(Session ses) throws
HibernateException {
            Query query=null;
            int count=0;
            //create Query object
            query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
            //set param values
            query.setParameter("lname",surName);
            count=query.executeUpdate();
            return count;
        }
    }); */
    count=ht.execute(ses->{
        Query query=null;
        int cnt=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname",surName);
        cnt=query.executeUpdate();
        return cnt;
    });

    return count;
} //method
```

```

@Override
public List<EmployeeHLO> getEmployeesByEmailDomain(String domain) {
    List<EmployeeHLO> listHLO=null;
    listHLO=(List<EmployeeHLO>)
    ht.findByNameParam(HQL_GET_EMPS_BY_EMAIL_DOMAIN, "dmn","%" +domain);
    return listHLO;
}

@Override
public List<EmployeeHLO> getEmployeesByLastName(String surName1,
String surName2, String surName3) {
    List<EmployeeHLO> listHLO=null;
    //get Access to Named HQL Query
    listHLO=(List<EmployeeHLO>)
    ht.findByNameQueryAndNamedParam("GET EMP DETAILS BY LNAME",
                                     new
String[] {"surname1","surname2","surname3"},
                                     new
String[] {surName1,surName2,surName3});
    return listHLO;
}

/* private class MyHibernateCallback implements
HibernateCallback<Integer>{
    private String surname;

    public MyHibernateCallback(String surName){
        this.surname=surName;
    }

    @Override
    public Integer doInHibernate(Session ses) throws
HibernateException {
        Query query=null;
        int count=0;
        //create Query object
        query=ses.createQuery(HQL_DELETE_EMPS_BY_LASTNAME);
        //set param values
        query.setParameter("lname", surname);
        count=query.executeUpdate();
        return count;
    } //method

} //inner class */
}

```

EmployeeService.java:-

```

package com.nt.service;

import java.util.List;

```

```
import com.nt.dto.EmployeeDTO;

public interface EmployeeService {
    public String registerEmployee(EmployeeDTO dto);
    public List<EmployeeDTO> fetchAllEmployees();
    public String modifyEmailByNo(int no,String newEmail);
    public String removeEmployeeByLastName(String surName);
    public List<EmployeeDTO> fetchEmployeesByEmailDomain(String
domain);
    public List<EmployeeDTO> fetchEmployeesByLastName(String
surname1,String surname2,String surname3);
}
```

EmployeeServiceImpl.java:-

```
package com.nt.service;

import java.util.ArrayList;
import java.util.List;
import javax.annotation.Resource;
import org.springframework.beans.BeanUtils;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.EmployeeDAO;
import com.nt.domain.EmployeeHLO;
import com.nt.dto.EmployeeDTO;

@Service("empService")
@Transactional(propagation=Propagation.REQUIRED,readOnly=false,transactio
nManager="hbTxMgmr")
public class EmployeeServiceImpl implements EmployeeService {
    @Resource
    private EmployeeDAO dao;

    @Override
    public String registerEmployee(EmployeeDTO dto) {
        EmployeeHLO hlo=null;
        int result=0;
        //Convert DTO To BO
        hlo=new EmployeeHLO();
        BeanUtils.copyProperties(dto,hlo);
        //use DAO
        result=dao.insert(hlo);
        if(result==0)
            return "Registration failed";
        else
            return "Employee Registered with Number:"+result;
    } //method
```

```
@Override
public List<EmployeeDTO> fetchAllEmployees() {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getAllEmployees();
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method

@Override
public String modifyEmailByNo(int no, String newEmail) {
    int count=0;
    //use DAO
    count=dao.updateEmployeeEmail(no,newEmail);
    if(count==0)
        return "Record not updated";
    else
        return "Record updated";
} //method

@Override
public String removeEmployeeByLastName(String surName) {
    int count=0;
    //use dao
    count=dao.deleteEmployeeByLastName(surName);
    if(count==0)
        return "Employee(s) Not Deleted";
    else
        return "Employee(s) Deleted";
} //method

@Override
public List<EmployeeDTO> fetchEmployeesByEmailDomain(String domain)
{
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByEmailDomain(domain);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
    });
}
```

```
        listDTO.add(dto);
    });
    return listDTO;
}

@Override
public List<EmployeeDTO> fetchEmployeesByLastName(String surname1,
String surname2, String surname3) {
    List<EmployeeDTO> listDTO=new ArrayList();
    List<EmployeeHLO> listHLO=null;

    //use DAO
    listHLO=dao.getEmployeesByLastName(surname1, surname2,
surname3);
    //convert listHLO to ListDTO
    listHLO.forEach(hlo->{
        EmployeeDTO dto=new EmployeeDTO();
        BeanUtils.copyProperties(hlo,dto);
        listDTO.add(dto);
    });
    return listDTO;
} //method

} //class
```

BootOrmCurdOperationsApplication.java:-

```
package com.nt.test;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.Import;
import com.nt.config.AOPConfig;
import com.nt.config.PersistenceConfig;
import com.nt.config.ServiceConfig;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

@SpringBootApplication
@Import({PersistenceConfig.class,ServiceConfig.class,AOPConfig.class})
public class BootOrmCurdOperationsApplication {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;

        ctx=SpringApplication.run(BootOrmCurdOperationsApplication.class,
args);
        //get Service bean
```

```
        service=ctx.getBean("empService",EmployeeService.class);  
        //preare DTO  
        dto=new EmployeeDTO();  
        dto.setFirstName("Raja"); dto.setLastName("ramesh");  
dto.setEmail("raja@x.com");  
        //invoke methods  
        System.out.println(service.registerEmployee(dto));  
        //close container  
        ((ConfigurableApplicationContext) ctx).close();  
  
    }  
}
```

application.properties:-

```
#jdbc data soruce for oracle (Tomcat)  
spring.datasource.driver-class-name=oracle.jdbc.driver.OracleDriver  
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe  
spring.datasource.username=system  
spring.datasource.password=manager
```

=====00000000=====

Spring Batch

Spring Batch Processing is defined as the Processing of Data without Interaction or interruption

Detail Information:-

1. Spring Batch is an open-source framework for batch processing. Batch Processing in simple terms, refers to running bulk operations that could run for hours on end without needing human intervention. Consider Enterprise level operations that involve say, reading from or writing into or updating millions of database records. Spring Batch provides the framework to have such jobs running with minimum human involvement.

2. It is light-weight, comprehensive, favors POJO-based development approach and comes with all the features that spring offers. Besides, it also exposes a number of classes and APIs that could be exploited say for transaction management, for reading and writing data.

3. Spring Batch provides reusable functions that are essential in processing large volumes of records, including logging/tracing, transaction management, job processing statistics, job restart, skip, resource management, logging, tracing, conversion of data, interfaces, etc. By using these diverse techniques, the framework takes care of the performance and the scalability while processing the records.

4. Spring Batch also provides more advanced technical services and features that will enable extremely high-volume and high performance batch jobs through optimization and partitioning techniques.

5. Spring Batch Application executes a series of jobs (iterative or in parallel), where input data is read, processed and written without any interaction. We are going to see how Spring Batch can help us with this purpose.

6. Normally a batch application can be divided in three main parts: ① Reading the data (from a database, file system, etc.)

Copyright © 2016 by Nataraz Classes <http://www.nataraz.in/>

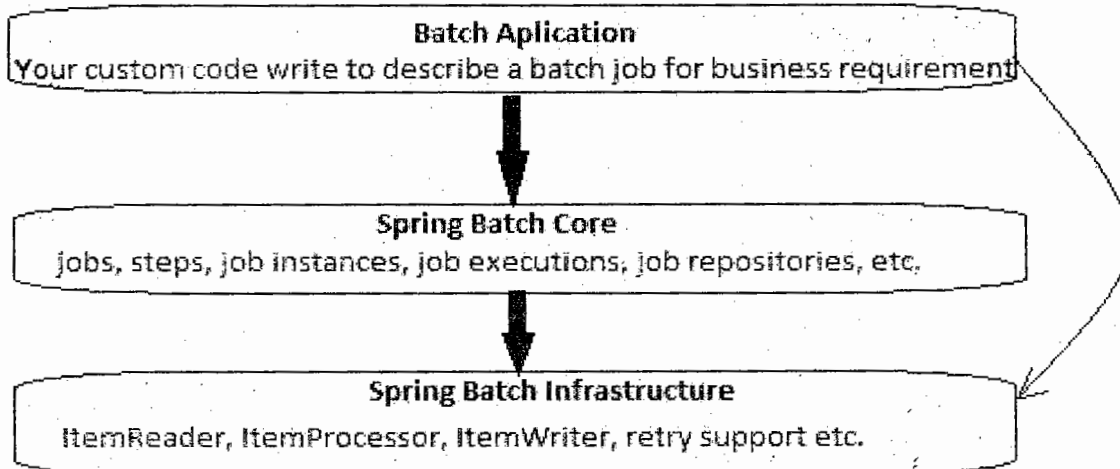
② Processing the data (filtering, grouping, calculating, validating...) ③ Writing the data (to a database, reporting, distributing...)

7. Spring Batch contains features and abstractions for automating these basic steps and allowing the application programmers to configure them, repeat them, retry them, stop them, executing them as a single element or grouped (transaction management), etc.

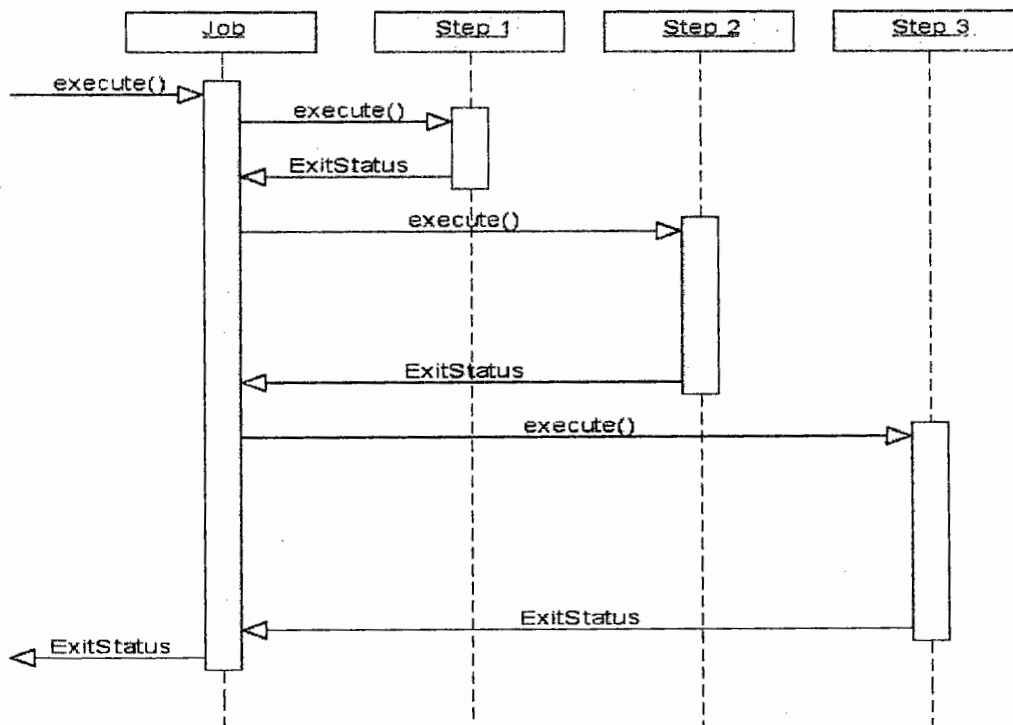
8. It also contains classes and interfaces for the main data formats, industry standards and providers like XML, CSV, SQL, Mongo DB, etc.

9. Examples :- Month-end calculations notices or correspondence , periodic application of complex business rules processed repetitively across very large data sets (e.g. insurance benefit determination or rate adjustments)

HIGH LEVEL ARCHITECTURE

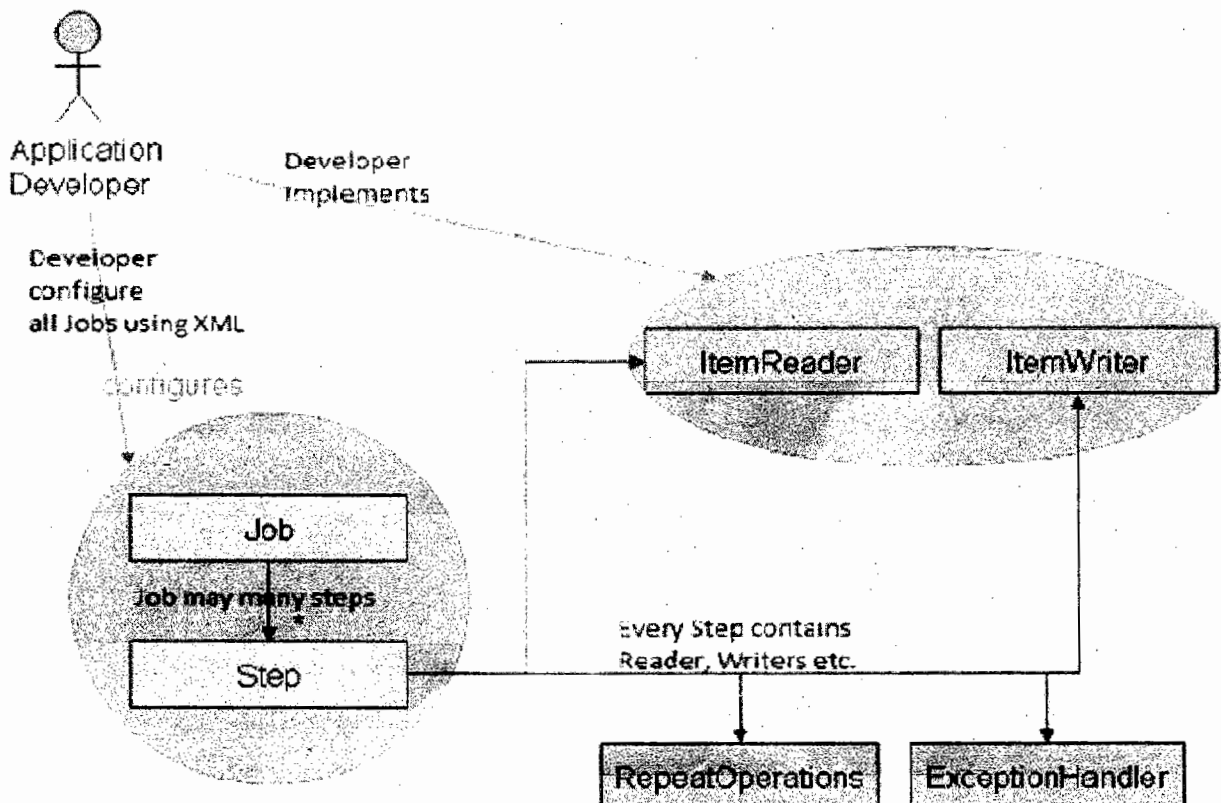


In above figure, the top of the hierarchy is the batch application itself. This is whatever batch processing application you want to write. It depends on the Spring Batch core module, which primarily provides a runtime environment for your batch jobs. Both the batch app and the core module in turn depend upon an infrastructure module that provides classes useful for both building and running batch apps.



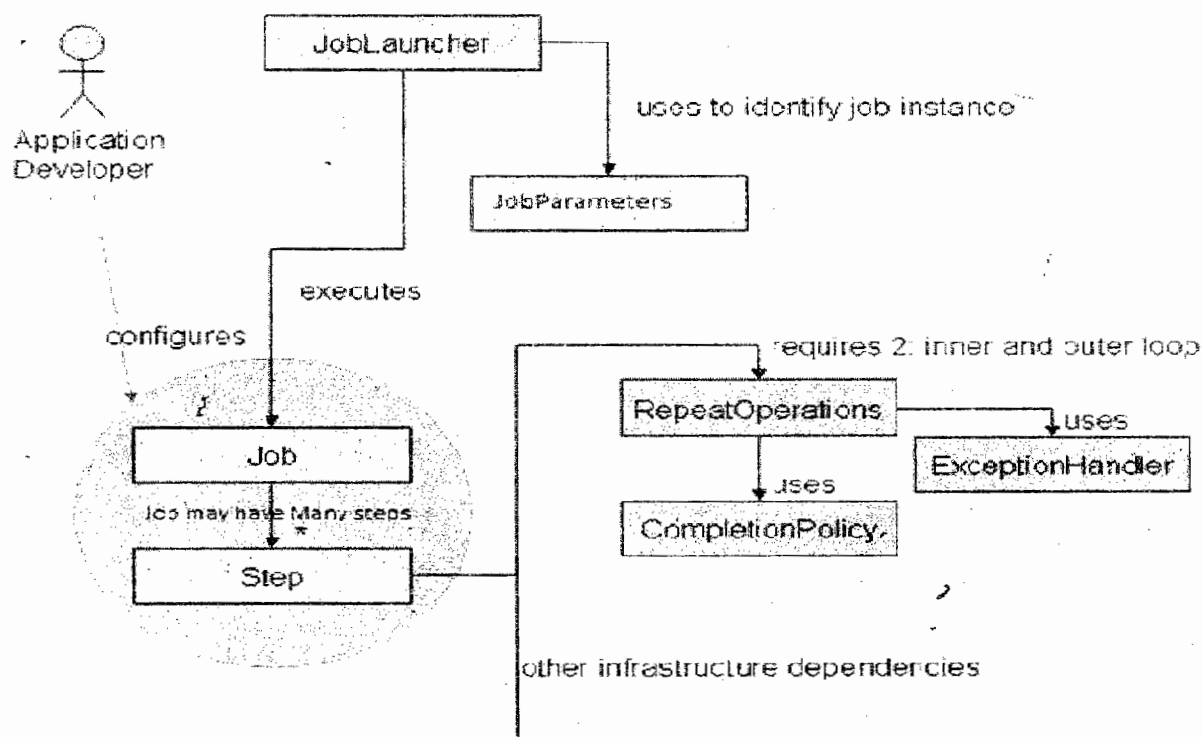
As we see in above figure a hypothetical three-step job, though obviously a job can have arbitrarily many steps. The steps are typically sequential, though as of Spring Batch 2.0 it's possible to define conditional flows (e.g., execute step 2 if step 1 succeeds; otherwise execute step 3). Within any given step, the basic process is as follows: read a bunch of "items" (e.g., database rows, XML elements, and lines in a flat file— whatever), process them, and write them out somewhere to make it convenient for subsequent steps to work with the result.

Overview of the Spring Batch Core - The Spring Batch Core Domain consists of an API for launching, monitoring and managing batch jobs.



The figure above shows the central parts of the core domain and its main touch points with the batch application developer (*Job* and *Step*). To launch a job there is a *JobLauncher* interface that can be used to simplify the launching for dumb clients like JMX or a command line.

A Job is composed of a list of Steps, each of which is executed in turn by the Job. The *Step* is a central strategy in the Spring Batch Core. Implementations of Step are responsible for sharing the work out, but in ways that the configuration doesn't need to be aware of. For instance, the same or very similar Step configuration might be used in a simple in-process sequential executor, or in a multi-threaded implementation, or one that delegates to remote calls to a distributed system.



A Job can be re-used to create multiple job instances and this is reflected in the figure above showing an extended picture of the core domain. When a Job is launched it first checks to see if a job with the same *JobParameters* was already executed. We expect one of the following outcomes, depending on the Job:

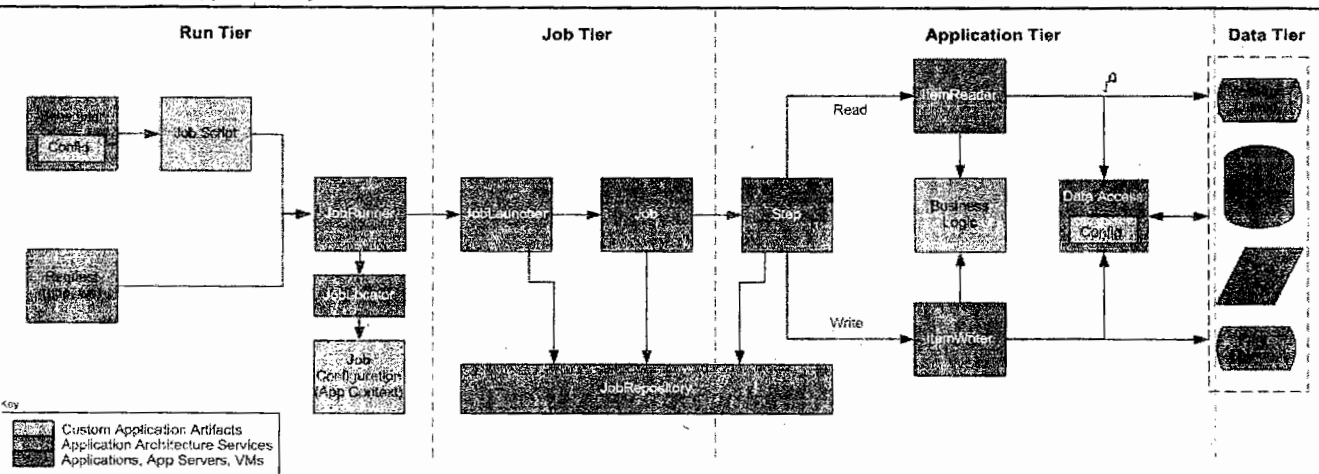
- If the job was not previously launched then it can be created and executed. A new *JobInstance* is created and stored in a repository (usually a database). A new *JobExecution* is also created to track the progress of this particular execution.

If the job was previously launched and failed the Job is responsible for indicating whether it believes it is restartable (is a restart legal and expected). There is a flag for this purpose on the Job. If there was a previous failure - maybe the operator has fixed some bad input and wants to run it again - then we might want to restart the previous job.

If the job was previously launched with the same *JobParameters* and completed successfully, then it is an error to restart it. An ad-hoc request needs to be distinguished from previous runs by adding a unique job parameter. In either case a new *JobExecution* is created and stored to monitor this execution of the *JobInstance*.

Spring Batch Launch Environment

Batch Application Style – Interactions and Services



The application style is organized into four logical tiers, which include **Run**, **Job**, **Application**, and **Data** tiers. The primary goal for organizing an application according to the tiers is to embed what is known as "**separation of concerns**" within the system. Effective separation of concerns results in reducing the impact of change to the system.

Run Tier: The Run Tier is concerned with the scheduling and launching of the application. A vendor product is typically used in this tier to allow time-based and interdependent scheduling of batch jobs as well as providing parallel processing capabilities.

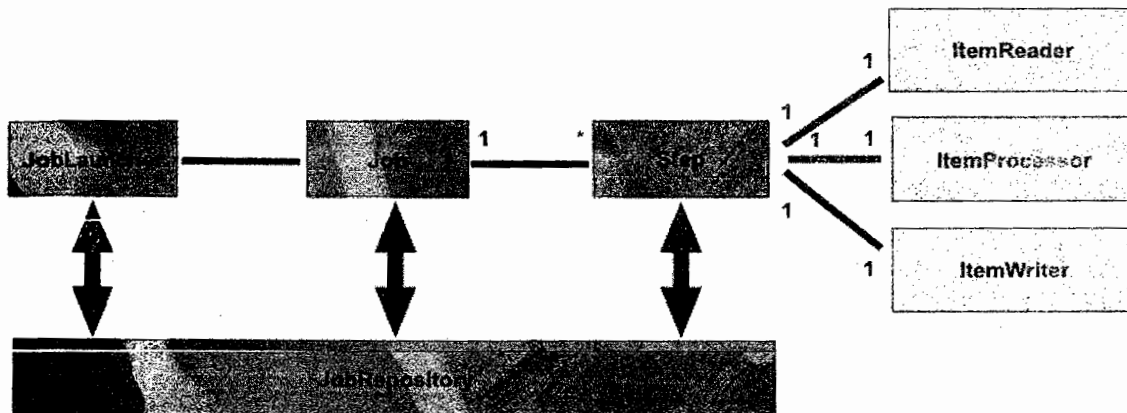
Job Tier: The Job Tier is responsible for the overall execution of a batch job. It sequentially executes batch steps, ensuring that all steps are in the correct state and all appropriate policies are enforced.

Application Tier: The Application Tier contains components required to execute the program. It contains specific modules that address the required batch functionality and enforces policies around a module execution (e.g., commit intervals, capture of statistics, etc.)

Data Tier: The Data Tier provides the integration with the physical data sources that might include databases, files, or queues. Note: In some cases the Job tier can

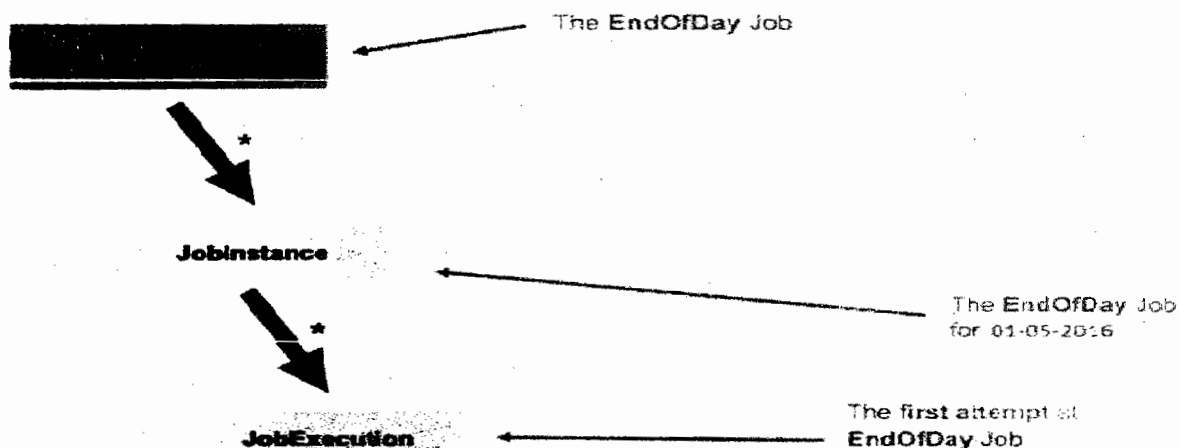
be completely missing and in other cases one job script can start several batch job instances.

Spring Batch: Core Concepts



The diagram above highlights the key concepts that make up the domain language of batch. A *Job* has one to many steps, which has exactly one *ItemReader*, *ItemProcessor*, and *ItemWriter*. A job needs to be launched (*JobLauncher*), and Meta data about the currently running process needs to be stored (*JobRepository*).

Job- A Job is an entity that encapsulates an entire batch process. As is common with other spring projects, a Job will be wired together via an XML configuration file. This file may be referred to as the "job configuration". However, Job is just the top of an overall hierarchy



In Spring Batch, a Job is simply a container for Steps. It combines multiple steps that belong

logically together in a flow and allows for configuration of properties global to all steps, such as restartability. The job configuration contains:

- The simple name of the job
- Definition and ordering of Steps
- Whether or not the job is restartable

A default simple implementation of the `Job` interface is provided by Spring Batch in the form of the `SimpleJob` class which creates some standard functionality on top of `Job`, however the batch namespace abstracts away the need to instantiate it directly. Instead, the `<job>` tag can be used:

```
<job id="myEmpExpireJob">
  <!-- Step bean details omitted for clarity -->
  <step id="readEmployeeData" next="writeEmployeeData"></step>
  <step id="writeEmployeeData" next="employeeDataProcess"></step>
  <step id="employeeDataProcess"></step>
</job>
```

JobInstance

A `JobInstance` refers to the concept of a logical job run. Let's consider a batch job that should be run once at the end of the day, such as the 'EndOfDay' job from the diagram above. There is one 'EndOfDay' Job, but each individual run of the Job must be tracked separately. In the case of this job, there will be one logical `JobInstance` per day. For example, there will be a January 1st run, and a January 2nd run. If the January 1st run fails the first time and is run again the next day, it is still the January 1st run.

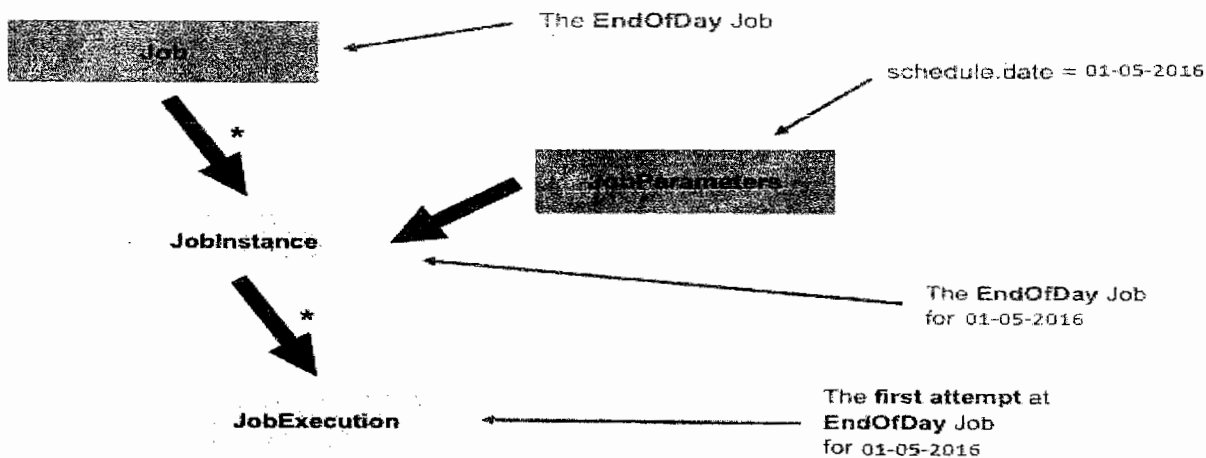
JobParameters

having discussed `JobInstance` and how it differs from `Job`, the natural question to ask is :-

Q :- how is one `JobInstance` distinguished one from another?

Ans :-The answer is: `JobParameters`.

`JobParameters` is a set of parameters used to start a batch job. They can be used for identification or even as reference data during the run:



Spring Batch ItemReader and ItemWriter:-

we will discuss about the three most important interfaces of spring batch and an overview of Spring Batch item reader and writer with a sample application. One of the important goals of a batch processing framework is to read large amounts of data, perform some business processing/transformation and write out the result. **Spring Batch Framework** supports this bulk reading, processing and writing using three key interfaces: **ItemReader**, **ItemProcessor** and **ItemWriter**.

Popular Spring Tutorials

1. ItemReader is the means for providing data from many different types of input. ItemReader interface is the means for reading bulk data in a bulk processing system. There are many different implementations of ItemReader interface. All implementations are expected to be stateful and will be called multiple times for each batch, with each call to read() returning a different value and finally returning null when all input data is exhausted. Below are few frequently used implementations of ItemReader.

ItemReader Implementation	Description
FlatFileItemReader	Reads lines of data from input file. Typically read line describe records with fields of data defined by fixed positions in the file or delimited by some special character (e.g. Comma (,), Pipe () etc).
JdbcCursorItemReader	Opens a JDBC cursor and continually retrieves the next row in the ResultSet.
StoredProcedureItemReader	Executes a stored procedure and then reads the returned cursor and continually retrieves the next row in the ResultSet.

All the above implementations override the read() method from the ItemReader interface. The read method defines the most essential contract of the ItemReader. It returns one item or null if no more items are left. An item might represent a line in a file, a row in a database and so on.

There are many more possibilities, but we'll focus on the basic ones for this chapter. A complete list of all available *ItemReaders* are

1. AmqpItemReader
2. AggregateItemReader
3. FlatFileItemReader
4. HibernateCursorItemReader
5. HibernatePagingItemReader
6. IbatisPagingItemReader
7. ItemReaderAdapter
8. JdbcCursorItemReader

9. JdbcPagingItemReader
10. JmsItemReader
11. JpaPagingItemReader
12. ListItemReader
13. MongoItemReader
14. Neo4jItemReader
15. RepositoryItemReader
16. StoredProcedureItemReader
17. StaxEventItemReader

We can see that Spring Batch already provides readers for many of the formatting standards and database industry providers. It is recommended to use the abstractions provided by Spring Batch in your applications rather than creating your own ones.

ItemReader is a basic interface for generic input operations:

```
1. public interface ItemReader<T> {  
2.  
3.     T read() throws Exception, UnexpectedInputException, ParseException;  
4.  
5. }
```

2. ItemWriter is similar in functionality to an ItemReader, but with inverse operations. ItemWriter is a interface for generic output operations. Implementation class will be responsible for serializing objects as necessary. Resources still need to be located, opened and closed but they differ in that an ItemWriter writes out, rather than reading in. For databases these may be inserts or updates.

The write method defines the most essential contract of the ItemWriter. It will attempt to write out the list of items passed in as long as it is open. As it is expected that items will be 'batched' together into a chunk and then output, the interface accepts a list of items, rather than an item by itself. Once the items are written out, any flushing that may be necessary can be performed before returning from the write method.

ItemWriter Implementation

Description

FlatFileItemWriter

Writes data to a file or stream. Uses buffered writer to improve performance.

StaxEventItemWriter

An implementation of ItemWriter which uses StAX and Marshaller for serializing object to XML.

A complete list of all available **ItemWriter** are :-

1. AbstractItemStreamItemWriter

2. AmqpItemWriter
3. CompositeItemWriter
4. FlatFileItemWriter
5. GemfireItemWriter
6. HibernateItemWriter
7. IbatisBatchItemWriter
8. ItemWriterAdapter
9. JdbcBatchItemWriter
10. JmsItemWriter
11. JpaItemWriter
12. MimeMessageItemWriter
13. MongoItemWriter
14. Neo4jItemWriter
15. StaxEventItemWriter
16. RepositoryItemWriter

We can see that Spring Batch already provides Writers for many of the formatting standards and database industry providers. It is recommended to use the abstractions provided by Spring Batch in your applications rather than creating your own ones.

As with *ItemReader*, *ItemWriter* is a fairly generic interface:

```
1 public interface ItemWriter<T> {  
2  
3     void write(List<? extends T> items) throws Exception;  
4  
5 }
```

As with read on *ItemReader*, write provides the basic contract of *ItemWriter*; it will attempt to write out the list of items passed in as long as it is open. Because it is generally expected that items will be 'batched' together into a chunk and then output, the interface accepts a list of items, rather than an item by itself. After writing out the list, any flushing that may be necessary can be performed before returning from the write method. For example, if writing to a Hibernate DAO, multiple calls to write can be made, one for each item. The writer can then call close on the hibernate Session before returning.

ItemProcessor—

The *ItemReader* and *ItemWriter* interfaces are both very useful for their specific tasks, but what if you want to insert business logic before writing? One option for both reading and writing is to use the composite pattern: create an *ItemWriter* that contains another *ItemWriter*, or an *ItemReader* that contains another *ItemReader*. For example:


```
public class CompositeItemWriter<T> implements ItemWriter<T> {

    ItemWriter<T> itemWriter;

    public CompositeItemWriter(ItemWriter<T> itemWriter) {
        this.itemWriter = itemWriter;
    }

    public void write(List<? extends T> items) throws Exception

        //Add business logic here
        itemWriter.write(item);
    }

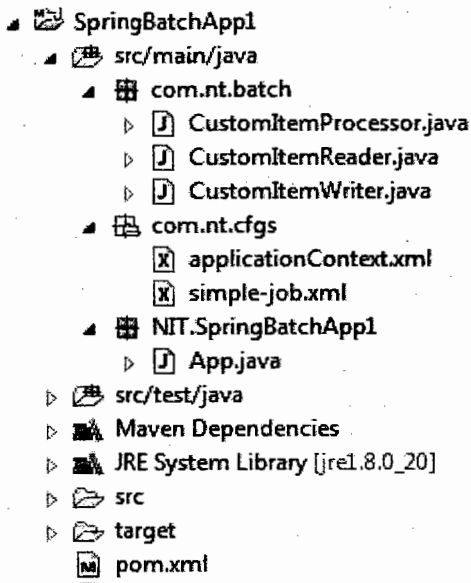
    public void setDelegate(ItemWriter<T> itemWriter){
        this.itemWriter = itemWriter;
    }
}
```

The class above contains another *ItemWriter* to which it *delegates* after having provided some business logic. This pattern could easily be used for an *ItemReader* as well, perhaps to obtain more reference data based upon the input that was provided by the main *ItemReader*. It is also useful if you need to control the call to write yourself. However, if you only want to 'transform' the item passed in for writing before it is actually written, there isn't much need to call write yourself: you just want to modify the item. For this scenario, Spring Batch provides the *ItemProcessor* interface:

```
1. public interface ItemProcessor<I, O> {
2.
3.     O process(I item) throws Exception;
4. }
```

An *ItemProcessor* is very simple; given one object, transform it and return another. The provided object may or may not be of the same type. The point is that business logic may be applied within process, and is completely up to the developer to create. An *ItemProcessor* can be wired directly into a step, For example, assuming an *ItemReader* provides a class of type Foo, and it needs to be converted to type Bar before being written out. An *ItemProcessor* can be written that performs the conversion:

```
}
```



CustomItemProcessor.java

```
package com.nt.batch;
import org.springframework.batch.item.ItemProcessor;

public class CustomItemProcessor implements ItemProcessor<String, String> {

    @Override
    public String process(String bookNameWithoutAuthor) throws Exception {
        System.out.println("ItemProcessor:process(-)");
        String bookNameWithAuthor = "Book Name - "+bookNameWithoutAuthor+" |
Author Name - ";

        if("Core Java".equalsIgnoreCase(bookNameWithoutAuthor)){
            bookNameWithAuthor += "Srinivas";
        }elseif("Design Patterns".equalsIgnoreCase(bookNameWithoutAuthor)){
            bookNameWithAuthor += "raja ";
        }elseif("Advance Java".equalsIgnoreCase(bookNameWithoutAuthor)){
            bookNameWithAuthor += "ravi ";
        }elseif("Spring FrameWork".equalsIgnoreCase(bookNameWithoutAuthor)){
            bookNameWithAuthor += "karan";
        }elseif("Hibernate Framework".equalsIgnoreCase(bookNameWithoutAuthor)){
            bookNameWithAuthor += "rani";
        }
        return bookNameWithAuthor;
    }
}
```

CustomItemReader.java

```
package com.nt.batch;

import java.util.List;
import org.springframework.batch.item.ItemReader;
import org.springframework.batch.item.ParseException;
import org.springframework.batch.item.UnexpectedInputException;

public class CustomItemReader implements ItemReader<String>{
    private List<String> bookNameList;
    private int bookCount = 0;

    @Override
    public String read() throws Exception, UnexpectedInputException,
        ParseException {
        System.out.println("ItemReader::read()");
        if(bookCount < bookNameList.size()){
            return bookNameList.get(bookCount++);
        }else{
            return null;
        }
    }
    /*public List<String> getUserNamelist() {
        return bookNameList;
    }*/
    public void setBookNameList(List<String> bookNameList) {
        this.bookNameList = bookNameList;
    }
}
```

CustomItemWriter.java

```
package com.nt.batch;

import java.util.List;
import org.springframework.batch.item.ItemWriter;

public class CustomItemWriter implements ItemWriter<String> {

    @Override
    public void write(List<? extends String> bookNameWithAuthor) throws Exception {
        System.out.println("ItemWriter: write(-)");
        System.out.println(bookNameWithAuthor);
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd">

    <bean id="transactionManager"
          class="org.springframework.batch.support.transaction.ResourcelessTransactionManager"/>

    <bean id="jobRepository"
          class="org.springframework.batch.core.repository.support.MapJobRepositoryFactoryBean">
        <property name="transactionManager" ref="transactionManager"/>
    </bean>

    <bean id="jobLauncher"
          class="org.springframework.batch.core.launch.support.SimpleJobLauncher">
        <property name="jobRepository" ref="jobRepository"/>
    </bean>

    <bean id="simpleJob"
          class="org.springframework.batch.core.job.SimpleJob" abstract="true">
        <property name="jobRepository" ref="jobRepository" />
    </bean>

</beans>
```

simple-job.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:batch="http://www.springframework.org/schema/batch"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
```

```
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd
http://www.springframework.org/schema/batch
http://www.springframework.org/schema/batch/spring-batch-3.0.xsd">

    <import resource="applicationContext.xml"/>

    <bean id="customReader" class="com.nt.batch.CustomItemReader">
<property name="bookNameList">
<list>
<value>Core Java</value>
<value>Design Patterns</value>
<value>Advance Java</value>
<value>Spring FrameWork</value>
<value>Hibernate Framework</value>
</list>
</property>
</bean>

<bean id="customProcessor" class="com.nt.batch.CustomItemProcessor" />

<bean id="customWriter" class="com.nt.batch.CustomItemWriter" />

<batch:job id="mySimpleJob" job-repository="jobRepository" parent="simpleJob">
    <batch:step id="step1">
        <batch:tasklet transaction-manager="transactionManager">
            <batch:chunk reader="customReader" processor="customProcessor"
                writer="customWriter" commit-interval="2"/>
        </batch:tasklet>
    </batch:step>
</batch:job>
</beans>
```

App.java

```
package NIT.SpringBatchApp1;

/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );
    }
}
```

pom.xml

```
<project
xmlns="http://maven.apache.org/POM/4.0.0"xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<groupId>NIT</groupId>
<artifactId>SpringBatchApp1</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>jar</packaging>

<name>SpringBatchApp1</name>
<url>http://maven.apache.org</url>

<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>4.2.2.RELEASE</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework/spring-aop -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-aop</artifactId>
<version>4.2.2.RELEASE</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.springframework/spring-context-support -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context-support</artifactId>
<version>4.2.2.RELEASE</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.springframework.batch/spring-batch-core -->
<dependency>
```

```
<groupId>org.springframework.batch</groupId>
<artifactId>spring-batch-core</artifactId>
<version>3.0.7.RELEASE</version>
</dependency>

</dependencies>
</project>
```

- ▼  SpringBatchApp2-DBToCsvMysql
 - ▼  src/main/java
 - ▼  com.nt.batch
 - >  ExamResultItemProcessor.java
 - ▼  com.nt.cfgs
 -  persistence-beans.xml
 -  spring-batch-beans.xml
 - ▼  com.nt.model
 - >  ExamResult.java
 - ▼  com.nt.rowmapper
 - >  ExamResultRowMapper.java
 - ▼  com.nt.test
 - >  SpringDBToCsvTest.java
 - >  src/test/java
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 - ▼  csv
 -  SuperBrains.csv
 - >  src
 - >  target
 -  MYSQL_Script_Readme.txt
 -  pom.xml
 -  Spring Batch .pdf

ExamResultItemProcessor.java:-

```
package com.nt.batch;

import org.springframework.batch.item.ItemProcessor;
import com.nt.model.ExamResult;

public class ExamResultItemProcessor implements ItemProcessor<ExamResult,
ExamResult>{

    public ExamResult process(ExamResult result) throws Exception {
        System.out.println("Processing result :"+result);

        /*
         * Only return results which are more than 80%
         */
        if(result.getPercentage() < 80){
```

```
        return null;
    }

    return result;
}
}
```

persistence-beans.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:batch="http://www.springframework.org/schema/batch"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-
        4.0.xsd">

    <bean id="dbcpDs" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="com.mysql.jdbc.Driver"
    />
        <property name="url"
value="jdbc:mysql://localhost:3306/EXAM_DATA" />
        <property name="username" value="root" />
        <property name="password" value="root" />
    </bean>
</beans>
```

spring-batch-beans.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:batch="http://www.springframework.org/schema/batch"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/batch
        http://www.springframework.org/schema/batch/spring-batch-3.0.xsd
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd">

    <import resource="persistence-beans.xml"/>

    <!-- JobRepository and JobLauncher are configuration/setup classes -
->
    <bean id="jobRepository"

        class="org.springframework.batch.core.repository.support.MapJobRepos
itoryFactoryBean" />

    <bean id="jobLauncher"

        class="org.springframework.batch.core.launch.support.SimpleJobLaunch
er">
        <property name="jobRepository" ref="jobRepository" />
    </bean>
```



```
by
    <!-- ItemReader which reads from database and returns the row mapped
    rowMapper -->
    <bean id="dbItemReader"
    class="org.springframework.batch.item.database.JdbcCursorItemReader"
    >
        <property name="dataSource" ref="dbcpDs" />
        <property name="sql"
            value="SELECT ID, SEMESTER, DOB, PERCENTAGE FROM
EXAM_RESULT" />
        <property name="rowMapper">
            <bean class="com.nt.rowmapper.ExamResultRowMapper"/>
        </property>
    </bean>

    <!-- ItemWriter writes a line into output CSV file -->
    <bean id="flatFileItemWriter"
class="org.springframework.batch.item.file.FlatFileItemWriter">
        <property name="resource" value="file:csv/SuperBrains.csv" />
        <property name="lineAggregator">
            <!-- An Aggregator which converts an object into
delimited list of strings -->
            <bean
                class="org.springframework.batch.item.file.transform.DelimitedLineAggregator">
                <property name="delimiter" value="," />
                <property name="fieldExtractor">
                    <!-- Extractor which returns the value of
beans property through reflection -->
                    <bean
                        class="org.springframework.batch.item.file.transform.BeanWrapperFieldExtractor">
                        <property name="names" value="id, sem,
percentage, dob" />
                    </bean>
                </property>
            </bean>
        </property>
    </bean>

    <!-- Optional ItemProcessor to perform business logic/filtering on
the input
    records -->
    <bean id="itemProcessor"
class="com.nt.batch.ExamResultItemProcessor" />
```

```
<!-- Step will need a transaction manager -->
<bean id="transactionManager"

    class="org.springframework.batch.support.transaction.ResourcelessTransactionManager" />

<!-- Actual Job -->
<batch:job id="examResultJob">
    <batch:step id="step1">
        <batch:tasklet transaction-manager="transactionManager">
            <batch:chunk reader="dbItemReader"
writer="flatFileItemWriter"
                        processor="itemProcessor" commit-interval="2"
/>
        </batch:tasklet>
    </batch:step>
</batch:job>

</beans>
```

ExamResult.java:-

```
package com.nt.model;

import java.sql.Date;

public class ExamResult {
    private int id;
    private int sem;
    private Date dob;
    private double percentage;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public int getSem() {
        return sem;
    }

    public void setSem(int sem) {
        this.sem = sem;
    }

    public Date getDob() {
```

```
        return dob;
    }

    public void setDob(Date dob) {
        this.dob = dob;
    }

    public double getPercentage() {
        return percentage;
    }

    public void setPercentage(double percentage) {
        this.percentage = percentage;
    }

    @Override
    public String toString() {
        return "ExamResult [id=" + id + ", sem=" + sem + ", dob=" +
dob + ", percentage=" + percentage + "]";
    }
}
```

ExamResultRowMapper.java:-

```
package com.nt.rowmapper;

import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.jdbc.core.RowMapper;
import com.nt.model.ExamResult;

public class ExamResultRowMapper implements RowMapper<ExamResult>{

    public ExamResult mapRow(ResultSet rs, int rowNum) throws
SQLException {
        ExamResult result =null;
        result=new ExamResult();
        result.setId(rs.getInt("id"));
        result.setSem(rs.getInt("Semester"));
        result.setDob(rs.getDate("dob"));
        result.setPercentage(rs.getDouble("percentage"));
        return result;
    }
}
```

SpringDBToCsvTest.java:-

```
package com.nt.test;

import org.springframework.batch.core.Job;
```

```
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobExecutionException;
import org.springframework.batch.core.JobParameters;
import org.springframework.batch.core.launch.JobLauncher;
import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContext;

public class SpringDBToCsvTest {

    @SuppressWarnings("resource")
    public static void main(String[] args){
        ApplicationContext ctx =null;
        JobLauncher jobLauncher=null;
        JobExecution execution=null;
        Job job=null;

        ctx=new ClassPathXmlApplicationContext("com/nt/cfgs/spring-
batch-beans.xml");

        jobLauncher = (JobLauncher) ctx.getBean("jobLauncher");
        job = (Job) ctx.getBean("examResultJob");
        try {
            execution = jobLauncher.run(job, new JobParameters());
            System.out.println("Job Exit Status : "+
execution.getStatus());
        } catch (JobExecutionException e) {
            System.out.println("Job ExamResult failed");
            e.printStackTrace();
        }
    }
} //main
} //class
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>nit</groupId>
    <artifactId>SpringBatchDatabaseToCsv</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <name>SpringBatchDatabaseToCsv</name>
    <url>http://maven.apache.org</url>

    <dependencies>
```

```
<!--  
https://mvnrepository.com/artifact/org.springframework/spring-context -->  
    <dependency>  
        <groupId>org.springframework</groupId>  
        <artifactId>spring-context</artifactId>  
        <version>4.3.9.RELEASE</version>  
    </dependency>  
  
    <!--  
https://mvnrepository.com/artifact/org.springframework/spring-context-  
support -->  
    <dependency>  
        <groupId>org.springframework</groupId>  
        <artifactId>spring-context-support</artifactId>  
        <version>4.3.9.RELEASE</version>  
    </dependency>  
  
    <dependency>  
        <groupId>org.springframework</groupId>  
        <artifactId>spring-core</artifactId>  
        <version>4.3.9.RELEASE</version>  
    </dependency>  
    <dependency>  
        <groupId>org.springframework</groupId>  
        <artifactId>spring-jdbc</artifactId>  
        <version>4.3.9.RELEASE</version>  
    </dependency>  
  
    <dependency>  
        <groupId>org.springframework.batch</groupId>  
        <artifactId>spring-batch-core</artifactId>  
        <version>3.0.8.RELEASE</version>  
    </dependency>  
    <dependency>  
        <groupId>org.springframework.batch</groupId>  
        <artifactId>spring-batch-infrastructure</artifactId>  
        <version>3.0.8.RELEASE</version>  
    </dependency>  
  
    <!--  
https://mvnrepository.com/artifact/org.apache.commons/commons-dbcp2 -->  
    <dependency>  
        <groupId>org.apache.commons</groupId>  
        <artifactId>commons-dbcp2</artifactId>  
        <version>2.1.1</version>  
    </dependency>  
  
    <dependency>  
        <groupId>mysql</groupId>
```

```
<artifactId>mysql-connector-java</artifactId>
<version>5.1.38</version>
</dependency>
</dependencies>
</project>
```

How to create Procedure in MySql DataBase:-

MYSQL_Script_Readme.txt:-

Step-1(Create Database)

```
=====
create Database EXAM_DATA;
```

Step-2(Use Database)

```
=====
use EXAM_DATA;
```

Step-3 (Create Table)

```
=====
CREATE TABLE `EXAM_RESULT`
(
  `id`          bigint(20) NOT NULL          AUTO_INCREMENT,
  `dob`         timestamp NOT NULL          DEFAULT CURRENT_TIMESTAMP,
  `Semester`    int(11)          DEFAULT NULL,
  `percentage`  float          DEFAULT NULL,

  PRIMARY KEY (`id`)
);
```

Step-4 (Create Procedure to Insert)

```
=====
DELIMITER $$
CREATE PROCEDURE generate_EXAM_RESULT()
BEGIN
  DECLARE i INT DEFAULT 0;

  WHILE i < 500000 DO
    INSERT INTO `EXAM_RESULT` (`dob`, `percentage`, `Semester`) VALUES (
      FROM_UNIXTIME(UNIX_TIMESTAMP('2000-01-01
01:00:00')+FLOOR(RAND()*31536000)),
      ROUND(RAND()*100,2),
      1
    );
    SET i = i + 1;
  END WHILE;
END$$
```

DELIMITER ;

Step-5 (Call Procedure to Insert 50K Data)=> Takes Least 30-45 Min to Insert the records

```
=====
CALL generate_EXAM_RESULT();
```

Step-6 (Check the Records)

```
=====
select * from EXAM_RESULT;
```

Step-7 /Optional (Remove Procedure if you dont need it any more)

```
=====
DROP PROCEDURE generate_EXAM_RESULT;
```

- ▼ SpringBatchApp3-CSVTOXML
 - ▼ src/main/java
 - ▼ com.nt.adapter
 - > SQLDateAdapter.java
 - ▼ com.nt.batch
 - > ExamResultItemProcessor.java
 - ▼ com.nt.cfgs
 - > spring-batch-beans.xml
 - ▼ com.nt.csv
 - > SuperBrains.csv
 - ▼ com.nt.listener
 - > ExamResultJobListener.java
 - ▼ com.nt.mapper
 - > ExamResultFieldSetMapper.java
 - ▼ com.nt.model
 - > ExamResult.java
 - ▼ com.nt.test
 - > SpringCsvToXmlTest.java
 - > src/test/java
 - > JRE System Library [JavaSE-1.8]
 - > Maven Dependencies
 - > src
 - > target
 - ▼ xml
 - > MugUpMasters.xml
 - > pom.xml

ExamResult.java:-

```
package com.nt.model;

import java.sql.Date;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.adapters.XmlJavaTypeAdapter;

@XmlRootElement(name = "ExamResult")
```

```
public class ExamResult {

    private int id;
    private int sem;
    private Date dob;
    private double percentage;

    @XmlElement(name = "id")
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    @XmlElement(name = "sem")
    public int getSem() {
        return sem;
    }

    public void setSem(int sem) {
        this.sem = sem;
    }

    @XmlElement(name = "dob")
    @XmlJavaTypeAdapter(type = Date.class, value =
com.nt.adapter.SQLDateAdapter.class)
    public Date getDob() {
        return dob;
    }

    public void setDob(Date dob) {
        this.dob = dob;
    }

    @XmlElement(name = "percentage")
    public double getPercentage() {
        return percentage;
    }

    public void setPercentage(double percentage) {
        this.percentage = percentage;
    }

    @Override
    public String toString() {
        return "ExamResult [id=" + id + ", sem=" + sem + ", dob=" +
dob + ", percentage=" + percentage + "];"
    }
}
```



```
}
```

SQLDateAdapter.java:-

```
package com.nt.adapter;

import java.sql.Date;
import javax.xml.bind.annotation.adapters.XmlAdapter;

public class SQLDateAdapter extends XmlAdapter<String, Date>{

    public Date unmarshal(String d) throws Exception {
        return java.sql.Date.valueOf(d);
    }

    public String marshal(Date d) throws Exception {
        return d.toString();
    }

}
```

ExamResultItemProcessor.java:-

```
package com.nt.batch;

import org.springframework.batch.item.ItemProcessor;
import com.nt.model.ExamResult;

public class ExamResultItemProcessor implements ItemProcessor<ExamResult, ExamResult>{

    public ExamResult process(ExamResult result) throws Exception {
        System.out.println("Processing result :"+result);

        /*
         * Only return results which are more than 90%
         */
        if(result.getPercentage() < 90){
            return null;
        }

        return result;
    }

}
```

spring-batch-beans.xml:-

```

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:batch="http://www.springframework.org/schema/batch"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/batch
    http://www.springframework.org/schema/batch/spring-batch-3.0.xsd
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-4.0.xsd">

  <!-- JobRepository and JobLauncher are configuration/setup classes -
->
  <bean id="jobRepository"

    class="org.springframework.batch.core.repository.support.MapJobRepos
itoryFactoryBean" />

  <bean id="jobLauncher"

    class="org.springframework.batch.core.launch.support.SimpleJobLaunch
er">
    <property name="jobRepository" ref="jobRepository" />
  </bean>

  <!-- ItemReader reads a complete line one by one from input file -->
  <bean id="flatFileItemReader"
class="org.springframework.batch.item.file.FlatFileItemReader"
    scope="step">
    <property name="resource"
value="classpath:com/nt/csv/SuperBrains.csv" />

    <property name="LineMapper">
      <bean
class="org.springframework.batch.item.file.mapping.DefaultLineMapper">
        <property name="lineTokenizer">
          <!-- A tokenizer class to be used when items
in input record are separated
by specific characters -->
          <bean

            class="org.springframework.batch.item.file.transform.DelimitedLineTo
kenizer">
              <property name="delimiter" value="," />
            </bean>
          </property>
          <property name="fieldSetMapper">
            <!-- Mapper which maps each individual items
in a record to properties
in POJO -->
            <bean

```

```
class="com.nt.mapper.ExamResultFieldSetMapper" />
        </property>
    </bean>
</property>
</bean>

<!-- XML ItemWriter which writes the data in XML format -->
<bean id="xmlItemWriter"
class="org.springframework.batch.item.xml.StaxEventItemWriter">

    <property name="resource" value="file:xml/MugUpMasters.xml" />

    <property name="rootTagName" value="TopResults" />
    <property name="marshaller">
        <bean
class="org.springframework.xml.jaxb.Jaxb2Marshaller">
            <property name="classesToBeBound">
                <list>
                    <value>com.nt.model.ExamResult</value>
                </list>
            </property>
        </bean>
    </property>
</bean>

<!-- Optional ItemProcessor to perform business logic/filtering on
the input records -->
<bean id="itemProcessor"
class="com.nt.batch.ExamResultItemProcessor" />

<!-- Cfg Job Listener -->
<bean id="jobListener"
class="com.nt.listener.ExamResultJobListener"/>

<!-- Step will need a transaction manager -->
<bean id="transactionManager"

class="org.springframework.batch.support.transaction.ResourceLessTra
nsactionManager" />

<!-- Actual Job -->
<batch:job id="examResultJob">
    <batch:step id="step1">
        <batch:tasklet transaction-manager="transactionManager">
            <batch:chunk reader="flatFileItemReader"
writer="xmlItemWriter"
processor="itemProcessor" commit-interval="10"
```

```
</>
        </batch:tasklet>
    </batch:step>
<batch:listeners>
    <batch:listener ref="jobListener"/>
</batch:listeners>
</batch:job>

</beans>
```

ExamResultJobListener.java:-

```
package com.nt.listener;

import java.util.Date;
import java.util.List;
import org.springframework.batch.core.BatchStatus;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobExecutionListener;

public class ExamResultJobListener implements JobExecutionListener{

    private long startTime, stopTime;

    public void beforeJob(JobExecution jobExecution) {
        startTime = System.currentTimeMillis();
        System.out.println("ExamResult Job started at :"+new Date());
    }

    public void afterJob(JobExecution jobExecution) {
        List<Throwable> exceptionList=null;
        stopTime = System.currentTimeMillis();
        System.out.println("ExamResult Job stoped at :"+new Date());
        System.out.println("Total time taken in millis :"+(stopTime-
startTime));

        if(jobExecution.getStatus() == BatchStatus.COMPLETED){
            System.out.println("ExamResult job completed
successfully");
        }
        else if(jobExecution.getStatus() == BatchStatus.FAILED){
            System.out.println("ExamResult job failed with following
exceptions ");

            exceptionList= jobExecution.getAllFailureExceptions();
            for(Throwable th : exceptionList){
                System.err.println("exception :"+
th.getLocalizedMessage());
            }
        }
    }
}
```

ExamResultFieldSetMapper.java:-

```
package com.nt.mapper;

import org.springframework.batch.item.file.mapping.FieldSetMapper;
import org.springframework.batch.item.file.transform.FieldSet;
import org.springframework.validation.BindException;
import com.nt.model.ExamResult;

public class ExamResultFieldSetMapper implements
FieldSetMapper<ExamResult>{

    public ExamResult mapFieldSet(FieldSet fieldSet) throws
BindException {
        ExamResult result =null;
        result=new ExamResult();

        result.setId(fieldSet.readInt(0));
        result.setSem(fieldSet.readInt(1));
        result.setPercentage(fieldSet.readDouble(2));
        result.setDob(new
java.sql.Date(fieldSet.readDate(3).getTime()));
        return result;
    }
}
```

SpringCsvToXmlTest.java:-

```
package com.nt.test;

import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobExecutionException;
import org.springframework.batch.core.JobParameters;
import org.springframework.batch.core.launch.JobLauncher;
import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContext;

public class SpringCsvToXmlTest {

    @SuppressWarnings("resource")
    public static void main(String areg[]) {

        ApplicationContext ctx = null;
        JobLauncher jobLauncher = null;
        JobExecution execution = null;
        Job job = null;
    }
}
```

```

    ctx = new ClassPathXmlApplicationContext("com/nt/cfgs/spring-
batch-beans.xml");

    jobLauncher = (JobLauncher) ctx.getBean("jobLauncher");
    job = (Job) ctx.getBean("examResultJob");

    try {
        System.out.println("----- " + jobLauncher);

        execution = jobLauncher.run(job, new JobParameters());
        System.out.println("Job Exit Status : " +
execution.getStatus());
    }
    catch (JobExecutionException e) {
        System.out.println("Job ExamResult failed");
        e.printStackTrace();
    }
}
}

```

pom.xml:-

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>nit</groupId>
    <artifactId>SpringBatchDatabaseToCsv</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <name>SpringBatchDatabaseToCsv</name>
    <url>http://maven.apache.org</url>

    <dependencies>

        <!--
https://mvnrepository.com/artifact/org.springframework/spring-context -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>4.3.9.RELEASE</version>
        </dependency>

        <!--
https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
        <dependency>
            <groupId>org.springframework</groupId>

```

```
<artifactId>spring-context-support</artifactId>
<version>4.3.9.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-jdbc</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework.batch</groupId>
  <artifactId>spring-batch-core</artifactId>
  <version>3.0.8.RELEASE</version>
</dependency>
<dependency>
  <groupId>org.springframework.batch</groupId>
  <artifactId>spring-batch-infrastructure</artifactId>
  <version>3.0.8.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-oxm</artifactId>
  <version>4.3.9.RELEASE</version>
</dependency>

</dependencies>
</project>
```

springBatchApp4-XMLToDB

- src/main/java
 - com.nt.adapter
 - SQLDateAdapter.java
 - com.nt.batch
 - ExamResultItemProcessor.java
 - com.nt.batchSetter
 - ExamResultItemPreparedStatementSetter.java
 - com.nt.cfgs
 - persistence-beans.xml
 - spring-batch-beans.xml
 - com.nt.model
 - ExamResult.java
 - com.nt.test
 - SpringXMLToDBBatchTest.java
 - com.nt.xml
 - MugUpMasters.xml
- src/test/java
- JRE System Library [JavaSE-1.8]
- Maven Dependencies
- src
- target
- pom.xml

ExamResult.java:-

```
package com.nt.model;

import java.sql.Date;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlRootElement;
import javax.xml.bind.annotation.adapters.XmlJavaTypeAdapter;

@XmlRootElement(name = "ExamResult")
public class ExamResult {

    private int id;
    private int sem;
    private Date dob;
    private double percentage;

    @XmlElement(name = "id")
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    @XmlElement(name = "sem")
    public int getSem() {
        return sem;
    }
}
```



```
public void setSem(int sem) {
    this.sem = sem;
}

@XmlElement(name = "dob")
@XmlJavaTypeAdapter(type = Date.class, value =
com.nt.adapter.SQLDateAdapter.class)
public Date getDob() {
    return dob;
}

public void setDob(Date dob) {
    this.dob = dob;
}

@XmlElement(name = "percentage")
public double getPercentage() {
    return percentage;
}

public void setPercentage(double percentage) {
    this.percentage = percentage;
}

@Override
public String toString() {
    return "ExamResult [id=" + id + ", sem=" + sem + ", dob=" +
dob + ", percentage=" + percentage + "];"
}
}
```

SQLDateAdapter:-

```
package com.nt.adapter;

import java.sql.Date;
import javax.xml.bind.annotation.adapters.XmlAdapter;

public class SQLDateAdapter extends XmlAdapter<String, Date>{
    public Date unmarshal(String d) throws Exception {
        return java.sql.Date.valueOf(d);
    }

    public String marshal(Date d) throws Exception {
        return d.toString();
    }
}
```

ExamResultItemProcessor.java:-

```
package com.nt.batch;

import org.springframework.batch.item.ItemProcessor;
import com.nt.model.ExamResult;

public class ExamResultItemProcessor implements ItemProcessor<ExamResult, ExamResult>{
    public ExamResult process(ExamResult result) throws Exception {
        System.out.println("Processing result :"+result);
        /*
         * Only return results which are more than 95%
         */
        if(result.getPercentage() <95){
            return null;
        }

        return result;
    }
}
```

ExamResultItemPreparedStatementSetter.java:-

```
package com.nt.batchSetter;

import java.sql.PreparedStatement;
import java.sql.SQLException;
import org.springframework.batch.item.database.ItemPreparedStatementSetter;
import com.nt.model.ExamResult;

public class ExamResultItemPreparedStatementSetter implements ItemPreparedStatementSetter<ExamResult> {

    public void setValues(ExamResult result, PreparedStatement ps)
    throws SQLException {
        ps.setInt(1, result.getId());
        ps.setInt(2,result.getSem());
        ps.setDouble(3, result.getPercentage());
        ps.setDate(4,result.getDob() );
    }
}
```

persistence-beans.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:batch="http://www.springframework.org/schema/batch"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans-
4.0.xsd">

    <bean id="dbcpDs" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName"
value="oracle.jdbc.driver.OracleDriver" />
        <property name="url"
value="jdbc:oracle:thin:@localhost:1521:xe" />
        <property name="username" value="system" />
        <property name="password" value="manager" />
    </bean>
</beans>
```

spring-batch-beans.xml:-

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:batch="http://www.springframework.org/schema/batch"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/batch
http://www.springframework.org/schema/batch/spring-batch-
3.0.xsd
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-
4.0.xsd">

    <import resource="persistence-beans.xml"/>

    <!-- JobRepository and JobLauncher are configuration/setup classes -
->
    <bean id="jobRepository"
class="org.springframework.batch.core.repository.support.MapJobRepository
FactoryBean" />

    <bean id="jobLauncher"
class="org.springframework.batch.core.launch.support.SimpleJobLaunch
er">
        <property name="jobRepository" ref="jobRepository" />
    </bean>

    <!-- ItemReader which reads data from XML file -->
    <bean id="xmlItemReader"
class="org.springframework.batch.item.xml.StaxEventItemReader">
        <property name="resource"
value="classpath:com/nt/xml/MugUpMasters.xml" />
        <property name="fragmentRootElementName" value="ExamResult" />
        <property name="unmarshaller">
            <bean
class="org.springframework.xml.jaxb.Jaxb2Marshaller">
                <property name="classesToBeBound">
                    <list>
```

```

        <value>com.nt.model.ExamResult</value>
    </list>
</property>
</bean>
</property>
</bean>

<!-- ItemWriter which writes data to database -->
<bean id="databaseItemWriter"
class="org.springframework.batch.item.database.JdbcBatchItemWriter">
    <property name="dataSource" ref="dbcpDs" />
    <property name="sql">
        <value>
            <![CDATA[
                insert into EXAM_RESULT values (?, ?, ?, ?)
            ]]>
        </value>
    </property>

    <!-- We need a custom setter to set values to Query param by
getting values ExamResult object -->
    <property name="ItemPreparedStatementSetter">
        <bean
class="com.nt.batchSetter.ExamResultItemPreparedStatementSetter"/>
    </property>
</bean>

<!-- Optional ItemProcessor to perform business logic/filtering on
the input records -->
<bean id="itemProcessor"
class="com.nt.batch.ExamResultItemProcessor" />

<!-- Step will need a transaction manager -->
<bean id="transactionManager"
class="org.springframework.batch.support.transaction.ResourceLessTransact
ionManager" />

<!-- Actual Job -->
<batch:job id="examResultJob">
    <batch:step id="step1">
        <batch:tasklet transaction-manager="transactionManager">
            <batch:chunk reader="xmlItemReader"
writer="databaseItemWriter"
processor="itemProcessor" commit-interval="10"
/>
        </batch:tasklet>
    </batch:step>

```

```
</batch:job>

</beans>
```

SpringXMLToDBBatchTest.java:-

```
package com.nt.test;

import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobExecutionException;
import org.springframework.batch.core.JobParameters;
import org.springframework.batch.core.launch.JobLauncher;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class SpringXMLToDBBatchTest {

    @SuppressWarnings("resource")
    public static void main(String areg[]) {

        ApplicationContext ctx = null;
        JobLauncher jobLauncher = null;
        JobExecution execution = null;
        Job job = null;

        ctx = new ClassPathXmlApplicationContext("com/nt/cfgs/spring-
batch-beans.xml");

        jobLauncher = (JobLauncher) ctx.getBean("jobLauncher");
        job = (Job) ctx.getBean("examResultJob");

        try {
            System.out.println("----- " + jobLauncher);

            execution = jobLauncher.run(job, new JobParameters());
            System.out.println("Job Exit Status : " +
execution.getStatus());
        }
        catch (JobExecutionException e) {
            System.out.println("Job ExamResult failed");
            e.printStackTrace();
        }
    }
}
```

pom.xml:-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<groupId>nit</groupId>
<artifactId>SpringBatchDatabaseToCsv</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>SpringBatchDatabaseToCsv</name>
<url>http://maven.apache.org</url>

<dependencies>

    <!--
https://mvnrepository.com/artifact/org.springframework/spring-context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>4.3.9.RELEASE</version>
    </dependency>

    <!--
https://mvnrepository.com/artifact/org.springframework/spring-context-
support -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context-support</artifactId>
        <version>4.3.9.RELEASE</version>
    </dependency>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-core</artifactId>
        <version>4.3.9.RELEASE</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>4.3.9.RELEASE</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.batch</groupId>
        <artifactId>spring-batch-core</artifactId>
        <version>3.0.8.RELEASE</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.batch</groupId>
        <artifactId>spring-batch-infrastructure</artifactId>
```

```
        <version>3.0.8.RELEASE</version>
    </dependency>

    <!--
https://mvnrepository.com/artifact/org.apache.commons/commons-dbc2 -->
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-dbc2</artifactId>
        <version>2.1.1</version>
    </dependency>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-oxm</artifactId>
        <version>4.3.9.RELEASE</version>
    </dependency>

</dependencies>
</project>
```

@@All the best my dear students@@

=====000=====

SPRING FAQs

Q. Explain DI or IOC pattern.

A: Dependency injection (DI) is a programming design pattern and architectural model, sometimes also referred to as inversion of control or IOC, although technically speaking, dependency injection specifically refers to an implementation of a particular form of IOC. Dependency Injection describes the situation where one object uses a second object to provide a particular capacity. For example, being passed a database connection as an argument to the constructor instead of creating one internally. The term "Dependency injection" is a misnomer, since it is not a dependency that is injected, rather it is a provider of some capability or resource that is injected. There are three common forms of dependency injection: setter-, constructor- and interface-based injection. Dependency injection is a way to achieve loose coupling. Inversion of control (IOC) relates to the way in which an object obtains references to its dependencies. This is often done by a lookup method. The advantage of inversion of control is that it decouples objects from specific lookup mechanisms and implementations of the objects it depends on. As a result, more flexibility is obtained for production applications as well as for testing.

Q. What are the different IOC containers available?

A. Spring is an IOC container. Other IOC containers are HiveMind, Avalon, PicoContainer.

Q. What are the different types of dependency injection. Explain with examples.

A: There are two types of dependency injection: setter injection and constructor injection. Setter Injection: Normally in all the java beans, we will use setter and getter method to set and get the value of property as follows:

```
public class namebean {  
    String name;  
    public void setName(String a) {  
        name = a; }  
    public String getName() {  
        return name; }  
}
```

We will create an instance of the bean 'namebean' (say bean1) and set property as `bean1.setName("tom");` Here in setter injection, we will set the property 'name' in spring configuration file as shown below:

```
<bean id="bean1" class="namebean">  
    <property name="name">  
        <value>tom</value>  
    </property>  
</bean>
```

The subelement <value> sets the 'name' property by calling the set method as `setName("tom");` This process is called setter injection.

To set properties that reference other beans <ref>, subelement of <property> is used as shown below,


```
<bean id="bean1" class="bean1impl">
  <property name="game">
    <ref bean="bean2"/>
  </property>
</bean>
<bean id="bean2" class="bean2impl" />
```

Constructor Injection: For constructor injection, we use constructor with parameters as shown below,

```
public class namebean {
    String name;
    public namebean(String a) {
        name = a;
    }
}
```

We will set the property 'name' while creating an instance of the bean 'namebean' as namebean bean1 = new namebean("tom");

Here we use the <constructor-arg> element to set the the property by constructor injection as

```
<bean id="bean1" class="namebean">
  <constructor-arg>
    <value>My Bean Value</value>
  </constructor-arg>
</bean>
```

Q. What is spring? What are the various parts of spring framework? What are the different persistence frameworks which could be used with spring?

A. Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development. The Spring modules are built on top of the core container, which defines how beans are created, configured, and managed, as shown in the following figure. Each of the modules (or components) that comprise the Spring framework can stand on its own or be implemented jointly with one or more of the others. The functionality of each component is as follows:

The Core Container: The core container provides the essential functionality of the Spring framework. A primary component of the core container is the BeanFactory, an implementation of the Factory pattern. The BeanFactory applies the Inversion of Control (IOC) pattern to separate an application's configuration and dependency specification from the actual application code.

Spring Context: The Spring context is a configuration file that provides context information to the Spring framework. The Spring context includes enterprise services such as JNDI, EJB, e-mail, internalization, validation, and scheduling functionality.

Spring AOP: The Spring AOP module integrates aspect-oriented programming functionality directly into the Spring framework, through its configuration management feature. As a result you can easily AOP-enable any object managed by the Spring framework. The Spring AOP module provides transaction management services for objects in any Spring-based application. With Spring AOP you can incorporate declarative transaction management into your applications without relying on EJB components.

Spring DAO: The Spring JDBC DAO abstraction layer offers a meaningful exception hierarchy for managing the exception handling and error messages thrown by different database vendors. The exception hierarchy simplifies error handling and greatly reduces the amount of exception code you need to write, such as opening and closing connections. Spring DAO's JDBC-oriented exceptions comply to its generic DAO exception hierarchy.

Spring ORM: The Spring framework plugs into several ORM frameworks to provide its Object Relational tool, including JDO, Hibernate, and iBatis SQL Maps. All of these comply to Spring's generic transaction and DAO exception hierarchies.

Spring Web Module: The Web context module builds on top of the application context module, providing contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

Spring MVC Framework: The Model-View-Controller (MVC) framework is a full-featured MVC implementation for building Web applications. The MVC framework is highly configurable via strategy interfaces and accommodates numerous view technologies including JSP, Velocity, Tiles, iText, and POI.

Q. What is AOP? How does it relate with IOC? What are different tools to utilize AOP?

A: Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules. AOP and IOC are complementary technologies in that both apply a modular approach to complex problems in enterprise application development. In a typical object-oriented development approach you might implement logging functionality by putting logger statements in all your methods and Java classes. In an AOP approach you would instead modularize the logging services and apply them declaratively to the components that required logging. The advantage, of course, is that the Java class doesn't need to know about the existence of the logging service or concern itself with any related code. As a result, application code written using Spring AOP is loosely coupled. The best tool to utilize AOP to its capability is AspectJ. However AspectJ works at the byte code level and you need to use AspectJ compiler to get the aop features built into your compiled code. Nevertheless AOP functionality is fully integrated into the Spring context for transaction management, logging, and various other features. In general any AOP framework control aspects in three possible ways:

Joinpoints: Points in a program's execution. For example, joinpoints could define calls to specific methods in a class.

Pointcuts: Program constructs to designate joinpoints and collect specific context at those points

Advices: Code that runs upon meeting certain conditions. For example, an advice could log a message before executing a joinpoint

Q. What are the advantages of spring framework?

A.

- 1. Spring has layed architecture. Use what you need and leave you don't need now.*
- 2. Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continous integration and testability.*
- 3. Dependency Injection and Inversion of Control Simplifies JDBC (Read the first question.)*
- 4. Open source and no vendor lock-in.*

Q. Can you name a tool which could provide the initial ant files and directory structure for a new spring project.

A: Appfuse or equinox.

Q. Explain BeanFactory in spring.

A: Bean factory is an implementation of the factory design pattern and its function is to create and dispense beans. As the bean factory knows about many objects within an application, it is able to create association between collaborating objects as they are instantiated. This removes the burden of configuration from the bean and the client. There are several implementation of BeanFactory. The most useful one is "org.springframework.beans.factory.xml.XmlBeanFactory" It loads its beans based on the definition contained in an XML file. To create an XmlBeanFactory, pass a InputStream to the constructor. The resource will provide the XML to the factory. BeanFactory factory = new XmlBeanFactory(new FileInputStream("myBean.xml"));

This line tells the bean factory to read the bean definition from the XML file. The bean definition includes the description of beans and their properties. But the bean factory doesn't instantiate the bean yet. To retrieve a bean from a 'BeanFactory', the getBean() method is called. When getBean() method is called, factory will instantiate the bean and begin setting the bean's properties using dependency injection. myBean bean1 = (myBean)factory.getBean("myBean");

Q. Explain the role of ApplicationContext in spring.

A. While Bean Factory is used for simple applications, the Application Context is spring's more advanced container. Like 'BeanFactory' it can be used to load bean definitions, wire beans together and dispense beans upon request. It also provide

- 1) a means for resolving text messages, including support for internationalization.
- 2) a generic way to load file resources.
- 3) events to beans that are registered as listeners.

Because of additional functionality, 'Application Context' is preferred over a BeanFactory. Only when the resource is scarce like mobile devices, 'BeanFactory' is used. The three commonly used implementation of 'Application Context' are

1. ClassPathXmlApplicationContext : It Loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code
ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
2. FileSystemXmlApplicationContext : It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code
ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");
3. XmlWebApplicationContext : It loads context definition from an XML file contained within a web application.

Q. How does Spring supports DAO in hibernate?

A. Spring's HibernateDaoSupport class is a convenient super class for Hibernate DAOs. It has handy methods you can call to get a Hibernate Session, or a SessionFactory. The most convenient method is getHibernateTemplate(), which returns a HibernateTemplate. This template wraps Hibernate checked exceptions with runtime exceptions, allowing your DAO interfaces to be Hibernate exception-free.

Example:

```
public class UserDAOHibernate extends HibernateDaoSupport {

    public User getUser(Long id) {
        return (User) getHibernateTemplate().get(User.class, id);
    }

    public void saveUser(User user) {
        getHibernateTemplate().saveOrUpdate(user);
        if (log.isDebugEnabled()) {
            log.debug("userId set to: " + user.getID());
        }
    }

    public void removeUser(Long id) {
        Object user = getHibernateTemplate().load(User.class, id);
        getHibernateTemplate().delete(user);
    }

}
```

Q. What are the id generator classes in hibernate?

increment: It generates identifiers of type long, short or int that are unique only when no other process is inserting data into the same table. It should not be used in the clustered environment.

identity: It supports identity columns in DB2, MySQL, MS SQL Server, Sybase and HypersonicSQL. The returned identifier is of type long, short or int.

sequence: The sequence generator uses a sequence in DB2, PostgreSQL, Oracle, SAP DB, McKoi or a generator in Interbase. The returned identifier is of type long, short or int.

hilo: The hilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a table and column (by default hibernate_unique_key and next_hi respectively) as a source of hi values. The hi/lo algorithm generates identifiers that are unique only for a particular database. Do not use this generator with connections enlisted with JTA or with a user-supplied connection.

seqhilo: The seqhilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a named database sequence.

uuid: The uuid generator uses a 128-bit UUID algorithm to generate identifiers of type String, unique within a network (the IP address is used). The UUID is encoded as a String of hexadecimal digits of length 32.

guid: It uses a database-generated GUID String on MS SQL Server and MySQL.

native: It picks identity, sequence or hilo depending upon the capabilities of the underlying database.

assigned: Lets the application to assign an identifier to the object before save() is called. This is the default strategy if no <generator> element is specified.

select: retrieves a primary key assigned by a database trigger by selecting the row by some unique key and retrieving the primary key value.

foreign: uses the identifier of another associated object. Usually used in conjunction with a <one-to-one> primary key association.

Q. How is a typical spring implementation look like?

A. For a typical Spring Application we need the following files

1. An interface that defines the functions.
2. An Implementation that contains properties, its setter and getter methods, functions etc.,
3. A XML file called Spring configuration file.
4. Client program that uses the function.

Q. How do you define hibernate mapping file in spring?

A. Add the hibernate mapping file entry in mapping resource inside Spring's applicationContext.xml file in the web/WEB-INF directory.

```
<property name="mappingResources">
  <list>
    <value>org/appfuse/model/User.hbm.xml</value>
  </list>
</property>
```

Q. How do you configure spring in a web application?

A. It is very easy to configure any J2EE-based web application to use Spring. At the very least, you can simply add Spring's ContextLoaderListener to your web.xml file:

```
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
```

Q. Can you have xyz.xml file instead of applicationContext.xml?

A. ContextLoaderListener is a ServletContextListener that initializes when your webapp starts up. By default, it looks for Spring's configuration file at WEB-INF/applicationContext.xml. You can change this default value by specifying a <context-param> element named "contextConfigLocation."

Example:

```
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener

  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/xyz.xml</param-value>
  </context-param>
</listener-class>
</listener>
```

Q. How do you configure your database driver in spring?

A. Using datasource "org.springframework.jdbc.datasource.DriverManagerDataSource".

Example:

```
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName">
    <value>org.hsqldb.jdbcDriver</value>
  </property>
  <property name="url">
    <value>jdbc:hsqldb:db/appfuse</value>
  </property>
```

```
<property name="username"><value>sa</value></property>
<property name="password"><value></value></property>
</bean>
```

Q. How can you configure JNDI instead of datasource in spring applicationContext.xml?

A. Using "org.springframework.jndi.JndiObjectFactoryBean". Example:

```
<bean id="dataSource" class="org.springframework.jndi.JndiObjectFactoryBean">
  <property name="jndiName">
    <value>java:comp/env/jdbc/appfuse</value>
  </property>
</bean>
```

Q. What are the key benefits of Hibernate?

A: These are the key benefits of Hibernate:

- Transparent persistence based on POJOs without byte code processing
- Powerful object-oriented hibernate query language
- Descriptive O/R Mapping through mapping file.
- Automatic primary key generation
- Hibernate cache : Session Level, Query and Second level cache.
- Performance: Lazy initialization, Outer join fetching, Batch fetching

Q. What is hibernate session and session factory? How do you configure sessionFactory in spring configuration file?

A. Hibernate Session is the main runtime interface between a Java application and Hibernate. SessionFactory allows applications to create hibernate session by reading hibernate configurations file hibernate.configure.xml.

```
// Initialize the Hibernate environment
Configuration configure = new Configuration().configure();
// Create the session factory
SessionFactory factory = configure.buildSessionFactory();
// Obtain the new session object
Session session = factory.openSession();
```

The call to Configuration().configure() loads the hibernate.configure.xml configuration file and initializes the Hibernate environment. Once the configuration is initialized, you can make any additional modifications you desire programmatically. However, you must make these modifications prior to creating the SessionFactory instance. An instance of SessionFactory is typically created once and used to create all sessions related to a given context.

The main function of the Session is to offer create, read and delete operations for instances of mapped entity classes. Instances may exist in one of three states:

transient: never persistent, not associated with any Session

persistent: associated with a unique Session

detached: previously persistent, not associated with any Session

A Hibernate Session object represents a single unit-of-work for a given data store and is opened by a SessionFactory instance. You must close Sessions when all work for a transaction is completed.

The following illustrates a typical Hibernate session:

```
Session session = null;
UserInfo user = null;
```

```
Transaction tx = null;
try {
    session = factory.openSession();
    tx = session.beginTransaction();
    user = (UserInfo)session.load(UserInfo.class, id);
    tx.commit();
} catch (Exception e) {
    if (tx != null) {
        try {
            tx.rollback();
        } catch (HibernateException e1) {
            throw new DAOException(e1.toString());
        }
    }
    throw new DAOException(e.toString());
} finally {
    if (session != null) {
        try {
            session.close();
        } catch (HibernateException e) {}
    }
}
```

Q. What is the difference between hibernate get and load methods?

A. The load() method is older; get() was added to Hibernate's API due to user request. The difference is trivial:

The following Hibernate code snippet retrieves a User object from the database: `User user = (User) session.get(User.class, userID);`

The `get()` method is special because the identifier uniquely identifies a single instance of a class. Hence it's common for applications to use the identifier as a convenient handle to a persistent object. Retrieval by identifier can use the cache when retrieving an object, avoiding a database hit if the object is already cached.

Hibernate also provides a `load()` method: `User user = (User) session.load(User.class, userID);`

If `load()` can't find the object in the cache or database, an exception is thrown. The `load()` method never returns null. The `get()` method returns

null if the object can't be found. The `load()` method may return a proxy instead of a real persistent instance. A proxy is a placeholder instance of a runtime-generated subclass (through cglib or Javassist) of a mapped persistent class, it can initialize itself if any method is called that is not the mapped database identifier getter-method. On the other hand, `get()` never returns a proxy.

Choosing between `get()` and `load()` is easy: If you're certain the persistent object exists, and nonexistence would be considered exceptional, `load()` is a good option. If you aren't certain there is a persistent instance with the given

identifier, use `get()` and test the return value to see if it's null. Using `load()` has a further implication: The application may retrieve a valid reference (a proxy) to a persistent instance without hitting the database to retrieve its persistent state. So `load()` might not throw an exception when it doesn't find the persistent object in the cache or database; the exception would be thrown later, when the proxy is accessed.

Q. What type of transaction management is supported in hibernate?

A. Hibernate communicates with the database via a JDBC Connection; hence it must support both managed and non-managed transactions.

non-managed in web containers:

```
<bean id="transactionManager"
class="org.springframework.orm.hibernate.HibernateTransactionManager">
  <property name="sessionFactory">
    <ref local="sessionFactory"/>
  </property>
</bean>
```

managed in application server using JTA:

```
<bean id="transactionManager"
class="org.springframework.transaction.jta.JtaTransactionManager">
  <property name="sessionFactory">
    <ref local="sessionFactory"/>
  </property>
</bean>
```

Q. What is lazy loading and how do you achieve that in hibernate?

A. Lazy setting decides whether to load child objects while loading the Parent Object. You need to specify parent class.Lazy = true in hibernate mapping file. By default the lazy loading of the child objects is true. This make sure that the child objects are not loaded unless they are explicitly invoked in the application by calling getChild() method on parent. In this case hibernate issues a fresh database call to load the child when getChild() is actually called on the Parent object. But in some cases you do need to load the child objects when parent is loaded. Just make the lazy=false and hibernate will load the child when parent is loaded from the database. Examples: Address child of User class can be made lazy if it is not required frequently. But you may need to load the Author object for Book parent whenever you deal with the book for online bookshop. Hibernate does not support lazy initialization for detached objects. Access to a lazy association outside of the context of an open Hibernate session will result in an exception.

Q. What are the different fetching strategy in Hibernate?

A. Hibernate3 defines the following fetching strategies:

Join fetching - Hibernate retrieves the associated instance or collection in the same SELECT, using an OUTER JOIN.

Select fetching - a second SELECT is used to retrieve the associated entity or collection. Unless you explicitly disable lazy fetching by specifying lazy="false", this second select will only be executed when you actually access the association.

Subselect fetching - a second SELECT is used to retrieve the associated collections for all entities retrieved in a previous query or fetch. Unless you explicitly disable lazy fetching by specifying lazy="false", this second select will only be executed when you actually access the association.

Batch fetching - an optimization strategy for select fetching - Hibernate retrieves a batch of entity instances or collections in a single SELECT, by specifying a list of primary keys or foreign keys. For more details read short primer on fetching strategy at <http://www.hibernate.org/315.html>

Q. What are different types of cache hibernate supports ?

A. Caching is widely used for optimizing database applications. Hibernate uses two different caches for objects: first-level cache and second-level cache. First-level cache is associated with the Session object, while second-level cache is associated with the Session Factory object. By default, Hibernate uses first-level cache on a per-transaction basis. Hibernate uses this cache mainly to reduce the number of SQL queries it needs to generate within a given transaction. For example, if an object is modified several times within the same transaction, Hibernate will generate only one SQL UPDATE statement at the end of the transaction, containing all the modifications. To reduce database traffic, second-level cache keeps loaded objects at the Session Factory level between transactions. These objects are available to the whole application, not just to the user running the query. This way, each time a query returns an object that is already loaded in the cache, one or more database transactions potentially are avoided. In addition, you can use a query-level cache if you need to cache actual query results, rather than just persistent objects. The query cache should always be used in conjunction with the second-level cache. Hibernate supports the following open-source cache implementations out-of-the-box:

- **EHCache** is a fast, lightweight, and easy-to-use in-process cache. It supports read-only and read/write caching, and memory- and disk-based caching. However, it does not support clustering.
- **OSCache** is another open-source caching solution. It is part of a larger package, which also provides caching functionalities for JSP pages or arbitrary objects. It is a powerful and flexible package, which, like EHCache, supports read-only and read/write caching, and memory- and disk-based caching. It also provides basic support for clustering via either JavaGroups or JMS.
- **SwarmCache** is a simple cluster-based caching solution based on JavaGroups. It supports read-only or nonstrict read/write caching (the next section explains this term). This type of cache is appropriate for applications that typically have many more read operations than write operations.
- **JBoss TreeCache** is a powerful replicated (synchronous or asynchronous) and transactional cache. Use this solution if you really need a true transaction-capable caching architecture.
- **Commercial Tangosol Coherence** cache.

Q. What are the different caching strategies?

A. The following four caching strategies are available:

- **Read-only:** This strategy is useful for data that is read frequently but never updated. This is by far the simplest and best-performing cache strategy.
- **Read/write:** Read/write caches may be appropriate if your data needs to be updated. They carry more overhead than read-only caches. In non-JTA environments, each transaction should be completed when `Session.close()` or `Session.disconnect()` is called.
- **Nonstrict read/write:** This strategy does not guarantee that two transactions won't simultaneously modify the same data. Therefore, it may be most appropriate for data that is read often but only occasionally modified.
- **Transactional:** This is a fully transactional cache that may be used only in a JTA environment.

Q. How do you configure 2nd level cache in hibernate?

A. To activate second-level caching, you need to define the `hibernate.cache.provider_class` property in the `hibernate.config.xml` file as follows: `<hibernate-configuration>`

```
<session-factory>  
  <property  
name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>  
  </session-factory>  
</hibernate-configuration>
```

By default, the second-level cache is activated and uses the EHCache provider.

To use the query cache you must first enable it by setting the property

`hibernate.cache.use_query_cache` to true in `hibernate.properties`.

Q. What is the difference between sorted and ordered collection in hibernate?

A. A sorted collection is sorted in-memory using java comparator, while order collection is ordered at the database level using order by clause.

Q. What are the types of inheritance models and describe how they work like vertical inheritance and horizontal?

A. There are three types of inheritance mapping in hibernate :

Example: Let us take the simple example of 3 java classes. Class Manager and Worker are inherited from Employee Abstract class.

1. Table per concrete class with unions : In this case there will be 2 tables. Tables: Manager, Worker [all common attributes will be duplicated]

2. Table per class hierarchy: Single Table can be mapped to a class hierarchy. There will be only one table in database called 'Employee' that will represent all the attributes required for all 3 classes. But it needs some discriminating column to differentiate between Manager and worker;

3. Table per subclass: In this case there will be 3 tables represent Employee, Manager and Worker

Additional Questions:

1. What is IOC (or Dependency Injection)?

The basic concept of the Inversion of Control pattern (also known as dependency injection) is that you do not create your objects but describe how they should be created. You don't directly connect your components and services together in code but describe which services are needed by which components in a configuration file. A container (in the case of the Spring framework, the IOC container) is then responsible for hooking it all up.

i.e., Applying IoC, objects are given their dependencies at creation time by some external entity that coordinates each object in the system. That is, dependencies are injected into objects. So, IoC means an inversion of responsibility with regard to how an object obtains references to collaborating objects.

2. What are the different types of IOC (dependency injection) ?

There are three types of dependency injection:

- **Constructor Injection** (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- **Setter Injection** (e.g. Spring): Dependencies are assigned through JavaBeans properties (ex: setter methods).
- **Interface Injection** (e.g. Avalon): Injection is done through an interface.

Note: Spring supports only Constructor and Setter Injection

3. What are the benefits of IOC (Dependency Injection)?

Benefits of IOC (Dependency Injection) are as follows:

- Minimizes the amount of code in your application. With IOC containers you do not care about how services are created and how you get references to the ones you need. You can also easily add additional services by adding a new constructor or a setter method with little or no extra configuration.
- Make your application more testable by not requiring any singletons or JNDI lookup mechanisms in your unit test cases. IOC containers make unit testing and switching implementations very easy by manually allowing you to inject your own objects into the object under test.
- Loose coupling is promoted with minimal effort and least intrusive mechanism. The factory design pattern is more intrusive because components or services need to be requested explicitly whereas in IOC the dependency is injected into requesting piece of code. Also some containers promote the design to interfaces not to implementations design concept by encouraging managed objects to implement a well-defined service interface of your own.
- IOC containers support eager instantiation and lazy loading of services. Containers also provide support for instantiation of managed objects, cyclical dependencies, life cycles management, and dependency resolution between managed objects etc.

4. What is Spring ?

Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development.

5. What are the advantages of Spring framework?

The advantages of Spring are as follows:

- Spring has layered architecture. Use what you need and leave you don't need now.
- Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
- Dependency Injection and Inversion of Control Simplifies JDBC
- Open source and no vendor lock-in.

6. What are features of Spring ?

Lightweight:

spring is lightweight when it comes to size and transparency. The basic version of spring framework is around 1MB. And the processing overhead is also very negligible.

Inversion of control (IOC):

Loose coupling is achieved in spring using the technique Inversion of Control. The objects give their dependencies instead of creating or looking for dependent objects.

Aspect oriented (AOP):

Spring supports Aspect oriented programming and enables cohesive development by separating application business logic from system services.

Container:

Spring contains and manages the life cycle and configuration of application objects.

MVC Framework:

Spring comes with MVC web application framework, built on core Spring functionality. This framework is highly configurable via strategy interfaces, and accommodates multiple view technologies like JSP, Velocity, Tiles, iText, and POI. But other frameworks can be easily used instead of Spring MVC Framework.

Transaction Management:

Spring framework provides a generic abstraction layer for transaction management. This allowing the developer to add the pluggable transaction managers, and making it easy to demarcate transactions without dealing with low-level issues. Spring's transaction support is not tied to J2EE environments and it can be also used in container less environments.

JDBC Exception Handling:

The JDBC abstraction layer of the Spring offers a meaningful exception hierarchy, which simplifies the error handling strategy. Integration with Hibernate, JDO, and iBATIS: Spring provides best Integration services with Hibernate, JDO and iBATIS contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

7. What is web module?

This module is built on the application context module, providing a context that is appropriate for web-based applications. This module also contains support for several web-oriented tasks such as transparently handling multipart requests for file uploads and programmatic binding of request parameters to your business objects. It also contains integration support with Jakarta Struts.

8. What are the types of Dependency Injection Spring supports?

Setter Injection:

Setter-based DI is realized by calling setter methods on your beans after invoking a no-argument constructor or no-argument static factory method to instantiate your bean.

Constructor Injection:

Constructor-based DI is realized by invoking a constructor with a number of arguments, each representing a collaborator.

9. What is Bean Factory ?

A BeanFactory is like a factory class that contains a collection of beans. The BeanFactory holds Bean Definitions of multiple beans within itself and then instantiates the bean whenever asked for by clients.

- BeanFactory is able to create associations between collaborating objects as they are instantiated. This removes the burden of configuration from bean itself and the beans client.
- BeanFactory also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

10. What is Application Context?

A bean factory is fine to simple applications, but to take advantage of the full power of the Spring framework, you may want to move up to Spring's more advanced container, the application context. On the surface, an application context is same as a bean factory. Both load bean definitions, wire beans together, and dispense beans upon request. But it also provides:

- A means for resolving text messages, including support for internationalization.
- A generic way to load file resources.
- Events to beans that are registered as listeners

11. What is the difference between Bean Factory and Application Context ?

On the surface, an application context is same as a bean factory. But application context offers much more..

- Application contexts provide a means for resolving text messages, including support for i18n of those messages.
- Application contexts provide a generic way to load file resources, such as images.
- Application contexts can publish events to beans that are registered as listeners.
- Certain operations on the container or beans in the container, which have to be handled in a programmatic fashion with a bean factory, can be handled declaratively in an application context.
- ResourceLoader support: Spring's Resource interface is a flexible generic abstraction for handling low-level resources. An application context itself is a ResourceLoader, Hence provides an application with access to deployment-specific Resource instances.
- MessageSource support: The application context implements MessageSource, an interface used to obtain localized messages, with the actual implementation being pluggable

12. What are the common implementations of the Application Context ?

The three commonly used implementation of 'Application Context' are

- ClassPathXmlApplicationContext : It Loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code .
- ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
- FileSystemXmlApplicationContext : It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code .
- ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");
- XmlWebApplicationContext : It loads context definition from an XML file contained within a

web application.

13. How is a typical spring implementation look like ?

For a typical Spring Application we need the following files:

- An interface that defines the functions.
- An Implementation that contains properties, its setter and getter methods, functions etc.,
- Spring AOP (Aspect Oriented Programming)
- A XML file called Spring configuration file.
- Client program that uses the function.

14. What is the typical Bean life cycle in Spring Bean Factory Container ?

Bean life cycle in Spring Bean Factory Container is as follows:

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition
- If the bean implements the BeanNameAware interface, the factory calls `setBeanName()` passing the bean's ID.
- If the bean implements the BeanFactoryAware interface, the factory calls `setBeanFactory()`, passing an instance of itself.
- If there are any BeanPostProcessors associated with the bean, their `postProcessBeforeInitialization()` methods will be called.
- If an init-method is specified for the bean, it will be called.
- Finally, if there are any BeanPostProcessors associated with the bean, their `postProcessAfterInitialization()` methods will be called.

15. What do you mean by Bean wiring ?

The act of creating associations between application components (beans) within the Spring container is referred to as Bean wiring.

16. What do you mean by Auto Wiring?

The Spring container is able to autowire relationships between collaborating beans. This means that it is possible to automatically let Spring resolve collaborators (other beans) for your bean by inspecting the contents of the BeanFactory. The autowiring functionality has five modes.

- no
- `byName`
- `byType`
- `constructor`
- `autodirect`

17. What is DelegatingVariableResolver?

Spring provides a custom JavaServer Faces VariableResolver implementation that extends the standard Java Server Faces managed beans mechanism which lets you use JSR169 and Spring together. This variable resolver is called as DelegatingVariableResolver

18. How to integrate Java Server Faces (JSR169) with Spring?

JSR169 and Spring do share some of the same features, most noticeably in the area of IOC services. By declaring JSR169 managed-beans in the `faces-config.xml` configuration file, you allow the FacesServlet to instantiate that bean at startup. Your

JSPRINGFRAMEWORK pages have access to these beans and all of their properties. We can integrate JSPRINGFRAMEWORK and Spring in two ways:

- *DelegatingVariableResolver*: Spring comes with a JSPRINGFRAMEWORK variable resolver that lets you use JSPRINGFRAMEWORK and Spring together.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<faces-config>
<application>
<variable-resolver>
    org.springframework.web.jspringframework.DelegatingVariableResolver
</variable-resolver>
</application>
</faces-config>
```

The *DelegatingVariableResolver* will first delegate value lookups to the default resolver of the underlying JSPRINGFRAMEWORK implementation, and then to Spring's 'business context' *WebApplicationContext*. This allows one to easily inject dependencies into one's JSPRINGFRAMEWORK-managed beans.

- *FacesContextUtils*: custom *VariableResolver* works well when mapping one's properties to beans in *faces-config.xml*, but at times one may need to grab a bean explicitly. The *FacesContextUtils* class makes this easy. It is similar to *WebApplicationContextUtils*, except that it takes a *FacesContext* parameter rather than a *ServletContext* parameter.

```
ApplicationContext                               ctx                               =
FacesContextUtils.getWebApplicationContext(FacesContext.getCurrentInstance());
```

19. What is Java Server Faces (JSPRINGFRAMEWORK) - Spring integration mechanism?

Spring provides a custom JavaServer Faces *VariableResolver* implementation that extends the standard JavaServer Faces managed beans mechanism. When asked to resolve a variable name, the following algorithm is performed:

- Does a bean with the specified name already exist in some scope (request, session, application)? If so, return it
- Is there a standard JavaServer Faces managed bean definition for this variable name? If so, invoke it in the usual way, and return the bean that was created.
- Is there configuration information for this variable name in the Spring *WebApplicationContext* for this application? If so, use it to create and configure an instance, and return that instance to the caller.
- If there is no managed bean or Spring definition for this variable name, return null instead.
- *BeanFactory* also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

As a result of this algorithm, you can transparently use either JavaServer Faces or Spring facilities to create beans on demand.

20. What is Significance of JSPRINGFRAMEWORK- Spring integration ?

Spring - JSPRINGFRAMEWORK integration is useful when an event handler wishes to explicitly invoke the bean factory to create beans on demand, such as a bean that encapsulates the business logic to be performed when a submit button is pressed.

21. How to integrate your Struts application with Spring?

To integrate your Struts application with Spring, we have two options:

- Configure Spring to manage your Actions as beans, using the `ContextLoaderPlugin`, and set their dependencies in a Spring context file.
- Subclass Spring's `ActionSupport` classes and grab your Spring-managed beans explicitly using a `getWebApplicationContext()` method.

22. What are ORM's Spring supports ?

Spring supports the following ORM's :

- Hibernate
- iBatis
- JPA (Java Persistence API)
- TopLink
- JDO (Java Data Objects)
- OJB

23. What are the ways to access Hibernate using Spring ?

There are two approaches to Spring's Hibernate integration:

- Inversion of Control with a `HibernateTemplate` and Callback
- Extending `HibernateDaoSupport` and Applying an AOP Interceptor

24. How to integrate Spring and Hibernate using `HibernateDaoSupport`?

Spring and Hibernate can integrate using Spring's `SessionFactory` called `LocalSessionFactory`. The integration process is of 3 steps.

- Configure the `Hibernate SessionFactory`
- Extend your DAO Implementation from `HibernateDaoSupport`
- Wire in Transaction Support with AOP

25. What are Bean scopes in Spring Framework ?

The Spring Framework supports exactly five scopes (of which three are available only if you are using a web-aware `ApplicationContext`). The scopes supported are listed below:

singleton

Scopes a single bean definition to a single object instance per Spring IoC container.

prototype

Scopes a single bean definition to any number of object instances.

request

Scopes a single bean definition to the lifecycle of a single HTTP request; that is each and every HTTP request will have its own instance of a bean created off the back of a single bean definition. Only valid in the context of a web-aware Spring `ApplicationContext`.

session

Scopes a single bean definition to the lifecycle of a HTTP Session. Only valid in the context of a web-aware Spring `ApplicationContext`.

global session

Scopes a single bean definition to the lifecycle of a global HTTP Session. Typically only valid when used in a portlet context. Only valid in the context of a web-aware Spring `ApplicationContext`.

26. What is AOP?

Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules.

27. How the AOP used in Spring?

AOP is used in the Spring Framework: To provide declarative enterprise services, especially as a replacement for EJB declarative services. The most important such service is declarative transaction management, which builds on the Spring Framework's transaction abstraction. To allow users to implement custom aspects, complementing their use of OOP with AOP.

28. What do you mean by Aspect ?

A modularization of a concern that cuts across multiple objects. Transaction management is a good example of a crosscutting concern in J2EE applications. In Spring AOP, aspects are implemented using regular classes (the schema-based approach) or regular classes annotated with the `@Aspect` annotation (`@AspectJ` style).

29. What do you mean by JointPoint?

A point during the execution of a program, such as the execution of a method or the handling of an exception. In Spring AOP, a join point always represents a method execution.

30. What do you mean by Advice?

Action taken by an aspect at a particular join point. Different types of advice include "around," "before" and "after" advice. Many AOP frameworks, including Spring, model an advice as an interceptor, maintaining a chain of interceptors "around" the join point.

31. What are the types of Advice?

Types of advice:

- **Before advice:** Advice that executes before a join point, but which does not have the ability to prevent execution flow proceeding to the join point (unless it throws an exception).
- **After returning advice:** Advice to be executed after a join point completes normally: for example, if a method returns without throwing an exception.
- **After throwing advice:** Advice to be executed if a method exits by throwing an exception.
- **After (finally) advice:** Advice to be executed regardless of the means by which a join point exits (normal or exceptional return).
- **Around advice:** Advice that surrounds a join point such as a method invocation. This is the most powerful kind of advice. Around advice can perform custom behavior before and after the method invocation. It is also responsible for choosing whether to proceed to the join point or to shortcut the advised method execution by returning its own return value or throwing an exception

32. What are the types of the transaction management Spring supports ?

Spring Framework supports:

- Programmatic transaction management.
- Declarative transaction management.

33. What are the benefits of the Spring Framework transaction management ?

The Spring Framework provides a consistent abstraction for transaction management that delivers the following benefits:

- Provides a consistent programming model across different transaction APIs such as JTA, JDBC, Hibernate, JPA, and JDO.
- Supports declarative transaction management.
- Provides a simpler API for programmatic transaction management than a number of complex transaction APIs such as JTA.
- Integrates very well with Spring's various data access abstractions.

34. Why most users of the Spring Framework choose declarative transaction management ?

Most users of the Spring Framework choose declarative transaction management because it is the option with the least impact on application code, and hence is most consistent with the ideals of a non-invasive lightweight container.

35. Explain the similarities and differences between EJB CMT and the Spring Framework's declarative transaction management?

The basic approach is similar: it is possible to specify transaction behavior (or lack of it) down to individual method level. It is possible to make a `setRollbackOnly()` call within a transaction context if necessary. The differences are:

- Unlike EJB CMT, which is tied to JTA, the Spring Framework's declarative transaction management works in any environment. It can work with JDBC, JDO, Hibernate or other transactions under the covers, with configuration changes only.
- The Spring Framework enables declarative transaction management to be applied to any class, not merely special classes such as EJBs.
- The Spring Framework offers declarative rollback rules: this is a feature with no EJB equivalent. Both programmatic and declarative support for rollback rules is provided.
- The Spring Framework gives you an opportunity to customize transactional behavior, using AOP. With EJB CMT, you have no way to influence the container's transaction management other than `setRollbackOnly()`.
- The Spring Framework does not support propagation of transaction contexts across remote calls, as do high-end application servers.

36. What are object/relational mapping integration module?

Spring also supports for using of an object/relational mapping (ORM) tool over straight JDBC by providing the ORM module. Spring provide support to tie into several popular ORM frameworks, including Hibernate, JDO, and iBATIS SQL Maps. Spring's transaction management supports each of these ORM frameworks as well as JDBC.

37. When to use programmatic and declarative transaction management ?

Programmatic transaction management is usually a good idea only if you have a small number of transactional operations. On the other hand, if your application has numerous transactional operations, declarative transaction management is usually worthwhile. It keeps transaction management out of business logic, and is not difficult to configure.

38. Explain about the Spring DAO support ?

The Data Access Object (DAO) support in Spring is aimed at making it easy to work with data access technologies like JDBC, Hibernate or JDO in a consistent way. This allows one to switch between the persistence technologies fairly easily and it also allows one to code without worrying about catching exceptions that are specific to each technology.

39. What are the exceptions thrown by the Spring DAO classes ?

Spring DAO classes throw exceptions which are subclasses of `org.springframework.dao.DataAccessException`. Spring provides a convenient translation from technology-specific exceptions like `SQLException` to its own exception class hierarchy with the `DataAccessException` as the root exception. These exceptions wrap the original exception.

40. What is `SQLExceptionTranslator` ?

`SQLExceptionTranslator`, is an interface to be implemented by classes that can translate between `SQLExceptions` and Spring's own data-access-strategy-agnostic `org.springframework.dao.DataAccessException`.

41. What is Spring's `JdbcTemplate` ?

Spring's `JdbcTemplate` is central class to interact with a database through JDBC. `JdbcTemplate` provides many convenience methods for doing things such as converting database data into primitives or objects, executing prepared and callable statements, and providing custom database error handling.

```
JdbcTemplate template = new JdbcTemplate(myDataSource);
```

42. What is `PreparedStatementCreator` ?

`PreparedStatementCreator`:

- Is one of the most common used interfaces for writing data to database.
- Has one method – `createPreparedStatement(Connection)`
- Responsible for creating a `PreparedStatement`.
- Does not need to handle `SQLExceptions`.

43. What is `SQLProvider` ?

`SQLProvider`:

- Has one method – `getSql()`
- Typically implemented by `PreparedStatementCreator` implementers.
- Useful for debugging.

44. What is `RowCallbackHandler` ?

The `RowCallbackHandler` interface extracts values from each row of a `ResultSet`.

- Has one method – `processRow(ResultSet)`
- Called for each row in `ResultSet`.
- Typically stateful.

45. What are the differences between EJB and Spring ?

Transaction management

EJB:

- Must use a JTA transaction manager.

- Supports transactions that span remote method calls.

Spring:

- Supports multiple transaction environments through its PlatformTransactionManager interface, including JTA, Hibernate, JDO, and JDBC.
- Does not natively support distributed transactions—it must be used with a JTA transaction manager.

Declarative transaction support

EJB:

- Can define transactions declaratively through the deployment descriptor.
- Can define transaction behavior per method or per class by using the wildcard character *.
- Cannot declaratively define rollback behavior—this must be done programmatically.

Spring:

- Can define transactions declaratively through the Spring configuration file or through class metadata.
- Can define which methods to apply transaction behavior explicitly or by using regular expressions.
- Can declaratively define rollback behavior per method and per exception type.

Persistence

EJB:

Supports programmatic bean-managed persistence and declarative container managed persistence.

Spring:

Provides a framework for integrating with several persistence technologies, including JDBC, Hibernate, JDO, and iBATIS.

Declarative security

EJB:

- Supports declarative security through users and roles. The management and implementation of users and roles is container specific.
- Declarative security is configured in the deployment descriptor.

Spring:

- No security implementation out-of-the box.
- Acegi, an open source security framework built on top of Spring, provides declarative security through the Spring configuration file or class metadata.

Distributed computing

EJB:

Provides container-managed remote method calls.

Spring:

Provides proxying for remote calls via RMI, JAX-RPC, and web services.

46 . Do I need any other SOAP framework to run Spring Web Services?

You don't need any other SOAP framework to use Spring Web services, though it can use some of the features of Axis 1 and 2.

47 . I get NAMESPACE_ERR exceptions when using Spring-WS. What can I do about it?

If you get the following Exception:

NAMESPACE_ERR: An attempt is made to create or change an object in a way which is incorrect with regard to namespaces.

Most often, this exception is related to an older version of Xalan being used. Make sure to upgrade to 2.7.0.

48 .Does Spring-WS run under Java 1.3?

Spring Web Services requires Java 1.4 or higher.

49. Does Spring-WS work under Java 1.4?

Spring Web Services works under Java 1.4, but it requires some effort to make it work. Java 1.4 is bundled with the older XML parser Crimson, which does not handle namespaces correctly. Additionally, it is bundled with an older version of Xalan, which also has problems. Unfortunately, placing newer versions of these on the class path does not override them. .

The only solution that works is to add newer versions of Xerces and Xalan in the lib/endorsed directory of your JDK, as explained in those FAQs (i.e.\$JAVA_HOME/lib/endorsed). The following libraries are known to work with Java 1.4.2:

Library	Version
Xerces	2.8.1
Xalan	2.7.0
XML-APIs	1.3.04
SAAJ	1.2

If you want to use WS-Security, note that the XwsSecurityInterceptor requires Java 5, because an underlying library (XWSS) requires it. Instead, you can use the Wss4jSecurityInterceptor.

50 .Does Spring-WS work under Java 1.6?

Java 1.6 ships with SAAJ 1.3, JAXB 2.0, and JAXP 1.4 (a custom version of Xerces and Xalan). Overriding these libraries by putting different version on the classpath will result in various classloading issues, or exceptions in org.apache.xml.serializer.ToXMLSAXHandler. The only option for using more recent versions is to put the newer version in the endorsed directory (see above).

51 . Why do the Spring-WS unit tests fail under Mac OS X?

For some reason, Apple decided to include a Java 1.4 compatibility jar with their JDK 1.5. This jar includes the XML parsers which were included in Java 1.4. No other JDK distribution does this, so it is unclear what the purpose of this compatibility jar is.

The jar can be found at /System/Library/Frameworks/JavaVM.framework/Versions/1.5.0/Classes/.compatibility/14compatibility.jar. You can safely remove or rename it, and the tests will run again.

52 . What is SAAJ?

SAAJ is the SOAP with Attachments API for Java. Like most Java EE libraries, it consists of a set of interfaces (saaj-api.jar), and implementations (saaj-impl.jar). When running in a Application Server, the implementation is typically provided by the application server. Previously, SAAJ has been part of JAXM, but it has been released as a separate API as part of the , and also as part of J2EE 1.4. SAAJ is generally known as the package javax.xml.soap.

Spring-WS uses this standard SAAJ library to create representations of SOAP messages. Alternatively, it can use

59. Why does Spring-WS only support contract-first?

Not that Spring-WS only requires you to write the XSD; the WSDL can be generated from that.

60 . How do I retrieve the WSDL from a Service?

The &WSDL query parameter does not work.

The `&WSDL` query parameter is a way to get a WSDL of a class. In SWS, a service is generally not implemented as a single class, but as a collection of endpoints.

There are two ways to expose a WSDL:

- Simply add the WSDL to the root of the WAR, and the file is served normally. This has the disadvantage that the "location" attribute in the WSDL is static, i.e. it does not necessarily reflect the host name of the server. You can transpringframeworkorm locations by using a `WsdDefinitionHandlerAdapter`.
- Use the `MessageDispatcherServlet`, which is done in the samples. Every `WsdDefinition` listed in the `*-servlet.xml` will be exposed under the bean name. So if you define a `WsdDefinition` named `echo`, it will be exposed as `echo.wsd` (i.e. `http://localhost:8080/echo/echo.wsd`).

61 What is web module?

Spring comes with a full-featured MVC framework for building web applications. Although Spring can easily be integrated with other MVC frameworks, such as Struts, Spring's MVC framework uses IoC to provide for a clean separation of controller logic from business objects. It also allows you to declaratively bind request parameters to your business objects. It also can take advantage of any of Spring's other services, such as I18N messaging and validation.

62 What is a BeanFactory?

A `BeanFactory` is an implementation of the factory pattern that applies Inversion of Control to separate the application's configuration and dependencies from the actual application code.

63 What is AOP Alliance?

AOP Alliance is an open-source project whose goal is to promote adoption of AOP and interoperability among different AOP implementations by defining a common set of interfaces and components.

64 What is Spring configuration file?

Spring configuration file is an XML file. This file contains the classes information and describes how these classes are configured and introduced to each other.

65 .What does a simple spring application contain?

These applications are like any Java application. They are made up of several classes, each performing a specific purpose within the application. But these classes are configured and introduced to each other through an XML file. This XML file describes how to configure the classes, known as the Spring configuration file.

66. What is XmlBeanFactory?

`BeanFactory` has many implementations in Spring. But one of the most useful one is `org.springframework.beans.factory.xml.XmlBeanFactory`, which loads its beans based on the definitions contained in an XML file. To create an `XmlBeanFactory`, pass a `java.io.InputStream` to the constructor. The `InputStream` will provide the XML to the factory. For example, the following code snippet uses a `java.io.FileInputStream` to provide a bean definition XML file to `XmlBeanFactory`.

```
BeanFactory factory = new XmlBeanFactory(new FileInputStream("beans.xml"));
```

To retrieve the bean from a `BeanFactory`, call the `getBean()` method by passing the name of the bean you want to retrieve.

```
MyBean myBean = (MyBean) factory.getBean("myBean")
```

67 . What are important ApplicationContext implementations in spring framework?

- `ClassPathXmlApplicationContext` – This context loads a context definition from an XML file

located in the class path, treating context definition files as class path resources.

- *FileSystemXmlApplicationContext* – This context loads a context definition from an XML file in the filesystem.
- *XmlWebApplicationContext* – This context loads the context definitions from an XML file contained within a web application.

68 . Explain Bean lifecycle in Spring framework?

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition.
- If the bean implements the *BeanNameAware* interface, the factory calls *setBeanName()* passing the bean's ID.
- If the bean implements the *BeanFactoryAware* interface, the factory calls *setBeanFactory()*, passing an instance of itself.
- If there are any *BeanPostProcessors* associated with the bean, their *postProcessBeforeInitialization()* methods will be called.
- If an *init-method* is specified for the bean, it will be called.
- Finally, if there are any *BeanPostProcessors* associated with the bean, their *postProcessAfterInitialization()* methods will be called.

69 What is bean wiring?

Combining together beans within the Spring container is known as bean wiring or wiring. When wiring beans, you should tell the container what beans are needed and how the container should use dependency injection to tie them together.

70. How do add a bean in spring application?

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">
<beans>
<bean id="foo" class="com.act.Foo"/>
<bean id="bar" class="com.act.Bar"/>
</beans>
```

71. What are singleton beans and how can you create prototype beans?

Beans defined in spring framework are singleton beans. There is an attribute in bean tag named 'singleton' if specified true then bean becomes singleton and if set to false then the bean becomes a prototype bean. By default it is set to true. So, all the beans in spring framework are by default singleton beans.

```
<beans>
<bean id="bar" class="com.act.Foo" singleton="false"/>
</beans>
```

72 . What are the important beans lifecycle methods?

There are two important bean lifecycle methods. The first one is *setup* which is called when the bean is loaded in to the container. The second method is the *teardown* method which is called when the bean is unloaded from the container.

73 . How can you override beans default lifecycle methods?

The bean tag has two more important attributes with which you can define your own custom initialization and destroy methods. Here I have shown a small demonstration. Two new methods `fooSetup` and `fooTeardown` are to be added to your `Foo` class.

```
<beans>
<bean id="bar" class="com.act.Foo" init-method="fooSetup" destroy="fooTeardown"/>
</beans>
```

74 . What are Inner Beans?

When wiring beans, if a bean element is embedded to a property tag directly, then that bean is said to the Inner Bean. The drawback of this bean is that it cannot be reused anywhere else.

75. What are the different types of bean injections?

There are two types of bean injections.

- By setter
- By constructor

76. What is Auto wiring?

You can wire the beans as you wish. But spring framework also does this work for you. It can auto wire the related beans together. All you have to do is just set the `autowire` attribute of bean tag to an `autowire` type.

```
<beans>
<bean id="bar" class="com.act.Foo" Autowire="autowire type"/>
</beans>
```

77 . What are different types of Autowire types?

There are four different types by which autowiring can be done.

?byName
?byType
?constructor
?autodetect

78. What are the different types of events related to Listeners?

There are a lot of events related to `ApplicationContext` of spring framework. All the events are subclasses of `org.springframework.context.Application-Event`. They are

- `ContextClosedEvent` – This is fired when the context is closed.
- `ContextRefreshedEvent` – This is fired when the context is initialized or refreshed.
- `RequestHandledEvent` – This is fired when the web context handles any request.

79. What is an Aspect?

An aspect is the cross-cutting functionality that you are implementing. It is the aspect of your application you are modularizing. An example of an aspect is logging. Logging is something that is required throughout an application. However, because applications tend to be broken down into layers based on functionality, reusing a logging module through inheritance does not make sense. However, you can create a logging aspect and apply it throughout your application using AOP.

80 . What is a Jointpoint?

A jointpoint is a point in the execution of the application where an aspect can be plugged in. This point could be a method being called, an exception being thrown, or even a field being modified. These are the points where your aspect's code can be inserted into the normal flow of your

application to add new behavior.

81. What is an Advice?

Advice is the implementation of an aspect. It is something like telling your application of a new behavior. Generally, and advice is inserted into an application at joinpoints.

82. What is a Pointcut?

A pointcut is something that defines at what joinpoints an advice should be applied. Advices can be applied at any joinpoint that is supported by the AOP framework. These Pointcuts allow you to specify where the advice can be applied.

83. What is an Introduction in AOP?

An introduction allows the user to add new methods or attributes to an existing class. This can then be introduced to an existing class without having to change the structure of the class, but give them the new behavior and state.

84. What is a Target?

A target is the class that is being advised. The class can be a third party class or your own class to which you want to add your own custom behavior. By using the concepts of AOP, the target class is free to center on its major concern, unaware to any advice that is being applied.

85. What is a Proxy?

A proxy is an object that is created after applying advice to a target object. When you think of client objects the target object and the proxy object are the same.

86. What is meant by Weaving?

The process of applying aspects to a target object to create a new proxy object is called as Weaving. The aspects are woven into the target object at the specified joinpoints.

87. What are the different points where weaving can be applied?

- Compile Time
- Classload Time
- Runtime

88. What are the different advice types in spring?

- Around : Intercepts the calls to the target method
- Before : This is called before the target method is invoked
- After : This is called after the target method is returned
- Throws : This is called when the target method throws an exception
- Around : `org.aopalliance.intercept.MethodInterceptor`
- Before : `org.springframework.aop.BeforeAdvice`
- After : `org.springframework.aop.AfterReturningAdvice`
- Throws : `org.springframework.aop.ThrowsAdvice`

89. What are the different types of AutoProxying?

- `BeanNameAutoProxyCreator`
- `DefaultAdvisorAutoProxyCreator`

- Metadata autoproxying

90. What kind of exceptions those spring DAO classes throw?

The spring DAO class does not throw any technology related exceptions such as SQLException. They throw exceptions which are subclasses of DataAccessException.

91. What is DataAccessException?

DataAccessException is a RuntimeException. This is an Unchecked Exception. The user is not forced to handle these kinds of exceptions.

92. How can you configure a bean to get DataSource from JNDI?

```
<bean id="dataSource" class="org.springframework.jndi.JndiObjectFactoryBean">
<property name="jndiName">
<value>java:comp/env/jdbc/myDatasource</value>
</property>
</bean>
```

93. How can you create a DataSource connection pool?

```
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
<property name="driver">
<value>${db.driver}</value>
</property>
<property name="url">
<value>${db.url}</value>
</property>
<property name="username">
<value>${db.username}</value>
</property>
<property name="password">
<value>${db.password}</value>
</property>
</bean>
```

94. How JDBC can be used more efficiently in spring framework?

JDBC can be used more efficiently with the help of a template class provided by spring framework called as JdbcTemplate.

95. How JdbcTemplate can be used?

With use of Spring JDBC framework the burden of resource management and error handling is reduced a lot. So it leaves developers to write the statements and queries to get the data to and from the database.

```
JdbcTemplate template = new JdbcTemplate(myDataSource);
```

A simple DAO class looks like this.

```
public class StudentDaoJdbc implements StudentDao {
    private JdbcTemplate jdbcTemplate;

    public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

```
}  
more..  
}
```

The configuration is shown below.

```
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">  
  <property name="dataSource">  
    <ref bean="dataSource"/>  
  </property>  
</bean>  
<bean id="studentDao" class="StudentDaoJdbc">  
  <property name="jdbcTemplate">  
    <ref bean="jdbcTemplate"/>  
  </property>  
</bean>  
<bean id="courseDao" class="CourseDaoJdbc">  
  <property name="jdbcTemplate">  
    <ref bean="jdbcTemplate"/>  
  </property>  
</bean>
```

96. How do you write data to backend in spring using JdbcTemplate?

The JdbcTemplate uses several of these callbacks when writing data to the database. The usefulness you will find in each of these interfaces will vary. There are two simple interfaces. One is PreparedStatementCreator and the other interface is BatchPreparedStatementSetter.

97. Explain about PreparedStatementCreator?

PreparedStatementCreator is one of the most common used interfaces for writing data to database. The interface has one method createPreparedStatement().

```
PreparedStatement createPreparedStatement(Connection conn)  
    throws SQLException;
```

When this interface is implemented, we should create and return a PreparedStatement from the Connection argument, and the exception handling is automatically taken care off. When this interface is implemented, another interface SqlProvider is also implemented which has a method called getSql() which is used to provide sql Strings to JdbcTemplate.

98. Explain about BatchPreparedStatementSetter?

If the user what to update more than one row at a shot then he can go for BatchPreparedStatementSetter. This interface provides two methods

```
setValues(PreparedStatement ps, int i) throws SQLException;  
int getBatchSize();
```

The getBatchSize() tells the JdbcTemplate class how many statements to create. And this also determines how many times setValues() will be called.

99. Explain about RowCallbackHandler and why it is used?

In order to navigate through the records we generally go for ResultSet. But spring provides an interface that handles this entire burden and leaves the user to decide what to do with each row. The interface provided by spring is RowCallbackHandler. There is a method processRow() which

needs to be implemented so that it is applicable for each and every row.

```
void processRow(java.sql.ResultSet rs);
```

100. What is JDBC abstraction and DAO module?

Using this module we can keep up the database code clean and simple, and prevent problems that result from a failure to close database resources. A new layer of meaningful exceptions on top of the error messages given by several database servers is brought in this module. In addition, this module uses Spring's AOP module to provide transaction management services for objects in a Spring application.